Программа расчёта уровня сформированности военно-профессиональных компетенций на основе модели полифункционального контроля

(«ПФК ВПК»)

Исходный текст программы для ЭВМ

Файл index.html · 567 Кб · 4783 строк

```
<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
<meta name="theme-color" content="#f2f2f7" media="(prefers-color-scheme: light)">
<meta name="theme-color" content="#000000" media="(prefers-color-scheme: dark)">
<title>ПФК ВПК — Программа расчёта уровня сформированности военно-профессиональных компетенций на основе модели полифункционального контроля</title>
<meta name="description" content="Автономное приложение для расчёта уровня сформированности военно-профессиональных компетенций по модели полифункционального контроля">
<style>
/* Оформление в стиле iOS: системные цвета, «стеклянные» материалы, сгруппированные списки, капсульные кнопки.
   Светлая и тёмная тема — по настройке системы. Логика приложения от оформления не зависит. */
:root{
  color-scheme:light dark;
  --bg:#f2f2f7; --bg-2:#e9e9f0; --surface:#ffffff; --surface-2:#f2f2f7; --elev:#ffffff;
  --line:rgba(60,60,67,.14); --line-2:rgba(60,60,67,.24);
  --text:#000000; --muted:rgba(60,60,67,.62); --faint:rgba(60,60,67,.42);
  --fill:rgba(118,118,128,.12); --fill-2:rgba(118,118,128,.20);
  --accent:#007AFF; --accent-ink:#ffffff; --accent-soft:rgba(0,122,255,.12);
  --danger:#FF3B30; --danger-ink:#D70015; --danger-soft:rgba(255,59,48,.12);
  --warn:#C93400; --warn-c:#FF9500; --warn-soft:rgba(255,149,0,.14);
  --ok:#248A3D; --ok-c:#34C759; --ok-soft:rgba(52,199,89,.14);
  --lv-high:#34C759; --lv-mid:#007AFF; --lv-low:#FF9500; --lv-crit:#FF3B30;
  --glass:rgba(255,255,255,.72); --glass-line:rgba(255,255,255,.65); --glass-shadow:0 8px 32px rgba(0,0,0,.10),0 1px 0 rgba(255,255,255,.6) inset;
  --grid:rgba(60,60,67,.12);
  --s1:4px; --s2:8px; --s3:12px; --s4:16px; --s5:20px; --s6:32px;
  --r:22px; --r-sm:12px; --shadow:0 1px 2px rgba(0,0,0,.04),0 4px 16px rgba(0,0,0,.04);
  --font:-apple-system,BlinkMacSystemFont,"SF Pro Text","SF Pro Display","Helvetica Neue","Segoe UI",system-ui,Roboto,Arial,sans-serif;
  --mono:ui-monospace,"SF Mono",Menlo,Consolas,"Courier New",monospace;
  --ease:cubic-bezier(.32,.72,0,1);
}
@media (prefers-color-scheme:dark){:root{
  --bg:#000000; --bg-2:#0c0c0e; --surface:#1c1c1e; --surface-2:#2c2c2e; --elev:#2c2c2e;
  --line:rgba(84,84,88,.55); --line-2:rgba(84,84,88,.8);
  --text:#ffffff; --muted:rgba(235,235,245,.62); --faint:rgba(235,235,245,.38);
  --fill:rgba(118,118,128,.24); --fill-2:rgba(118,118,128,.36);
  --accent:#0A84FF; --accent-soft:rgba(10,132,255,.20);
  --danger:#FF453A; --danger-ink:#FF6961; --danger-soft:rgba(255,69,58,.20);
  --warn:#FFB340; --warn-c:#FF9F0A; --warn-soft:rgba(255,159,10,.18);
  --ok:#30DB5B; --ok-c:#30D158; --ok-soft:rgba(48,209,88,.18);
  --lv-high:#30D158; --lv-mid:#0A84FF; --lv-low:#FF9F0A; --lv-crit:#FF453A;
  --glass:rgba(30,30,32,.70); --glass-line:rgba(255,255,255,.10); --glass-shadow:0 8px 32px rgba(0,0,0,.5),0 1px 0 rgba(255,255,255,.08) inset;
  --grid:rgba(235,235,245,.12); --shadow:none;
}}
*{box-sizing:border-box;-webkit-tap-highlight-color:transparent}
html{-webkit-text-size-adjust:100%}
html,body{margin:0;background:var(--bg);color:var(--text);font:16px/1.45 var(--font);letter-spacing:-.01em;-webkit-font-smoothing:antialiased}
a{color:var(--accent);text-decoration:none}
a:hover{text-decoration:underline}
h1{font-size:34px;font-weight:700;letter-spacing:-.025em;margin:4px 0 var(--s3);line-height:1.15}
h2{font-size:22px;font-weight:700;letter-spacing:-.02em;margin:var(--s5) 0 var(--s3)}
h3{font-size:17px;font-weight:600;margin:var(--s4) 0 var(--s2)}
p{margin:0 0 var(--s3)}
small,.muted{color:var(--muted)}
.mono{font-family:var(--mono)}
::selection{background:var(--accent-soft)}

/* ---------- Каркас ---------- */
.app{display:grid;grid-template-columns:300px 1fr;min-height:100vh}
.side{position:sticky;top:0;height:100vh;overflow:auto;padding:12px 10px 16px 12px;scrollbar-width:thin}
.side::-webkit-scrollbar{width:6px}.side::-webkit-scrollbar-thumb{background:var(--fill-2);border-radius:3px}
.brand{display:flex;align-items:center;gap:12px;font-weight:700;font-size:17px;letter-spacing:-.02em;line-height:1.2;padding:8px 8px 14px}
.brand small{display:block;color:var(--muted);font-weight:400;font-size:12px;letter-spacing:0;margin-top:2px}
```

```
.brand .logo{flex:none;width:40px;height:40px;border-radius:11px;display:grid;place-items:center;background:linear-gradient(160deg,#5AC8FA,#007AFF 55%,#5856D6);box-shadow:0 4px 12px rgba(0,122,255,.35)}
.brand .logo svg{width:24px;height:24px;stroke:#fff;stroke-width:2;fill:none;stroke-linecap:round;stroke-linejoin:round}
.nav-search{padding:0 4px 10px}
.nav-search input{background:var(--fill);border:none;border-radius:10px;padding:8px 12px 8px 32px;font-size:15px;
  background-image:url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 24 24' fill='none' stroke='%238e8e93' stroke-width='2.4' stroke-linecap='round'%3E%3Ccircle cx='11' cy='11' r='7'/%3E%3Cpath d='m20 20-3.5-3.5'/%3E%3C/svg%3E");background-repeat:no-repeat;background-position:10px 50%;background-size:15px}
.nav .sep{font-size:13px;font-weight:600;color:var(--muted);margin:16px 12px 6px;letter-spacing:0}
.nav .grp{background:var(--surface);border-radius:14px;overflow:hidden;margin:0 4px 6px;box-shadow:var(--shadow)}
.nav a{display:flex;align-items:center;gap:12px;color:var(--text);text-decoration:none;padding:7px 12px 7px 10px;font-size:15px;position:relative;min-height:44px;transition:background .15s}
.nav a+a::before{content:"";position:absolute;left:50px;right:0;top:0;border-top:.5px solid var(--line)}
.nav a:hover{background:var(--fill)}
.nav a.active{background:var(--accent);color:#fff}
.nav a.active+a::before,.nav a.active::before{border-color:transparent}
.nav a .t{flex:1;min-width:0}
.ic{flex:none;width:29px;height:29px;border-radius:7px;background:var(--c,#8e8e93);display:grid;place-items:center}
.ic svg{width:17px;height:17px;stroke:#fff;stroke-width:2;fill:none;stroke-linecap:round;stroke-linejoin:round}
.nav a.active .ic{background:rgba(255,255,255,.22)}
.nav a.hide{display:none}
.nav .grp.empty-grp,.nav .sep.empty-grp{display:none}
.storage{margin:14px 12px 0;font-size:12px;color:var(--muted)}
.main{padding:28px 40px 60px;max-width:1240px;width:100%;min-width:0}
.topbar{display:none}
.side-scrim{display:none}
.tabbar{display:none}

/* ---------- Карточки и сетки ---------- */
.card{background:var(--surface);border:none;border-radius:var(--r);box-shadow:var(--shadow);padding:20px 22px;margin-bottom:var(--s4)}
.card h2:first-child,.card h3:first-child{margin-top:0}
.card>h2{font-size:20px}
.grid{display:grid;gap:var(--s4)}
.g2{grid-template-columns:repeat(2,minmax(0,1fr))}
.g3{grid-template-columns:repeat(3,minmax(0,1fr))}
.g4{grid-template-columns:repeat(4,minmax(0,1fr))}
.row{display:flex;gap:var(--s2);flex-wrap:wrap;align-items:center}
.spacer{flex:1}
.kpi{background:var(--surface);border:none;border-radius:20px;padding:16px 18px;box-shadow:var(--shadow)}
.kpi .l{font-size:13px;color:var(--muted);font-weight:600;letter-spacing:0}
.kpi .v{font-size:34px;font-weight:700;letter-spacing:-.03em;font-variant-numeric:tabular-nums;line-height:1.15;margin-top:4px}
.kpi .s{font-size:14px;color:var(--muted)}

/* ---------- Быстрые действия главной ---------- */
.quick{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:12px;margin:8px 0 20px}
.quick a{display:flex;flex-direction:column;gap:10px;padding:16px;border-radius:20px;background:var(--surface);color:var(--text);text-decoration:none;font-weight:600;font-size:15px;box-shadow:var(--shadow);transition:transform .2s var(--ease)}
.quick a:hover{text-decoration:none;transform:scale(1.02)}
.quick a:active{transform:scale(.97)}
.quick a small{font-weight:400;font-size:13px}
.quick .ic{width:36px;height:36px;border-radius:10px}
.quick .ic svg{width:20px;height:20px}
.eyebrow{font-size:13px;font-weight:600;color:var(--muted);text-transform:uppercase;letter-spacing:.02em;margin:0}

/* ---------- Кнопки (капсулы) ---------- */
.btn{display:inline-flex;align-items:center;justify-content:center;gap:6px;border:none;background:var(--fill);color:var(--accent);border-radius:999px;padding:9px 18px;font:inherit;font-size:15px;font-weight:600;cursor:pointer;text-decoration:none;line-height:1.25;letter-spacing:-.01em;transition:transform .15s var(--ease),filter .15s,background .15s;min-height:36px}
.btn:hover{background:var(--fill-2);text-decoration:none}
.btn:active{transform:scale(.96)}
.btn.primary{background:var(--accent);color:#fff;box-shadow:0 2px 10px rgba(0,122,255,.28)}
.btn.primary:hover{filter:brightness(1.08);background:var(--accent)}
```

```
.btn.danger{color:var(--danger);background:var(--danger-soft)}
.btn.danger:hover{background:var(--danger-soft);filter:brightness(.97)}
.btn.sm{padding:5px 12px;font-size:13px;min-height:28px}
.btn[disabled]{opacity:.4;cursor:not-allowed;transform:none}
:focus-visible{outline:3px solid color-mix(in srgb,var(--accent) 55%,transparent);outline-offset:2px}

/* ---------- Поля ввода ---------- */
label.f{display:block;font-size:13px;color:var(--muted);margin:0 0 5px 4px;font-weight:500}
input[type=text],input[type=number],input[type=date],input[type=search],input[type=password],select,textarea{width:100%;font:inherit;font-size:16px;padding:10px 14px;border:1px solid transparent;border-radius:12px;background:var(--fill);color:var(--text);transition:background .15s,border-color .15s,box-shadow .15s;-webkit-appearance:none;appearance:none}
input:focus,select:focus,textarea:focus{outline:none;background:var(--surface);border-color:var(--accent);box-shadow:0 0 0 4px var(--accent-soft)}
select{padding-right:36px;background-image:url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 24 24' fill='none' stroke='%238e8e93' stroke-width='2.5' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpath d='m7 9 5-5 5 5M7 15l5 5 5-5'/%3E%3C/svg%3E");background-repeat:no-repeat;background-position:right 12px center;background-size:14px}
select[multiple]{background-image:none;padding-right:14px}
.row>input,.row>select{min-width:0;flex:1 1 0}
.grid>*{min-width:0}
input[type=date]{min-width:0}
input::placeholder,textarea::placeholder{color:var(--faint)}
textarea{min-height:84px;resize:vertical}
input.cell{width:64px;padding:6px 6px;text-align:center;border-radius:9px;font-size:15px}
input.invalid,select.invalid{border-color:var(--danger);background:var(--danger-soft)}
input[type=checkbox],input[type=radio]{accent-color:var(--accent);width:18px;height:18px;vertical-align:-3px}
input[type=file]{font:inherit;font-size:14px;color:var(--muted)}
input[type=file]::file-selector-button{font:inherit;font-weight:600;border:none;border-radius:999px;background:var(--fill);color:var(--accent);padding:8px 16px;margin-right:10px;cursor:pointer}
input[type=range]{accent-color:var(--accent)}
.field{margin-bottom:var(--s4)}
.hint{font-size:13px;color:var(--muted);margin-top:4px}

/* ---------- Таблицы (сгруппированный список) ---------- */
.table-wrap{overflow-x:auto;-webkit-overflow-scrolling:touch;border-radius:14px}
table{border-collapse:separate;border-spacing:0;width:100%;font-size:15px}
th,td{border-bottom:.5px solid var(--line);padding:10px 10px;text-align:left;vertical-align:top}
th{background:transparent;font-weight:600;font-size:12px;color:var(--muted);white-space:nowrap;text-transform:uppercase;letter-spacing:.02em;border-bottom:.5px solid var(--line-2)}
tbody tr:last-child td{border-bottom:none}
td.num,th.num{text-align:right;font-variant-numeric:tabular-nums;white-space:nowrap}
td.c,th.c{text-align:center}
tr.click{cursor:pointer;transition:background .12s}
tr.click:hover td{background:var(--fill)}
tr.click:active td{background:var(--fill-2)}
tfoot td{font-weight:700;background:var(--surface-2);border-top:.5px solid var(--line-2)}

/* ---------- Метки ---------- */
.badge{display:inline-block;padding:3px 10px;border-radius:999px;font-size:12px;font-weight:600;border:none;white-space:nowrap}
.b-draft{background:var(--warn-soft);color:var(--warn)}
.b-final{background:var(--ok-soft);color:var(--ok)}
.b-info{background:var(--accent-soft);color:var(--accent)}
.lv{display:inline-block;padding:3px 11px;border-radius:999px;font-size:13px;font-weight:600;color:#fff;white-space:nowrap}
.lv-high{background:var(--lv-high)} .lv-mid{background:var(--lv-mid)} .lv-low{background:var(--lv-low)} .lv-crit{background:var(--lv-crit)}
.lv-none{background:var(--fill);color:var(--muted)}

/* ---------- Уведомления ---------- */
.note{border:none;background:var(--fill);padding:12px 14px 12px 40px;border-radius:14px;font-size:14.5px;margin:var(--s2) 0;position:relative}
.note::before{content:"i";position:absolute;left:12px;top:12px;width:18px;height:18px;border-radius:50%;background:var(--muted);color:var(--surface);font:700 12px/18px var(--font);text-align:center}
.note.warn{background:var(--warn-soft)} .note.warn::before{content:"!";background:var(--warn-c);color:#fff}
.note.err{background:var(--danger-soft)} .note.err::before{content:"!";background:var(--danger);color:#fff}
.note.ok{background:var(--ok-soft)} .note.ok::before{content:"✓";background:var(--ok-c);color:#fff;font-size:11px}
.src{font-size:12px;color:var(--faint)}

```

```
/* ---------- Шаги мастера ---------- */
.steps{display:flex;gap:6px;flex-wrap:wrap;margin:var(--s3) 0 var(--s4);padding:4px;list-style:none;background:var(--fill);border-radius:999px;width:fit-content;max-width:100%}
.steps li{font-size:14px;font-weight:500;padding:6px 14px;border-radius:999px;color:var(--muted)}
.steps li.cur{background:var(--surface);color:var(--text);font-weight:600;box-shadow:0 2px 8px rgba(0,0,0,.12)}
.steps li.done{color:var(--accent)}

/* ---------- Оценки: сегментированный переключатель ---------- */
.gradebar{display:inline-flex;background:var(--fill);border:none;border-radius:10px;padding:2px;gap:2px}
.gradebar label{position:relative;cursor:pointer}
.gradebar input{position:absolute;opacity:0;width:1px;height:1px;left:50%;top:50%;margin:0;pointer-events:none}
.gradebar span{display:block;min-width:36px;text-align:center;padding:5px 0;font-weight:600;font-size:15px;border:none;border-radius:8px;background:transparent;color:var(--text);transition:background .15s,color .15s,box-shadow .15s}
.gradebar label:hover span{background:var(--fill)}
.gradebar input:checked+span{background:var(--accent);color:#fff;box-shadow:0 2px 6px rgba(0,122,255,.35)}
.gradebar input:focus-visible+span{outline:2px solid var(--accent);outline-offset:1px}
.gradebar .clr span{min-width:30px;color:var(--faint);font-weight:400}
tr.missing td:first-child{box-shadow:inset 3px 0 0 var(--warn-c)}
.progress{height:6px;background:var(--fill);border-radius:3px;overflow:hidden;margin-top:6px}
.progress>i{display:block;height:100%;background:var(--accent);border-radius:3px}
.bar-track{flex:1;background:var(--fill);border-radius:5px;height:16px;overflow:hidden}
.empty{text-align:center;padding:40px 16px;color:var(--muted)}
.empty .btn{margin-top:var(--s3)}

/* ---------- Вкладки: сегментированный контрол ---------- */
.tabs{display:flex;gap:2px;background:var(--fill);border:none;border-radius:12px;padding:3px;margin-bottom:var(--s4);flex-wrap:wrap;width:fit-content;max-width:100%}
.tabs a{padding:6px 14px;text-decoration:none;color:var(--text);border:none;border-radius:9px;font-size:14px;font-weight:500;transition:background .15s}
.tabs a:hover{background:var(--fill);text-decoration:none}
.tabs a.active{background:var(--surface);color:var(--text);font-weight:600;box-shadow:0 2px 8px rgba(0,0,0,.12),0 0 0 .5px rgba(0,0,0,.04)}
.formula{font-family:"New York",Cambria,"Times New Roman",serif;font-size:17px;background:var(--fill);border:none;border-radius:10px;padding:6px 12px;display:inline-block;margin:var(--s1) 0}
dl.spec{display:grid;grid-template-columns:200px 1fr;gap:6px var(--s3);margin:var(--s2) 0;font-size:15px}
dl.spec dt{color:var(--muted)} dl.spec dd{margin:0}
details{border-radius:12px}
details>summary{cursor:pointer;color:var(--accent);font-weight:600;list-style:none;display:flex;align-items:center;gap:6px}
details>summary::-webkit-details-marker{display:none}
details>summary::before{content:"";width:8px;height:8px;border-right:2px solid currentColor;border-bottom:2px solid currentColor;transform:rotate(-45deg);transition:transform .2s var(--ease);flex:none;margin:0 4px 2px 2px}
details[open]>summary::before{transform:rotate(45deg)}
details.calc{margin-top:var(--s3)}

/* ---------- Графики ---------- */
.chart{width:100%;height:auto;display:block}
.chart text{font:11px var(--font);fill:var(--muted)}
.chart line[stroke="#e5e7eb"],.chart line[stroke="#f1f3f5"],.chart polygon[stroke="#e5e7eb"]{stroke:var(--grid)}
.chart line[stroke="#9aa3ad"]{stroke:var(--line-2)}
.legend{display:flex;gap:var(--s4);flex-wrap:wrap;font-size:13px;color:var(--muted);margin-top:var(--s2)}
.legend i{display:inline-block;width:12px;height:4px;vertical-align:middle;margin-right:6px;border-radius:2px}

/* ---------- Всплывающие уведомления (капсулы сверху) ---------- */
.toast-wrap{position:fixed;left:50%;top:14px;transform:translateX(-50%);display:flex;flex-direction:column;align-items:center;gap:8px;z-index:60;pointer-events:none;width:max-content;max-width:calc(100% - 32px)}
.toast{background:var(--glass);-webkit-backdrop-filter:blur(30px) saturate(1.8);backdrop-filter:blur(30px) saturate(1.8);color:var(--text);padding:11px 20px;font-size:15px;font-weight:500;box-shadow:var(--glass-shadow);border:.5px solid var(--glass-line);max-width:100%;border-radius:24px;animation:toastIn .45s var(--ease)}
.toast.err{color:var(--danger-ink)}
@keyframes toastIn{from{opacity:0;transform:translateY(-16px) scale(.9)}to{opacity:1;transform:none}}

/* ---------- Диалоги (листы) ---------- */
```

```
dialog{border:none;border-radius:28px;padding:0;max-width:540px;width:calc(100% - 32px);background:var(--elev);color:var(--text);box-shadow:0 24px 80px rgba(0,0,0,.30)}
dialog[open]{animation:sheetIn .4s var(--ease)}
@keyframes sheetIn{from{opacity:0;transform:scale(.94) translateY(10px)}to{opacity:1;transform:none}}
dialog::backdrop{background:rgba(0,0,0,.32);-webkit-backdrop-filter:blur(2px);backdrop-filter:blur(2px)}
dialog .dh{padding:24px 24px 0;font-weight:700;font-size:20px;letter-spacing:-.02em}
dialog .db{padding:10px 24px 12px;color:var(--text);max-height:65vh;overflow:auto}
dialog .df{padding:12px 20px 20px;display:flex;gap:var(--s2);justify-content:flex-end;flex-wrap:wrap}
dialog .df .btn{min-width:110px}

/* ---------- Учебный режим ---------- */
.mode-banner{position:sticky;top:0;z-index:35;display:flex;align-items:center;gap:10px;flex-wrap:wrap;padding:8px 20px;font-size:14px;background:color-mix(in srgb,var(--warn-c) 18%,var(--bg));color:var(--text);border-bottom:.5px solid var(--line);-webkit-backdrop-filter:blur(20px);backdrop-filter:blur(20px)}
.mode-banner[hidden]{display:none}
.mode-banner .dot{width:9px;height:9px;border-radius:50%;background:var(--warn-c);box-shadow:0 0 0 4px var(--warn-soft)}

/* ---------- Печатные формы — всегда «бумага» ---------- */
.report{background:#fff;color:#000}
.report h1{font-size:22px}
.only-print{display:none}
.blank{font-family:"Times New Roman",serif;font-size:14px;color:#000;background:#fff;letter-spacing:0}
.card .blank{border-radius:6px;padding:16px;box-shadow:0 0 0 .5px var(--line)}
.blank p{margin:6px 0}
table.bl{border-collapse:collapse;width:100%;margin:6px 0;font-size:13px}
table.bl th,table.bl td{border:1px solid #000;padding:3px 5px;background:none;white-space:normal;color:#000;text-transform:none;letter-spacing:0;font-size:13px}
abbr[title]{text-decoration:underline dotted;cursor:help;text-underline-offset:2px}
@media print{abbr[title]{text-decoration:none}}

/* ---------- Навигация v2: разделы, вкладки, «Назад» ---------- */
.nav .t small{display:block;font-size:12px;color:var(--muted);line-height:1.25;margin-top:1px}
.nav a.active .t small{color:rgba(255,255,255,.8)}
.nav a.sec.open:not(.active){background:var(--fill)}
.nav .sub{padding:4px 0 8px 52px;background:var(--fill)}
.nav .sub a{min-height:34px;padding:6px 12px;font-size:14px;border-radius:9px;margin-right:8px}
.nav .sub a::before{display:none}
.nav .sub-l{font-size:11px;font-weight:600;text-transform:uppercase;letter-spacing:.03em;color:var(--muted);padding:8px 12px 2px}
.backbar{display:flex;align-items:center;gap:10px;margin:-6px 0 10px;min-height:36px}
.backbar .back{display:inline-flex;align-items:center;gap:2px;color:var(--accent);font-size:17px;font-weight:500;text-decoration:none;padding:6px 10px 6px 2px;border-radius:10px;max-width:70%}
.backbar .back span{white-space:nowrap;overflow:hidden;text-overflow:ellipsis}
.backbar .back:hover{background:var(--fill);text-decoration:none}
.backbar svg{width:24px;height:24px;stroke:currentColor;stroke-width:2.6;fill:none;stroke-linecap:round;stroke-linejoin:round;flex:none}
.backbar .crumb{margin-left:auto;font-size:13px;color:var(--muted)}
.main:focus{outline:none}
@media (min-width:761px){.sectabs{display:none!important}}
.sectabs{display:flex;flex-wrap:wrap;gap:8px 18px;margin:0 0 18px;align-items:flex-end}
.sectabs .tg{display:flex;flex-direction:column;gap:4px;min-width:0;max-width:100%}
.sectabs .tg-l{font-size:11px;font-weight:600;text-transform:uppercase;letter-spacing:.03em;color:var(--muted);margin-left:8px}
.sectabs .tabs{margin:0}
.toast.has-act{display:flex;align-items:center;gap:14px;pointer-events:auto}
.toast-btn{border:none;background:var(--accent);color:#fff;font:600 14px var(--font);border-radius:999px;padding:6px 14px;cursor:pointer}
.busy{position:fixed;inset:0;z-index:80;display:grid;place-items:center;background:rgba(0,0,0,.18)}
.busy[hidden]{display:none}
.busy-box{display:flex;align-items:center;gap:12px;padding:16px 22px;border-radius:20px;background:var(--glass);-webkit-backdrop-filter:blur(30px);backdrop-filter:blur(30px);box-shadow:var(--glass-shadow);font-weight:600}
.spin{width:22px;height:22px;border-radius:50%;border:3px solid var(--fill-2);border-top-color:var(--accent);animation:spin .8s linear infinite}
@keyframes spin{to{transform:rotate(360deg)}}
.state-ic{width:52px;height:52px;margin:0 auto 10px;border-radius:50%;display:grid;place-items:center;background:var(--fill);color:var(--muted);font-size:24px;font-weight:700}
.state-ic.err{background:var(--danger-soft);color:var(--danger)}
.btn .bi{width:18px;height:18px;stroke:currentColor;stroke-width:2.4;fill:none;stroke-linecap:round;margin-left:-4px}
.obj-head{display:flex;align-items:flex-start;gap:12px;flex-wrap:wrap;margin-bottom:6px}
.obj-head h1{margin:0}
```

```
.obj-head .avatar{width:52px;height:52px;border-radius:50%;display:grid;place-items:center;background:linear-gradient(160deg,#8e8ef0,#5856D6);color:#fff;font-weight:700;font-size:20px;flex:none}
.obj-head .meta{flex:1;min-width:200px}
.quickadd{display:grid;grid-template-columns:minmax(0,2fr) minmax(0,1fr) auto;gap:10px;align-items:end}
.onboard ol{margin:0;padding:0;list-style:none;counter-reset:st}
.onboard li{display:flex;gap:14px;align-items:flex-start;padding:12px 0;border-top:.5px solid var(--line)}
.onboard li:first-child{border-top:none}
.onboard .n{flex:none;width:30px;height:30px;border-radius:50%;display:grid;place-items:center;background:var(--accent-soft);color:var(--accent);font-weight:700}
.onboard li.done .n{background:var(--ok-c);color:#fff}
.onboard li.done b{text-decoration:line-through;color:var(--muted)}
.pagehead{display:flex;align-items:center;gap:10px;flex-wrap:wrap;margin-bottom:4px}
.pagehead h1{margin:0}
.pagehead .spacer{flex:1}
@media (max-width:760px){
  .quickadd{grid-template-columns:1fr}
  .backbar .crumb{display:none}
  .sectabs{flex-wrap:nowrap;overflow-x:auto;margin:0 -16px 14px;padding:0 16px;scrollbar-width:none}
  .sectabs::-webkit-scrollbar{display:none}
  .sectabs .tg{flex:none}
  .sectabs .tabs{flex-wrap:nowrap;width:max-content}
  .sectabs .tabs a{white-space:nowrap}
  .tabbar a.active{color:var(--accent);background:var(--fill)}
  nav.tabs{flex-wrap:nowrap;overflow-x:auto;width:auto;scrollbar-width:none}
  nav.tabs::-webkit-scrollbar{display:none}
  nav.tabs a{white-space:nowrap;flex:none}
}

@media (prefers-reduced-motion:reduce){*,*::before,*::after{animation:none!important;transition:none!important}}

/* ---------- Планшет ---------- */
@media (max-width:1100px){.g4,.quick{grid-template-columns:repeat(2,minmax(0,1fr))}.g3{grid-template-columns:1fr 1fr}.app{grid-template-columns:272px 1fr}.main{padding:24px 24px 60px}}

/* ---------- Телефон: «большой заголовок», плавающая панель вкладок, разделы — лист ---------- */
@media (max-width:760px){
  html,body{font-size:16px}
  .app{grid-template-columns:1fr}
  .side{position:fixed;left:8px;right:8px;top:auto;bottom:8px;height:auto;max-height:82vh;z-index:45;border-radius:30px;padding:10px 8px 14px;
    background:var(--bg);box-shadow:0 -10px 60px rgba(0,0,0,.25);transform:translateY(calc(100% + 24px));transition:transform .45s var(--ease);visibility:hidden}
  .side::before{content:"";display:block;width:38px;height:5px;border-radius:3px;background:var(--fill-2);margin:2px auto 8px}
  .side.open{transform:none;visibility:visible}
  .side-scrim{display:block;position:fixed;inset:0;background:rgba(0,0,0,.3);z-index:44;opacity:0;pointer-events:none;transition:opacity .3s}
  .side.open~.side-scrim{opacity:1;pointer-events:auto}
  .side-scrim[hidden]{display:block}
  .topbar{display:flex;align-items:center;justify-content:center;position:sticky;top:0;z-index:30;padding:12px 16px;font-size:17px;
    background:var(--glass);-webkit-backdrop-filter:blur(24px) saturate(1.8);backdrop-filter:blur(24px) saturate(1.8);border-bottom:.5px solid var(--line)}
  .mode-banner{position:static;padding:8px 16px;font-size:13px}
  .tabbar{display:flex;position:fixed;left:12px;right:12px;bottom:max(12px,env(safe-area-inset-bottom));z-index:40;justify-content:space-around;padding:6px;border-radius:999px;
    background:var(--glass);-webkit-backdrop-filter:blur(30px) saturate(1.9);backdrop-filter:blur(30px) saturate(1.9);border:.5px solid var(--glass-line);box-shadow:var(--glass-shadow)}
  .tabbar a,.tabbar button{flex:1;display:flex;flex-direction:column;align-items:center;gap:1px;padding:6px 2px 4px;border-radius:999px;border:none;background:none;color:var(--text);font:600 10px/1.2 var(--font);text-decoration:none;cursor:pointer;min-width:0}
  .tabbar span{white-space:nowrap;overflow:hidden;text-overflow:ellipsis;max-width:100%}
  .tabbar svg{width:24px;height:24px;stroke:currentColor;stroke-width:1.9;fill:none;stroke-linecap:round;stroke-linejoin:round}
  .tabbar a.active{color:var(--accent);background:var(--fill)}
  .main{padding:8px 16px 110px}
  h1{font-size:30px}
  .g2,.g3,.g4{grid-template-columns:1fr}
  .quick{grid-template-columns:1fr 1fr;gap:10px}
  .card{padding:16px;border-radius:20px}
  dl.spec{grid-template-columns:1fr}
  .kpi .v{font-size:28px}
```

```
  .toast-wrap{top:56px}
  dialog{margin:auto auto 8px;max-width:none;width:calc(100% - 16px);border-radius:30px}
  dialog[open]{animation:sheetUp .45s var(--ease)}
  dialog .df .btn{flex:1}
  .steps,.tabs{width:auto}
}
@keyframes sheetUp{from{transform:translateY(60%);opacity:.4}to{transform:none;opacity:1}}

/* ======================================================================
   «Жидкое стекло»: полупрозрачные материалы с размытием и насыщением,
   блик по верхней кромке, свечение под курсором, пружинные анимации.
   Без поддержки backdrop-filter — плотные непрозрачные поверхности.
   ====================================================================== */
:root{
  --spring:cubic-bezier(.34,1.56,.64,1);
  --g-bg:rgba(255,255,255,.58); --g-bg-2:rgba(255,255,255,.42); --g-bg-hi:rgba(255,255,255,.78);
  --g-edge:rgba(255,255,255,.75); --g-edge-2:rgba(255,255,255,.25);
  --g-shadow:0 1px 0 rgba(255,255,255,.85) inset,0 -1px 0 rgba(255,255,255,.25) inset,0 10px 30px -12px rgba(30,40,90,.22),0 2px 6px rgba(30,40,90,.06);
  --g-blur:blur(28px) saturate(185%);
  --glow:0 0 0 1px rgba(0,122,255,.25),0 8px 28px -6px rgba(0,122,255,.55);
  --hl:rgba(255,255,255,.75);
  --mesh-1:#b9d2ff; --mesh-2:#ffcfe6; --mesh-3:#c4f0df; --mesh-4:#ddd0ff;
}
@media (prefers-color-scheme:dark){:root{
  --g-bg:rgba(44,44,52,.52); --g-bg-2:rgba(44,44,52,.36); --g-bg-hi:rgba(70,70,82,.7);
  --g-edge:rgba(255,255,255,.16); --g-edge-2:rgba(255,255,255,.05);
  --g-shadow:0 1px 0 rgba(255,255,255,.14) inset,0 -1px 0 rgba(255,255,255,.04) inset,0 14px 36px -12px rgba(0,0,0,.7),0 2px 6px rgba(0,0,0,.3);
  --glow:0 0 0 1px rgba(10,132,255,.45),0 8px 30px -6px rgba(10,132,255,.7);
  --hl:rgba(255,255,255,.22);
  --mesh-1:#14254d; --mesh-2:#3a1636; --mesh-3:#0d3329; --mesh-4:#261b4a;
}}
/* Цветная подложка, которую «преломляет» стекло */
html{background:var(--bg)}
body{background:transparent}
body::before{content:"";position:fixed;inset:-20vmax;z-index:-1;pointer-events:none;
  background:radial-gradient(38vmax 38vmax at 12% 18%,var(--mesh-1),transparent 70%),radial-gradient(34vmax 34vmax at 88% 12%,var(--mesh-2),transparent 70%),
  radial-gradient(40vmax 40vmax at 80% 85%,var(--mesh-3),transparent 70%),radial-gradient(36vmax 36vmax at 18% 88%,var(--mesh-4),transparent 70%);
  animation:meshDrift 38s ease-in-out infinite alternate}
@keyframes meshDrift{0%{transform:translate3d(0,0,0) rotate(0deg)}50%{transform:translate3d(-3vmax,2vmax,0) rotate(6deg)}100%{transform:translate3d(2vmax,-2vmax,0) rotate(-4deg)}}

/* Материал стекла */
.card,.kpi,.quick a,.nav .grp,.tabs,.steps,.gradebar,.toast,.busy-box,.bulkbar,.backbar .back,.tabbar,.mode-banner{
  background:var(--g-bg);-webkit-backdrop-filter:var(--g-blur);backdrop-filter:var(--g-blur);
  border:1px solid var(--g-edge);box-shadow:var(--g-shadow)}
.card,.kpi,.quick a{position:relative;overflow:hidden;isolation:isolate}
.card::before,.kpi::before,.quick a::before{content:"";position:absolute;inset:0;border-radius:inherit;pointer-events:none;z-index:-1;
  background:linear-gradient(160deg,var(--hl) 0%,transparent 38%,transparent 70%,var(--g-edge-2) 100%);opacity:.55}
.card .card,.card .kpi{background:var(--g-bg-2)}
.table-wrap{background:transparent}
.side{background:transparent}
@media (min-width:761px){
  .side{top:12px;height:calc(100vh - 24px);margin:12px 0 12px 12px;border-radius:30px;padding:14px 10px 16px;
    background:var(--g-bg-2);-webkit-backdrop-filter:var(--g-blur);backdrop-filter:var(--g-blur);border:1px solid var(--g-edge);box-shadow:var(--g-shadow)}
  .app{grid-template-columns:312px 1fr}
}
.nav .grp{border-radius:18px;background:var(--g-bg-hi)}
.nav .sub{background:transparent}
dialog{background:var(--g-bg-hi);-webkit-backdrop-filter:blur(40px) saturate(190%);backdrop-filter:blur(40px) saturate(190%);border:1px solid var(--g-edge);box-shadow:var(--g-shadow),0 30px 90px rgba(0,0,0,.3)}
dialog::backdrop{background:rgba(10,14,30,.22);-webkit-backdrop-filter:blur(8px) saturate(120%);backdrop-filter:blur(8px) saturate(120%)}
input[type=text],input[type=number],input[type=date],input[type=search],input[type=password],select,textarea{background-color:var(--g-bg-2);border:1px solid var(--g-edge-2);box-shadow:0 1px 2px rgba(0,0,0,.0
```

```
4) inset}
input:hover,select:hover,textarea:hover{border-color:var(--g-edge)}
input:focus,select:focus,textarea:focus{background-color:var(--g-bg-hi);box-shadow:0 0 0 4px var(--accent-soft),0 6px 20px -8px rgba(0,122,255,.45)}

/* Кнопки: стеклянная капсула, блик, свечение под курсором */
.btn,.tabs a,.nav a,.quick a,.tabbar a,.tabbar button,.toast-btn,.backbar .back,.gradebar span{
  position:relative;overflow:hidden;isolation:isolate;
  transition:transform .35s var(--spring),box-shadow .3s var(--ease),background-color .25s,color .2s,filter .25s}
.btn{background:var(--g-bg);border:1px solid var(--g-edge);box-shadow:var(--g-shadow)}
.btn::after,.tabs a::after,.nav a::after,.quick a::after,.tabbar a::after,.tabbar button::after,.backbar .back::after,.gradebar span::after{
  content:"";position:absolute;inset:0;border-radius:inherit;pointer-events:none;z-index:-1;opacity:0;transition:opacity .3s;
  background:radial-gradient(140px circle at var(--mx,50%) var(--my,50%),rgba(255,255,255,.55),rgba(255,255,255,0) 60%)}
.btn:hover,.quick a:hover{transform:translateY(-2px)}
.btn:hover::after,.tabs a:hover::after,.nav a:hover::after,.quick a:hover::after,.tabbar a:hover::after,.tabbar button:hover::after,.backbar .back:hover::after,.gradebar label:hover span::after{opacity:1}
.btn:hover{background:var(--g-bg-hi);box-shadow:var(--g-shadow),0 10px 26px -10px rgba(0,122,255,.45);text-decoration:none}
.btn:active,.quick a:active,.tabs a:active,.nav a:active,.tabbar a:active,.tabbar button:active{transform:scale(.95);transition-duration:.12s}
.btn.primary{background:linear-gradient(180deg,#3d9bff 0%,#007AFF 55%,#0062d6 100%);border-color:rgba(255,255,255,.35);color:#fff;
  box-shadow:0 1px 0 rgba(255,255,255,.45) inset,0 -1px 0 rgba(0,0,0,.12) inset,0 6px 18px -6px rgba(0,122,255,.65)}
.btn.primary:hover{background:linear-gradient(180deg,#4fa6ff 0%,#0a84ff 55%,#006ae6 100%);box-shadow:0 1px 0 rgba(255,255,255,.5) inset,var(--glow);filter:none}
.btn.primary::after{background:radial-gradient(120px circle at var(--mx,50%) var(--my,50%),rgba(255,255,255,.45),rgba(255,255,255,0) 60%)}
.btn.danger{background:linear-gradient(180deg,rgba(255,59,48,.10),rgba(255,59,48,.16));border-color:rgba(255,59,48,.28)}
.btn.danger:hover{background:linear-gradient(180deg,rgba(255,59,48,.16),rgba(255,59,48,.24));box-shadow:var(--g-shadow),0 0 0 1px rgba(255,59,48,.3),0 8px 26px -8px rgba(255,59,48,.6)}
.btn[disabled]:hover{transform:none;box-shadow:var(--g-shadow)}
.quick a:hover{box-shadow:var(--g-shadow),0 18px 40px -16px rgba(30,40,120,.35)}
.quick a .ic{transition:transform .45s var(--spring)}
.quick a:hover .ic{transform:scale(1.12) rotate(-6deg)}
.kpi{transition:transform .35s var(--spring),box-shadow .3s}
.kpi .lv{white-space:normal}
.kpi:hover{transform:translateY(-2px)}

/* Меню */
.nav a:hover{background:rgba(255,255,255,.35)}
@media (prefers-color-scheme:dark){.nav a:hover{background:rgba(255,255,255,.07)}}
.nav a.active{background:linear-gradient(180deg,#3d9bff,#007AFF);box-shadow:0 1px 0 rgba(255,255,255,.35) inset,0 6px 16px -6px rgba(0,122,255,.6)}
.nav .sub a.active{color:#fff}
.nav a .ic{transition:transform .4s var(--spring)}
.nav a:hover .ic{transform:scale(1.1)}
.nav .sub{animation:subOpen .45s var(--ease)}
@keyframes subOpen{from{opacity:0;transform:translateY(-6px)}to{opacity:1;transform:none}}

/* Вкладки и сегменты: активный сегмент — «капля» стекла */
.tabs{background:var(--g-bg-2)}
.tabs a.active,.steps li.cur{background:var(--g-bg-hi);box-shadow:0 1px 0 rgba(255,255,255,.8) inset,0 4px 14px -4px rgba(0,0,0,.18);animation:pillIn .45s var(--spring)}
@keyframes pillIn{from{transform:scale(.88);opacity:.6}to{transform:none;opacity:1}}
.gradebar input:checked+span{background:linear-gradient(180deg,#3d9bff,#007AFF);box-shadow:0 1px 0 rgba(255,255,255,.4) inset,0 4px 12px -3px rgba(0,122,255,.6);animation:pillIn .4s var(--spring)}

/* Таблицы */
tr.click td{transition:background .2s}
tr.click:hover td{background:rgba(0,122,255,.06)}
tr.selected td{background:rgba(0,122,255,.10)}
th{background:transparent}

/* Появление экрана и карточек */
.main>*{animation:enter .42s var(--ease) both}
.main>*:nth-child(2){animation-delay:.03s}.main>*:nth-child(3){animation-delay:.06s}.main>*:nth-child(4){animation-delay:.09s}
```

```
.main>*:nth-child(5){animation-delay:.12s}.main>*:nth-child(6){animation-delay:.15s}.main>*:nth-child(n+7){animation-delay:.18s}
@keyframes enter{from{opacity:0;transform:translateY(14px) scale(.985);filter:blur(3px)}to{opacity:1;transform:none;filter:none}}
.grid>.card,.grid>.kpi,.quick>a{animation:enter .48s var(--ease) both}
.grid>*:nth-child(2),.quick>a:nth-child(2){animation-delay:.05s}.grid>*:nth-child(3),.quick>a:nth-child(3){animation-delay:.1s}.grid>*:nth-child(4),.quick>a:nth-child(4){animation-delay:.15s}
.no-anim .main>*,.no-anim .grid>*,.no-anim .quick>a{animation:none}
.lv,.badge{transition:transform .3s var(--spring)}
.lv:hover,.badge:hover{transform:scale(1.06)}
.chart path,.chart polyline{transition:opacity .3s}
.note{background:var(--g-bg-2);-webkit-backdrop-filter:var(--g-blur);backdrop-filter:var(--g-blur);border:1px solid var(--g-edge-2)}
.note.warn{background:linear-gradient(180deg,rgba(255,149,0,.13),rgba(255,149,0,.08))}
.note.err{background:linear-gradient(180deg,rgba(255,59,48,.13),rgba(255,59,48,.08))}
.note.ok{background:linear-gradient(180deg,rgba(52,199,89,.13),rgba(52,199,89,.08))}

/* Сообщения, панель массовых действий, мобильная панель */
.toast{background:var(--g-bg-hi);animation:toastIn .6s var(--spring)}
.bulkbar{position:sticky;top:12px;z-index:20;display:flex;align-items:center;gap:8px;flex-wrap:wrap;padding:10px 14px;border-radius:18px;margin-bottom:12px;animation:pillIn .45s var(--spring)}
.bulkbar[hidden]{display:none}
.tabbar{background:var(--g-bg)}
.tabbar a.active{background:var(--g-bg-hi);box-shadow:0 1px 0 rgba(255,255,255,.8) inset,0 4px 12px -4px rgba(0,0,0,.2);animation:pillIn .45s var(--spring)}
.backbar .back{border-radius:999px;padding:6px 14px 6px 6px}
.backbar .back:hover{background:var(--g-bg-hi);transform:translateX(-2px)}
.backbar .back svg{transition:transform .35s var(--spring)}
.backbar .back:hover svg{transform:translateX(-3px)}
.mode-banner{border-left:none;border-right:none;border-top:none}

/* Раздел «Отчёты» */
.card h2 .ic{display:inline-grid;width:28px;height:28px;vertical-align:-6px;margin-right:8px}
.rep-item{display:flex;align-items:center;gap:10px;flex-wrap:wrap;padding:12px 0;border-top:.5px solid var(--line)}
.rep-item:first-of-type{border-top:none}
.rep-t{flex:1;min-width:200px;display:flex;flex-direction:column}
.rep-t .muted{font-size:13.5px}

/* Профиль: сильные и слабые стороны */
.sw{display:grid;grid-template-columns:1fr 1fr;gap:14px}
.sw-col{border-radius:18px;padding:14px 16px;border:1px solid var(--g-edge-2)}
.sw-col.plus{background:linear-gradient(180deg,rgba(52,199,89,.14),rgba(52,199,89,.05))}
.sw-col.minus{background:linear-gradient(180deg,rgba(255,149,0,.15),rgba(255,149,0,.05))}
.sw-col h3{margin:0 0 8px}
.sw-col ul{margin:0;padding-left:18px}
.sw-col li{margin:6px 0}
.vpk-desc{font-size:13px;color:var(--muted);line-height:1.35;margin-top:2px}
.narr p{margin:0 0 8px}
@media (max-width:760px){.sw{grid-template-columns:1fr}}

@supports not ((backdrop-filter:blur(1px)) or (-webkit-backdrop-filter:blur(1px))){
  :root{--g-bg:var(--surface);--g-bg-2:var(--surface);--g-bg-hi:var(--surface)}
}
@media (prefers-reduced-motion:reduce){body::before{animation:none}.main>*,.grid>*,.quick>a{animation:none!important}}
@media print{
  body::before{display:none}
  .card,.kpi,.note{background:#fff!important;-webkit-backdrop-filter:none!important;backdrop-filter:none!important;box-shadow:none!important;border:none!important}
  .card::before,.kpi::before{display:none}
  .main>*,.grid>*{animation:none!important;opacity:1!important;transform:none!important;filter:none!important}
  .sw-col{border:1px solid #999;background:none!important}
}

@media print{
  :root{color-scheme:light}
  body{background:#fff;color:#000}
  .side,.topbar,.tabbar,.side-scrim,.mode-banner,.no-print,.toast-wrap{display:none!important}
  .app{display:block}
  .main{padding:0;max-width:none}
  .card{box-shadow:none;border:none;padding:0;margin-bottom:14px;break-inside:avoid;background:#fff;color:#000;border-radius:0}
```

```
  .card .blank{box-shadow:none;padding:0}
  .page-break{break-after:page}
  .only-print{display:block} span.only-print{display:inline}
  .lv{color:#000!important;background:none!important;border:1px solid #000}
  a{color:#000;text-decoration:none}
  h1{font-size:22px}
  table{font-size:11.5px} th{background:#eee!important;color:#000;text-transform:none}
  th,td{border-bottom:1px solid #999}
  .note{background:none;border:1px solid #999}.note::before{display:none}.note{padding-left:12px}
}
</style>
</head>
<body>
<header class="topbar no-print"><strong id="topTitle">ПФК ВПК</strong></header>
<div id="modeBanner" class="mode-banner no-print" hidden><span class="dot" aria-hidden="true"></span><span><b>Учебный режим</b> — отдельное хранилище, рабочие данные не затрагиваются.</span> <button type="button" class="btn sm" data-act="mode-main">Вернуться к рабочим данным</button></div>
<div class="app">
  <aside class="side" id="side" aria-label="Навигация">
    <div class="brand"><span class="logo" aria-hidden="true"><svg class="i" viewBox="0 0 24 24" aria-hidden="true"><path d="M12 21a9 9 0 1 0 0-18 9 9 0 0 0 0 0 18zM12 16a4 4 0 1 0 0-8 4 4 0 0 0 0 0 8zM12 12h.01"/></svg></span><div>ПФК ВПК<small>Расчёт уровня сформированности военно-профессиональных компетенций</small></div></div>
    <div class="nav-search"><input type="search" id="navSearch" placeholder="Найти раздел или функцию" aria-label="Поиск раздела" autocomplete="off"></div>
    <nav class="nav" id="nav" aria-label="Разделы"></nav>
    <div class="storage" id="storageStatus"></div>
  </aside>
  <div class="side-scrim no-print" id="sideScrim" hidden></div>
  <main class="main" id="main" tabindex="-1"></main>
</div>
<nav class="tabbar no-print" aria-label="Разделы"><a href="#/" data-sec="home"><svg class="i" viewBox="0 0 24 24" aria-hidden="true"><path d="M3 10.5 12 3l9 7.5V20a1 1 0 0 1-1 1h-5v-6H9v6H4a1 1 0 0 1-1-1z"/></svg><span>Главная</span></a><a href="#/history" data-sec="assess"><svg class="i" viewBox="0 0 24 24" aria-hidden="true"><path d="M4 20h4L19 9l-4-4L4 16zM14 6l4 4"/></svg><span>Оценки</span></a><a href="#/objects" data-sec="people"><svg class="i" viewBox="0 0 24 24" aria-hidden="true"><path d="M9 11a4 4 0 1 0 0-8 4 4 0 0 0 0 8zM2 21v-1a6 6 0 0 1 12 0v1M17 11a3 3 0 1 0 0-6M22 21v-1a5 5 0 0 0-4-4.9"/></svg><span>Курсанты</span></a><a href="#/summary" data-sec="analysis"><svg class="i" viewBox="0 0 24 24" aria-hidden="true"><path d="M4 20h16M7 16l3-5 3 3 4-7"/></svg><span>Анализ</span></a><button type="button" id="menuBtn" aria-controls="side" aria-expanded="false"><svg class="i" viewBox="0 0 24 24" aria-hidden="true"><path d="M8 6h12M8 12h12M8 18h12M4 6h.01M4 12h.01M4 18h.01"/></svg><span>Ещё</span></button></nav>
<div id="busy" class="busy" hidden role="status" aria-live="assertive"><div class="busy-box"><span class="spin" aria-hidden="true"></span><span id="busyMsg">Выполняется…</span></div></div>
<div class="toast-wrap" id="toasts" role="status" aria-live="polite"></div>
<dialog id="dlg"><div class="dh" id="dlgTitle"></div><div class="db" id="dlgBody"></div><div class="df" id="dlgFoot"></div></dialog>
<noscript><p style="padding:24px">Для работы приложения необходимо включить JavaScript.</p></noscript>
<script>
"use strict";
/* ======================================================================
   МОДУЛЬ 1. КОНФИГУРАЦИЯ МЕТОДИКИ (версия правил расчёта)
   Все значения перенесены из исходных документов; основание
   указана у каждого элемента. Модуль не зависит от DOM.
   ====================================================================== */
const METHOD = (() => {
  const VERSION = '1.0.0';

  /* Основания расчёта (обобщённо). Ключи используются только внутри программы. */
  const SOURCES = {
    S1: { short: 'Методика оценки по отзывам', full: 'Методика определения уровня сформированности военно-профессиональных компетенций выпускников по отзывам из войск после 1 и 2 лет службы: показатель сформированности, шкалы уровней, форма отзыва.' },
    S2: { short: 'Методика оценки по практике', full: 'Методика определения уровня сформированности военно-профессиональных компетенций курсантов по результатам войсковой практики: анкета экспертной оценки и порядок расчёта.' },
    S3: { short: 'Модель полифункционального контроля', full: 'Модель полифункционального контроля сформированности военно-профессиональных компетенций: структура компетенций, уровни сформированности, функции, виды и формы контроля.' },
    S4: { short: 'Экспертное определение коэффициентов', full: 'Методика экспертного определения коэффициентов значимости компетенций и контрольные примеры расчёта.' },
    S5: { short: 'Модель прогнозирования подготовленности', full: 'Математическая модель прогнозирования уровня подготовленности обучающихся по дисциплинам учебного плана и оптимизации плана.' },
    S6: { short: 'Схема модели контроля', full: 'Схема модели полифункционального контроля: этапы подготовки, блоки контроля, математический, результативный и выходной блоки.' },
```

```
    S7: { short: 'Схема функций и форм контроля', full: 'Схема комплекса функций контроля и форм контроля военно-профессиональных компетенций.' }
  };


  /* Уровни сформированности: методика */
  const LEVELS = [
    { code: 'high', name: 'высокий', mark: 'отлично', descr: 'творческо-профессиональный: творческий, инициативный подход, способность к лидерству, принятию решений и надёжному выполнению задач в сложных, неопределённых условиях' },
    { code: 'mid', name: 'средний', mark: 'хорошо', descr: 'функциональный: стабильное и качественное выполнение обязанностей в большинстве типовых и частично нестандартных ситуаций' },
    { code: 'low', name: 'низкий', mark: 'удовлетворительно', descr: 'адаптивный: способность действовать по известному образцу в стандартных, предсказуемых ситуациях' },
    { code: 'crit', name: 'критический', mark: 'неудовлетворительно', descr: 'репродуктивный: неспособность к самостоятельной деятельности, выполнение действий только по прямым указаниям' }
  ];

  /* Шкалы уровней. mins — нижние границы для high, mid, low (critical — ниже low).
     decimals — до скольких знаков округляется значение перед сравнением
     (границы в методике заданы с 3 знаками; null — без округления).
     На стыке перекрывающихся границ (в методике «≤» с обеих сторон) — верхний уровень
     (принятое правило). */
  const SCALES = {
    bsr: { id: 'bsr', title: 'Средний балл за освоение компетенции (bcp)', mins: [4.5, 3.75, 2.7], decimals: null,
      src: 'методика',
      text: ['4,5 ≤ bcp ≤ 5', '3,75 ≤ bcp < 4,5', '2,7 ≤ bcp < 3,75', 'bcp < 2,7'] },
    psfkVpk16: { id: 'psfkVpk16', title: 'Суммарный показатель Псфк — ВПК (16 компетенций, отзыв)', mins: [0.272, 0.235, 0.169], decimals: 3,
      src: 'методика',
      text: ['0,272 ≤ Псфк ≤ 0,313', '0,235 ≤ Псфк ≤ 0,272', '0,169 ≤ Псфк ≤ 0,235', 'Псфк ≤ 0,169'],
      note: 'Граница 0,272 не согласуется с правилом bcp × ΣКзнк / n, по которому получены остальные границы (4,5 × 1,00001 / 16 = 0,281). Применяется значение из таблицы методики.' },
    psfkPsk5: { id: 'psfkPsk5', title: 'Суммарный показатель Псфк — ПСК (5 компетенций, отзыв)', mins: [0.289, 0.241, 0.173], decimals: 3,
      src: 'методика',
      text: ['0,289 ≤ Псфк ≤ 0,321', '0,241 ≤ Псфк ≤ 0,289', '0,173 ≤ Псфк ≤ 0,241', 'Псфк ≤ 0,173'] },
    psfkTotal21: { id: 'psfkTotal21', title: 'Итоговая оценка сформированности компетенций (ВПК + ПСК)', mins: [0.284, 0.236, 0.170], decimals: 3,
      src: 'методика',
      text: ['0,284 ≤ Псфк ≤ 0,315', '0,236 ≤ Псфк ≤ 0,284', '0,170 ≤ Псфк ≤ 0,236', 'Псфк ≤ 0,170'],
      note: 'Формула итоговой оценки в методике не приведена. По принятому правилу итог = среднее арифметическое всех 21 Псфк (ВПК и ПСК): этому правилу соответствуют границы 0,236 и 0,170 таблицы.' }
  };

  /* Наборы компетенций с коэффициентами значимости Кзнк.
     Кзнк получены методом экспертных оценок (методика: λ = Σs / ΣΣs). */
  const VPK16 = [
    ['ВПК-1', 'Способность в служебной деятельности руководствоваться и строго соблюдать нормативные правовые акты Российской Федерации и уметь при принятии решений учитывать правовые, социальные, экономические, экологические и иные последствия', 0.06355, 'общие'],
    ['ВПК-2', 'Способность иметь высокий уровень военно-профессиональной подготовки, постоянно совершенствовать личный профессиональный, интеллектуальный, духовный, нравственный и физический уровень развития, поддерживать образцовый внешний вид и строевую выправку', 0.06288, 'общие'],
    ['ВПК-3', 'Способность быть бдительным, лично исполнительным, требовательным, инициативным и самостоятельным в служебно-боевой деятельности, хранить государственную тайну', 0.06488, 'общие'],
    ['ВПК-4', 'Способность обеспечить постоянную боевую и мобилизационную готовность подразделения', 0.06689, 'командная деятельность'],
    ['ВПК-5', 'Способность к руководству подразделением при выполнении задач служебно-боевой деятельности в мирное и в военное время', 0.06421, 'командная деятельность'],
    ['ВПК-6', 'Способность подводить итоги служебно-боевой деятельности подразделения', 0.05619, 'командная деятельность'],
    ['ВПК-7', 'Способность следить за внешним видом и строевой выправкой подчинённых военнослужащих, выполнением ими правил ношения военной формы одежды, правильной подгонкой снаряжения и обмундирования', 0.06087, 'командная деятельность'],
    ['ВПК-8', 'Способность проводить учебные занятия с подчинёнными и следить за их правильным обучением', 0.06421, 'педагогическая деятельность'],
    ['ВПК-9', 'Способность постоянно совершенствовать физическую тренированность личного состава и организовывать спортивно-массовую работу и работу по военно-спортивному комплексу', 0.06355, 'педагогическая деятельность'],
    ['ВПК-10', 'Способность организовывать и проводить мероприятия военно-политической работы при выполнении задач служебно-боевой деятельности подразделения', 0.05284, 'воспитательная деятельность'],
```

```
    ['ВПК-11', 'Способность организовывать своевременное получение, правильную эксплуатацию вооружения, военной и специальной техники, другого военного имущества подразделения, проверять его наличие, состояние и учёт, документировать результаты осмотра (проверки), принимать меры по предупреждению поломок и аварий', 0.06355, 'военно-техническая деятельность'],
    ['ВПК-12', 'Способность лично владеть вооружением, военной и специальной техникой, состоящими на вооружении подразделения', 0.06555, 'военно-техническая деятельность'],
    ['ВПК-13', 'Способность своевременно обеспечивать военнослужащих подразделения довольствием в соответствии с положенными нормами, заботиться о быте подчинённых и вникать в их нужды, следить за соблюдением ими правил личной гигиены', 0.06355, 'административно-хозяйственная деятельность'],
    ['ВПК-14', 'Способность вести учёт личного состава подразделения', 0.06355, 'административно-хозяйственная деятельность'],
    ['ВПК-15', 'Способность руководить ротным хозяйством', 0.05819, 'административно-хозяйственная деятельность'],
    ['ВПК-16', 'Способность обеспечивать выполнение личным составом требований безопасности военной службы при несении боевой службы, проведении занятий, стрельб, учений, работ и других мероприятий повседневной деятельности', 0.06555, 'административно-хозяйственная деятельность']
  ].map(([id, text, k, group]) => ({ id, text, k, group }));

  const PSK5 = [
    ['ПСК-1', 'Способностью соблюдать общепризнанные принципы и нормы международного гуманитарного права', 0.06455],
    ['ПСК-2', 'Способностью проводить правовое обучение и юридическое консультирование личного состава', 0.06455],
    ['ПСК-3', 'Способностью осуществлять отбор кандидатов для приёма на военную службу по контракту, а также для поступления в военные институты', 0.06451],
    ['ПСК-4', 'Способностью представлять солдат и сержантов (старшин) к присвоению очередных воинских званий, выдвигать достойных для замещения вакантных должностей', 0.06421],
    ['ПСК-5', 'Способность выполнять должностные обязанности в составе юридического подразделения воинской части, соединения и объединения ФСВНГ', 0.06355]
  ].map(([id, text, k]) => ({ id, text, k, group: 'профессионально-специализированные' }));

  /* Формулировки компетенций анкеты войсковой практики */
  const PRACTICE_TEXT = {
    'ВПК-1': 'способен в служебной деятельности руководствоваться и строго соблюдать Конституцию РФ, федеральные конституционные законы, федеральные законы, общевоинские уставы ВС РФ и иные нормативные правовые акты РФ, ФС ВНГ РФ и уметь при принятии решений учитывать правовые, социальные, экономические, экологические и иные последствия',
    'ВПК-2': 'способен иметь высокий уровень военно-профессиональной подготовки, постоянно совершенствовать личный профессиональный, интеллектуальный, духовный, нравственный и физический уровень развития, поддерживать образцовый внешний вид и строевую выправку',
    'ВПК-3': 'способен быть бдительным, лично исполнительным, требовательным, инициативным и самостоятельным в служебно-боевой деятельности, хранить государственную тайну',
    'ВПК-4': 'способен обеспечить постоянную боевую готовность подразделения',
    'ВПК-5': 'способен к руководству подразделением при выполнении задач служебно-боевой деятельности в мирное время',
    'ВПК-6': 'способен подводить итоги служебно-боевой деятельности подразделения',
    'ВПК-7': 'способен следить за внешним видом и строевой выправкой подчинённых ему военнослужащих, выполнением ими правил ношения военной формы одежды, правильной подгонкой снаряжения и обмундирования',
    'ВПК-8': 'способен проводить учебные занятия с подчинёнными и следить за их правильным обучением',
    'ВПК-9': 'способен постоянно совершенствовать физическую тренированность личного состава и организовывать спортивно-массовую работу и работу по военно-спортивному комплексу',
    'ВПК-10': 'способен организовывать и проводить мероприятия морально-психологического обеспечения служебно-боевой деятельности подразделения',
    'ВПК-11': 'способен организовывать своевременное получение, правильную эксплуатацию вооружения, военной и специальной техники и другого военного имущества подразделения, проверять его наличие, состояние и учёт, документировать результаты осмотра (проверки), принимать меры по предупреждению поломок и аварий',
    'ВПК-12': 'способен лично владеть вооружением, автомобильной, бронетанковой и специальной техникой, состоящими на вооружении подразделения',
    'ВПК-16': 'способен обеспечивать выполнение личным составом требований безопасности военной службы при несении боевой службы, проведении занятий, стрельб, учений, работ и других мероприятий повседневной деятельности'
  };
  const VPK13 = Object.keys(PRACTICE_TEXT).map(id => {
    const base = VPK16.find(c => c.id === id);
    return { id, text: PRACTICE_TEXT[id], k: base.k, group: base.group };
  });

  /* ОК и ПК анкеты практики: оцениваются по 5-балльной системе, но Кзнк для них
     в методике не определены, и программа ЭВМ их не обрабатывает. */
  const PRACTICE_OKPK = [
    ['ОК-4', 'способен выполнять профессиональные задачи в соответствии с нормами морали, профессиональной этики и служебного этикета'],
    ['ОК-5', 'способен работать в коллективе, толерантно воспринимая социальные, культурные, конфессиональные и иные различия, предупреждать и конструктивно разрешать конфликтные ситуации в процессе профессио
```

```
нальной деятельности'],
    ['ОК-6', 'способен проявлять психологическую устойчивость в сложных и экстремальных условиях, применять методы эмоциональной и когнитивной регуляции для оптимизации собственной деятельности и психологического состояния'],
    ['ОК-7', 'способен к логическому мышлению, аргументированно и ясно строить устную и письменную речь, вести полемику и дискуссии'],
    ['ОК-9', 'способен организовывать свою жизнь в соответствии с социально значимыми представлениями о здоровом образе жизни'],
    ['ОК-10', 'способен осуществлять письменную и устную коммуникацию на русском языке'],
    ['ПК-4', 'способен квалифицированно применять нормативные правовые акты в профессиональной деятельности'],
    ['ПК-6', 'способен квалифицированно толковать нормативные правовые акты'],
    ['ПК-8', 'способен соблюдать и защищать права и свободы человека и гражданина'],
    ['ПК-12', 'способен осуществлять профилактику, предупреждение правонарушений, коррупционных проявлений, выявлять и устранять причины и условия, способствующие их совершению'],
    ['ПК-14', 'способен осуществлять действия по силовому пресечению правонарушения, использовать для решения профессиональных задач специальную технику, оружие, специальные средства'],
    ['ПК-15', 'способен применять при решении профессиональных задач психологические методы, средства и приёмы'],
    ['ПК-16', 'способен соблюдать в профессиональной деятельности требования нормативных правовых актов в области защиты государственной тайны и информационной безопасности, обеспечивать соблюдение режима секретности'],
    ['ПК-19', 'способен организовывать работу малого коллектива исполнителей, планировать и организовывать служебную деятельность исполнителей, осуществлять контроль и учёт её результатов']
  ].map(([id, text]) => ({ id, text }));

  const COEF_SETS = {
    'src-vpk16': { id: 'src-vpk16', builtin: true, name: 'ВПК (16) — отзыв на выпускника', items: VPK16, scale: 'psfkVpk16',
      src: 'методика (колонка «Кзнк»); значения подтверждаются расчётом по методике (λ = Σs/1495)' },
    'src-psk5': { id: 'src-psk5', builtin: true, name: 'ПСК (5) — отзыв на выпускника, специальность 40.05.01', items: PSK5, scale: 'psfkPsk5',
      src: 'методика (раздел ПСК, колонка «Кзнк»). Методика получения этих Кзнк в методике не приведена.' },
    'src-vpk13': { id: 'src-vpk13', builtin: true, name: 'ВПК (13) — войсковая практика 4 (5) курса', items: VPK13, scale: null,
      src: 'методика (программа ЭВМ: 13 строк). Пороги Псфк для набора из 13 ВПК в методике не установлены.' }
  };

  /* Отзыв на выпускника: разделы II–V. В расчёт Псфк не входят. */
  const PERSONAL_QUALITIES = [
    ['lead', 'Лидерские', 'воля, организаторские способности, требовательность к себе и подчинённым, умение мобилизовать их на выполнение поставленных задач'],
    ['will', 'Волевые', 'целеустремлённость, выдержка, решительность, настойчивость, умение преодолевать тяготы и лишения, подчинение личных интересов службе'],
    ['bus', 'Деловые', 'личная дисциплинированность, трудолюбие, исполнительность, самостоятельность, инициатива, принципиальность, способность принимать решения в экстремальной ситуации'],
    ['moral', 'Нравственный облик', 'моральная устойчивость, честность, правдивость, скромность, тактичность, культура поведения'],
    ['dir', 'Военно-профессиональная направленность личности', 'способность дорожить офицерской честью и достоинством, позитивное отношение к избранной профессии, чувство воинского долга, стремление к профессиональному совершенствованию'],
    ['comm', 'Коммуникативные', 'коммуникабельность, умение владеть собой, отношение к критике и замечаниям командиров и начальников']
  ].map(([id, name, text]) => ({ id, name, text }));
  const QUALITY_SCALE = ['1 — качества не развиты или отсутствуют', '2 — уровень развития, обеспечивающий минимальную деятельность', '3 — удовлетворительный (средний) уровень развития', '4 — уровень развития выше среднего', '5 — высокий уровень развития оцениваемых качеств'];
  const COMPETENCE_SCALE = ['1 — низкий уровень сформированности', '2 — ниже среднего', '3 — средний уровень', '4 — выше среднего', '5 — высокий уровень сформированности'];
  const PERSONAL_TRAINING = { 1: ['Правовая', 'Специальная', 'Огневая', 'Физическая', 'Военно-политическая'], 2: ['Правовая', 'Служебная', 'Огневая', 'Физическая', 'Военно-политическая'] };
  const ACTIVITY_KINDS = ['Командная', 'Педагогико-воспитательная', 'Военно-техническая', 'Социально-правовая', 'Административно-хозяйственная'];
  const MARKS4 = ['отл.', 'хор.', 'удовл.', 'неуд.'];
  const CONCLUSIONS = ['Назначен на вышестоящую воинскую должность', 'Достоин выдвижения на вышестоящую должность', 'Занимаемой должности соответствует', 'С занимаемой должностью справляется с трудом, требует контроля', 'Занимаемой должности не соответствует'];

  /* Шапка отзыва. Сведения хранятся только для отчёта и анализа отзывов. */
  const REVIEW_HEADER = [
    ['rank', 'Воинское звание'], ['gradYear', 'Год выпуска'], ['faculty', 'Факультет'], ['company', 'Рота'], ['platoon', 'Взвод'],
```

```
    ['primaryPosition', 'Первичная должность по прибытии в часть (подразделение)'], ['position', 'Занимаемая штатная должность в настоящее время'],
    ['organ', 'Округ (орган военного управления)'], ['city', 'Город'], ['unitNo', 'Номер войсковой части']
  ];
  const SERVICE_CHANGES = [['', 'нет'], ['transfer', 'перевод (указать, в какую в/ч убыл)'], ['dismissal', 'увольнение (указать основание)']];
  const REVIEW_SIGNERS = [['direct', 'Непосредственный начальник'], ['commander', 'Командир войсковой части («СОГЛАСЕН»)'], ['deputy', 'Заместитель командира по ВПР']];

  /* «Колонка выпускника» — заполняется выпускником, в расчёт не входит */
  const GRADUATE_COLUMN = [
    { id: 'q1', text: 'Какие практические и теоретические трудности возникли у Вас в начале служебной деятельности, в период становления?' },
    { id: 'q2', text: 'Что, по Вашему мнению, недостаточно изучено Вами в институте?' },
    { id: 'q3', text: 'Как Вы считаете, обеспечила ли Вам практикуемая в институте система обучения достаточный уровень профессиональной подготовленности как военного специалиста?',
      options: ['Да, обеспечила', 'Скорее да, чем нет', 'Скорее нет, чем да', 'Нет, не обеспечила', 'Затрудняюсь ответить'] },
    { id: 'q4', text: 'Что, по Вашему мнению, можно было бы исключить из программы обучения курсантов, а что добавить?' },
    { id: 'q5', text: 'Изменились ли Ваши взгляды на выбор профессии офицера в настоящее время?',
      options: ['Интерес к избранной профессии заметно повысился', 'По-прежнему убеждён в правильности выбора', 'В выборе профессии испытываю некоторую разочарованность', 'Считаю поступление в институт неправильным шагом', 'Намерен уволиться из войск', 'Свой вариант ответа'], own: true },
    { id: 'q6', text: 'Как Вы считаете, какие условия в большей степени повлияли на Ваше обучение и каким образом?' },
    { id: 'q7', text: 'Ваши предложения по улучшению учебно-воспитательного процесса в институте' }
  ];

  /* Пояснения терминов для подсказок интерфейса (по тексту методики) */
  const GLOSSARY = {
    'ВПК': 'Военно-профессиональные компетенции — набор требуемых результатов освоения программы специалитета по квалификационным требованиям.',
    'ПСК': 'Профессионально-специализированные компетенции по специальности.',
    'Кзнк': 'Коэффициент значимости компетенции — получен методом экспертных оценок (ранжирование экспертами), λ = Σs / ΣΣs.',
    'Псфк': 'Показатель сформированности компетенции: Псфк = Оценка × Кзнк. Суммарный показатель — среднее арифметическое по набору.',
    'bcp': 'Средний балл за освоение компетенций: среднее арифметическое оценок 1–5.',
    'Степень сформированности': 'bcp / 5 × 100 %.',
    'g₀': 'Относительный уровень подготовленности обучающегося на начальной стадии обучения, 0…1.',
    'g': 'Уровень (степень) подготовленности обучающегося после изучения дисциплины, 0…1: чем ближе к 1, тем более подготовлен.',
    'm': 'Относительный уровень методической подготовки преподавателя M / Mmax.',
    'c': 'Способность обучающегося к овладению учебным материалом, 0 ≤ c < 1.',
    'p': 'Интенсивность процесса обучения (скорость обучения в единицу времени), ч⁻¹.',
    'Y': 'Уровень компетентности выпускника: отношение взвешенной суммы компетентностных признаков к эталону.',
    'W': 'Коэффициент конкордации — средняя степень согласованности мнений экспертов, 0…1.',
    'χ²': 'Критерий согласования Пирсона для проверки значимости коэффициента конкордации.'
  };

  /* ---------- Организация контроля ---------- */
  const CONTROL = {
    functions: [
      { id: 'diag', group: 'Регулятивно-оценочные', name: 'Диагностическая', where: 'Анкеты практики, отзывы, журнал (диагностика), g₀' },
      { id: 'anal', group: 'Регулятивно-оценочные', name: 'Аналитико-информационная', where: 'Сводная по группе, анализ отзывов, выпуск ↔ войска, сравнение групп, карта этапов' },
      { id: 'mgmt', group: 'Регулятивно-оценочные', name: 'Управленческая', where: 'Коррекция: оптимизация учебного плана, прогноз g, расхождения' },
      { id: 'teach', group: 'Развивающе-формирующие', name: 'Обучающая', where: 'Журнал форм контроля (мониторинг)' },
      { id: 'motiv', group: 'Развивающе-формирующие', name: 'Мотивационная', where: 'Открытые критерии: справочник, отчёты для курсанта' },
      { id: 'educ', group: 'Развивающе-формирующие', name: 'Воспитательная', where: 'Личностные качества в отзыве (раздел II)' }
    ],
    kinds: [['diagnostic', 'Диагностический'], ['current', 'Текущий'], ['periodic', 'Периодический (тематический)'], ['intermediate', 'Промежуточный (рубежный)'], ['final', 'Итоговый']],
    types: [['external', 'Внешний'], ['mutual', 'Взаимный'], ['self', 'Самоконтроль']],
    methodFamilies: [['traditional', 'Традиционные'], ['complex', 'Комплексные военно-профессиональные']],
```

```
    methods: {
      traditional: ['Устный', 'Письменный', 'Программируемый', 'Комбинируемый', 'Практический'],
      complex: ['В составе воинской части', 'В составе подразделений', 'В составе экипажа (расчёта)']
    },
    /* Формы контроля: методика (18 форм) и формы блока контроля схемы модели.
       block — блок схемы по умолчанию: monitoring (мониторинг) или diagnostics (диагностика). */
    forms: [
      ['kursovaya', 'Курсовая работа', 'forms', 'monitoring'], ['exam', 'Экзамен', 'forms', 'diagnostics'], ['zachet', 'Зачёт', 'forms', 'diagnostics'],
      ['test', 'Тест (тестирование)', 'forms', 'diagnostics'], ['portfolio', 'Портфолио', 'forms', 'diagnostics'], ['letuchka', 'Тактическая летучка', 'forms', 'monitoring'],
      ['gruppovoe', 'Групповое упражнение', 'forms', 'monitoring'], ['normativy', 'Нормативы по боевой и специальной подготовке', 'forms', 'monitoring'],
      ['kshu', 'КШУ (командно-штабное учение)', 'forms', 'monitoring'], ['ksht', 'КШТ (командно-штабная тренировка)', 'forms', 'monitoring'],
      ['tsz', 'ТСЗ (тактико-строевое занятие)', 'forms', 'monitoring'], ['kshvi', 'КШВИ', 'forms', 'monitoring'], ['strelba', 'Боевая стрельба', 'forms', 'monitoring'],
      ['opros', 'Устный опрос', 'forms', 'monitoring'], ['kontrolnaya', 'Контрольная работа', 'forms', 'monitoring'], ['stazhirovka', 'Войсковая стажировка', 'forms', 'diagnostics'],
      ['samost', 'Самостоятельная работа', 'forms', 'monitoring'], ['vozhdenie', 'Вождение боевых машин', 'forms', 'monitoring'],
      ['ege', 'ЕГЭ', 'model', 'diagnostics'], ['profotbor', 'Профессиональный отбор', 'model', 'diagnostics'], ['vhod', 'Входная диагностика', 'model', 'diagnostics'],
      ['praktika', 'Оценка за практику', 'model', 'diagnostics'], ['nir', 'Отчёт НИР', 'model', 'diagnostics'], ['diplom', 'Дипломная работа', 'model', 'diagnostics'],
      ['gia', 'ГИА', 'model', 'diagnostics'], ['pdp', 'Оценки за занятия по ПДП', 'model', 'monitoring'], ['sbory', 'Учебные сборы выпускников', 'model', 'diagnostics'],
      ['kpz', 'Контрольно-проверочные занятия', 'model', 'diagnostics'], ['klass', 'Итоговая проверка испытания на классность', 'model', 'diagnostics']
    ].map(([id, name, src, block]) => ({ id, name, src, block })),
    /* Связи «вид контроля → форма» и «группа методов → форма» прочитаны со стрелок схемы
       визуально (часть линий пересекается) — проверяются и правятся пользователем. */
    links: {
      diagnostic: ['test'],
      current: ['letuchka', 'gruppovoe', 'opros', 'kontrolnaya'],
      periodic: ['kontrolnaya'],
      intermediate: ['kursovaya', 'kontrolnaya'],
      final: ['exam', 'zachet', 'test', 'portfolio'],
      traditional: ['kshu', 'ksht', 'tsz', 'kshvi', 'strelba', 'opros', 'kontrolnaya', 'stazhirovka', 'gruppovoe'],
      complex: ['kshu', 'ksht', 'tsz', 'kshvi', 'strelba', 'kontrolnaya', 'stazhirovka', 'samost', 'vozhdenie', 'normativy']
    },
    markTypes: [['grade', 'оценка 2–5'], ['pass', 'зачёт / незачёт'], ['score', 'баллы (число)'], ['text', 'текст (без числа)']],
    /* Этапы подготовки по схеме модели */
    stages: [
      { id: 0, name: 'Абитуриент', semesters: [] }, { id: 1, name: '1 курс', semesters: [1, 2] }, { id: 2, name: '2 курс', semesters: [3, 4] },
      { id: 3, name: '3 курс', semesters: [5, 6] }, { id: 4, name: '4 курс', semesters: [7, 8] }, { id: 5, name: '5 курс', semesters: [9, 10] },
      { id: 6, name: 'Выпускник', semesters: [] }]
  };
  /* Паспорт компетенции: уровни, достижимые на курсах (методика — пример для ВПК-11) */
  const PASSPORTS = { 'ВПК-11': { 1: ['low', 'crit'], 2: ['mid', 'low'], 3: ['mid', 'low'], 4: ['high', 'mid', 'low'], 5: ['high', 'mid', 'low'] } };
  /* Сведения абитуриента (входной блок модели) */
  const APPLICANT = {
    education: ['школа', 'колледж', 'лицей', 'кадетский корпус / училище', 'иное'],
    flags: [['commissariat', 'направлен военным комиссариатом'], ['camp', 'учебный сбор'], ['service', 'проходил военную службу'], ['combat', 'участие в боевых действиях']]
  };

  /* Модель подготовленности: параметры по умолчанию */
  const G_MODEL = {
    p24: 1e-4, /* методика: p = 1/(365·24 ч) ≈ 10⁻⁴ ч⁻¹; использован в контрольном примере */
    p21: 0.005, /* методика: «Принимаем g0 = 0,5, p = 0,005 час⁻¹» (для графиков) */
    xmaxExpert: 6 /* методика: s_ij = x_max − x_ij, x_max = 6 */
  };

  /* Справочные (не участвующие в расчётах) данные */
  const STAGE_WEIGHTS = [
    ['1–2 курс (общевоинский)', 'Базовые ВПК', 'Комплексные тактические задачи (КТЗ-1)', 0.2],
    ['3 курс (специальный)', 'Специальные ВПК', 'КТЗ-2, тренажёрные комплексы', 0.3],
```

```
    ['4 курс (интеграционный)', 'Интегративные ВПК', 'Полковые учения, штабные игры', 0.4],
    ['5 курс (завершающий)', 'Комплексные ВПК', 'Выпускная КТЗ, государственный экзамен', 0.1]
  ];
  const COMPONENTS = [
    ['Когнитивный', 'уровень военно-теоретических знаний', 'системность, полнота, точность', 'тестирование, экзамен'],
    ['Операционально-деятельностный', 'профессиональные умения и навыки', 'выполнение нормативов, тактические действия', 'практические задания'],
    ['Мотивационно-ценностный', 'профессиональная направленность', 'устойчивость мотивации, ценностные ориентации', 'анкетирование, наблюдение'],
    ['Личностно-волевой', 'профессионально значимые качества', 'ответственность, стрессоустойчивость', 'экспертная оценка'],
    ['Рефлексивный', 'самоанализ профессиональной деятельности', 'способность к самокоррекции', 'рефлексивные отчёты']
  ];

  return { VERSION, SOURCES, LEVELS, SCALES, COEF_SETS, PRACTICE_OKPK, PERSONAL_QUALITIES, QUALITY_SCALE, COMPETENCE_SCALE,
    PERSONAL_TRAINING, ACTIVITY_KINDS, MARKS4, CONCLUSIONS, G_MODEL, STAGE_WEIGHTS, COMPONENTS,
    REVIEW_HEADER, SERVICE_CHANGES, REVIEW_SIGNERS, GRADUATE_COLUMN, GLOSSARY, CONTROL, APPLICANT, PASSPORTS };
})();

/* ======================================================================
   МОДУЛЬ 2. ВАЛИДАЦИЯ И ВЫЧИСЛЕНИЯ
   Чистые функции: одинаковые входные данные и снимок методики дают
   одинаковый результат. Промежуточные значения не округляются;
   округление выполняется только при сравнении с границами уровней
   (как задано в шкале) и при отображении.
   ====================================================================== */
const Calc = (() => {
  class CalcError extends Error {}

  function roundTo(x, d) {
    const f = Math.pow(10, d);
    return Math.round(Number((x * f).toPrecision(12))) / f;
  }
  const isNum = v => typeof v === 'number' && Number.isFinite(v);
  const isGrade = (v, min = 1, max = 5) => Number.isInteger(v) && v >= min && v <= max;
  const sum = a => a.reduce((s, x) => s + x, 0);

  /* Определение уровня по шкале (METHOD.SCALES.*) */
  function levelBy(scale, value) {
    if (!scale || !isNum(value)) return null;
    const v = scale.decimals == null ? value : roundTo(value, scale.decimals);
    const codes = ['high', 'mid', 'low'];
    for (let i = 0; i < 3; i++) if (v >= scale.mins[i]) return { code: codes[i], compared: v, scale: scale.id };
    return { code: 'crit', compared: v, scale: scale.id };
  }

  /* Псфк = Оценка × Кзнк по каждой компетенции набора.
     Минимум/максимум показателя — при оценке 2 и 5.
     Степень сформированности в % = Оценка / 5 × 100. */
  function competenceSet(items, grades) {
    if (!Array.isArray(items) || !items.length) throw new CalcError('Пустой набор компетенций');
    const rows = [], missing = [], invalid = [];
    for (const it of items) {
      if (!isNum(it.k) || it.k <= 0) throw new CalcError(`Некорректный Кзнк для ${it.id}`);
      const raw = grades ? grades[it.id] : undefined;
      if (raw === undefined || raw === null || raw === '') { missing.push(it.id); rows.push({ id: it.id, k: it.k, grade: null }); continue; }
      if (!isGrade(raw)) { invalid.push(it.id); rows.push({ id: it.id, k: it.k, grade: null }); continue; }
      rows.push({ id: it.id, k: it.k, grade: raw, psfk: raw * it.k, pct: raw / 5 * 100, min: 2 * it.k, max: 5 * it.k });
    }
    const complete = !missing.length && !invalid.length;
    const res = { rows, n: items.length, missing, invalid, complete, sumK: sum(items.map(i => i.k)) };
    if (complete) {
      res.sumGrade = sum(rows.map(r => r.grade));
      res.sumPsfk = sum(rows.map(r => r.psfk));
      res.meanGrade = res.sumGrade / res.n;          /* bcp */
      res.meanPct = res.meanGrade / 5 * 100;
```

```
      res.meanPsfk = res.sumPsfk / res.n; /* суммарный показатель — среднее арифметическое */
      res.maxMeanPsfk = 5 * res.sumK / res.n;
      res.relPsfk = res.meanPsfk / res.maxMeanPsfk * 100;
    }
    return res;
  }

  /* Отзыв на выпускника. input: {vpk:{id:grade}, pskEnabled, psk:{id:grade}} */
  function graduate(snapshot, input) {
    const S = snapshot.scales;
    const vpkSet = snapshot.sets.vpk, pskSet = snapshot.sets.psk;
    const vpk = competenceSet(vpkSet.items, input.vpk || {});
    const out = { vpk, psk: null, total: null, warnings: [] };
    if (vpk.complete) {
      out.vpk.levelBsr = levelBy(S.bsr, vpk.meanGrade);
      out.vpk.levelPsfk = vpkSet.scale ? levelBy(S[vpkSet.scale], vpk.meanPsfk) : null;
    }
    if (input.pskEnabled) {
      const psk = competenceSet(pskSet.items, input.psk || {});
      out.psk = psk;
      if (psk.complete) {
        psk.levelBsr = levelBy(S.bsr, psk.meanGrade);
        psk.levelPsfk = pskSet.scale ? levelBy(S[pskSet.scale], psk.meanPsfk) : null;
      }
      if (vpk.complete && psk.complete) {
        const n = vpk.n + psk.n;
        const total = { n, meanPsfk: (vpk.sumPsfk + psk.sumPsfk) / n, meanGrade: (vpk.sumGrade + psk.sum
Grade) / n };
        total.levelBsr = levelBy(S.bsr, total.meanGrade);
        total.levelPsfk = (vpkSet.scale && pskSet.scale) ? levelBy(S.psfkTotal21, total.meanPsfk) : nul
l;
        out.total = total;
      }
    }
    if (out.vpk.levelPsfk && out.vpk.levelBsr && out.vpk.levelPsfk.code !== out.vpk.levelBsr.code)
      out.warnings.push('Уровень ВПК по таблице Псфк не совпадает с уровнем по среднему баллу bcp (см. п
римечание к шкале).');
    if (out.psk && out.psk.levelPsfk && out.psk.levelBsr && out.psk.levelPsfk.code !== out.psk.levelBsr.
code)
      out.warnings.push('Уровень ПСК по таблице Псфк не совпадает с уровнем по среднему баллу bcp.');
    if (out.total && out.total.levelPsfk && out.total.levelBsr && out.total.levelPsfk.code !== out.tota
l.levelBsr.code)
      out.warnings.push('Итоговый уровень по таблице Псфк не совпадает с уровнем по среднему баллу bc
p.');
    out.complete = vpk.complete && (!input.pskEnabled || (out.psk && out.psk.complete));
    return out;
  }

  /* Анкета экспертной оценки на войсковой практике. input: {vpk:{}, okpk:{}} */
  function practice(snapshot, input) {
    const set = snapshot.sets.vpk;
    const vpk = competenceSet(set.items, input.vpk || {});
    if (vpk.complete) {
      vpk.levelBsr = levelBy(snapshot.scales.bsr, vpk.meanGrade);
      vpk.levelPsfk = set.scale ? levelBy(snapshot.scales[set.scale], vpk.meanPsfk) : null;
    }
    const okpk = [];
    for (const it of snapshot.okpk || []) {
      const g = input.okpk ? input.okpk[it.id] : undefined;
      okpk.push({ id: it.id, grade: isGrade(g) ? g : null });
    }
    return { vpk, okpk, okpkMissing: okpk.filter(r => r.grade === null).map(r => r.id), complete: vpk.co
mplete, warnings: [] };
  }

  /* Ранжирование («Место», методика): по убыванию Псфк, равные значения — одно место,
     следующее место = 1 + число строго больших значений. */
  function rank(values, eps = 1e-12) {
    return values.map(v => v == null ? null : 1 + values.filter(w => w != null && w > v + eps).length);
  }

  /* Формула (22), методика: g0 = (Σx + Σy) / (5·(L1 + L2)) */
  function g0(xs, ys) {
    const all = [...(xs || []), ...(ys || [])];
```

```
    if (!all.length) throw new CalcError('Нет оценок для расчёта g0');
    for (const v of all) if (!isGrade(v, 1, 5)) throw new CalcError(`Недопустимая оценка: ${v} (допустимо целое 1…5)`);
    const L1 = (xs || []).length, L2 = (ys || []).length;
    const sx = sum(xs || []), sy = sum(ys || []);
    return { L1, L2, sumX: sx, sumY: sy, g0: (sx + sy) / (5 * (L1 + L2)) };
  }

  /* Формулы (24) и (21), методика:
     mode '24': g = 1 − (1 − g0)·exp(−p·(m·T1 + T2)/(1 − c))
     mode '21': g = 1 − (1 − g0)·exp(−p·(m·T1/(1 − c1) + T2/(1 − c2))) */
  function gLevel({ g0: G0, m, T1, T2, c, c1, c2, p, mode }) {
    const chk = (v, name, lo, hi, hiOpen) => {
      if (!isNum(v)) throw new CalcError(`Не задано значение ${name}`);
      if (v < lo || (hiOpen ? v >= hi : v > hi)) throw new CalcError(`${name} = ${v} вне допустимого диапазона`);
    };
    chk(G0, 'g0', 0, 1); chk(m, 'm', 0, 1, false); chk(T1, 'T1', 0, Infinity); chk(T2, 'T2', 0, Infinity);
    if (!isNum(p) || p <= 0) throw new CalcError('Параметр p должен быть > 0');
    let x;
    if (mode === '21') { chk(c1, 'c1', 0, 1, true); chk(c2, 'c2', 0, 1, true); x = p * (m * T1 / (1 - c1) + T2 / (1 - c2)); }
    else { chk(c, 'c', 0, 1, true); x = p * (m * T1 + T2) / (1 - c); }
    return { exponent: -x, g: 1 - (1 - G0) * Math.exp(-x) };
  }

  /* Алгоритм: для каждой дисциплины g0 — по формуле (22) для
     дисциплин без предшествующих, иначе g_ot — среднее g предшествующих (23). */
  function trajectory({ disciplines, g0: G0, abilities, p, mode }) {
    if (!Array.isArray(disciplines) || !disciplines.length) throw new CalcError('Учебный план не содержит дисциплин');
    const idx = new Map(disciplines.map((d, i) => [d.id, i]));
    const res = [];
    disciplines.forEach((d, i) => {
      for (const pid of d.preds || []) {
        if (!idx.has(pid)) throw new CalcError(`Дисциплина «${d.name}»: неизвестная предшествующая дисциплина`);
        if (idx.get(pid) >= i) throw new CalcError(`Дисциплина «${d.name}» должна опираться только на дисциплины с меньшими номерами`);
      }
      const ab = (abilities || {})[d.id] || {};
      let start, startRule;
      if (!d.preds || !d.preds.length) { start = G0; startRule = '22'; }
      else { start = sum(d.preds.map(pid => res[idx.get(pid)].g)) / d.preds.length; startRule = '23'; }
      const r = gLevel({ g0: start, m: d.m, T1: d.T1, T2: d.T2, c: ab.c, c1: ab.c1, c2: ab.c2, p, mode });
      res.push({ id: d.id, name: d.name, start, startRule, exponent: r.exponent, g: r.g });
    });
    return res;
  }

  /* Уровень компетентности: Y = Σ aᵢ·k_jᵢ / Σ aᵢ·kᵢ */
  function yLevel(features, values) {
    if (!Array.isArray(features) || !features.length) throw new CalcError('Не заданы признаки');
    let num = 0, den = 0; const rows = [];
    for (const f of features) {
      if (!isNum(f.a) || f.a < 0) throw new CalcError(`Вес aᵢ признака «${f.name}» некорректен`);
      if (!isNum(f.k)) throw new CalcError(`Эталон kᵢ признака «${f.name}» не задан`);
      const v = values ? values[f.id] : undefined;
      if (!isNum(v)) throw new CalcError(`Не задано значение признака «${f.name}»`);
      rows.push({ id: f.id, a: f.a, k: f.k, v, av: f.a * v, ak: f.a * f.k });
      num += f.a * v; den += f.a * f.k;
    }
    if (den === 0) throw new CalcError('Знаменатель Σaᵢkᵢ равен нулю');
    return { rows, num, den, Y: num / den };
  }

  /* ---------- Экспертная оценка значимости ---------- */
  function lnGamma(z) {
    const c = [76.18009172947146, -86.50532032941677, 24.01409824083091, -1.231739572450155, 0.1208650973866179e-2, -0.5395239384953e-5];
    let x = z, y = z, t = x + 5.5; t -= (x + 0.5) * Math.log(t);
    let s = 1.000000000190015; for (const ci of c) s += ci / ++y;
```

```
    return -t + Math.log(2.5066282746310005 * s / x);
  }
  function gammaP(a, x) { /* регуляризованная неполная гамма-функция */
    if (x <= 0) return 0;
    if (x < a + 1) { let ap = a, s = 1 / a, d = s; for (let n = 0; n < 500; n++) { ap++; d *= x / ap; s += d; if (Math.abs(d) < Math.abs(s) * 1e-15) break; } return s * Math.exp(-x + a * Math.log(x) - lnGamma(a)); }
    let b = x + 1 - a, c = 1e300, d = 1 / b, h = d;
    for (let i = 1; i < 500; i++) { const an = -i * (i - a); b += 2; d = an * d + b; if (Math.abs(d) < 1e-300) d = 1e-300; c = b + an / c; if (Math.abs(c) < 1e-300) c = 1e-300; d = 1 / d; const del = d * c; h *= del; if (Math.abs(del - 1) < 1e-15) break; }
    return 1 - Math.exp(-x + a * Math.log(x) - lnGamma(a)) * h;
  }
  function chi2Cdf(x, df) { return gammaP(df / 2, x / 2); }
  function chi2Inv(prob, df) {
    let lo = 0, hi = Math.max(10, df * 10);
    while (chi2Cdf(hi, df) < prob) hi *= 2;
    for (let i = 0; i < 200; i++) { const mid = (lo + hi) / 2; if (chi2Cdf(mid, df) < prob) lo = mid; else hi = mid; }
    return (lo + hi) / 2;
  }

  /* matrix[i][j] — ранг i-го фактора (компетенции) у j-го эксперта (исходная шкала 1…xmax).
     allowZero — только для воспроизведения контрольного примера методики, где пустая
     ячейка (фактор 16, эксперт 19) обработана как 0. */
  function expert(matrix, xmax, alpha = 0.05, allowZero = false) {
    const n = matrix.length; if (n < 2) throw new CalcError('Нужно не менее 2 факторов');
    const m = matrix[0].length; if (m < 1) throw new CalcError('Нужен хотя бы 1 эксперт');
    if (!isNum(xmax)) throw new CalcError('Не задан x_max');
    for (let i = 0; i < n; i++) {
      if (matrix[i].length !== m) throw new CalcError('Матрица должна быть прямоугольной');
      for (let j = 0; j < m; j++) {
        const v = matrix[i][j];
        if (!Number.isInteger(v) || v < (allowZero ? 0 : 1) || v > xmax) throw new CalcError(`Фактор ${i + 1}, эксперт ${j + 1}: недопустимое значение «${v ?? ''}» (целое 1…${xmax})`);
      }
    }
    /* Этап 3: переформирование рангов (средние ранги для связанных) и T_j */
    const R = Array.from({ length: n }, () => Array(m).fill(0));
    const T = [];
    for (let j = 0; j < m; j++) {
      const col = matrix.map((row, i) => ({ i, v: row[j] })).sort((a, b) => a.v - b.v);
      let pos = 0, tj = 0;
      while (pos < n) {
        let end = pos; while (end + 1 < n && col[end + 1].v === col[pos].v) end++;
        const t = end - pos + 1, avg = (pos + 1 + end + 1) / 2;
        for (let q = pos; q <= end; q++) R[col[q].i][j] = avg;
        if (t > 1) tj += t * t * t - t;
        pos = end + 1;
      }
      T.push(tj / 12);
    }
    const sums = R.map(r => sum(r));
    const mean = m * (n + 1) / 2;
    const d = sums.map(s => s - mean);
    const S = sum(d.map(x => x * x));
    const sumT = sum(T);
    const denW = m * m * (n * n * n - n) / 12 - m * sumT;
    const denChi = m * n * (n + 1) / 12 - sumT / (n - 1);
    if (denW <= 0 || denChi <= 0) throw new CalcError('Все оценки экспертов совпадают — коэффициент конкордации не определён');
    const W = S / denW;
    const chi2 = S / denChi;
    const df = n - 1;
    const chi2crit = chi2Inv(1 - alpha, df);
    /* Этап 7: s_ij = x_max − x_ij, λ_i = Σ_j s_ij / ΣΣ s */
    const s = matrix.map(row => row.map(v => xmax - v));
    const sRow = s.map(r => sum(r));
    const sTot = sum(sRow);
    if (sTot <= 0) throw new CalcError('Сумма преобразованных рангов равна нулю');
    const lambda = sRow.map(x => x / sTot);
    const order = sums.map((v, i) => ({ i, v })).sort((a, b) => a.v - b.v || a.i - b.i).map(o => o.i);
    return { n, m, ranks: R, sums, mean, d, S, T, sumT, W, chi2, df, alpha, chi2crit, significant: chi2 >= chi2crit, s, sRow, sTot, lambda, order };
```

```
  }

  /* ---------- Сопоставление оценок при выпуске и в войсках ----------
     Сравниваются оценки одних и тех же компетенций (по индексу). Правила признания
     оценки «подтверждённой» методика не задаёт — выводятся разности. */
  function followup(rowsA, rowsB) {
    const b = new Map((rowsB || []).filter(r => isGrade(r.grade)).map(r => [r.id, r]));
    const rows = (rowsA || []).filter(r => isGrade(r.grade) && b.has(r.id)).map(r => ({ id: r.id, a: r.grade, b: b.get(r.id).grade, diff: b.get(r.id).grade - r.grade }));
    if (!rows.length) return { rows, n: 0 };
    const n = rows.length;
    return { rows, n, meanA: sum(rows.map(r => r.a)) / n, meanB: sum(rows.map(r => r.b)) / n, meanDiff: sum(rows.map(r => r.diff)) / n,
      higher: rows.filter(r => r.diff > 0).length, lower: rows.filter(r => r.diff < 0).length, same: rows.filter(r => r.diff === 0).length };
  }

  /* ---------- Описание группы: средние значения (методика — средний балл групп) ---------- */
  function groupStats(items) {
    const ok = (items || []).filter(x => isNum(x.meanGrade) && isNum(x.meanPsfk));
    const counts = { high: 0, mid: 0, low: 0, crit: 0 };
    ok.forEach(x => { if (x.level && counts[x.level] !== undefined) counts[x.level]++; });
    const marks = k => ok.map(x => Number(x[k])).filter(v => isGrade(v, 2, 5));
    const mean = a => a.length ? sum(a) / a.length : null;
    return { n: ok.length, meanGrade: mean(ok.map(x => x.meanGrade)), meanPsfk: mean(ok.map(x => x.meanPsfk)),
      minPsfk: ok.length ? Math.min(...ok.map(x => x.meanPsfk)) : null, maxPsfk: ok.length ? Math.max(...ok.map(x => x.meanPsfk)) : null,
      meanLeader: mean(marks('leader')), meanCommission: mean(marks('commission')), counts };
  }

  /* ---------- Правило расхождения (задаётся пользователем; в методике не формализовано) ----------
     Уровень переводится в оценку: высокий 5, средний 4, низкий 3, критический 2. */
  const MARK_OF_LEVEL = { high: 5, mid: 4, low: 3, crit: 2 };
  function discrepancy(levelCode, mark, minDiff) {
    const m = Number(mark);
    if (!levelCode || !isGrade(m, 2, 5)) return null;
    const derived = MARK_OF_LEVEL[levelCode], diff = m - derived;
    return { derived, mark: m, diff, flag: Math.abs(diff) >= (minDiff || 1) };
  }

  /* ---------- Чувствительность модели g ---------- */
  function sensitivity({ g0: G0, p, mode, c, mValues, shares, T1from, T1to, T1step }) {
    if (!(T1step > 0) || !(T1to >= T1from) || T1from < 0) throw new CalcError('Некорректный диапазон T1');
    const T1s = []; for (let t = T1from; t <= T1to + 1e-9; t += T1step) T1s.push(Number(t.toFixed(6)));
    if (T1s.length > 500) throw new CalcError('Слишком много точек (> 500)');
    const g = (m, T1, share) => gLevel({ g0: G0, m, T1, T2: T1 * share, c, c1: c, c2: c, p, mode }).g;
    const byM = mValues.map(m => ({ m, values: T1s.map(T => g(m, T, shares[0] || 0)) }));
    const mMid = mValues[Math.floor(mValues.length / 2)];
    const byShare = shares.map(s => ({ share: s, values: T1s.map(T => g(mMid, T, s)) }));
    const base = T1s.map(T => g(mMid, T, 0));
    const ratio = shares.filter(s => s > 0).map(s => ({ share: s, values: T1s.map((T, i) => g(mMid, T, s) / base[i]) }));
    return { T1s, byM, byShare, ratio, mMid };
  }

  /* ---------- Оптимизация учебного плана методом случайного поиска ---------- */
  function rng(seed) { let a = seed >>> 0; return () => { a |= 0; a = a + 0x6D2B79F5 | 0; let t = Math.imul(a ^ a >>> 15, 1 | a); t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t; return ((t ^ t >>> 14) >>> 0) / 4294967296; }; }
  /* Целевая функция: средний по обучающимся уровень g по выпускающим дисциплинам */
  function planScore(disciplines, T, students, p) {
    const fin = disciplines.map((d, i) => d.final ? i : -1).filter(i => i >= 0);
    let s = 0;
    for (const st of students) {
      const ds = disciplines.map((d, i) => ({ ...d, T1: T[i][0], T2: T[i][1] }));
      const ab = Object.fromEntries(disciplines.map(d => [d.id, { c: st.c }]));
      const r = trajectory({ disciplines: ds, g0: st.g0, abilities: ab, p, mode: '24' });
      s += sum(fin.map(i => r[i].g)) / fin.length;
    }
    return s / students.length;
  }
  function optimizePlan({ disciplines, limits, gamma, students, p, iterations, seed }) {
```

```
    if (!disciplines || !disciplines.length) throw new CalcError('Учебный план не содержит дисциплин');
    if (!disciplines.some(d => d.final)) throw new CalcError('Отметьте хотя бы одну выпускающую дисципли
ну (по ней предусмотрена ГИА)');
    if (!students || !students.length) throw new CalcError('Не заданы обучающиеся');
    if (gamma != null && !(gamma > 1)) throw new CalcError('γ должно быть больше 1');
    if (!(iterations >= 1 && iterations <= 200000)) throw new CalcError('Число итераций: 1…200 000');
    const sem = new Map();
    disciplines.forEach((d, i) => {
      if (!Number.isInteger(d.semester) || d.semester < 1) throw new CalcError(`Дисциплина «${d.name}»:
не указан семестр`);
      for (const k of ['T1min', 'T2min']) if (!isNum(d[k]) || d[k] < 0) throw new CalcError(`Дисциплина
«${d.name}»: некорректное ${k}`);
      if (!sem.has(d.semester)) sem.set(d.semester, []); sem.get(d.semester).push(i);
    });
    const minTot = d => gamma != null ? Math.max(d.T1min * gamma / (gamma - 1), d.T2min * gamma) : d.T1m
in + d.T2min;
    for (const [k, idx] of sem) {
      const lim = limits[k]; if (!isNum(lim) || lim <= 0) throw new CalcError(`Не задан лимит времени дл
я семестра ${k}`);
      const need = sum(idx.map(i => minTot(disciplines[i])));
      if (need > lim + 1e-9) throw new CalcError(`Семестр ${k}: сумма минимальных объёмов ${roundTo(nee
d, 1)} ч больше лимита ${lim} ч`);
    }
    const rand = rng(seed || 1);
    const sample = () => {
      const T = disciplines.map(() => [0, 0]);
      for (const [k, idx] of sem) {
        const surplus = limits[k] - sum(idx.map(i => minTot(disciplines[i])));
        const slots = gamma != null ? idx.length : idx.length * 2;
        const w = Array.from({ length: slots }, () => -Math.log(1 - rand())); const ws = sum(w);
        idx.forEach((i, q) => {
          const d = disciplines[i];
          if (gamma != null) { const tot = minTot(d) + surplus * w[q] / ws; T[i] = [tot - tot / gamma, t
ot / gamma]; }
          else T[i] = [d.T1min + surplus * w[2 * q] / ws, d.T2min + surplus * w[2 * q + 1] / ws];
        });
      }
      return T;
    };
    const baseT = disciplines.map(d => [d.T1, d.T2]);
    const baseScore = planScore(disciplines, baseT, students, p);
    const semUse = [...sem].map(([k, idx]) => ({ semester: k, limit: limits[k], used: sum(idx.map(i => b
aseT[i][0] + baseT[i][1])) }));
    let best = null, bestScore = -Infinity; const trace = [];
    for (let it = 1; it <= iterations; it++) {
      const T = sample(); const sc = planScore(disciplines, T, students, p);
      if (sc > bestScore) { bestScore = sc; best = T; trace.push({ it, score: sc }); }
    }
    return { best, bestScore, baseT, baseScore, semUse, trace, iterations, seed: seed || 1 };
  }

  /* ---------- Журнал форм контроля: проверка записи и эмпирические показатели (методика,
     результативный блок: «эмпирические оценочные показатели»). Правил свёртки в уровень ВПК
     методика не содержит — показатели только описательные. */
  function validateRecord(r) {
    const errs = [];
    if (!r.objectId) errs.push('не выбран объект');
    if (!Number.isInteger(r.stage) || r.stage < 0 || r.stage > 6) errs.push('этап');
    if (r.semester != null && r.semester !== '' && !(Number.isInteger(r.semester) && r.semester >= 1 &&
r.semester <= 10)) errs.push('семестр 1…10');
    if (!r.form) errs.push('форма контроля');
    if (r.markType === 'grade' && !isGrade(r.mark, 2, 5)) errs.push('оценка 2…5');
    if (r.markType === 'pass' && !['зачтено', 'не зачтено'].includes(r.mark)) errs.push('зачтено / не за
чтено');
    if (r.markType === 'score' && !isNum(r.mark)) errs.push('баллы — число');
    if (r.markType === 'text' && !String(r.mark || '').trim()) errs.push('текст результата');
    return errs;
  }
  function journalStats(records) {
    const rs = records || [];
    const g = rs.filter(r => r.markType === 'grade' && isGrade(r.mark, 2, 5)).map(r => r.mark);
    const p = rs.filter(r => r.markType === 'pass');
    const dist = { 5: 0, 4: 0, 3: 0, 2: 0 }; g.forEach(v => dist[v]++);
    return { n: rs.length, grades: g.length, meanGrade: g.length ? sum(g) / g.length : null, dist,
```

```
      pass: p.length, passed: p.filter(r => r.mark === 'зачтено').length,
      monitoring: rs.filter(r => r.block === 'monitoring').length, diagnostics: rs.filter(r => r.block =
== 'diagnostics').length };
  }

  /* ---------- Статистическое направление (методика: корреляционный и регрессионный анализ без
     указания конкретных методов). Применяются стандартные коэффициенты Пирсона и Спирмена. */
  function pearson(xs, ys) {
    const n = Math.min(xs.length, ys.length); if (n < 3) return null;
    const mx = sum(xs) / n, my = sum(ys) / n;
    let sxy = 0, sxx = 0, syy = 0;
    for (let i = 0; i < n; i++) { const a = xs[i] - mx, b = ys[i] - my; sxy += a * b; sxx += a * a; syy
+= b * b; }
    if (sxx === 0 || syy === 0) return null;
    const r = sxy / Math.sqrt(sxx * syy), slope = sxy / sxx;
    return { n, r, slope, intercept: my - slope * mx };
  }
  function avgRanks(a) {
    const idx = a.map((v, i) => [v, i]).sort((p, q) => p[0] - q[0]); const r = Array(a.length);
    for (let i = 0; i < idx.length;) { let j = i; while (j + 1 < idx.length && idx[j + 1][0] === idx[i]
[0]) j++; for (let k = i; k <= j; k++) r[idx[k][1]] = (i + j) / 2 + 1; i = j + 1; }
    return r;
  }
  function spearman(xs, ys) { const p = pearson(avgRanks(xs), avgRanks(ys)); return p ? { n: p.n, rho:
p.r } : null; }

  /* Уровень отдельной компетенции по её оценке — шкала «средний балл за освоение компетенции»:
     5 → высокий, 4 → средний, 3 → низкий, 1–2 → критический. */
  function levelOfGrade(g, bsrScale) { return isGrade(g) ? levelBy(bsrScale, g).code : null; }
  /* Паспорт компетенции: допустимые уровни по курсам */
  function passportCheck(rows, passports, course, bsrScale) {
    const out = [];
    for (const r of rows || []) {
      const p = passports[r.id]; if (!p || !p[course]) continue;
      const lv = levelOfGrade(r.grade, bsrScale); if (!lv) continue;
      if (!p[course].includes(lv)) out.push({ id: r.id, grade: r.grade, level: lv, allowed: p[course]
});
    }
    return out;
  }

  return { CalcError, roundTo, isGrade, levelBy, competenceSet, graduate, practice, rank, g0, gLevel, tr
ajectory, yLevel, expert, chi2Inv, chi2Cdf,
    followup, groupStats, discrepancy, MARK_OF_LEVEL, sensitivity, optimizePlan, planScore, validateReco
rd, journalStats, pearson, spearman, levelOfGrade, passportCheck };
})();

/* ======================================================================
   МОДУЛЬ 3. ТЕСТЫ ВЫЧИСЛИТЕЛЬНОГО МОДУЛЯ
   Запускаются в приложении (раздел «Самопроверка») и в Node.js.
   Контрольные примеры взяты из исходных документов.
   ====================================================================== */
const SelfTests = (() => {
  function snapshotFor(vpkSetId, pskSetId) {
    const S = METHOD.COEF_SETS;
    return { scales: METHOD.SCALES, okpk: METHOD.PRACTICE_OKPK,
      sets: { vpk: S[vpkSetId], psk: pskSetId ? S[pskSetId] : null } };
  }
  const gradesOf = (items, arr) => Object.fromEntries(items.map((it, i) => [it.id, arr[i]]));
  const allOf = (items, v) => gradesOf(items, items.map(() => v));

  function run() {
    const results = [];
    const t = (group, name, fn) => {
      try { const r = fn(); results.push({ group, name, ok: r === true || r === undefined, detail: r ===
true || r === undefined ? '' : String(r) }); }
      catch (e) { results.push({ group, name, ok: false, detail: 'Исключение: ' + e.message }); }
    };
    const near = (a, b, eps, label = '') => {
      if (typeof a !== 'number' || Math.abs(a - b) > eps) throw new Error(`${label} ожидалось ${b}, полу
чено ${a}`);
      return true;
    };
```

```
1259     const throws = (fn) => { try { fn(); } catch (e) { if (e instanceof Calc.CalcError) return true; thr
     ow e; } throw new Error('ожидалась ошибка валидации'); };
1260
1261     const pr = snapshotFor('src-vpk13');
1262     const v13 = METHOD.COEF_SETS['src-vpk13'].items;
1263     const gr = snapshotFor('src-vpk16', 'src-psk5');
1264     const v16 = METHOD.COEF_SETS['src-vpk16'].items, p5 = METHOD.COEF_SETS['src-psk5'].items;
1265
1266     /* --- Псфк: контрольные примеры --- */
1267     t('Псфк (практика)', 'Программа ЭВМ, контрольный пример 1: ср. оценка 4,0; уровень 0,251 (80%)', ()
     => {
1268       const r = Calc.practice(pr, { vpk: gradesOf(v13, [4, 3, 5, 5, 4, 4, 3, 5, 4, 4, 4, 3, 4]) });
1269       const exp = [0.2542, 0.18864, 0.3244, 0.33445, 0.25684, 0.22476, 0.18261, 0.32105, 0.2542, 0.2113
     6, 0.2542, 0.19665, 0.2622];
1270       r.vpk.rows.forEach((row, i) => near(row.psfk, exp[i], 1e-9, row.id));
1271       near(r.vpk.meanGrade, 4.0, 1e-12, 'ср. оценка'); near(Calc.roundTo(r.vpk.meanPsfk, 3), 0.251, 0,
     'Псфк'); near(r.vpk.meanPct, 80, 1e-9, '%');
1272       return true;
1273     });
1274     t('Псфк (практика)', 'Контрольный пример 2: построчные Псфк, мин., макс.; итог 4,62 / 92,31 / 0,28
     9', () => {
1275       const r = Calc.practice(pr, { vpk: gradesOf(v13, [4, 4, 5, 5, 4, 4, 5, 5, 5, 5, 5, 5, 4]) });
1276       const ps = [0.254, 0.252, 0.324, 0.334, 0.257, 0.225, 0.304, 0.321, 0.318, 0.264, 0.318, 0.328, 0.
     262];
1277       const mn = [0.127, 0.126, 0.130, 0.134, 0.128, 0.112, 0.122, 0.128, 0.127, 0.106, 0.127, 0.131, 0.
     131];
1278       const mx = [0.318, 0.314, 0.324, 0.334, 0.321, 0.281, 0.304, 0.321, 0.318, 0.264, 0.318, 0.328, 0.
     328];
1279       r.vpk.rows.forEach((row, i) => { near(Calc.roundTo(row.psfk, 3), ps[i], 0, row.id + ' Псфк'); near
     (Calc.roundTo(row.min, 3), mn[i], 0, row.id + ' мин'); near(Calc.roundTo(row.max, 3), mx[i], 0, row.id +
     ' макс'); });
1280       near(Calc.roundTo(r.vpk.meanGrade, 2), 4.62, 0); near(Calc.roundTo(r.vpk.meanPct, 2), 92.31, 0); n
     ear(Calc.roundTo(r.vpk.meanPsfk, 3), 0.289, 0);
1281       return true;
1282     });
1283     t('Псфк (практика)', 'курсанты №2–6 и «все 5» (0,294; 0,251; 0,294; 0,270; 0,289; 0,313)', () => {
1284       const cases = [
1285         [[5, 5, 5, 5, 4, 5, 5, 4, 5, 5, 4, 4, 5], 0.294, 4.69],
1286         [[4, 3, 5, 5, 4, 4, 3, 5, 4, 4, 4, 3, 4], 0.251, 4.00],
1287         [[4, 5, 5, 5, 5, 4, 4, 5, 5, 5, 5, 5, 4], 0.294, 4.69],
1288         [[5, 4, 4, 3, 4, 5, 4, 4, 5, 4, 5, 5, 4], 0.270, 4.31],
1289         [[5, 5, 5, 4, 5, 4, 4, 4, 5, 5, 4, 5, 5], 0.289, 4.62],
1290         [[5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5], 0.313, 5.00]];
1291       for (const [g, ps, bs] of cases) { const r = Calc.practice(pr, { vpk: gradesOf(v13, g) }); near(Ca
     lc.roundTo(r.vpk.meanPsfk, 3), ps, 0, 'Псфк'); near(Calc.roundTo(r.vpk.meanGrade, 2), bs, 0, 'bcp'); }
1292       return true;
1293     });
1294
1295     /* --- Отзыв на выпускника --- */
1296     t('Отзыв', 'Все оценки 5: ВПК 0,3125→«высокий»; ПСК 0,32137→0,321→«высокий»; итог 0,3146→«высокий»',
     () => {
1297       const r = Calc.graduate(gr, { vpk: allOf(v16, 5), pskEnabled: true, psk: allOf(p5, 5) });
1298       near(r.vpk.meanPsfk, 0.312503125, 1e-12); near(r.psk.meanPsfk, 0.32137, 1e-12); near(r.total.meanP
     sfk, 0.31461428571, 1e-9);
1299       if (r.vpk.levelPsfk.code !== 'high' || r.psk.levelPsfk.code !== 'high' || r.total.levelPsfk.code !
     == 'high') return 'уровень не «высокий»';
1300       return r.warnings.length === 0 || 'неожиданные предупреждения';
1301     });
1302     t('Отзыв', 'Все оценки 1: все уровни «критический»', () => {
1303       const r = Calc.graduate(gr, { vpk: allOf(v16, 1), pskEnabled: true, psk: allOf(p5, 1) });
1304       return ['vpk', 'psk', 'total'].every(k => r[k].levelPsfk.code === 'crit' && r[k].levelBsr.code ===
     'crit') || 'ожидался «критический»';
1305     });
1306     t('Отзыв', 'Все оценки 3: bcp = 3 («низкий»), Псфк ВПК 0,1875 («низкий»)', () => {
1307       const r = Calc.graduate(gr, { vpk: allOf(v16, 3), pskEnabled: false });
1308       near(r.vpk.meanPsfk, 0.1875019, 1e-6);
1309       return (r.vpk.levelBsr.code === 'low' && r.vpk.levelPsfk.code === 'low' && r.total === null) || 'н
     еверный уровень';
1310     });
1311     t('Отзыв', 'ВПК-1…6 = 5, остальные 4: bcp = 4,375 («средний»), Псфк = 0,2738 («высокий» по границе
     0,272) — выдаётся предупреждение о расхождении', () => {
1312       const g = v16.map((_, i) => i < 6 ? 5 : 4);
1313       const r = Calc.graduate(gr, { vpk: gradesOf(v16, g), pskEnabled: false });
```

```
1314      return (r.vpk.levelPsfk.code === 'high' && r.vpk.levelBsr.code === 'mid' && r.warnings.length === 1) || `Псфк ${r.vpk.meanPsfk}`;
1315    });
1316    t('Отзыв', 'ПСК отключены — итоговая оценка не рассчитывается', () => Calc.graduate(gr, { vpk: allOf(v16, 4), pskEnabled: false }).total === null || 'итог не null');
1317    t('Отзыв', 'Итог = среднее всех 21 Псфк (решение по неоднозначности)', () => {
1318      const r = Calc.graduate(gr, { vpk: allOf(v16, 4), pskEnabled: true, psk: allOf(p5, 3) });
1319      return near(r.total.meanPsfk, (r.vpk.sumPsfk + r.psk.sumPsfk) / 21, 1e-15);
1320    });
1321
1322    /* --- Пропуски и недопустимые значения --- */
1323    t('Валидация', 'Пропущенная оценка: расчёт не завершается, агрегаты не вычисляются', () => {
1324      const g = allOf(v16, 4); delete g['ВПК-7'];
1325      const r = Calc.graduate(gr, { vpk: g, pskEnabled: false });
1326      return (!r.complete && r.vpk.missing[0] === 'ВПК-7' && r.vpk.meanPsfk === undefined) || 'пропуск не обнаружен';
1327    });
1328    t('Валидация', 'Недопустимые оценки 0, 6, 3.5, "4" отклоняются', () => {
1329      for (const bad of [0, 6, 3.5, '4', NaN]) { const g = allOf(v16, 4); g['ВПК-1'] = bad; const r = Calc.graduate(gr, { vpk: g }); if (r.complete || r.vpk.invalid[0] !== 'ВПК-1') return 'не отклонено: ' + bad; }
1330      return true;
1331    });
1332    t('Валидация', 'Пустая строка = пропуск', () => { const g = allOf(v13, 5); g['ВПК-16'] = ''; return Calc.practice(pr, { vpk: g }).vpk.missing[0] === 'ВПК-16'; });
1333
1334    /* --- Шкалы уровней (границы) --- */
1335    t('Шкалы', 'bcp: 4,5→высокий; 4,4999→средний; 3,75→средний; 2,7→низкий; 2,6999→критический; 5→высокий', () => {
1336      const S = METHOD.SCALES.bsr; const cases = [[4.5, 'high'], [4.4999, 'mid'], [3.75, 'mid'], [3.7499, 'low'], [2.7, 'low'], [2.6999, 'crit'], [5, 'high'], [1, 'crit']];
1337      for (const [v, c] of cases) if (Calc.levelBy(S, v).code !== c) return `${v}: ${Calc.levelBy(S, v).code}`;
1338      return true;
1339    });
1340    t('Шкалы', 'Псфк ВПК: 0,272→высокий; 0,2715 (≈0,272)→высокий; 0,2714→средний; 0,169→низкий; 0,1684→критический', () => {
1341      const S = METHOD.SCALES.psfkVpk16; const cases = [[0.272, 'high'], [0.2715, 'high'], [0.2714, 'mid'], [0.235, 'mid'], [0.169, 'low'], [0.1684, 'crit']];
1342      for (const [v, c] of cases) if (Calc.levelBy(S, v).code !== c) return `${v}: ${Calc.levelBy(S, v).code}`;
1343      return true;
1344    });
1345    t('Шкалы', 'Итоговая: 0,284 высокий; 0,236 средний; 0,170 низкий; 0,1694 критический', () => {
1346      const S = METHOD.SCALES.psfkTotal21; const cases = [[0.284, 'high'], [0.236, 'mid'], [0.17, 'low'], [0.1694, 'crit']];
1347      for (const [v, c] of cases) if (Calc.levelBy(S, v).code !== c) return `${v}: ${Calc.levelBy(S, v).code}`;
1348      return true;
1349    });
1350
1351    /* --- Ранжирование --- */
1352    t('Ранжирование', 'Равные значения — одно место, далее пропуск мест', () => JSON.stringify(Calc.rank([5, 3, 5, 1, null])) === '[1,3,1,4,null]' || JSON.stringify(Calc.rank([5, 3, 5, 1, null])));
1353
1354    /* --- g0 (22) --- */
1355    t('g0 (22)', 'Пример К1: (3·11+4·6+5·1)+(4·1+5·2) / 5·21 = 76/105 = 0,7238 (в методике напечатано 0,713 — арифметическое расхождение)', () => {
1356      const xs = [...Array(11).fill(3), ...Array(6).fill(4), 5], ys = [4, 5, 5];
1357      const r = Calc.g0(xs, ys); near(r.g0, 76 / 105, 1e-12); return (r.L1 === 18 && r.L2 === 3) || 'L1/L2';
1358    });
1359    t('g0 (22)', 'Все оценки 5 → g0 = 1; все 1 → 0,2', () => near(Calc.g0([5, 5], [5]).g0, 1, 0) && near(Calc.g0([1], []).g0, 0.2, 1e-15));
1360    t('g0 (22)', 'Пустые данные и оценка 6 — ошибка', () => throws(() => Calc.g0([], [])) && throws(() => Calc.g0([6], [])));
1361
1362    /* --- g (24), алгоритм --- */
1363    const plan = [
1364      { id: 'D1', name: 'D1', T1: 100, T2: 80, m: 0.8, preds: [] },
1365      { id: 'D2', name: 'D2', T1: 100, T2: 70, m: 0.8, preds: [] },
1366      { id: 'D3', name: 'D3', T1: 120, T2: 90, m: 0.8, preds: ['D1'] },
1367      { id: 'D4', name: 'D4', T1: 80, T2: 50, m: 0.8, preds: ['D1'] },
1368      { id: 'D5', name: 'D5', T1: 100, T2: 80, m: 0.8, preds: ['D2'] },
```

```
      { id: 'D6', name: 'D6', T1: 90, T2: 70, m: 0.8, preds: ['D4', 'D5'] }];
    const abK1 = { D1: { c: 0.5 }, D2: { c: 0.5 }, D3: { c: 0.4 }, D4: { c: 0.4 }, D5: { c: 0.3 }, D6: { c: 0.4 } };
    t('g (24)', 'Отдельные шаги примера K1 с входами, как в тексте методики: g1 = 0,722; g4 = 0,727; g5 = 0,728; g6 = 0,734; g3 = 0,731 (от неокруглённого g1)', () => {
      const f = (g0, T1, T2, c) => Calc.roundTo(Calc.gLevel({ g0, m: 0.8, T1, T2, c, p: 1e-4, mode: '24' }).g, 3);
      near(f(0.713, 100, 80, 0.5), 0.722, 0);
      const g1 = Calc.gLevel({ g0: 0.713, m: 0.8, T1: 100, T2: 80, c: 0.5, p: 1e-4 }).g; near(f(g1, 120, 90, 0.4), 0.731, 0); near(f(0.722, 80, 50, 0.4), 0.727, 0);
      near(f(0.722, 100, 80, 0.3), 0.728, 0); near(f(0.7275, 90, 70, 0.4), 0.734, 0); return true;
    });
    t('g (24)', 'Полный алгоритм для K1 ( →) без промежуточного округления: отклонение ≤ 0,0006', () => {
      const r = Calc.trajectory({ disciplines: plan, g0: 0.713, abilities: abK1, p: 1e-4, mode: '24' });
      const exp = [0.722, 0.722, 0.731, 0.727, 0.728, 0.734];
      r.forEach((x, i) => near(x.g, exp[i], 0.0006, x.id));
      return (r[5].startRule === '23' && near(r[5].start, (r[3].g + r[4].g) / 2, 1e-15)) || 'g_ot';
    });
    t('g (21)', 'При c1 = c2 = c формула (21) совпадает с (24)', () => {
      const a = Calc.gLevel({ g0: 0.6, m: 0.7, T1: 120, T2: 60, c: 0.5, p: 0.005, mode: '24' }).g;
      const b = Calc.gLevel({ g0: 0.6, m: 0.7, T1: 120, T2: 60, c1: 0.5, c2: 0.5, p: 0.005, mode: '21' }).g;
      return near(a, b, 1e-15);
    });
    t('g', 'T1 = T2 = 0 → g = g0 (f(0,0) = f0, методика)', () => near(Calc.gLevel({ g0: 0.55, m: 0.8, T1: 0, T2: 0, c: 0.3, p: 1e-4 }).g, 0.55, 1e-15));
    t('g', 'Недопустимые c = 1, g0 = 1,2, p = 0, ссылка на последующую дисциплину — ошибка', () =>
      throws(() => Calc.gLevel({ g0: 0.5, m: 0.8, T1: 10, T2: 0, c: 1, p: 1e-4 })) &&
      throws(() => Calc.gLevel({ g0: 1.2, m: 0.8, T1: 10, T2: 0, c: 0.2, p: 1e-4 })) &&
      throws(() => Calc.gLevel({ g0: 0.5, m: 0.8, T1: 10, T2: 0, c: 0.2, p: 0 })) &&
      throws(() => Calc.trajectory({ disciplines: [{ id: 'A', name: 'A', T1: 1, T2: 1, m: 0.5, preds: ['B'] }, { id: 'B', name: 'B', T1: 1, T2: 1, m: 0.5, preds: [] }], g0: 0.5, abilities: { A: { c: 0.1 }, B: { c: 0.1 } }, p: 1e-4 })));

    /* --- Y --- */
    t('Y', 'Y = Σaᵢkⱼᵢ / Σaᵢkᵢ: совпадение с эталоном → 1; (1·4+2·5)/(1·5+2·5) = 0,9333', () => {
      const F = [{ id: 'a', name: 'a', a: 1, k: 5 }, { id: 'b', name: 'b', a: 2, k: 5 }];
      return near(Calc.yLevel(F, { a: 5, b: 5 }).Y, 1, 1e-15) && near(Calc.yLevel(F, { a: 4, b: 5 }).Y, 14 / 15, 1e-15);
    });
    t('Y', 'Нулевой знаменатель и пропуск значения — ошибка', () =>
      throws(() => Calc.yLevel([{ id: 'a', name: 'a', a: 0, k: 5 }], { a: 1 })) && throws(() => Calc.yLevel([{ id: 'a', name: 'a', a: 1, k: 5 }], {})));

    /* --- Экспертная оценка --- */
    const RAW = [
      [1, 1, 1, 1, 2, 1, 1, 1, 2, 4, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 3, 1, 2, 1, 1, 4, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      [2, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      [2, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, 1, 1, 1], [2, 3, 1, 2, 1, 1, 3, 1, 2, 3, 1, 6, 2, 1, 1, 1, 1, 2, 1, 1],
      [1, 2, 2, 1, 1, 2, 1, 2, 1, 3, 1, 1, 3, 1, 1, 1, 1, 2, 1, 1], [1, 1, 2, 1, 1, 2, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, 1, 1, 1],
      [2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, 1, 1, 1], [2, 3, 2, 2, 1, 1, 1, 1, 3, 4, 1, 3, 5, 1, 2, 1, 3, 2, 1, 2],
      [2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 2, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 1, 1, 1, 1],
      [1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 1, 2, 3, 1, 1, 1, 1, 1, 1, 1], [1, 1, 2, 2, 1, 1, 1, 1, 1, 3, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1],
      [1, 1, 2, 2, 1, 1, 3, 1, 2, 3, 1, 1, 3, 2, 2, 1, 1, 2, 1, 2], [1, 2, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 0, 1]];
    t('Экспертная оценка', 'суммы рангов, S = 12175, ΣT = 3715,5, W ≈ 0,2, χ² = 59,21, χ²крит(15; 0,05) = 24,996', () => {
      const r = Calc.expert(RAW, 6, 0.05, true);
      const sums = [159, 159.5, 157.5, 133.5, 159, 204, 189, 159, 166, 241.5, 168.5, 143, 162, 160.5, 214.5, 143.5];
      r.sums.forEach((s, i) => near(s, sums[i], 1e-12, 'x' + (i + 1)));
      near(r.S, 12175, 1e-9, 'S'); near(r.sumT, 3715.5, 1e-9, 'ΣT'); near(Calc.roundTo(r.W, 1), 0.2, 0, 'W'); near(r.chi2, 59.2073, 1e-3, 'χ²');
      near(r.chi2crit, 24.99579, 1e-4, 'χ²крит'); return r.significant || 'незначим';
    });
    t('Экспертная оценка', 'λ = Σs/ΣΣs совпадает с Кзнк методики (5 знаков), ΣΣs = 1495', () => {
```

```
      const r = Calc.expert(RAW, 6, 0.05, true);
      near(r.sTot, 1495, 0, 'ΣΣs');
      r.lambda.forEach((l, i) => near(Calc.roundTo(l, 5), v16[i].k, 0, v16[i].id));
      return r.order[0] === 3 || 'первым по значимости должен быть x4';
    });
    t('Экспертная оценка', 'Пустая ячейка без режима воспроизведения — ошибка валидации', () => throws(() => Calc.expert(RAW, 6)));
    t('Экспертная оценка', 'χ²: обратная функция — табличные значения (df=1: 3,841; df=10: 18,307)', () => near(Calc.chi2Inv(0.95, 1), 3.8415, 1e-3) && near(Calc.chi2Inv(0.95, 10), 18.307, 1e-3));

    /* --- Воспроизводимость --- */
    t('Воспроизводимость', 'Повторный расчёт с теми же данными даёт идентичный результат', () => {
      const inp = { vpk: gradesOf(v16, v16.map((_, i) => 1 + (i % 5))), pskEnabled: true, psk: allOf(p5, 4) };
      return JSON.stringify(Calc.graduate(gr, inp)) === JSON.stringify(Calc.graduate(gr, JSON.parse(JSON.stringify(inp)))) || 'различие';
    });
    /* --- Аналитика --- */
    t('Выпуск ↔ войска', 'Сопоставление по общим компетенциям: разности и средние', () => {
      const r = Calc.followup([{ id: 'ВПК-1', grade: 4 }, { id: 'ВПК-2', grade: 5 }, { id: 'ВПК-3', grade: null }], [{ id: 'ВПК-1', grade: 5 }, { id: 'ВПК-2', grade: 3 }, { id: 'ВПК-3', grade: 4 }, { id: 'ВПК-13', grade: 4 }]);
      return (r.n === 2 && r.higher === 1 && r.lower === 1 && near(r.meanDiff, -0.5, 1e-15)) || JSON.stringify(r);
    });
    t('Группы', 'Средние по группе и распределение уровней; оценки вне 2…5 не учитываются', () => {
      const r = Calc.groupStats([{ meanGrade: 4, meanPsfk: 0.25, level: 'mid', leader: '5' }, { meanGrade: 5, meanPsfk: 0.31, level: 'high', leader: '' }]);
      return (r.n === 2 && near(r.meanGrade, 4.5, 1e-15) && near(r.meanPsfk, 0.28, 1e-15) && r.counts.high === 1 && r.meanLeader === 5) || JSON.stringify(r);
    });
    t('Расхождения', 'Правило пользователя: «высокий»→5 против оценки 4 — расхождение 1; совпадение — нет; некорректная оценка — нет данных', () => {
      const a = Calc.discrepancy('high', '4', 1), b = Calc.discrepancy('mid', 4, 1), c = Calc.discrepancy('mid', '', 1);
      return (a.flag && a.diff === -1 && !b.flag && c === null) || 'ошибка правила';
    });
    t('Чувствительность g', 'g растёт с T1 и m; самостоятельная работа повышает g (отношение > 1)', () => {
      const r = Calc.sensitivity({ g0: 0.5, p: 0.005, mode: '21', c: 0.5, mValues: [0.7, 0.8, 0.9], shares: [0, 0.5, 1], T1from: 50, T1to: 200, T1step: 10 });
      const inc = a => a.every((v, i) => i === 0 || v > a[i - 1]);
      return (r.byM.every(s => inc(s.values)) && r.byM[2].values[0] > r.byM[0].values[0] && r.ratio.every(s => s.values.every(v => v > 1))) || 'монотонность нарушена';
    });
    const optPlan = [
      { id: 'A', name: 'A', T1: 60, T2: 20, m: 0.8, preds: [], semester: 1, T1min: 30, T2min: 10, final: false },
      { id: 'B', name: 'B', T1: 60, T2: 20, m: 0.8, preds: [], semester: 1, T1min: 30, T2min: 10, final: false },
      { id: 'C', name: 'C', T1: 80, T2: 40, m: 0.8, preds: ['A', 'B'], semester: 2, T1min: 40, T2min: 20, final: true }];
    t('Оптимизация плана', 'Ограничения (26), (27) выполняются, сумма по семестру = лимит; при γ: (T1+T2)/T2 = γ', () => {
      const r = Calc.optimizePlan({ disciplines: optPlan, limits: { 1: 200, 2: 150 }, gamma: 3, students: [{ g0: 0.7, c: 0.5 }], p: 1e-4, iterations: 300, seed: 7 });
      const T = r.best;
      const okMin = T.every((x, i) => x[0] >= optPlan[i].T1min - 1e-9 && x[1] >= optPlan[i].T2min - 1e-9);
      const okSum = Math.abs(T[0][0] + T[0][1] + T[1][0] + T[1][1] - 200) < 1e-6 && Math.abs(T[2][0] + T[2][1] - 150) < 1e-6;
      const okG = T.every(x => Math.abs((x[0] + x[1]) / x[1] - 3) < 1e-9);
      return (okMin && okSum && okG && r.bestScore > 0) || JSON.stringify(T);
    });
    t('Оптимизация плана', 'Одинаковое зерно — одинаковый результат; превышение лимита — ошибка; нет выпускающих — ошибка', () => {
      const args = { disciplines: optPlan, limits: { 1: 200, 2: 150 }, gamma: null, students: [{ g0: 0.7, c: 0.5 }], p: 1e-4, iterations: 200, seed: 42 };
      const a = Calc.optimizePlan(args), b = Calc.optimizePlan(args);
      return (JSON.stringify(a.best) === JSON.stringify(b.best)) && throws(() => Calc.optimizePlan({ ...args, limits: { 1: 50, 2: 150 } }))
        && throws(() => Calc.optimizePlan({ ...args, disciplines: optPlan.map(d => ({ ...d, final: false })) }));
    });
```

```
1470     t('Журнал контроля', 'Проверка записи: оценка 2…5, «зачтено», баллы — число, этап и семестр', () =>
     {
1471       const base = { objectId: 'x', stage: 1, semester: 2, form: 'exam', markType: 'grade', mark: 5 };
1472       return Calc.validateRecord(base).length === 0 && Calc.validateRecord({ ...base, mark: 1 }).length
     === 1 && Calc.validateRecord({ ...base, markType: 'pass', mark: 'да' }).length === 1
1473         && Calc.validateRecord({ ...base, markType: 'score', mark: 87 }).length === 0 && Calc.validateRe
     cord({ ...base, semester: 11 }).length === 1 && Calc.validateRecord({ ...base, stage: 7 }).length === 1;
1474     });
1475     t('Журнал контроля', 'Эмпирические показатели: средний балл по оценкам 2…5, доля зачтённых; баллы и
     текст в средний не входят', () => {
1476       const r = Calc.journalStats([{ markType: 'grade', mark: 5, block: 'monitoring' }, { markType: 'gra
     de', mark: 4, block: 'diagnostics' }, { markType: 'pass', mark: 'зачтено' }, { markType: 'pass', mark:
     'не зачтено' }, { markType: 'score', mark: 90 }]);
1477       return (r.n === 5 && near(r.meanGrade, 4.5, 1e-15) && r.pass === 2 && r.passed === 1 && r.dist[5]
     === 1 && r.monitoring === 1) || JSON.stringify(r);
1478     });
1479     t('Справочник', 'Связи схемы ссылаются только на существующие формы контроля', () => {
1480       const ids = new Set(METHOD.CONTROL.forms.map(f => f.id));
1481       return Object.values(METHOD.CONTROL.links).every(a => a.every(id => ids.has(id))) || 'неизвестная
     форма';
1482     });
1483     t('Статистика', 'Пирсон: r = 1 для линейной связи, −1 для обратной; Спирмен с одинаковыми рангами; м
     енее 3 пар или нулевая дисперсия — нет результата', () => {
1484       return near(Calc.pearson([1, 2, 3, 4], [2, 4, 6, 8]).r, 1, 1e-12) && near(Calc.pearson([1, 2, 3],
     [3, 2, 1]).r, -1, 1e-12)
1485         && near(Calc.pearson([1, 2, 3, 4, 5], [2, 1, 4, 3, 5]).r, 0.8, 1e-12) && near(Calc.spearman([1,
     2, 2, 3], [10, 20, 20, 30]).rho, 1, 1e-12)
1486         && Calc.pearson([1, 2], [1, 2]) === null && Calc.pearson([1, 1, 1], [1, 2, 3]) === null;
1487     });
1488     t('Паспорт компетенции', 'методика (ВПК-11): «высокий» на 3 курсе и «критический» на 2 курсе недопус
     тимы; «средний» на 4 курсе допустим', () => {
1489       const S = METHOD.SCALES.bsr, P = METHOD.PASSPORTS;
1490       const a = Calc.passportCheck([{ id: 'ВПК-11', grade: 5 }], P, 3, S), b = Calc.passportCheck([{ id:
     'ВПК-11', grade: 2 }], P, 2, S), c = Calc.passportCheck([{ id: 'ВПК-11', grade: 4 }, { id: 'ВПК-1', grad
     e: 1 }], P, 4, S);
1491       return (a.length === 1 && a[0].level === 'high' && b.length === 1 && b[0].level === 'crit' && c.le
     ngth === 0) || JSON.stringify([a, b, c]);
1492     });
1493     t('Паспорт компетенции', 'Уровень компетенции по оценке: 5 высокий, 4 средний, 3 низкий, 2 и 1 крити
     ческий', () => [5, 4, 3, 2, 1].map(g => Calc.levelOfGrade(g, METHOD.SCALES.bsr)).join() === 'high,mid,lo
     w,crit,crit');
1494     t('Справочник', 'Сумма Кзнк ВПК (16) = 1,00001; ПСК (5) = 0,32137', () => near(v16.reduce((s, c) =>
     s + c.k, 0), 1.00001, 1e-12) && near(p5.reduce((s, c) => s + c.k, 0), 0.32137, 1e-12));
1495     return results;
1496   }
1497   return { run };
1498 })();
1499
1500 /* ======================================================================
1501    МОДУЛЬ 4. МОДЕЛЬ ДАННЫХ И ХРАНЕНИЕ
1502    Данные хранятся в localStorage браузера (ключ STORE_KEY). При запуске
1503    через file:// браузер может ограничить или не сохранять хранилище —
1504    поэтому предусмотрены резервное копирование и восстановление (JSON).
1505    ====================================================================== */
1506 const Store = (() => {
1507   const KEY = 'pfk-vpk-calculator-v1';
1508   /* Режим: основной (рабочие данные) или учебный (отдельное хранилище) */
1509   let mode = (() => { try { return localStorage.getItem('pfk-vpk-mode') === 'demo' ? 'demo' : 'main'; }
     catch (e) { return 'main'; } })();
1510   const key = () => mode === 'demo' ? KEY + '-demo' : KEY;
1511   function setMode(m) { mode = m === 'demo' ? 'demo' : 'main'; try { localStorage.setItem('pfk-vpk-mod
     e', mode); } catch (e) { /* хранилище недоступно */ } }
1512   const SCHEMA = 1;
1513   let status = { available: true, error: null, lastSaved: null };
1514
1515   const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2, 8);
1516   const nowIso = () => new Date().toISOString();
1517   const empty = () => ({ schema: SCHEMA, objects: [], assessments: [], coefSets: [], expertStudies: [],
     curricula: [], yModels: [], reviewSent: {}, settings: {}, practiceReports: [], controlRecords: [], tras
     h: [], meta: { createdAt: nowIso(), lastBackupAt: null } });
1518
1519   function probe() {
1520     try { const k = KEY + '-probe'; localStorage.setItem(k, '1'); localStorage.removeItem(k); return tru
     e; }
```

```
    catch (e) { status.available = false; status.error = e.message; return false; }
  }
  function load() {
    if (!probe()) return empty();
    try {
      const raw = localStorage.getItem(key());
      if (!raw) return empty();
      const data = JSON.parse(raw);
      const v = validate(data);
      if (!v.ok) { status.error = 'Сохранённые данные повреждены: ' + v.error; return empty(); }
      return normalize(data);
    } catch (e) { status.error = 'Не удалось прочитать данные: ' + e.message; return empty(); }
  }
  function save(state) {
    try { localStorage.setItem(key(), JSON.stringify(state)); status.available = true; status.error = nu
ll; status.lastSaved = new Date(); return true; }
    catch (e) { status.available = false; status.error = e.message; return false; }
  }
  function normalize(d) {
    const e = empty();
    for (const k of Object.keys(e)) if (d[k] === undefined) d[k] = e[k];
    return d;
  }
  function validate(d) {
    if (!d || typeof d !== 'object') return { ok: false, error: 'не объект' };
    if (d.schema === undefined) return { ok: false, error: 'файл не является резервной копией этого прил
ожения' };
    if (d.schema !== SCHEMA) return { ok: false, error: `неподдерживаемая версия формата данных (${d.sch
ema})` };
    for (const k of ['objects', 'assessments']) if (!Array.isArray(d[k])) return { ok: false, error: `не
т массива ${k}` };
    for (const o of d.objects) if (!o || typeof o.id !== 'string' || typeof o.name !== 'string') return
{ ok: false, error: 'некорректная запись объекта оценки' };
    for (const a of d.assessments) if (!a || typeof a.id !== 'string' || typeof a.type !== 'string' || !
a.input || !a.snapshot) return { ok: false, error: 'некорректная запись оценки' };
    return { ok: true };
  }
  function exportPayload(state) {
    return { app: 'ПФК-ВПК калькулятор', methodVersion: METHOD.VERSION, exportedAt: nowIso(), data: stat
e };
  }
  function parseImport(text) {
    let obj; try { obj = JSON.parse(text); } catch (e) { return { ok: false, error: 'Файл не является ко
ректным JSON' }; }
    const data = obj && obj.data ? obj.data : obj;
    const v = validate(data);
    return v.ok ? { ok: true, data: normalize(data), info: obj } : v;
  }
  return { load, save, setMode, mode: () => mode, status: () => status, uid, nowIso, empty, exportPayloa
d, parseImport, SCHEMA };
})();

/* Модель предметной области: типы оценок, снимки методики, расчёт записи */
const Model = (() => {
  const TYPES = {
    graduate: { title: 'Отзыв на выпускника (1–2 год службы)', short: 'Отзыв', src: 'методика' },
    practice: { title: 'Анкета экспертной оценки: войсковая практика', short: 'Практика', src: 'методик
a' },
    g0: { title: 'Начальный уровень подготовленности g₀', short: 'g₀', src: 'методика' },
    trajectory: { title: 'Уровень подготовленности по дисциплинам учебного плана', short: 'Траектория
g', src: 'методика' },
    ymodel: { title: 'Уровень компетентности выпускника Y', short: 'Y', src: 'методика (трактовка переме
нных — по методике)' }
  };

  function coefSet(state, id) {
    return METHOD.COEF_SETS[id] || state.coefSets.find(s => s.id === id) || null;
  }
  const clone = x => JSON.parse(JSON.stringify(x));

  /* Снимок методики сохраняется в каждой записи: изменение справочников
     не меняет ранее выполненные расчёты. */
  function snapshot(state, type, meta, input) {
```

```
    const base = { methodVersion: METHOD.VERSION, takenAt: Store.nowIso(), scales: clone(METHOD.SCALES) };
    if (type === 'graduate') {
      const v = coefSet(state, meta.vpkSetId || 'src-vpk16'), p = coefSet(state, meta.pskSetId || 'src-psk5');
      return { ...base, sets: { vpk: clone(v), psk: clone(p) } };
    }
    if (type === 'practice') return { ...base, sets: { vpk: clone(coefSet(state, meta.vpkSetId || 'src-vpk13')) }, okpk: clone(METHOD.PRACTICE_OKPK) };
    if (type === 'trajectory') { const c = state.curricula.find(x => x.id === input.curriculumId); return { ...base, curriculum: c ? clone(c) : null }; }
    if (type === 'ymodel') { const mdl = state.yModels.find(x => x.id === input.modelId); return { ...base, yModel: mdl ? clone(mdl) : null }; }
    return base;
  }

  function newAssessment(state, type, objectId) {
    const meta = { date: new Date().toISOString().slice(0, 10), evaluator: '' };
    let input = {};
    if (type === 'graduate') { Object.assign(meta, { period: 1, vpkSetId: 'src-vpk16', pskSetId: 'src-psk5' }); Object.assign(meta, { header: {}, serviceChange: '', serviceChangeNote: '', signers: {} }); input = { vpk: {}, pskEnabled: true, psk: {}, qualities: {}, training: {}, activity: {}, conclusion: '', notes: {}, graduateColumn: {} }; }
    if (type === 'practice') { Object.assign(meta, { course: 4, vpkSetId: 'src-vpk13', unit: '', evaluator2: '' }); input = { vpk: {}, okpk: {}, leaderMark: '', commissionMark: '' }; }
    if (type === 'g0') input = { x: { 1: 0, 2: 0, 3: 0, 4: 0, 5: 0 }, y: { 1: 0, 2: 0, 3: 0, 4: 0, 5: 0 } };
    if (type === 'trajectory') input = { curriculumId: '', g0: '', g0From: '', mode: '24', p: METHOD.G_MODEL.p24, abilities: {} };
    if (type === 'ymodel') input = { modelId: '', values: {} };
    const a = { id: Store.uid(), type, objectId, status: 'draft', createdAt: Store.nowIso(), updatedAt: Store.nowIso(), meta, input, snapshot: null, result: null, revisions: [] };
    a.snapshot = snapshot(state, type, meta, input);
    return a;
  }

  /* Для черновика: снимок пересобирается по текущим справочникам. Возвращает true,
     если параметры методики (наборы Кзнк, учебный план, модель Y) изменились. */
  function refreshDraftSnapshot(state, a) {
    if (a.status !== 'draft') return false;
    const fresh = snapshot(state, a.type, a.meta, a.input);
    const key = x => JSON.stringify({ ...x, takenAt: null });
    if (key(fresh) === key(a.snapshot)) return false;
    a.snapshot = fresh; return true;
  }

  const expand = counts => { const out = []; for (const g of [1, 2, 3, 4, 5]) { const n = Number(counts[g] || 0); for (let i = 0; i < n; i++) out.push(g); } return out; };

  /* Расчёт записи по сохранённым входным данным и снимку. Возвращает
     {ok, complete, result, errors[]} и никогда не бросает исключений наружу. */
  function compute(a) {
    try {
      const s = a.snapshot, inp = a.input;
      if (a.type === 'graduate') { const r = Calc.graduate(s, inp); return { ok: true, complete: r.complete, result: r, errors: [] }; }
      if (a.type === 'practice') { const r = Calc.practice(s, inp); return { ok: true, complete: r.complete, result: r, errors: [] }; }
      if (a.type === 'g0') {
        for (const side of ['x', 'y']) for (const g of [1, 2, 3, 4, 5]) { const v = Number(inp[side][g] || 0); if (!Number.isInteger(v) || v < 0) return { ok: false, complete: false, errors: ['Количество оценок должно быть целым неотрицательным числом'] }; }
        const r = Calc.g0(expand(inp.x), expand(inp.y)); return { ok: true, complete: true, result: r, errors: [] };
      }
      if (a.type === 'trajectory') {
        const c = s.curriculum; if (!c) return { ok: false, complete: false, errors: ['Не выбран учебный план'] };
        const g0v = typeof inp.g0 === 'number' ? inp.g0 : NaN;
        const r = Calc.trajectory({ disciplines: c.disciplines, g0: g0v, abilities: inp.abilities, p: Number(inp.p), mode: inp.mode });
        return { ok: true, complete: true, result: { rows: r, final: r[r.length - 1].g }, errors: [] };
      }
      if (a.type === 'ymodel') {
```

```
        const mdl = s.yModel; if (!mdl) return { ok: false, complete: false, errors: ['Не выбрана модель признаков'] };
        const r = Calc.yLevel(mdl.features, inp.values); return { ok: true, complete: true, result: r, errors: [] };
      }
      return { ok: false, complete: false, errors: ['Неизвестный тип оценки'] };
    } catch (e) {
      if (e instanceof Calc.CalcError) return { ok: false, complete: false, errors: [e.message] };
      return { ok: false, complete: false, errors: ['Внутренняя ошибка расчёта: ' + e.message] };
    }
  }

  /* Ключевой показатель записи для списков, сравнения и динамики */
  function headline(a) {
    const r = a.result; if (!r) return null;
    if (a.type === 'graduate') return { label: 'Псфк ВПК', value: r.vpk.meanPsfk, digits: 3, level: r.vpk.levelPsfk || r.vpk.levelBsr };
    if (a.type === 'practice') return { label: 'Псфк ВПК', value: r.vpk.meanPsfk, digits: 3, level: r.vpk.levelBsr };
    if (a.type === 'g0') return { label: 'g₀', value: r.g0, digits: 3 };
    if (a.type === 'trajectory') return { label: 'g (посл. дисц.)', value: r.final, digits: 3 };
    if (a.type === 'ymodel') return { label: 'Y', value: r.Y, digits: 3 };
    return null;
  }
  return { TYPES, coefSet, snapshot, refreshDraftSnapshot, newAssessment, compute, headline, clone };
})();

/* ======================================================================
   МОДУЛЬ 4а. ИНФОРМАЦИОННАЯ АРХИТЕКТУРА И НАВИГАЦИЯ
   Пять разделов (Главная · Оценки · Курсанты и выпускники · Анализ ·
   Методика) и сервисные пункты. Внутри раздела — вкладки, сгруппированные
   по смыслу. Детальные экраны получают кнопку «Назад» с названием экрана,
   на который она ведёт. Все прежние адреса (#/...) сохранены.
   ====================================================================== */
const ICONS = {
  house: 'M3 10.5 12 3l9 7.5V20a1 1 0 0 1-1 1h-5v-6H9v6H4a1 1 0 0 1-1-1z',
  plus: 'M12 5v14M5 12h14',
  edit: 'M4 20h4L19 9l-4-4L4 16zM14 6l4 4',
  people: 'M9 11a4 4 0 1 0 0-8 4 4 0 0 0 0 8zM2 21v-1a6 6 0 0 1 12 0v1M17 11a3 3 0 1 0 0-6M22 21v-1a5 5 0 0 0-4-4.9',
  chart: 'M4 20h16M7 16l3-5 3 3 4-7',
  book: 'M4 4h6a2 2 0 0 1 2 2v14a2 2 0 0 0-2-2H4zM20 4h-6a2 2 0 0 0-2 2v14a2 2 0 0 1 2-2h6z',
  disk: 'M5 3h11l3 3v15H5zM8 3v5h7V3M8 21v-7h8v7',
  trash: 'M4 7h16M10 11v6M14 11v6M6 7l1 14h10l1-14M9 7V4h6v3',
  help: 'M12 21a9 9 0 1 0 0-18 9 9 0 0 0 0 18zM9.5 9.5a2.5 2.5 0 1 1 3.5 2.3c-.6.3-1 .9-1 1.6V14M12 17v.5',
  list: 'M8 6h12M8 12h12M8 18h12M4 6h.01M4 12h.01M4 18h.01',
  target: 'M12 21a9 9 0 1 0 0-18 9 9 0 0 0 0 18zM12 16a4 4 0 1 0 0-8 4 4 0 0 0 0 8zM12 12h.01',
  back: 'M15 5l-7 7 7 7',
  doc: 'M6 3h8l4 4v14H6zM14 3v4h4M9 12h6M9 16h6',
  search: 'M11 18a7 7 0 1 0 0-14 7 7 0 0 0 0 14zM20 20l-3.5-3.5'
};
const svgIcon = (k, cls = 'i') => `<svg class="${cls}" viewBox="0 0 24 24" aria-hidden="true"><path d="${ICONS[k]}"/></svg>`;
const iconTile = (k, color) => `<span class="ic" style="--c:${color}">${svgIcon(k)}</span>`;

/* Разделы. pages: [адрес, название вкладки, краткое пояснение]. group — подпись группы вкладок. */
const NAV = [
  { id: 'home', title: 'Главная', icon: 'house', color: '#007AFF', href: '#/', groups: [] },
  { id: 'assess', title: 'Оценки', icon: 'edit', color: '#34C759', href: '#/history', groups: [
    { label: 'Ввести', pages: [['#/new', 'Новая оценка', 'анкета практики, отзыв, g₀, прогноз g, Y — пошагово'], ['#/batch', 'Ввод по группе', 'оценки всей группы в одной таблице'], ['#/import', 'Импорт из файла', 'загрузка оценок из Excel или CSV']] },
    { label: 'Просмотреть', pages: [['#/history', 'Все оценки', 'утверждённые оценки и черновики, фильтры'], ['#/journal', 'Журнал контроля', 'экзамены, зачёты, ЕГЭ, нормативы по этапам']] }] },
  { id: 'people', title: 'Курсанты и выпускники', short: 'Курсанты', icon: 'people', color: '#5856D6', href: '#/objects', groups: [] },
  { id: 'analysis', title: 'Анализ', icon: 'chart', color: '#FF9500', href: '#/summary', groups: [
    { label: 'Результаты', pages: [['#/summary', 'Сводная по группе', 'средние значения и место в группе'], ['#/risk', 'Группы риска', 'кто отстаёт и почему'], ['#/reviews', 'Отзывы из войск', 'итоги отзывов по органам управления'], ['#/followup', 'Практика и служба', 'сравнение оценок практики и отзыва'], ['#/groups', 'Сравнение групп', 'контрольная и экспериментальная группы'], ['#/stats', 'Статистика', 'корреляции показателей']] },
```

```
    { label: 'Прогноз g', pages: [['#/gbatch', 'Прогноз по группе', 'уровень подготовленности по дисциплинам'], ['#/sensitivity', 'Чувствительность', 'как g зависит от часов и мастерства'], ['#/optimize', 'Оптимизация плана', 'распределение часов по дисциплинам'], ['#/matrix', 'Дисциплины и ВПК', 'какие дисциплины формируют компетенции']] }] },
  { id: 'reports', title: 'Отчёты', icon: 'doc', color: '#FF2D55', href: '#/reports', groups: [
    { label: '', pages: [['#/reports', 'Все отчёты', 'по курсанту, по группе, документы, выгрузки — печать и PDF'], ['#/preports', 'Отчёты о практике', 'отчёты руководителя практики'], ['#/blanks', 'Бланки для печати', 'пустые отзывы и анкеты для заполнения в войсках']] }] },
  { id: 'method', title: 'Методика', icon: 'book', color: '#A2845E', href: '#/method', groups: [
    { label: '', pages: [['#/method', 'Справочник', 'модель, компетенции, шкалы, формулы, термины'], ['#/coef', 'Коэффициенты Кзнк', 'наборы коэффициентов и экспертиза'], ['#/passports', 'Паспорта компетенций', 'допустимые уровни по курсам'], ['#/curricula', 'Учебные планы', 'исходные данные для расчёта g'], ['#/ymodels', 'Модели Y', 'признаки, веса и эталон'], ['#/tests', 'Проверка расчётов', 'контрольные примеры методики']] }] }
];
const NAV_SERVICE = [['#/data', 'Данные и резервные копии', 'disk', '#007AFF'], ['#/trash', 'Корзина', 'trash', '#8E8E93']];
const allPages = () => NAV.flatMap(s => s.groups.flatMap(g => g.pages.map(p => ({ href: p[0], title: p[1], desc: p[2], section: s }))));

/* Детальные экраны: раздел и экран по умолчанию для кнопки «Назад» */
const DETAIL = [
  [/^\/a\/([\w-]+)\/report$/, 'assess', m => ({ href: '#/a/' + m[1], title: 'Результат оценки' })],
  [/^\/a\/([\w-]+)\/edit\/\d$/, 'assess', () => ({ href: '#/history', title: 'Все оценки' })],
  [/^\/a\/([\w-]+)$/, 'assess', m => { const a = state.assessments.find(x => x.id === m[1]); return a ? { href: '#/object/' + a.objectId, title: objName(a.objectId) } : { href: '#/history', title: 'Все оценки' }; }],
  [/^\/object\/([\w-]+)(\/\w+)?$/, 'people', () => ({ href: '#/objects', title: 'Курсанты и выпускники' })],
  [/^\/compare\/([\w-]+)\//, 'people', m => { const a = state.assessments.find(x => x.id === m[1]); return a ? { href: '#/object/' + a.objectId, title: objName(a.objectId) } : { href: '#/objects', title: 'Курсанты и выпускники' }; }],
  [/^\/followup\/[\w-]+$/, 'analysis', () => ({ href: '#/followup', title: 'Практика и служба' })],
  [/^\/journal\/sheet$/, 'assess', () => ({ href: '#/journal', title: 'Журнал контроля' })],
  [/^\/(coef|expert)\/[\w-]+$/, 'method', () => ({ href: '#/coef', title: 'Коэффициенты Кзнк' })],
  [/^\/curriculum\/[\w-]+$/, 'method', () => ({ href: '#/curricula', title: 'Учебные планы' })],
  [/^\/ymodel\/[\w-]+$/, 'method', () => ({ href: '#/ymodels', title: 'Модели Y' })],
  [/^\/preport\/[\w-]+$/, 'reports', () => ({ href: '#/preports', title: 'Отчёты о практике' })]
];
const pageOf = path => allPages().find(p => { const h = p.href.slice(1); return path === h || (h === '/method' && path.startsWith('/method/')); });
function locate(path) {
  if (path === '/') return { section: NAV[0], root: true };
  if (path === '/objects') return { section: NAV[2], root: true };
  const pg = pageOf(path); if (pg) return { section: pg.section, page: pg, root: true };
  for (const [re, sec, parent] of DETAIL) { const m = path.match(re); if (m) return { section: NAV.find(s => s.id === sec), detail: true, parent: parent(m) }; }
  const sv = NAV_SERVICE.find(s => s[0].slice(1) === path); if (sv) return { service: sv, root: true };
  return { root: true };
}

/* Стек переходов: «Назад» ведёт туда, откуда пользователь пришёл; без истории — к родительскому экрану. */
const navStack = [];
const stackKey = path => path.replace(/\/edit\/\d$/, '/edit');
function trackNav(path, loc, title) {
  const key = stackKey(path);
  if (loc.root) { navStack.length = 0; navStack.push({ key, path, title }); return; }
  const i = navStack.findIndex(x => x.key === key);
  if (i >= 0) { navStack.length = i + 1; navStack[i].path = path; navStack[i].title = title || navStack[i].title; }
  else navStack.push({ key, path, title });
}
function backTarget(path, loc) {
  const key = stackKey(path); const i = navStack.findIndex(x => x.key === key);
  const prev = i > 0 ? navStack[i - 1] : null;
  return prev ? { href: '#' + prev.path, title: prev.title } : loc.parent;
}

/* Боковое меню: пять разделов; у текущего раздела раскрыт список его экранов. */
function renderNav(path, loc) {
  const cur = loc.section ? loc.section.id : null;
  const item = (href, title, icon, color, active, extra = '') => `<a href="${href}" class="${active ? 'active' : ''}" ${active ? 'aria-current="page"' : ''} ${extra}>${iconTile(icon, color)}<span class="t">
```

```
${esc(title)}</span></a>`;
  const q = ($('#navSearch') || {}).value || '';
  let html;
  if (q.trim()) {
    const s = q.trim().toLowerCase();
    const hits = [...allPages(), { href: '#/objects', title: 'Курсанты и выпускники', desc: 'список, карточки, сведения', section: NAV[2] }, ...NAV_SERVICE.map(x => ({ href: x[0], title: x[1], desc: '', section: { icon: x[2], color: x[3], title: 'Сервис' } }))]
      .filter(p => (p.title + ' ' + p.desc + ' ' + p.section.title).toLowerCase().includes(s));
    html = hits.length ? `<div class="grp">${hits.map(p => `<a href="${p.href}">${iconTile(p.section.icon, p.section.color)}<span class="t">${esc(p.title)}<small>${esc(p.section.title)}${p.desc ? ' · ' + esc(p.desc) : ''}</small></span></a>`).join('')}</div>`
      : `<p class="muted" style="padding:8px 12px">Ничего не найдено. Попробуйте другое слово или откройте «Справка».</p>`;
  } else {
    html = `<div class="grp">${NAV.map(s => {
      const on = s.id === cur;
      const sub = on && s.groups.length ? `<div class="sub">${s.groups.map(g => `${g.label ? `<div class="sub-l">${esc(g.label)}</div>` : ''}${g.pages.map(p => `<a href="${p[0]}" class="${loc.page && loc.page.href === p[0] ? 'active' : ''}">${esc(p[1])}</a>`).join('')}`).join('')}</div>` : '';
      return item(s.href, s.title, s.icon, s.color, on && (loc.root && (!loc.page || !s.groups.length)), `data-sec="${s.id}"`).replace('class="', `class="sec ${on ? 'open ' : ''}`) + sub;
    }).join('')}</div>
    <div class="sep">Сервис</div><div class="grp">${NAV_SERVICE.map(x => item(x[0], x[1], x[2], x[3], loc.service === x)).join('')}<a href="#" data-act="help">${iconTile('help', '#636366')}<span class="t">Справка <small>клавиша F1</small></span></a><a href="#" data-act="tour">${iconTile('target', '#FF2D55')}<span class="t">Как пользоваться</span></a></div>`;
  }
  $('#nav').innerHTML = html;
  $$('.tabbar a').forEach(a => { const id = a.dataset.sec; a.classList.toggle('active', id === cur); });
}

/* Шапка экрана: «Назад» (на детальных экранах) и вкладки раздела (на экранах раздела) */
function pageChrome(path, loc) {
  let top = '';
  if (loc.detail) {
    const b = backTarget(path, loc);
    if (b) top += `<nav class="backbar no-print" aria-label="Возврат"><a class="back" href="${b.href}">${svgIcon('back')}<span>${esc(b.title || 'Назад')}</span></a><span class="crumb">${esc(loc.section.title)}</span></nav>`;
  }
  if (loc.root && loc.section && loc.section.groups.length) {
    top += `<nav class="sectabs no-print" aria-label="${esc(loc.section.title)}">${loc.section.groups.map(g => `<div class="tg">${g.label ? `<span class="tg-l">${esc(g.label)}</span>` : ''}<div class="tabs">${g.pages.map(p => `<a href="${p[0]}" class="${loc.page && loc.page.href === p[0] ? 'active' : ''}" ${loc.page && loc.page.href === p[0] ? 'aria-current="page"' : ''}>${esc(p[1])}</a>`).join('')}</div></div>`).join('')}</nav>`;
  }
  return top;
}

/* Заголовок для верхней строки и для стека */
function screenTitle(loc, main) {
  if (loc.page) return loc.page.title;
  if (loc.service) return loc.service[1];
  if (loc.section && loc.root) return loc.section.title;
  const h = main.querySelector('h1'); const t = h ? h.textContent.trim() : '';
  return t.length > 42 ? t.slice(0, 40) + '…' : (t || 'ПФК ВПК');
}

/* Всплывающее сообщение с действием («Отменить») */
function toastAction(msg, label, fn, ms = 6000) {
  const t = document.createElement('div'); t.className = 'toast has-act';
  t.innerHTML = `<span>${esc(msg)}</span><button type="button" class="toast-btn">${esc(label)}</button>`;
  t.querySelector('button').addEventListener('click', () => { t.remove(); fn(); });
  const box = $('#toasts'); while (box.children.length >= 3) box.firstChild.remove(); box.appendChild(t); setTimeout(() => t.remove(), ms);
}
/* Индикатор длительной операции: показывается, если операция идёт дольше 150 мс */
async function withBusy(msg, fn) {
  const b = $('#busy'); $('#busyMsg').textContent = msg || 'Выполняется…';
  const timer = setTimeout(() => { b.hidden = false; }, 150);
  await new Promise(r => requestAnimationFrame(() => setTimeout(r, 0)));
```

```
  try { return await fn(); }
  finally { clearTimeout(timer); b.hidden = true; }
}
/* Экран ошибки: показывается вместо раздела, если при его построении произошёл сбой */
function errorScreen(err) {
  return `<div class="card empty" role="alert"><div class="state-ic err">!</div><h1 style="font-size:24px">Не удалось открыть раздел</h1>
  <p>Данные не повреждены и сохранены. Попробуйте вернуться назад или открыть главную.</p>
  <div class="row" style="justify-content:center"><button class="btn" data-act="nav-back">Назад</button><a class="btn primary" href="#/">На главную</a></div>
  <details style="margin-top:16px;text-align:left"><summary>Подробности для разработчика</summary><pre class="mono" style="white-space:pre-wrap;font-size:12px">${esc(err && (err.stack || err.message) || err)}</pre></details></div>`;
}
/* Единое «пустое состояние»: что здесь будет и что сделать дальше */
const emptyState = (title, text, buttons = '') => `<div class="empty"><div class="state-ic">ø</div><h3 style="margin:0 0 6px">${esc(title)}</h3>${text ? `<p class="muted" style="max-width:520px;margin:0 auto 6px">${text}</p>` : ''}${buttons ? `<div class="row" style="justify-content:center;margin-top:8px">${buttons}</div>` : ''}</div>`;
const notFound = what => `<div class="card">${emptyState(what + ' — не найдено', 'Запись могла быть удалена (проверьте корзину) или ссылка устарела.', '<button class="btn" data-act="nav-back">Назад</button><a class="btn" href="#/trash">Корзина</a>')}</div>`;
/* Свечение под курсором на кнопках, вкладках и плитках (переменные --mx, --my для CSS) */
const GLOW_SEL = '.btn,.tabs a,.nav a,.quick a,.tabbar a,.tabbar button,.backbar .back,.gradebar label';
document.addEventListener('pointermove', e => {
  const el = e.target.closest && e.target.closest(GLOW_SEL); if (!el) return;
  const t = el.matches('.gradebar label') ? el.querySelector('span') : el; if (!t) return;
  const r = t.getBoundingClientRect(); t.style.setProperty('--mx', (e.clientX - r.left) + 'px'); t.style.setProperty('--my', (e.clientY - r.top) + 'px');
}, { passive: true });

/* ======================================================================
   МОДУЛЬ 5. УПРАВЛЕНИЕ ИНТЕРФЕЙСОМ: утилиты, маршрутизация, визуализация
   ====================================================================== */
const $ = (s, r = document) => r.querySelector(s);
const $$ = (s, r = document) => Array.from(r.querySelectorAll(s));
const esc = s => String(s ?? '').replace(/[&<>"']/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
const isNumV = x => typeof x === 'number' && Number.isFinite(x);
const fmt = (x, d = 3) => isNumV(x) ? Calc.roundTo(x, d).toLocaleString('ru-RU', { minimumFractionDigits: d, maximumFractionDigits: d }) : '—';
const fmtK = x => isNumV(x) ? x.toLocaleString('ru-RU', { minimumFractionDigits: 5, maximumFractionDigits: 5 }) : '—';
const fmtFull = x => isNumV(x) ? x.toLocaleString('ru-RU', { maximumFractionDigits: 6 }) : '—';
const fmtDate = s => { if (!s) return '—'; const d = new Date(s); return isNaN(d) ? esc(s) : d.toLocaleDateString('ru-RU'); };
const fmtDT = s => { if (!s) return '—'; const d = new Date(s); return isNaN(d) ? esc(s) : d.toLocaleString('ru-RU', { day: '2-digit', month: '2-digit', year: 'numeric', hour: '2-digit', minute: '2-digit' }); };
const parseNum = v => { if (v === '' || v == null) return NaN; return Number(String(v).replace(',', '.').trim()); };

let state = Store.load();
const ui = { editing: null, filters: { type: '', status: '', q: '', from: '', to: '' }, summary: { type: 'practice', group: '', sub: '' }, reviews: { period: '1', year: '', basis: 'total' }, follow: { group: '', course: '', period: '1' }, groupsCmp: { type: 'practice', sub: '', a: '', b: '', c: '' }, saveTimer: null, lastRoute: '' };

function persist() {
  const ok = Store.save(state);
  renderStorage();
  if (!ok) toast('Не удалось сохранить данные в браузере. Сделайте резервную копию (раздел «Данные»).', true);
  return ok;
}
function persistSoon() { clearTimeout(ui.saveTimer); ui.saveTimer = setTimeout(persist, 300); }
function renderStorage() {
  const st = Store.status(), el = $('#storageStatus');
  if (!el) return;
  el.innerHTML = st.available
    ? `Хранилище браузера: доступно${st.lastSaved ? '<br>Сохранено ' + st.lastSaved.toLocaleTimeString('ru-RU') : ''}`
    : `<span style="color:var(--danger)">Хранилище недоступно: данные не сохраняются между запусками. Используйте резервную копию.</span>`;
```

```
}
function toast(msg, err = false) {
  const t = document.createElement('div'); t.className = 'toast' + (err ? ' err' : ''); t.textContent = msg;
  const box = $('#toasts'); while (box.children.length >= 3) box.firstChild.remove(); box.appendChild(t); setTimeout(() => t.remove(), err ? 7000 : 3500);
}
/* Модальный диалог подтверждения. buttons: [{label, value, cls}] */
function dialog(title, bodyHtml, buttons) {
  return new Promise(resolve => {
    const d = $('#dlg');
    $('#dlgTitle').textContent = title; $('#dlgBody').innerHTML = bodyHtml;
    $('#dlgFoot').innerHTML = buttons.map((b, i) => `<button type="button" class="btn ${b.cls || ''}" data-i="${i}">${esc(b.label)}</button>`).join('');
    const done = v => { d.close(); resolve(v); };
    $$('#dlgFoot button').forEach(btn => btn.onclick = () => done(buttons[+btn.dataset.i].value));
    d.oncancel = e => { e.preventDefault(); done(null); };
    d.showModal();
    const last = $('#dlgFoot button:last-child'); if (last) last.focus();
  });
}
const confirmDlg = (title, body, okLabel = 'Подтвердить', danger = false) =>
  dialog(title, body, [{ label: 'Отмена', value: false }, { label: okLabel, value: true, cls: danger ? 'danger' : 'primary' }]);

function download(name, text, mime) {
  const blob = new Blob([text], { type: mime });
  const a = document.createElement('a'); a.href = URL.createObjectURL(blob); a.download = name;
  document.body.appendChild(a); a.click(); setTimeout(() => { URL.revokeObjectURL(a.href); a.remove(); }, 1000);
}
const csvCell = v => { const s = v == null ? '' : String(v); return /[;"\n\r]/.test(s) ? '"' + s.replace(/"/g, '""') + '"' : s; };
const csvNum = x => isNumV(x) ? String(x).replace('.', ',') : '';
const toCsv = rows => '' + rows.map(r => r.map(csvCell).join(';')).join('\r\n');

/* Подсказки терминов (METHOD.GLOSSARY) */
const term = (k, label) => `<abbr title="${esc(METHOD.GLOSSARY[k] || '')}">${esc(label || k)}</abbr>`;
const glossaryFor = label => { const tokens = String(label).split(/[\s:(),.]+/); const k = Object.keys(METHOD.GLOSSARY).find(k => k.includes(' ') ? label.includes(k) : tokens.includes(k)); return k ? METHOD.GLOSSARY[k] : ''; };
const levelInfo = code => METHOD.LEVELS.find(l => l.code === code);
function lvBadge(lv, withMark = true) {
  if (!lv) return '<span class="lv lv-none">не определяется</span>';
  const L = levelInfo(lv.code);
  return `<span class="lv lv-${lv.code}" title="${esc(L.descr)}">«${esc(L.name)}»${withMark ? ' (' + esc(L.mark) + ')' : ''}</span>`;
}
const objName = id => { const o = state.objects.find(x => x.id === id); return o ? o.name : '(объект удалён)'; };
const typeTitle = t => (Model.TYPES[t] || { title: t }).title;
const statusBadge = a => a.status === 'final' ? '<span class="badge b-final">утверждён</span>' : '<span class="badge b-draft">черновик</span>';
const srcRef = keys => `<span class="src">Основание: ${esc(keys)}</span>`;

/* ---------- Визуализация (SVG без внешних библиотек) ---------- */
function lineChart({ labels, series, yMin, yMax, yDigits = 3, height = 280, yTitle = '', ticks = 5 }) {
  const W = 760, H = height, pl = 56, pt = 14, pb = labels.some(l => String(l).length > 5) ? 64 : 34, pr = pb > 40 ? 48 : 16;
  const vals = series.flatMap(s => s.values.filter(isNumV));
  if (!vals.length) return '<p class="muted">Нет данных для графика.</p>';
  let lo = yMin ?? Math.min(...vals), hi = yMax ?? Math.max(...vals);
  if (hi === lo) { hi += 0.05; lo -= 0.05; }
  const iw = W - pl - pr, ih = H - pt - pb;
  const x = i => pl + (labels.length === 1 ? iw / 2 : i * iw / (labels.length - 1));
  const y = v => pt + ih - (v - lo) / (hi - lo) * ih;
  let g = '';
  for (let k = 0; k <= ticks; k++) {
    const v = lo + (hi - lo) * k / ticks, yy = y(v);
    g += `<line x1="${pl}" x2="${W - pr}" y1="${yy}" y2="${yy}" stroke="#e5e7eb"/><text x="${pl - 6}" y="${yy + 4}" text-anchor="end">${fmt(v, yDigits)}</text>`;
  }
  labels.forEach((l, i) => {
    const rot = pb > 40;
```

```
    g += `<line x1="${x(i)}" x2="${x(i)}" y1="${pt}" y2="${pt + ih}" stroke="#f1f3f5"/>`;
    g += rot ? `<text transform="translate(${x(i) + 4},${pt + ih + 10}) rotate(45)">${esc(l)}</text>` : `<text x="${x(i)}" y="${pt + ih + 18}" text-anchor="middle">${esc(l)}</text>`;
  });
  for (const s of series) {
    const pts = s.values.map((v, i) => isNumV(v) ? [x(i), y(v), v, i] : null).filter(Boolean);
    if (!pts.length) continue;
    g += `<polyline fill="none" stroke="${s.color}" stroke-width="2.2" ${s.dash ? 'stroke-dasharray="5 4"' : ''} points="${pts.map(p => p[0] + ',' + p[1]).join(' ')}"/>`;
    g += pts.map(p => `<circle cx="${p[0]}" cy="${p[1]}" r="3.6" fill="${s.color}"><title>${esc(s.name)} — ${esc(labels[p[3]])}: ${fmt(p[2], yDigits)}</title></circle>`).join('');
  }
  if (yTitle) g += `<text transform="translate(12,${pt + ih / 2}) rotate(-90)" text-anchor="middle">${esc(yTitle)}</text>`;
  const legend = `<div class="legend">${series.map(s => `<span><i style="background:${s.color}"></i>${esc(s.name)}</span>`).join('')}</div>`;
  return `<svg class="chart" viewBox="0 0 ${W} ${H}" role="img" aria-label="${esc(series.map(s => s.name).join(', '))}">${g}</svg>${legend}`;
}
function competenceChart(rows, title) {
  const ok = rows.filter(r => r.grade != null);
  if (!ok.length) return '';
  return lineChart({ labels: rows.map(r => r.id), yMin: 0, yMax: 0.4, yTitle: title || 'Показатель сформированности',
    series: [
      { name: 'Показатель сформированности компетенции', color: '#007AFF', values: rows.map(r => r.psfk) },
      { name: 'Минимальное значение (оценка 2)', color: '#FF9500', values: rows.map(r => r.min) },
      { name: 'Максимальное значение (оценка 5)', color: '#8E8E93', values: rows.map(r => r.max), dash: true }] });
}
function levelBars(counts) {
  const total = Object.values(counts).reduce((s, x) => s + x, 0) || 1;
  return METHOD.LEVELS.map(L => `<div class="row" style="margin:4px 0"><span style="width:130px">«${L.name}»</span>
    <div class="bar-track"><div class="lv-${L.code}" style="height:100%;width:${(counts[L.code] || 0) / total * 100}%"></div></div>
    <span class="num" style="width:40px;text-align:right">${counts[L.code] || 0}</span></div>`).join('');
}

/* ---------- Маршрутизация ---------- */
const routes = [];
const route = (re, fn) => routes.push([re, fn]);
function render() {
  const h = (location.hash || '#/').slice(1) || '/';
  const path = h.split('?')[0];
  const loc = locate(path);
  $('#side').classList.remove('open'); $('#menuBtn').setAttribute('aria-expanded', 'false');
  let html = null, after = null;
  try {
    for (const [re, fn] of routes) {
      const m = path.match(re);
      if (m) { const r = fn(...m.slice(1).map(decodeURIComponent)); if (typeof r === 'string') html = r; else if (r) { html = r.html; after = r.after; } break; }
    }
    if (html == null) html = `<div class="card">${emptyState('Страница не найдена', 'Возможно, ссылка устарела или запись была удалена.', '<a class="btn primary" href="#/">На главную</a>')}</div>`;
  } catch (err) { console.error(err); html = errorScreen(err); after = null; }
  const main = $('#main');
  document.body.classList.toggle('no-anim', path === ui.lastRoute);
  main.innerHTML = pageChrome(path, loc) + html;
  const title = screenTitle(loc, main);
  trackNav(path, loc, title);
  renderNav(path, loc);
  $('#topTitle').textContent = title;
  document.title = title + ' — ПФК ВПК';
  if (path !== ui.lastRoute) { window.scrollTo(0, 0); main.focus({ preventScroll: true }); ui.lastRoute = path; }
  if (after) { try { after(main); } catch (err) { console.error(err); toast('Часть экрана не загрузилась: ' + err.message, true); } }
}

/* ---------- Делегирование событий ---------- */
```

```
const actions = {};
actions['nav-back'] = () => history.back();
document.addEventListener('click', async e => {
  const el = e.target.closest('[data-act]');
  if (!el || el.disabled) return;
  if (el.tagName === 'TR' && e.target.closest('input,select,label,button,a')) return;
  const fn = actions[el.dataset.act];
  if (fn) { e.preventDefault(); await fn(el, e); }
});
function setPath(obj, path, val) {
  const parts = path.split('|'); let o = obj;
  for (let i = 0; i < parts.length - 1; i++) { if (o[parts[i]] == null || typeof o[parts[i]] !== 'objec
t') o[parts[i]] = {}; o = o[parts[i]]; }
  if (val === undefined) delete o[parts[parts.length - 1]]; else o[parts[parts.length - 1]] = val;
}
function getPath(obj, path) { return path.split('|').reduce((o, k) => o == null ? undefined : o[k], ob
j); }
/* Поля, привязанные к редактируемой записи: data-bind="input|vpk|ВПК-1", data-type="int|num|str|bool"
*/
function onBind(e) {
  const el = e.target; if (!el.dataset || !el.dataset.bind || !ui.editing) return;
  const a = state.assessments.find(x => x.id === ui.editing); if (!a || a.status !== 'draft') return;
  let v;
  const t = el.dataset.type || 'str';
  if (el.type === 'checkbox') v = el.checked;
  else if (t === 'int') v = el.value === '' ? undefined : (Number.isInteger(Number(el.value)) ? Number(e
l.value) : el.value);
  else if (t === 'num') { const n = parseNum(el.value); v = el.value === '' ? '' : (isNumV(n) ? n : el.v
alue); el.classList.toggle('invalid', el.value !== '' && !isNumV(n)); }
  else v = el.value;
  setPath(a, el.dataset.bind, v);
  if (el.dataset.resnap) a.snapshot = Model.snapshot(state, a.type, a.meta, a.input);
  a.updatedAt = Store.nowIso();
  persistSoon();
  if (el.dataset.rerender) render(); else updateProgress(a);
}
document.addEventListener('change', onBind);
document.addEventListener('input', e => { if (e.target.matches('input[type=text][data-bind],input[type=n
umber][data-bind],textarea[data-bind],input[type=date][data-bind]')) onBind(e); });

/* ---------- Главная панель ---------- */
/* Первые шаги: показываются, пока пользователь их не выполнит или не скроет */
function onboardingCard() {
  if (state.meta.onboardHidden) return '';
  const steps = [
    [state.objects.length > 0, 'Добавьте курсантов', 'по одному или списком из Excel', '<a class="btn s
m" href="#/objects">Добавить</a> <a class="btn sm" href="#/import">Импорт</a>'],
    [state.assessments.length > 0, 'Введите оценки', 'анкету практики, отзыв из войск или сразу всю груп
пу', '<a class="btn sm" href="#/new">Новая оценка</a> <a class="btn sm" href="#/batch">По группе</a>'],
    [state.assessments.some(a => a.status === 'final'), 'Утвердите и посмотрите результат', 'уровни, отч
ёты, группы риска, сводная', '<a class="btn sm" href="#/summary">Анализ</a>'],
    [!!state.meta.lastBackupAt, 'Сохраните резервную копию', 'данные хранятся только в этом браузере',
'<a class="btn sm" href="#/data">Сохранить</a>']];
  if (steps.every(x => x[0])) return '';
  return `<div class="card onboard no-print"><div class="pagehead"><h2 style="margin:0">С чего начать</h
2><span class="spacer"></span><button class="btn sm" data-act="ob-hide">Скрыть</button></div>
  <p class="muted" style="margin:4px 0 6px">Четыре шага до первого результата. ${state.objects.length ?
'' : 'Хотите сначала посмотреть, как всё работает? <a href="#" data-act="demo-load">Откройте демонстраци
онные данные</a> — они загрузятся в отдельный учебный режим и не смешаются с рабочими.'}</p>
  <ol>${steps.map(([done, t, d, b], i) => `<li class="${done ? 'done' : ''}"><span class="n">${done ?
'✓' : i + 1}</span><div style="flex:1"><b>${t}</b><div class="muted" style="font-size:14px">${d}</div></
div>${done ? '' : `<div class="row">${b}</div>`}</li>`).join('')}</ol></div>`;
}
actions['ob-hide'] = () => { state.meta.onboardHidden = true; persist(); render(); toastAction('Подсказк
и скрыты', 'Вернуть', () => { state.meta.onboardHidden = false; persist(); render(); }); };
/* Краткий тур: как устроено приложение */
actions.tour = () => dialog('Как пользоваться', `<ol style="padding-left:20px;margin:0">
  <li><b>Главная</b> — что требует внимания: незавершённые оценки, группы риска, резервная копия.</li>
  <li><b>Оценки</b> — ввод (новая оценка, по группе, импорт) и просмотр (все оценки, журнал контроля, от
чёты о практике, бланки).</li>
  <li><b>Курсанты и выпускники</b> — список; в карточке курсанта — его оценки, профиль, карта этапов, от
чёт и сведения.</li>
  <li><b>Анализ</b> — сводная по группе, группы риска, отзывы, сравнения, статистика и прогноз g.</li>
```

```
  <li><b>Методика</b> — справочник, коэффициенты, паспорта компетенций, учебные планы, проверка расчёто
в.</li></ol>
  <p style="margin-top:12px">Кнопка <b>«‹ Назад»</b> над заголовком всегда показывает, куда она ведёт. П
оиск в меню находит любой раздел по слову. <b>F1</b> — справка по текущему экрану. Удалённое попадает в
корзину на 30 дней, сразу после удаления его можно вернуть кнопкой «Отменить».</p>`, [{ label: 'Понятн
о', value: true, cls: 'primary' }]);
route(/^\/$/, () => {
  const fin = state.assessments.filter(a => a.status === 'final');
  const drafts = state.assessments.filter(a => a.status === 'draft');
  const last = [...fin].sort((a, b) => (b.finalizedAt || '').localeCompare(a.finalizedAt || '')).slice
(0, 6);
  const st = Store.status();
  const checks = [
    ['Коэффициенты значимости ВПК (16), ПСК (5), ВПК практики (13)', 'подтверждены методикой', 'ok'],
    ['Шкалы уровней bcp и Псфк (ВПК, ПСК, итог)', 'подтверждены методикой; граница 0,272 применяется дос
ловно (расхождение отмечено)', 'ok'],
    ['Пороги уровней Псфк для 13 ВПК (практика)', 'в методике не установлены — уровень определяется по b
cp', 'warn'],
    ['Учебные планы для расчёта g', state.curricula.length ? `создано: ${state.curricula.length}` : 'не
созданы — требуется ввод пользователем', state.curricula.length ? 'ok' : 'warn'],
    ['Модели признаков для Y', state.yModels.length ? `создано: ${state.yModels.length}` : 'не созданы —
признаки, веса и эталон в методике не заданы', state.yModels.length ? 'ok' : 'warn'],
    ['Резервная копия данных', state.meta.lastBackupAt ? 'последняя: ' + fmtDT(state.meta.lastBackupAt)
: 'не создавалась', state.meta.lastBackupAt ? 'ok' : 'warn']
  ];
  const ob = onboardingCard();
  const tiles = [['#/new', 'Новая оценка', 'анкета практики или отзыв', 'plus', '#34C759'], ['#/batch',
'Ввод по группе', 'вся группа в одной таблице', 'list', '#30B0C7'], ['#/objects', 'Курсанты и выпускник
и', 'список и карточки', 'people', '#5856D6'], ['#/risk', 'Группы риска', 'кто отстаёт и почему', 'char
t', '#FF3B30']];
  return `
  <p class="eyebrow">${new Date().toLocaleDateString('ru-RU', { weekday: 'long', day: 'numeric', month:
'long' })}</p>
  <h1>Обзор</h1>
  <p class="muted">Расчёт уровня сформированности военно-профессиональных компетенций по модели полифунк
ционального контроля. <a href="#" data-act="tour">Как пользоваться</a></p>
  ${!st.available ? '<div class="note err"><b>Хранилище браузера недоступно.</b> Данные будут потеряны п
осле закрытия страницы. Регулярно сохраняйте резервную копию (раздел «Данные и резервные копии»).</div>'
: ''}
  ${st.error && st.available ? `<div class="note warn">${esc(st.error)}</div>` : ''}
  ${ob}
  ${typeof backupReminder === 'function' ? backupReminder() : ''}
  ${typeof riskSummaryCard === 'function' ? riskSummaryCard() : ''}
  ${drafts.length ? `<div class="card"><div class="pagehead"><h2 style="margin:0">Незавершённые оценки
(${drafts.length})</h2><span class="spacer"></span><a class="btn sm" href="#/history?draft">Все черновик
и</a></div>
    <div class="table-wrap" style="margin-top:8px"><table><tbody>${[...drafts].sort((a, b) => (b.updated
At || '').localeCompare(a.updatedAt || '')).slice(0, 3).map(a => `<tr class="click" data-act="go" data-h
ref="#/a/${a.id}/edit/1"><td>${esc(objName(a.objectId))}</td><td>${esc(Model.TYPES[a.type].short)}</td><
td class="muted">изменён ${fmtDT(a.updatedAt)}</td><td class="num"><span class="btn sm">Продолжить</span
></td></tr>`).join('')}</tbody></table></div></div>` : ''}
  <div class="quick no-print">${tiles.map(([h, l, d, ic, c]) => `<a href="${h}">${iconTile(ic, c)}<span>
${l}<br><small class="muted">${d}</small></span></a>`).join('')}</div>
  <div class="grid g4" style="margin:0 0 16px">
    <div class="kpi"><div class="l">Курсанты и выпускники</div><div class="v">${state.objects.length}</d
iv><div class="s"><a href="#/objects">открыть список</a></div></div>
    <div class="kpi"><div class="l">Утверждённые оценки</div><div class="v">${fin.length}</div><div clas
s="s"><a href="#/history">все оценки</a></div></div>
    <div class="kpi"><div class="l">Черновики</div><div class="v">${drafts.length}</div><div class="s">
${drafts.length ? '<a href="#/history?draft">продолжить</a>' : 'нет незавершённых'}</div></div>
    <div class="kpi"><div class="l">Версия методики</div><div class="v" style="font-size:20px">${METHOD.
VERSION}</div><div class="s"><a href="#/method">справочник</a></div></div>
  </div>
  <div class="grid g2">
    <div class="card"><h2>Последние оценки</h2>
      ${last.length ? `<div class="table-wrap"><table><thead><tr><th>Дата</th><th>Объект</th><th>Тип</th
><th class="num">Показатель</th><th>Уровень</th></tr></thead><tbody>
      ${last.map(a => { const h = Model.headline(a); return `<tr class="click" data-act="go" data-href
="#/a/${a.id}"><td>${fmtDate(a.meta.date)}</td><td>${esc(objName(a.objectId))}</td><td>${esc(Model.TYPES
[a.type].short)}</td><td class="num">${h ? esc(h.label) + ' ' + fmt(h.value, h.digits) : '—'}</td><td>
${h && h.level !== undefined ? lvBadge(h.level, false) : '—'}</td></tr>`; }).join('')}
      </tbody></table></div>` : emptyState('Оценок пока нет', 'Здесь появятся последние утверждённые оце
нки.', '<a class="btn primary" href="#/new">Новая оценка</a>')}
    </div>
```

```
    <div class="card"><h2>Готовность методики</h2>
      ${checks.map(c => `<div class="note ${c[2]}"><b>${esc(c[0])}:</b> ${esc(c[1])}</div>`).join('')}
    </div>
  </div>
  <div class="card"><h2>Кратко о модели</h2>
    <p>Модель охватывает этапы подготовки от абитуриента до выпускника и первых двух лет службы и включает блоки: <b>полифункционального контроля</b> (мониторинг и диагностика), <b>математический</b> (определение уровня подготовленности на начальной стадии — g₀, после каждой дисциплины — g, уровень компетентности — Y), <b>ИКТ</b> (программы для ЭВМ), <b>результативный</b> и <b>выходной</b> (уровень сформированности ВПК: «высокий», «средний», «низкий», «критический»).</p>
    <p>Основной показатель: <span class="formula">Псфк = Оценка × Кзнк</span>; суммарный показатель — среднее арифметическое Псфк по набору компетенций.</p>
  </div>`;
});
actions.go = el => { location.hash = el.dataset.href; };

/* ---------- Объекты оценки ---------- */
const SPECIALTIES = ['40.05.01 Правовое обеспечение национальной безопасности', '37.05.02 Психология служебной деятельности', 'иная специальность'];
function objectForm(o, quick = false) {
  o = o || { name: '', group: '', specialty: '', year: '', notes: '' };
  const main = `<div class="field"><label class="f" for="of-name">Фамилия И.О. *</label><input type="text" id="of-name" value="${esc(o.name)}" maxlength="120" autocomplete="off" placeholder="Иванов И.И."></div>
    <div class="field"><label class="f" for="of-group">Группа / подразделение</label><input type="text" id="of-group" value="${esc(o.group)}" maxlength="80" placeholder="41 взвод" list="groupList"></div>
    <datalist id="groupList">${[...new Set(state.objects.map(x => x.group).filter(Boolean))].sort().map(g => `<option value="${esc(g)}">`).join('')}</datalist>`;
  if (quick) return `<div class="quickadd">${main}<div class="field"><button class="btn primary" data-act="obj-add">Добавить</button></div></div>
    <details><summary>Дополнительные сведения (необязательно)</summary><div style="margin-top:10px">${objectExtra(o)}</div></details>`;
  return `<div class="grid g2">${main}</div>${objectExtra(o)}`;
}
function objectExtra(o) {
  return `<div class="grid g2">
    <div class="field"><label class="f" for="of-spec">Специальность</label><select id="of-spec"><option value="">— не указана —</option>${SPECIALTIES.map(s => `<option ${o.specialty === s ? 'selected' : ''}>${esc(s)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="of-year">Год выпуска / курс</label><input type="text" id="of-year" value="${esc(o.year)}" maxlength="20"></div>
  </div>
  <div class="field"><label class="f" for="of-notes">Примечание</label><textarea id="of-notes" maxlength="500">${esc(o.notes)}</textarea></div>
  <details ${o.applicant && (o.applicant.education || o.applicant.experience || Object.values(o.applicant.flags || {}).some(Boolean)) ? 'open' : ''}><summary style="cursor:pointer;color:var(--accent);font-weight:600">Сведения абитуриента (входной блок модели)</summary>
    <div class="grid g2" style="margin-top:8px"><div class="field"><label class="f" for="of-edu">Образование до поступления</label><select id="of-edu"><option value="">— не указано —</option>${METHOD.APPLICANT.education.map(x => `<option ${(o.applicant || {}).education === x ? 'selected' : ''}>${esc(x)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="of-exp">Предыдущий опыт (кратко)</label><input type="text" id="of-exp" maxlength="200" value="${esc((o.applicant || {}).experience || '')}"></div></div>
    <div class="row">${METHOD.APPLICANT.flags.map(([k, l]) => `<label><input type="checkbox" class="of-flag" value="${k}" ${((o.applicant || {}).flags || {})[k] ? 'checked' : ''}> ${esc(l)}</label>`).join('')}</div>
    <p class="hint">Результаты ЕГЭ, профессионального отбора и входной диагностики вносятся в «Журнал контроля» на этапе «Абитуриент».</p></details>
  <p class="hint">Храните только сведения, необходимые для расчёта. Персональные данные сверх ФИО не требуются.</p>`;
}
function readObjectForm() {
  const name = $('#of-name').value.trim();
  if (!name) { $('#of-name').classList.add('invalid'); $('#of-name').focus(); toast('Укажите Фамилию И.О. или обозначение', true); return null; }
  const flags = {}; $$('.of-flag').forEach(c => { if (c.checked) flags[c.value] = true; });
  return { name, group: $('#of-group').value.trim(), specialty: $('#of-spec').value, year: $('#of-year').value.trim(), notes: $('#of-notes').value.trim(),
    applicant: { education: $('#of-edu') ? $('#of-edu').value : '', experience: $('#of-exp') ? $('#of-exp').value.trim() : '', flags } };
}
route(/^\/objects$/, () => {
  const list = [...state.objects].sort((a, b) => a.name.localeCompare(b.name, 'ru'));
```

```
  const groups = [...new Set(list.map(o => o.group).filter(Boolean))].sort((a, b) => a.localeCompare(b, 'ru', { numeric: true }));
  const F = ui.objF || (ui.objF = { q: '', group: '', sort: 'name' });
  if (!ui.objSel) ui.objSel = new Set();
  for (const id of [...ui.objSel]) if (!list.some(o => o.id === id)) ui.objSel.delete(id);
  const dups = typeof duplicateGroups === 'function' ? duplicateGroups() : [];
  const cnt = {}; state.assessments.forEach(a => { cnt[a.objectId] = (cnt[a.objectId] || 0) + 1; });
  const last = {}; state.assessments.filter(a => a.status === 'final').forEach(a => { const d = a.meta.date || ''; if (!last[a.objectId] || d > last[a.objectId].meta.date) last[a.objectId] = a; });
  const rows = list.filter(o => !F.group || o.group === F.group);
  if (F.sort === 'group') rows.sort((a, b) => (a.group || '').localeCompare(b.group || '', 'ru', { numeric: true }) || a.name.localeCompare(b.name, 'ru'));
  if (F.sort === 'count') rows.sort((a, b) => (cnt[b.id] || 0) - (cnt[a.id] || 0));
  return `<div class="pagehead"><h1>Курсанты и выпускники</h1><span class="spacer"></span>${list.length ? `<a class="btn" href="#/import">Импорт списка</a>` : ''}</div>
  <p class="muted">Все, по кому ведутся оценки. Нажмите на строку, чтобы открыть карточку: оценки, профиль, карта этапов, отчёт.</p>
  <div class="card no-print"><h2 style="font-size:17px;margin:0 0 10px">Добавить курсанта</h2>${objectForm(null, true)}</div>
  ${dups.length ? `<div class="note warn no-print">Найдены возможные дубли: ${dups.length} ${dups.length === 1 ? 'группа' : 'групп(ы)'} записей с одинаковыми ФИО. <a href="#dupCard" data-act="go-dups">Проверить и объединить</a></div>` : ''}
  <div class="card">${list.length ? `<div class="row no-print" style="margin-bottom:10px">
    <input type="search" id="objSearch" placeholder="Поиск по ФИО" style="max-width:260px" aria-label="Поиск по ФИО" value="${esc(F.q)}">
    <select data-objf="group" aria-label="Группа" style="max-width:200px"><option value="">Все группы (${list.length})</option>${groups.map(g => `<option ${F.group === g ? 'selected' : ''} value="${esc(g)}">${esc(g)} (${list.filter(o => o.group === g).length})</option>`).join('')}</select>
    <select data-objf="sort" aria-label="Сортировка" style="max-width:220px"><option value="name" ${F.sort === 'name' ? 'selected' : ''}>По алфавиту</option><option value="group" ${F.sort === 'group' ? 'selected' : ''}>По группе</option><option value="count" ${F.sort === 'count' ? 'selected' : ''}>По числу оценок</option></select>
    <span class="spacer"></span><span class="muted" id="objCount">Показано: ${rows.length}</span></div>
  <div class="bulkbar no-print" id="bulkBar" ${ui.objSel.size ? '' : 'hidden'}><span><b id="bulkN">${ui.objSel.size}</b> выбрано</span><span class="spacer"></span><button class="btn sm" data-act="bulk-group">Перевести в группу…</button><button class="btn sm danger" data-act="bulk-del">Удалить выбранных</button><button class="btn sm" data-act="bulk-clear">Снять выбор</button></div>
  <div class="table-wrap"><table id="objTable"><thead><tr><th class="c no-print" style="width:36px"><input type="checkbox" id="selAll" aria-label="Выбрать всех показанных"></th><th>Фамилия И.О.</th><th>Группа</th><th class="num">Оценок</th><th>Последний результат</th></tr></thead><tbody>
  ${rows.map(o => { const a = last[o.id], h = a && Model.headline(a); return `<tr class="click" data-act="go" data-href="#/object/${o.id}" data-s="${esc((o.name + ' ' + o.group).toLowerCase())}" ${F.q && !(o.name + ' ' + o.group).toLowerCase().includes(F.q.toLowerCase()) ? 'hidden' : ''}><td class="c no-print"><input type="checkbox" class="osel" value="${o.id}" ${ui.objSel.has(o.id) ? 'checked' : ''} aria-label="Выбрать ${esc(o.name)}"></td><td><b style="font-weight:600">${esc(o.name)}</b></td><td>${esc(o.group) || '—'}</td><td class="num">${cnt[o.id] || 0}</td><td>${h ? `${esc(Model.TYPES[a.type].short)} · ${h.level !== undefined ? lvBadge(h.level, false) : fmt(h.value, 3)}` : '<span class="muted">нет оценок</span>'}</td></tr>`; }).join('')}
  </tbody></table></div><div id="objNone" hidden>${emptyState('Никого не найдено', 'Измените условия поиска или выберите другую группу.')}</div>`
  : emptyState('Список пока пуст', 'Добавьте курсанта в форме выше, загрузите список из Excel или попробуйте приложение на демонстрационных данных.', '<a class="btn" href="#/import">Импорт из файла</a><button class="btn" data-act="demo-load">Демонстрационные данные</button>')}</div>
  ${dups.length ? `<div id="dupCard">${duplicatesCard()}</div>` : ''}`;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.objf; if (k) { ui.objF[k] = e.target.value; render(); } });
/* Выбор нескольких курсантов и действия над ними */
function bulkSync() {
  const n = ui.objSel.size; const bar = $('#bulkBar'); if (!bar) return;
  bar.hidden = !n; $('#bulkN').textContent = n;
  const vis = $$('#objTable tbody tr:not([hidden]) .osel'); const all = $('#selAll');
  if (all) { const c = vis.filter(x => x.checked).length; all.checked = !!vis.length && c === vis.length; all.indeterminate = c > 0 && c < vis.length; }
}
document.addEventListener('change', e => {
  if (e.target.classList.contains('osel')) { e.target.checked ? ui.objSel.add(e.target.value) : ui.objSel.delete(e.target.value); e.target.closest('tr').classList.toggle('selected', e.target.checked); bulkSync(); }
  if (e.target.id === 'selAll') { $$('#objTable tbody tr:not([hidden]) .osel').forEach(x => { x.checked = e.target.checked; e.target.checked ? ui.objSel.add(x.value) : ui.objSel.delete(x.value); x.closest('tr').classList.toggle('selected', x.checked); }); bulkSync(); }
});
actions['bulk-clear'] = () => { ui.objSel.clear(); render(); };
```

```
actions['bulk-group'] = async () => {
  const ids = [...ui.objSel]; if (!ids.length) return;
  const groups = groupList();
  const r = await dialog(`Перевести в группу: ${ids.length}`, `<div class="field"><label class="f" for
="bgName">Новая группа / подразделение</label><input type="text" id="bgName" list="bgList" maxlength="8
0" placeholder="например, 51 взвод" autocomplete="off"><datalist id="bgList">${groups.map(g => `<option
value="${esc(g)}">`).join('')}</datalist></div><p class="hint">Можно выбрать существующую группу или вве
сти новую. Оценки и журнал остаются у курсантов.</p>`,
    [{ label: 'Отмена', value: null }, { label: 'Перевести', value: 'ok', cls: 'primary' }]);
  if (r !== 'ok') return;
  const g = ($('#bgName') || {}).value; const name = (g || '').trim();
  if (!name) { toast('Группа не указана — перевод не выполнен', true); return; }
  const prev = ids.map(id => { const o = state.objects.find(x => x.id === id); return o ? [o, o.group] :
null; }).filter(Boolean);
  prev.forEach(([o]) => { o.group = name; });
  ui.objSel.clear(); persist(); render();
  toastAction(`Переведено в «${name}»: ${prev.length}`, 'Отменить', () => { prev.forEach(([o, old]) => {
o.group = old; }); persist(); render(); toast('Перевод отменён'); }, 8000);
};
actions['bulk-del'] = async () => {
  const objs = [...ui.objSel].map(id => state.objects.find(o => o.id === id)).filter(Boolean); if (!obj
s.length) return;
  const nA = state.assessments.filter(a => ui.objSel.has(a.objectId)).length;
  if (!await confirmDlg(`Удалить выбранных: ${objs.length}?`, `<p>${objs.slice(0, 6).map(o => esc(o.nam
e)).join(', ')}${objs.length > 6 ? ` и ещё ${objs.length - 6}` : ''}.</p><p>Вместе с ними в корзину пере
местятся их оценки (${nA}) и записи журнала. Корзина хранит удалённое 30 дней.</p>`, 'Удалить', true)) r
eturn;
  const tids = objs.map(o => { const as = state.assessments.filter(a => a.objectId === o.id), rs = (stat
e.controlRecords || []).filter(r => r.objectId === o.id); return toTrash('object', o.name, { object: o,
assessments: as, records: rs }); });
  const del = new Set(objs.map(o => o.id));
  state.objects = state.objects.filter(o => !del.has(o.id)); state.assessments = state.assessments.filte
r(a => !del.has(a.objectId)); state.controlRecords = (state.controlRecords || []).filter(r => !del.has
(r.objectId));
  ui.objSel.clear(); persist(); render();
  toastAction(`Удалено курсантов: ${objs.length}`, 'Отменить', () => { const n = tids.filter(id => resto
reTrash(id)).length; toast(`Восстановлено: ${n}`); render(); }, 10000);
};
actions['go-dups'] = () => { const c = $('#dupCard'); if (c) c.scrollIntoView({ behavior: 'smooth' });
};
document.addEventListener('input', e => {
  if (e.target.id === 'objSearch') { const q = e.target.value.toLowerCase().trim(); ui.objF.q = q; let n
= 0; $$('#objTable tbody tr').forEach(tr => { tr.hidden = !!q && !tr.dataset.s.includes(q); if (!tr.hidd
en) n++; }); $('#objCount').textContent = 'Показано: ' + n; $('#objNone').hidden = n > 0; }
});
actions['obj-add'] = async () => {
  const d = readObjectForm(); if (!d) return;
  if (state.objects.some(o => normName(o.name) === normName(d.name) && (o.group || '') === d.group) && !
await confirmDlg('Такой курсант уже есть', `<p>В группе «${esc(d.group || 'без группы')}» уже есть «${es
c(d.name)}». Добавить ещё одну запись?</p>`, 'Добавить')) return;
  const o = { id: Store.uid(), ...d, createdAt: Store.nowIso() }; state.objects.push(o); persist(); rend
er();
  toastAction(`Добавлен: ${d.name}`, 'Открыть карточку', () => { location.hash = '#/object/' + o.id; });
const n = $('#of-name'); if (n) n.focus();
};
route(/^\/object\/([\w-]+)$/, id => {
  const o = state.objects.find(x => x.id === id);
  if (!o) return objMissing();
  const list = state.assessments.filter(a => a.objectId === id).sort((a, b) => (a.meta.date || '').local
eCompare(b.meta.date || ''));
  const fin = list.filter(a => a.status === 'final');
  const dyn = ['practice', 'graduate', 'trajectory', 'g0', 'ymodel'].map(t => {
    const arr = fin.filter(a => a.type === t); if (arr.length < 1) return '';
    const label = a => (t === 'practice' ? (a.meta.course + ' курс') : t === 'graduate' ? (a.meta.period
+ ' год сл.') : fmtDate(a.meta.date));
    const h0 = Model.headline(arr[0]);
    return `<h3>${esc(typeTitle(t))}: ${esc(h0.label)}</h3>${arr.length > 1 ? lineChart({ labels: arr.ma
p(label), series: [{ name: h0.label, color: '#007AFF', values: arr.map(a => Model.headline(a).value) }],
yDigits: 3 }) : `<p class="muted">Одна оценка: ${fmt(h0.value, 3)}. Для динамики нужно не менее двух.</p
>`}`;
  }).join('');
  return objectChrome(o, 'overview') + `
  <div class="card"><h2>Оценки</h2>
```

```
  ${list.length ? `<div class="table-wrap"><table><thead><tr><th class="no-print">Сравн.</th><th>Дата</th><th>Тип</th><th>Параметры</th><th>Статус</th><th class="num">Показатель</th><th>Уровень</th></tr></thead><tbody>
  ${list.map(a => { const h = Model.headline(a); return `<tr><td class="no-print">${a.status === 'final' && (a.type === 'graduate' || a.type === 'practice') ? `<input type="checkbox" class="cmp" value="${a.id}" data-type="${a.type}" aria-label="Выбрать для сравнения">` : ''}</td><td><a href="#/a/${a.id}">${fmtDate(a.meta.date)}</a></td><td>${esc(Model.TYPES[a.type].short)}</td><td>${a.type === 'practice' ? a.meta.course + ' курс' : a.type === 'graduate' ? a.meta.period + ' год службы' : ''}</td><td>${statusBadge(a)}</td><td class="num">${h ? fmt(h.value, 3) : '—'}</td><td>${h && h.level !== undefined ? lvBadge(h.level, false) : '—'}</td></tr>`; }).join('')}
  </tbody></table></div><button class="btn sm no-print" style="margin-top:10px" data-act="compare">Сравнить выбранные (2 оценки одного типа)</button>` : emptyState('Оценок пока нет', 'Начните с анкеты войсковой практики или отзыва из войск.', `<a class="btn primary" href="#/new?obj=${o.id}">Новая оценка</a>`)}
  </div>
  <div class="card"><h2>Динамика</h2>${dyn || '<p class="muted">Нет утверждённых расчётов.</p>'}</div>
  `;
});
const objMissing = () => `<div class="card">${emptyState('Курсант не найден', 'Возможно, запись удалена — её можно восстановить из корзины.', '<a class="btn" href="#/trash">Корзина</a><a class="btn primary" href="#/objects">К списку</a>')}</div>`;
/* Шапка карточки курсанта: имя, группа, главное действие и вкладки */
function objectChrome(o, tab) {
  const ini = o.name.split(/\s+/).map(x => x[0] || '').join('').slice(0, 2).toUpperCase();
  const n = state.assessments.filter(a => a.objectId === o.id).length;
  const hasPair = typeof Calc !== 'undefined' && state.assessments.some(a => a.objectId === o.id && a.type === 'graduate' && a.status === 'final') && state.assessments.some(a => a.objectId === o.id && a.type === 'practice' && a.status === 'final');
  const tabs = [['overview', '', 'Оценки'], ['profile', '/profile', 'Профиль'], ['map', '/map', 'Карта этапов'], ['report', '/report', 'Отчёт'], ...(hasPair ? [['follow', null, 'Практика и служба']] : []), ['edit', '/edit', 'Сведения']];
  return `<div class="no-print"><div class="obj-head"><span class="avatar" aria-hidden="true">${esc(ini)}</span><div class="meta"><h1>${esc(o.name)}</h1><p class="muted" style="margin:2px 0 0">${esc(o.group || 'группа не указана')}${o.specialty ? ' · ' + esc(o.specialty) : ''}${o.year ? ' · ' + esc(o.year) : ''} · оценок: ${n}</p></div>
  <a class="btn primary" href="#/new?obj=${o.id}">${svgIcon('plus', 'bi')} Новая оценка</a></div>
  <nav class="tabs" aria-label="Карточка курсанта" style="margin-top:10px">${tabs.map(([k, suf, l]) => `<a href="${suf === null ? '#/followup/' + o.id : '#/object/' + o.id + suf}" class="${k === tab ? 'active' : ''}" ${k === tab ? 'aria-current="page"' : ''}>${l}</a>`).join('')}</nav></div>`;
}
route(/^\/object\/([\w-]+)\/edit$/, id => {
  const o = state.objects.find(x => x.id === id); if (!o) return objMissing();
  return objectChrome(o, 'edit') + `<div class="card no-print"><h2>Сведения о курсанте</h2>${objectForm(o)}<div class="row"><button class="btn primary" data-act="obj-save" data-id="${o.id}">Сохранить изменения</button><span class="spacer"></span></div></div>
  <div class="card no-print"><h2>Удаление</h2><p class="muted">Курсант, его оценки и записи журнала переместятся в корзину. В течение 30 дней их можно восстановить.</p><button class="btn danger" data-act="obj-del" data-id="${o.id}">Удалить курсанта</button></div>`;
});
/* Отчёт по объекту оценки: все утверждённые расчёты, динамика, сравнение по компетенциям */
route(/^\/object\/([\w-]+)\/report$/, id => {
  const o = state.objects.find(x => x.id === id);
  if (!o) return objMissing();
  const fin = state.assessments.filter(a => a.objectId === id && a.status === 'final').sort((a, b) => (a.meta.date || '').localeCompare(b.meta.date || ''));
  const lab = a => `${fmtDate(a.meta.date)}${a.type === 'practice' ? ', ' + a.meta.course + ' курс' : a.type === 'graduate' ? ', ' + a.meta.period + ' год' : ''}`;
  const matrix = t => {
    const arr = fin.filter(a => a.type === t); if (!arr.length) return '';
    const rowsOf = a => [a.result.vpk, a.result.psk].filter(Boolean).flatMap(x => x.rows);
    const ids = [...new Set(arr.flatMap(a => rowsOf(a).map(r => r.id)))];
    const cell = (a, id) => { const r = rowsOf(a).find(x => x.id === id); return r ? `${r.grade ?? '—'} / ${fmt(r.psfk, 3)}` : '—'; };
    return `<h3>${esc(typeTitle(t))}: оценка / Псфк по компетенциям</h3><div class="table-wrap"><table><thead><tr><th>Компетенция</th>${arr.map(a => `<th class="num">${esc(lab(a))}</th>`).join('')}</tr></thead><tbody>
      ${ids.map(cid => `<tr><td>${esc(cid)}</td>${arr.map(a => `<td class="num">${cell(a, cid)}</td>`).join('')}</tr>`).join('')}</tbody>
      <tfoot><tr><td>Суммарный показатель ВПК (bcp / Псфк)</td>${arr.map(a => `<td class="num">${fmt(a.result.vpk.meanGrade, 2)} / ${fmt(a.result.vpk.meanPsfk, 3)}</td>`).join('')}</tr></tfoot></table></div>
      ${arr.length > 1 ? lineChart({ labels: arr.map(lab), series: [{ name: 'Суммарный Псфк ВПК', color: '#007AFF', values: arr.map(a => a.result.vpk.meanPsfk) }] }) : ''}`;
  };
  return objectChrome(o, 'report') + `<div class="row no-print" style="margin-bottom:12px"><button class="btn primary" data-act="print">Печать / сохранить в PDF</button></div>
```

```
  <div class="report"><div class="card"><h1>Отчёт по объекту оценки</h1><dl class="spec"><dt>Фамилия И.О.</dt><dd>${esc(o.name)}</dd><dt>Группа / подразделение</dt><dd>${esc(o.group || '—')}</dd><dt>Специальность</dt><dd>${esc(o.specialty || '—')}</dd><dt>Год выпуска / курс</dt><dd>${esc(o.year || '—')}</dd><dt>Утверждённых расчётов</dt><dd>${fin.length}</dd></dl></div>
  <div class="card"><h2>Утверждённые расчёты</h2>${fin.length ? `<div class="table-wrap"><table><thead><tr><th>Дата</th><th>Вид оценки</th><th>Параметры</th><th>Оценщик</th><th class="num">Показатель</th><th>Уровень</th><th>Методика</th></tr></thead><tbody>
    ${fin.map(a => { const h = Model.headline(a); return `<tr><td>${fmtDate(a.meta.date)}</td><td>${esc(Model.TYPES[a.type].short)}</td><td>${a.type === 'practice' ? a.meta.course + ' курс' : a.type === 'graduate' ? a.meta.period + ' год службы' : ''}</td><td>${esc(a.meta.evaluator || '—')}</td><td class="num">${h ? esc(h.label) + ' ' + fmt(h.value, 3) : '—'}</td><td>${h && h.level !== undefined ? lvBadge(h.level) : '—'}</td><td>v${esc(a.snapshot.methodVersion)}</td></tr>`; }).join('')}</tbody></table></div>` : '<p class="muted">Утверждённых расчётов нет.</p>'}</div>
  ${fin.some(a => a.type === 'practice' || a.type === 'graduate') ? `<div class="card"><h2>Динамика сформированности компетенций</h2>${matrix('practice')}${matrix('graduate')}<p class="src">Сравнение значений Псфк корректно при одинаковом наборе Кзнк.</p></div>` : ''}
  ${fin.filter(a => a.type === 'trajectory' || a.type === 'g0' || a.type === 'ymodel').map(a => `<div class="card"><h2>${esc(typeTitle(a.type))} — ${fmtDate(a.meta.date)}</h2>${resultView(a, a.result)}</div>`).join('')}
  <p class="muted">Подробности каждого расчёта, формулы и основания — в отчёте по отдельной оценке. Сформировано ${fmtDT(new Date().toISOString())}.</p></div>`;
});
actions['obj-save'] = el => {
  const o = state.objects.find(x => x.id === el.dataset.id); const d = readObjectForm(); if (!o || !d) return;
  Object.assign(o, d); persist(); toast('Сведения сохранены'); render();
};
actions.compare = () => {
  const sel = $$('.cmp:checked');
  if (sel.length !== 2 || sel[0].dataset.type !== sel[1].dataset.type) { toast('Выберите ровно две утверждённые оценки одного типа', true); return; }
  location.hash = `#/compare/${sel[0].value}/${sel[1].value}`;
};
route(/^\/compare\/([\w-]+)\/([\w-]+)$/, (i1, i2) => {
  const A = state.assessments.find(a => a.id === i1), B = state.assessments.find(a => a.id === i2);
  if (!A || !B || !A.result || !B.result) return '<div class="card empty">Оценки не найдены или не утверждены.</div>';
  const [a, b] = [A, B].sort((x, y) => (x.meta.date || '').localeCompare(y.meta.date || ''));
  const sets = r => [r.vpk, r.psk].filter(Boolean);
  const rowsA = sets(a.result).flatMap(s => s.rows), rowsB = sets(b.result).flatMap(s => s.rows);
  const ids = [...new Set([...rowsA, ...rowsB].map(r => r.id))];
  const lab = x => `${fmtDate(x.meta.date)}${x.type === 'practice' ? ', ' + x.meta.course + ' курс' : x.type === 'graduate' ? ', ' + x.meta.period + ' год' : ''}`;
  return `<h1>Сравнение оценок</h1><p class="muted">${esc(objName(a.objectId))}${a.objectId !== b.objectId ? ' / ' + esc(objName(b.objectId)) : ''} · ${esc(typeTitle(a.type))}</p>
  <div class="card"><div class="table-wrap"><table><thead><tr><th>Компетенция</th><th class="num">Оценка (${esc(lab(a))})</th><th class="num">Оценка (${esc(lab(b))})</th><th class="num">Δ оценки</th><th class="num">Псфк 1</th><th class="num">Псфк 2</th><th class="num">Δ Псфк</th></tr></thead><tbody>
  ${ids.map(id => { const x = rowsA.find(r => r.id === id) || {}, y = rowsB.find(r => r.id === id) || {}; const dg = isNumV(x.grade) && isNumV(y.grade) ? y.grade - x.grade : null; const dp = isNumV(x.psfk) && isNumV(y.psfk) ? y.psfk - x.psfk : null;
    return `<tr><td>${esc(id)}</td><td class="num">${x.grade ?? '—'}</td><td class="num">${y.grade ?? '—'}</td><td class="num">${dg == null ? '—' : (dg > 0 ? '+' : '') + dg}</td><td class="num">${fmt(x.psfk, 4)}</td><td class="num">${fmt(y.psfk, 4)}</td><td class="num">${dp == null ? '—' : (dp > 0 ? '+' : '') + fmt(dp, 4)}</td></tr>`; }).join('')}
  </tbody><tfoot><tr><td>Суммарный показатель ВПК</td><td class="num">${fmt(a.result.vpk.meanGrade, 2)}</td><td class="num">${fmt(b.result.vpk.meanGrade, 2)}</td><td class="num">${fmt(b.result.vpk.meanGrade - a.result.vpk.meanGrade, 2)}</td><td class="num">${fmt(a.result.vpk.meanPsfk, 3)}</td><td class="num">${fmt(b.result.vpk.meanPsfk, 3)}</td><td class="num">${fmt(b.result.vpk.meanPsfk - a.result.vpk.meanPsfk, 3)}</td></tr></tfoot></table></div>
  ${a.snapshot.sets.vpk.id !== b.snapshot.sets.vpk.id ? '<div class="note warn">Оценки рассчитаны с разными наборами Кзнк — значения Псфк несопоставимы напрямую.</div>' : ''}</div>
  <div class="card"><h2>Псфк по компетенциям</h2>${lineChart({ labels: ids, yMin: 0, yMax: 0.4, series: [{ name: lab(a), color: '#8E8E93', values: ids.map(id => (rowsA.find(r => r.id === id) || {}).psfk) }, { name: lab(b), color: '#007AFF', values: ids.map(id => (rowsB.find(r => r.id === id) || {}).psfk) }] })}</div>`;
});

/* ======================================================================
   МОДУЛЬ 6. ПРОВЕДЕНИЕ ОЦЕНКИ, РЕЗУЛЬТАТЫ, ОТЧЁТ
   ====================================================================== */
const hashQuery = () => new URLSearchParams((location.hash.split('?')[1]) || '');
const findA = id => state.assessments.find(a => a.id === id);

```

```
/* ---------- Шаг 0: выбор объекта и типа ---------- */
route(/^\/new$/, () => {
  const pre = hashQuery().get('obj') || '';
  const objs = [...state.objects].sort((a, b) => a.name.localeCompare(b.name, 'ru'));
  return `<h1>Новая оценка</h1>
  <ol class="steps"><li class="cur">1. Объект и вид оценки</li><li>2. Параметры</li><li>3. Ввод данных</li><li>4. Проверка и расчёт</li></ol>
  <div class="card"><h2>Объект оценки</h2>
    ${objs.length ? `<div class="field"><label class="f" for="nw-obj">Выберите курсанта / выпускника</label><select id="nw-obj"><option value="">— выберите —</option>${objs.map(o => `<option value="${o.id}" ${o.id === pre ? 'selected' : ''}>${esc(o.name)}${o.group ? ' — ' + esc(o.group) : ''}</option>`).join('')}</select></div>` : '<div class="note warn">Объектов оценки нет. Добавьте объект ниже.</div>'}
    <details ${objs.length ? '' : 'open'}><summary class="btn sm" style="display:inline-flex">+ Добавить новый объект</summary><div style="margin-top:12px">${objectForm()}<button class="btn" data-act="nw-addobj">Добавить и выбрать</button></div></details>
  </div>
  <div class="card"><h2>Вид оценки</h2>
    <div class="grid g2" role="radiogroup" aria-label="Вид оценки">
    ${Object.entries(Model.TYPES).map(([k, t], i) => `<label class="kpi" style="cursor:pointer;display:block"><input type="radio" name="nw-type" value="${k}" ${i === 0 ? 'checked' : ''}> <b>${esc(t.title)}</b><div class="s">${esc(typeDescr(k))}</div><div class="src">${esc(t.src)}</div></label>`).join('')}
    </div>
  </div>
  <div class="row"><button class="btn primary" data-act="nw-create">Создать черновик и продолжить →</button><a class="btn" href="#/">Отмена</a></div>`;
});
function typeDescr(k) {
  return {
    graduate: 'Оценка командиром 16 ВПК и 5 ПСК по 5-балльной схеме; Псфк = Оценка × Кзнк; суммарные показатели и итоговая оценка; разделы II–V отзыва.',
    practice: 'Анкета руководителя практики (ОК, ПК, ВПК по 5-балльной схеме); расчёт Псфк по 13 ВПК как в программе ЭВМ; степень сформированности в %, минимум и максимум.',
    g0: 'Формула (22): относительный уровень подготовленности на начальной стадии по оценкам аттестата и вступительных испытаний.',
    trajectory: 'Алгоритм: уровень подготовленности после каждой дисциплины учебного плана, формулы (23), (24) или (21).',
    ymodel: 'Уровень компетентности как отношение взвешенной суммы признаков выпускника к эталону. Признаки, веса и эталон задаются пользователем.'
  }[k];
}
actions['nw-addobj'] = () => {
  const d = readObjectForm(); if (!d) return;
  const o = { id: Store.uid(), ...d, createdAt: Store.nowIso() }; state.objects.push(o); persist();
  location.hash = '#/new?obj=' + o.id; render(); toast('Объект добавлен');
};
actions['nw-create'] = () => {
  const obj = $('#nw-obj') ? $('#nw-obj').value : '';
  if (!obj) { toast('Выберите объект оценки', true); if ($('#nw-obj')) $('#nw-obj').focus(); return; }
  const type = ($('input[name=nw-type]:checked') || {}).value;
  const a = Model.newAssessment(state, type, obj);
  state.assessments.push(a); persist();
  location.hash = `#/a/${a.id}/edit/1`;
};

/* ---------- Мастер ввода ---------- */
function gradeBar(path, val, label) {
  return `<div class="gradebar" role="radiogroup" aria-label="${esc(label)}">${[1, 2, 3, 4, 5].map(g => `<label><input type="radio" name="gr-${esc(path)}" value="${g}" data-bind="${esc(path)}" data-type="int" ${val === g ? 'checked' : ''}><span>${g}</span></label>`).join('')}<label class="clr"><button type="button" class="btn sm" style="border:none;padding:5px 6px" data-act="clear-grade" data-path="${esc(path)}" title="Очистить" aria-label="Очистить оценку">×</button></label></div>`;
}
function gradeTable(items, base, values, opts = {}) {
  return `<div class="table-wrap"><table><thead><tr><th>Компетенция</th>${opts.showK ? `<th class="num">${term('Кзнк')}</th>` : ''}<th class="c">Оценка (1–5)</th></tr></thead><tbody>
  ${items.map(it => { const v = values ? values[it.id] : undefined; const path = base + '|' + it.id;
    const title = it.title || it.id;
    return `<tr data-row="${esc(path)}" class="${Calc.isGrade(v) ? '' : 'missing'}"><td><b>${esc(title)}</b><div class="muted" style="font-size:13px">${esc(it.text)}</div></td>${opts.showK ? `<td class="num">${fmtK(it.k)}</td>` : ''}<td class="c">${gradeBar(path, v, title)}</td></tr>`; }).join('')}
  </tbody></table></div>`;
}
actions['clear-grade'] = el => {
  const a = findA(ui.editing); if (!a || a.status !== 'draft') return;
```

```
  setPath(a, el.dataset.path, undefined); $$(`input[name="gr-${CSS.escape(el.dataset.path)}"]`).forEach(r => r.checked = false);
  a.updatedAt = Store.nowIso(); persistSoon(); updateProgress(a);
};
function countGrades(items, values) { return items.filter(it => Calc.isGrade((values || {})[it.id])).length; }
function updateProgress(a) {
  $$('tr[data-row]').forEach(tr => { const v = getPath(a, tr.dataset.row); tr.classList.toggle('missing', !Calc.isGrade(v)); });
  const el = $('#prog'); if (!el) return;
  const [done, total] = progressOf(a);
  el.innerHTML = `Заполнено оценок, участвующих в расчёте: <b>${done} из ${total}</b><div class="progress"><i style="width:${total ? done / total * 100 : 0}%"></i></div>`;
}
function progressOf(a) {
  const s = a.snapshot;
  if (a.type === 'graduate') { const v = s.sets.vpk.items, p = a.input.pskEnabled ? s.sets.psk.items : []; return [countGrades(v, a.input.vpk) + countGrades(p, a.input.psk), v.length + p.length]; }
  if (a.type === 'practice') return [countGrades(s.sets.vpk.items, a.input.vpk), s.sets.vpk.items.length];
  return [0, 0];
}
function setOptions(base, selected) {
  const opts = [METHOD.COEF_SETS[base], ...state.coefSets.filter(c => c.baseSetId === base)];
  return opts.map(c => `<option value="${c.id}" ${c.id === selected ? 'selected' : ''}>${esc(c.name)} ${c.builtin ? ' (методика)' : ' (пользовательский)'}</option>`).join('');
}
const fieldText = (label, path, val, type = 'text', hint = '') => `<div class="field"><label class="f">${esc(label)}</label><input type="${type}" data-bind="${path}" value="${esc(val ?? '')}">${hint ? `<div class="hint">${hint}</div>` : ''}</div>`;
const fieldArea = (label, path, val) => `<div class="field"><label class="f">${esc(label)}</label><textarea data-bind="${path}" maxlength="2000">${esc(val ?? '')}</textarea></div>`;
const fieldSelect = (label, path, val, options, extra = '') => `<div class="field"><label class="f">${esc(label)}</label><select data-bind="${path}" ${extra}>${options.map(o => { const [v, t] = Array.isArray(o) ? o : [o, o]; return `<option value="${esc(v)}" ${String(v) === String(val) ? 'selected' : ''}>${esc(t)}</option>`; }).join('')}</select></div>`;

const STEPS = {
  graduate: [['Параметры', a => `<div class="grid g2">
      ${fieldSelect('Период', 'meta|period', a.meta.period, [['1', 'по окончании первого года службы'], ['2', 'по окончании двух лет службы']], 'data-type="int" data-rerender="1"')}
      ${fieldText('Дата отзыва', 'meta|date', a.meta.date, 'date')}
      ${fieldText('Оценщик (непосредственный начальник)', 'meta|evaluator', a.meta.evaluator)}
    </div>
    <h3>Сведения о выпускнике (шапка отзыва)</h3>
    <div class="grid g3">${METHOD.REVIEW_HEADER.map(([k, l]) => fieldText(l, 'meta|header|' + k, (a.meta.header || {})[k] ?? (k === 'position' ? a.meta.position : ''))).join('')}</div>
    <div class="grid g2">
      ${fieldSelect('Перевод / увольнение', 'meta|serviceChange', a.meta.serviceChange || '', METHOD.SERVICE_CHANGES)}
      ${fieldText('В какую в/ч убыл / по каким основаниям уволен', 'meta|serviceChangeNote', a.meta.serviceChangeNote)}
    </div>
    <details><summary class="btn sm" style="display:inline-flex">Семейное положение (необязательно)</summary><div style="margin-top:8px">${fieldText('Семейное положение', 'meta|header|family', (a.meta.header || {}).family, 'text', 'Поле есть в форме отзыва, но для расчёта не требуется. Заполняйте только при необходимости.')}</div></details>
    <h3>Подписи</h3>
    <div class="grid g3">${METHOD.REVIEW_SIGNERS.map(([k, l]) => fieldText(l + ' (звание, фамилия)', 'meta|signers|' + k, (a.meta.signers || {})[k])).join('')}</div>
    <div class="grid g2">
      ${fieldSelect('Набор Кзнк для ВПК', 'meta|vpkSetId', a.meta.vpkSetId, [], 'data-resnap="1" data-rerender="1"').replace('</select>', setOptions('src-vpk16', a.meta.vpkSetId) + '</select>')}
      ${fieldSelect('Набор Кзнк для ПСК', 'meta|pskSetId', a.meta.pskSetId, [], 'data-resnap="1" data-rerender="1"').replace('</select>', setOptions('src-psk5', a.meta.pskSetId) + '</select>')}
    </div>
    <div class="field"><label><input type="checkbox" data-bind="input|pskEnabled" ${a.input.pskEnabled ? 'checked' : ''} data-rerender="1"> Оценивать профессионально-специализированные компетенции (ПСК)</label>
    <div class="hint">ПСК в форме отзыва приведены для специальности 40.05.01. Без ПСК итоговая оценка сформированности компетенций (ВПК + ПСК) не рассчитывается.</div></div>
    <div class="note">Оценка компетенций выставляется командиром по 5-балльной схеме: ${METHOD.COMPETENCE_SCALE.map(esc).join('; ')}. <span class="src">В тексте методики упомянута «четырёхбалльная шкала», но форма отзыва и пороги bcp предполагают 5-балльную — применяется форма.</span></div>`],
```

```
  ['Раздел I: компетенции', a => `<div id="prog" class="note"></div>
      <h3>Уровень сформированности военно-профессиональных компетенций (ВПК)</h3>${gradeTable(a.snapsho
t.sets.vpk.items, 'input|vpk', a.input.vpk, { showK: true })}
      ${a.input.pskEnabled ? `<h3>Уровень сформированности профессионально-специализированных компетенци
й (ПСК)</h3>${gradeTable(a.snapshot.sets.psk.items, 'input|psk', a.input.psk, { showK: true })}` : '<div
class="note">ПСК не оцениваются (отключено в параметрах).</div>'}
      <p class="src">Кзнк заполняются автоматически из набора «${esc(a.snapshot.sets.vpk.name)}» (в мето
дике — сотрудниками учебно-методического отдела).</p>`],
  ['Разделы II–V, колонка выпускника', a => `
      <h3>II. Личностные качества <span class="muted" style="font-weight:400">(в расчёт Псфк не входят)
</span></h3>
      <div class="note">${METHOD.QUALITY_SCALE.map(esc).join('; ')}</div>
      ${gradeTable(METHOD.PERSONAL_QUALITIES.map(q => ({ id: q.id, title: q.name, text: q.text })), 'inp
ut|qualities', a.input.qualities)}
      <h3>III. Уровень личной подготовки молодого офицера</h3>
      <div class="grid g3">${METHOD.PERSONAL_TRAINING[a.meta.period === 2 ? 2 : 1].map(s => fieldSelect
(s + ' подготовка', 'input|training|' + s, (a.input.training || {})[s], [['', '—'], ...METHOD.MARKS4])).
join('')}</div>
      ${fieldArea('Оценка подразделения (влияние на неё деятельности офицера)', 'input|notes|unit', a.in
put.notes.unit)}
      <div class="grid g2">${fieldArea('Количество поощрений (кем, сколько раз)', 'input|notes|rewards',
a.input.notes.rewards)}${fieldArea('Дисциплинарные проступки, взыскания (в т.ч. снятые)', 'input|notes|p
enalties', a.input.notes.penalties)}</div>
      ${fieldArea('Характерные недостатки в подготовке офицера и их причины', 'input|notes|shortcoming
s', a.input.notes.shortcomings)}
      ${fieldArea('Рекомендации по улучшению подготовки выпускников', 'input|notes|recommendations', a.i
nput.notes.recommendations)}
      <h3>IV. Общая оценка по видам профессиональной деятельности за год</h3>
      <div class="grid g3">${METHOD.ACTIVITY_KINDS.map(s => fieldSelect(s, 'input|activity|' + s, (a.inp
ut.activity || {})[s], [['', '—'], ...METHOD.MARKS4])).join('')}</div>
      <h3>V. Выводы по отзыву</h3>
      ${fieldSelect('Вывод командира', 'input|conclusion', a.input.conclusion, [['', '— не выбран —'],
...METHOD.CONCLUSIONS])}
      <h3>Колонка выпускника <span class="muted" style="font-weight:400">(заполняется выпускником; в рас
чёт не входит)</span></h3>
      ${METHOD.GRADUATE_COLUMN.map((q, i) => q.options
        ? fieldSelect(`${i + 1}. ${q.text}`, 'input|graduateColumn|' + q.id, (a.input.graduateColumn ||
{})[q.id] || '', [['', '— нет ответа —'], ...q.options]) + (q.own ? fieldText('Свой вариант ответа', 'in
put|graduateColumn|' + q.id + 'own', (a.input.graduateColumn || {})[q.id + 'own']) : '')
        : fieldArea(`${i + 1}. ${q.text}`, 'input|graduateColumn|' + q.id, (a.input.graduateColumn ||
{})[q.id])).join('')}
      <p class="src">Разделы II–V заполняются по форме методики; методика не содержит правил их числовой
обработки, поэтому они отображаются в отчёте без агрегирования.</p>`],
  ['Проверка и расчёт', a => reviewStep(a)]],

  practice: [['Параметры', a => `<div class="grid g2">
      ${fieldSelect('Курс (практика)', 'meta|course', a.meta.course, [['4', '4 курс'], ['5', '5 курс']],
'data-type="int"')}
      ${fieldText('Дата заполнения анкеты', 'meta|date', a.meta.date, 'date')}
      ${fieldText('Оценщик (командир подразделения — непосредственный руководитель практики)', 'meta|eva
luator', a.meta.evaluator)}
      ${fieldText('Руководитель практики от военного института (совместная оценка)', 'meta|evaluator2',
a.meta.evaluator2)}
      ${fieldText('Воинская часть / подразделение', 'meta|unit', a.meta.unit)}
      ${fieldSelect('Набор Кзнк для ВПК', 'meta|vpkSetId', a.meta.vpkSetId, [], 'data-resnap="1" data-re
render="1"').replace('</select>', setOptions('src-vpk13', a.meta.vpkSetId) + '</select>')}
    </div><div class="note">Анкета экспертной оценки уровня достижения компетенций курсантом — стажёром
командира взвода. Оценка от 1 до 5 по каждому индексу компетенции. Анкету заполняет руководитель практик
и от института совместно с командиром подразделения — одна согласованная оценка по каждой компетенции.</
div>`],
  ['Анкета', a => `<div id="prog" class="note"></div>
      <h3>Военно-профессиональные компетенции (участвуют в расчёте)</h3>${gradeTable(a.snapshot.sets.vp
k.items, 'input|vpk', a.input.vpk, { showK: true })}
      <h3>Общекультурные и профессиональные компетенции</h3>
      <div class="note warn">Оценки ОК и ПК фиксируются в анкете, но коэффициенты значимости для них в м
етодике не определены и программа ЭВМ их не обрабатывает — в расчёт Псфк не входят.</div>
      ${gradeTable(a.snapshot.okpk, 'input|okpk', a.input.okpk)}`],
  ['Итоги практики', a => `<div class="grid g2">
      ${fieldSelect('Оценка руководителя практики (отзыв)', 'input|leaderMark', a.input.leaderMark,
[['', '—'], ['5', '5 (отлично)'], ['4', '4 (хорошо)'], ['3', '3 (удовлетворительно)'], ['2', '2 (неудовл
етворительно)']])}
      ${fieldSelect('Итоговая оценка комиссии за практику', 'input|commissionMark', a.input.commissionMa
rk, [['', '—'], ['5', '5 (отлично)'], ['4', '4 (хорошо)'], ['3', '3 (удовлетворительно)'], ['2', '2 (неу
довлетворительно)']])}
```

```
    </div>${fieldArea('Примечание', 'input|note', a.input.note)}
    <p class="src">Итоговую оценку по четырёхбалльной системе выставляет комиссия с учётом результатов расчёта программы ЭВМ. Приложение не выводит итоговую оценку автоматически — правило её получения в методике не формализовано.</p>`],
  ['Проверка и расчёт', a => reviewStep(a)]],

  g0: [['Параметры', a => `<div class="grid g2">${fieldText('Дата', 'meta|date', a.meta.date, 'date')}${fieldText('Кто выполнил расчёт', 'meta|evaluator', a.meta.evaluator)}</div>
      <div class="note">Формула (22), методика. Оценки предполагаются по 5-балльной шкале (так в методике); результаты ЕГЭ в 100-балльной шкале формулой не предусмотрены.</div>`],
  ['Оценки', a => `<p>Укажите количество оценок каждого вида.</p><div class="table-wrap"><table><thead><tr><th>Оценка</th><th class="c">Аттестат о среднем образовании (x)</th><th class="c">Вступительные испытания (y)</th></tr></thead><tbody>
      ${[5, 4, 3, 2, 1].map(g => `<tr><td><b>${g}</b></td><td class="c"><input type="number" min="0" step="1" class="cell" data-bind="input|x|${g}" data-type="num" value="${esc(a.input.x[g] ?? 0)}" aria-label="Количество оценок ${g} в аттестате"></td><td class="c"><input type="number" min="0" step="1" class="cell" data-bind="input|y|${g}" data-type="num" value="${esc(a.input.y[g] ?? 0)}" aria-label="Количество оценок ${g} на вступительных испытаниях"></td></tr>`).join('')}
      </tbody></table></div>`],
  ['Проверка и расчёт', a => reviewStep(a)]],

  trajectory: [['Параметры', a => `${state.curricula.length ? '' : '<div class="note warn">Учебные планы не созданы. <a href="#/curricula">Создайте учебный план</a> — дисциплины, часы T1, T2, коэффициент m и предшествующие дисциплины.</div>'}
      ${fieldSelect('Учебный план', 'input|curriculumId', a.input.curriculumId, [['', '— выберите —'], ...state.curricula.map(c => [c.id, c.name])], 'data-resnap="1" data-rerender="1"')}
      <div class="grid g2">
      ${fieldSelect('Формула', 'input|mode', a.input.mode, [['24', '(24): g = 1 − (1 − g₀)·exp(−p·(m·T1 + T2)/(1 − c)) — рабочая формула и контрольный пример'], ['21', '(21): g = 1 − (1 − g₀)·exp(−p·(m·T1/(1 − c1) + T2/(1 − c2))) — формула модели (рис. модели)']], 'data-rerender="1"')}
      ${fieldText('Параметр p, ч⁻¹', 'input|p', a.input.p, 'text', 'По умолчанию 0,0001 (методика, контрольный пример). Для формулы (21) в методике принято p = 0,005.')}
      ${fieldText('Дата', 'meta|date', a.meta.date, 'date')}${fieldText('Кто выполнил расчёт', 'meta|evaluator', a.meta.evaluator)}</div>`.replace('data-bind="input|p"', 'data-bind="input|p" data-type="num"')],
  ['g₀ и способности', a => trajData(a)],
  ['Проверка и расчёт', a => reviewStep(a)]],

  ymodel: [['Параметры', a => `${state.yModels.length ? '' : '<div class="note warn">Модели признаков не созданы. <a href="#/ymodels">Создайте модель</a>: перечень признаков, веса aᵢ и эталонные значения kᵢ.</div>'}
      ${fieldSelect('Модель признаков', 'input|modelId', a.input.modelId, [['', '— выберите —'], ...state.yModels.map(c => [c.id, c.name])], 'data-resnap="1" data-rerender="1"')}
      <div class="grid g2">${fieldText('Дата', 'meta|date', a.meta.date, 'date')}${fieldText('Кто выполнил расчёт', 'meta|evaluator', a.meta.evaluator)}</div>
      <div class="note warn">Формула Y приведена только на схеме модели; смысл переменных принят по тексту методики: «взвешенная сумма компетентностных признаков по сравнению с эталоном». Уровни для Y в методике не установлены.</div>`],
  ['Значения признаков', a => { const m = a.snapshot.yModel; if (!m) return '<div class="note err">Модель признаков не выбрана (шаг 1).</div>';
      return `<div class="table-wrap"><table><thead><tr><th>Признак</th><th class="num">Вес aᵢ</th><th class="num">Эталон kᵢ</th><th class="c">Значение k_jᵢ</th></tr></thead><tbody>${m.features.map(f => `<tr><td>${esc(f.name)}<div class="muted" style="font-size:13px">${esc(f.note || '')}</div></td><td class="num">${fmtFull(f.a)}</td><td class="num">${fmtFull(f.k)}</td><td class="c"><input type="text" class="cell" style="width:90px" data-bind="input|values|${f.id}" data-type="num" value="${esc((a.input.values || {})[f.id] ?? '')}" aria-label="${esc(f.name)}"></td></tr>`).join('')}</tbody></table></div>`; }],
  ['Проверка и расчёт', a => reviewStep(a)]]
};

function trajData(a) {
  const c = a.snapshot.curriculum; if (!c) return '<div class="note err">Учебный план не выбран (шаг 1).</div>';
  const g0s = state.assessments.filter(x => x.type === 'g0' && x.status === 'final' && x.objectId === a.objectId);
  const mode = a.input.mode;
  return `<h3>Начальный уровень g₀</h3>
    <div class="grid g2"><div class="field"><label class="f">Значение g₀ (0…1)</label><input type="text" data-bind="input|g0" data-type="num" value="${esc(a.input.g0 ?? '')}"><div class="hint">Для дисциплин без предшествующих (формула 22).</div></div>
    <div class="field"><label class="f">Взять из утверждённого расчёта g₀</label><select data-act-change="take-g0"><option value="">—</option>${g0s.map(x => `<option value="${x.id}">${fmtDate(x.meta.date)}: g₀ = ${fmt(x.result.g0, 4)}</option>`).join('')}</select></div></div>
    <h3>Способности обучающегося по дисциплинам (${mode === '21' ? 'c1 — при работе с преподавателем, c2 — при самостоятельной работе' : 'c'}; 0 ≤ c < 1)</h3>
```

```
    <div class="table-wrap"><table><thead><tr><th>№</th><th>Дисциплина</th><th class="num">T1</th><th class="num">T2</th><th class="num">m</th><th>Опирается на</th>${mode === '21' ? '<th class="c">c1</th><th class="c">c2</th>' : '<th class="c">c</th>'}</tr></thead><tbody>
    ${c.disciplines.map((d, i) => { const ab = (a.input.abilities || {})[d.id] || {}; const cell = k => `<td class="c"><input type="text" class="cell" data-bind="input|abilities|${d.id}|${k}" data-type="num" value="${esc(ab[k] ?? '')}" aria-label="${k} для ${esc(d.name)}"></td>`;
      return `<tr><td>${i + 1}</td><td>${esc(d.name)}</td><td class="num">${fmtFull(d.T1)}</td><td class="num">${fmtFull(d.T2)}</td><td class="num">${fmtFull(d.m)}</td><td>${(d.preds || []).map(p => c.disciplines.findIndex(x => x.id === p) + 1).join(', ') || '—'}</td>${mode === '21' ? cell('c1') + cell('c2') : cell('c')}</tr>`; }).join('')}
    </tbody></table></div>`;
}
document.addEventListener('change', e => {
  if (e.target.dataset && e.target.dataset.actChange === 'take-g0' && e.target.value) {
    const a = findA(ui.editing), src = findA(e.target.value); if (!a || !src) return;
    a.input.g0 = src.result.g0; a.input.g0From = src.id; persist(); render(); toast('g₀ подставлено из расчёта');
  }
});

function reviewStep(a) {
  const r = Model.compute(a);
  const problems = [];
  if (!r.ok) problems.push(...r.errors);
  if (r.ok && (a.type === 'graduate' || a.type === 'practice')) {
    const res = r.result;
    const miss = [...res.vpk.missing, ...res.vpk.invalid, ...(res.psk ? [...res.psk.missing, ...res.psk.invalid] : [])];
    if (miss.length) problems.push(`Не выставлены оценки (участвуют в расчёте): ${miss.join(', ')}`);
  }
  const optional = optionalGaps(a);
  return `${problems.length ? `<div class="note err"><b>Расчёт не может быть утверждён:</b><ul>${problems.map(p => `<li>${esc(p)}</li>`).join('')}</ul></div>` : '<div class="note ok"><b>Данные для расчёта заполнены полностью.</b> Ниже — предварительный результат. Он станет окончательным только после нажатия «Утвердить расчёт».</div>'}
    ${optional.length ? `<div class="note warn">Не заполнены сведения, не участвующие в расчёте: ${esc(optional.join('; '))}.</div>` : ''}
    ${r.ok && r.complete ? `<div style="border:2px dashed var(--line-2);border-radius:8px;padding:12px 16px;margin-top:12px"><div class="badge b-draft" style="margin-bottom:8px">ПРЕДВАРИТЕЛЬНЫЙ РЕЗУЛЬТАТ</div>${resultView(a, r.result)}</div>` : ''}`;
}
function optionalGaps(a) {
  const g = [];
  if (a.type === 'graduate') {
    const q = METHOD.PERSONAL_QUALITIES.filter(x => !Calc.isGrade((a.input.qualities || {})[x.id])).length; if (q) g.push(`личностные качества (${q})`);
    if (!a.input.conclusion) g.push('вывод по отзыву');
    if (!a.meta.evaluator) g.push('оценщик');
  }
  if (a.type === 'practice') {
    const q = a.snapshot.okpk.filter(x => !Calc.isGrade((a.input.okpk || {})[x.id])).length; if (q) g.push(`оценки ОК/ПК (${q})`);
    if (!a.input.leaderMark) g.push('оценка руководителя практики');
    if (!a.meta.evaluator) g.push('оценщик');
  }
  return g;
}

route(/^\/a\/([\w-]+)\/edit\/(\d)$/, (id, stepS) => {
  const a = findA(id);
  if (!a) return notFound('Запись');
  if (a.status !== 'draft') { location.hash = '#/a/' + id; return ''; }
  ui.editing = id;
  const refreshed = Model.refreshDraftSnapshot(state, a);
  if (refreshed) { a.updatedAt = Store.nowIso(); persist(); ui.snapNote = id; }
  const steps = STEPS[a.type]; const step = Math.min(Math.max(1, +stepS), steps.length);
  const isLast = step === steps.length;
  const reopened = a.revisions.length > 0;
  return {
    html: `<div class="row"><h1 style="margin:0">${esc(typeTitle(a.type))}</h1><span class="spacer"></span>${statusBadge(a)}</div>
    <p class="muted">${esc(objName(a.objectId))}${reopened ? ` · новая редакция (ранее утверждённых: ${a.revisions.length})` : ''} · черновик сохраняется автоматически</p>
```

```
    <ol class="steps" aria-label="Шаги">${steps.map((s, i) => `<li class="${i + 1 === step ? 'cur' : i + 1 < step ? 'done' : ''}"><a href="#/a/${id}/edit/${i + 1}" style="color:inherit;text-decoration:none" ${i + 1 === step ? 'aria-current="step"' : ''}>${i + 1}. ${esc(s[0])}</a></li>`).join('')}</ol>
    ${ui.snapNote === id ? '<div class="note warn"><b>Параметры методики обновлены.</b> После создания черновика изменились используемые справочники (набор Кзнк, учебный план или модель признаков) — черновик пересчитан по их текущему состоянию. Проверьте введённые данные.</div>' : ''}
    <div class="card">${steps[step - 1][1](a)}</div>
    <div class="row no-print">
      ${step > 1 ? `<a class="btn" href="#/a/${id}/edit/${step - 1}">← Назад</a>` : ''}
      ${!isLast ? `<a class="btn primary" href="#/a/${id}/edit/${step + 1}">Далее →</a>` : `<button class="btn primary" data-act="finalize" data-id="${id}">Утвердить расчёт</button>`}
      <span class="spacer"></span>
      <a class="btn" href="#/history">Сохранить черновик и выйти</a>
      ${reopened ? `<button class="btn" data-act="cancel-edit" data-id="${id}">Отменить редактирование</button>` : `<button class="btn danger" data-act="del-a" data-id="${id}">Удалить черновик</button>`}
    </div>`,
    after: () => updateProgress(a)
  };
});

actions.finalize = async el => {
  const a = findA(el.dataset.id); if (!a) return;
  persist();
  const r = Model.compute(a);
  if (!r.ok || !r.complete) { toast('Расчёт не может быть утверждён: данные неполны или содержат ошибки', true); render(); return; }
  const gaps = optionalGaps(a);
  const body = `<p>После утверждения результат фиксируется вместе со снимком методики (версия ${METHOD.VERSION}). Изменить его можно только созданием новой редакции.</p>${gaps.length ? `<div class="note warn">Не заполнено (в расчёт не входит): ${esc(gaps.join('; '))}.</div>` : ''}`;
  if (!await confirmDlg('Утвердить расчёт?', body, 'Утвердить')) return;
  a.result = r.result; a.status = 'final'; a.finalizedAt = Store.nowIso(); a.updatedAt = a.finalizedAt;
  ui.editing = null; persist(); toast('Расчёт утверждён'); location.hash = '#/a/' + a.id;
};
actions['del-a'] = async el => {
  const a = findA(el.dataset.id); if (!a) return;
  if (!await confirmDlg(a.status === 'draft' ? 'Удалить черновик?' : 'Удалить расчёт?', 'Запись и история её редакций будут удалены без возможности восстановления.', 'Удалить', true)) return;
  state.assessments = state.assessments.filter(x => x.id !== a.id); ui.editing = null; persist(); toast('Удалено'); location.hash = '#/history';
};
actions['cancel-edit'] = async el => {
  const a = findA(el.dataset.id); if (!a || !a.revisions.length) return;
  if (!await confirmDlg('Отменить редактирование?', 'Изменения новой редакции будут отброшены; восстановится последний утверждённый расчёт.', 'Отменить изменения', true)) return;
  const last = a.revisions.pop();
  Object.assign(a, { meta: last.meta, input: last.input, snapshot: last.snapshot, result: last.result, finalizedAt: last.finalizedAt, status: 'final', updatedAt: Store.nowIso() });
  ui.editing = null; persist(); location.hash = '#/a/' + a.id;
};
actions['reopen'] = async el => {
  const a = findA(el.dataset.id); if (!a) return;
  if (!await confirmDlg('Создать новую редакцию?', 'Текущий утверждённый результат будет сохранён в истории редакций, а запись перейдёт в режим черновика. Снимок методики будет обновлён по текущим справочникам.', 'Создать редакцию')) return;
  a.revisions.push({ finalizedAt: a.finalizedAt, meta: Model.clone(a.meta), input: Model.clone(a.input), snapshot: a.snapshot, result: a.result });
  a.status = 'draft'; a.result = null; a.snapshot = Model.snapshot(state, a.type, a.meta, a.input); a.updatedAt = Store.nowIso();
  persist(); location.hash = `#/a/${a.id}/edit/1`;
};
actions['verify'] = el => {
  const a = findA(el.dataset.id); if (!a || !a.result) return;
  const r = Model.compute(a);
  const same = r.ok && JSON.stringify(r.result) === JSON.stringify(a.result);
  if (same) toast('Повторный расчёт по сохранённым данным и снимку методики дал идентичный результат');
  else toast('ВНИМАНИЕ: повторный расчёт отличается от сохранённого результата', true);
};
actions['export-one'] = el => {
  const a = findA(el.dataset.id); if (!a) return;
  const o = state.objects.find(x => x.id === a.objectId);
  download(`расчёт_${(o ? o.name : 'объект').replace(/[^\wА-Яа-яЁё.-]+/g, '_')}_${a.meta.date || ''}.json`, JSON.stringify({ app: 'ПФК-ВПК калькулятор', methodVersion: METHOD.VERSION, exportedAt: Store.nowIso(), object: o, assessment: a }, null, 2), 'application/json');
```

```
};

/* ---------- Представление результатов ---------- */
function kpi(label, value, sub) { const tip = glossaryFor(label); return `<div class="kpi"><div class="l"${tip ? ` title="${esc(tip)}" style="cursor:help"` : ''}>${esc(label)}${tip ? ' ⓘ' : ''}</div><div class="v">${value}</div><div class="s">${sub || ''}</div></div>`; }
function setTable(set, res, withMinMax) {
  return `<div class="table-wrap"><table><thead><tr><th>№</th><th>Показатель сформированности</th><th class="num">Оценка</th><th class="num">${term('Кзнк')}</th><th class="num">${term('Псфк')}</th><th class="num">${term('Степень сформированности', 'Степень, %')}</th>${withMinMax ? '<th class="num">Мин. (2)</th><th class="num">Макс. (5)</th>' : ''}</tr></thead><tbody>
  ${res.rows.map((r, i) => { const it = set.items[i]; return `<tr><td>${i + 1}</td><td><b>${esc(r.id)}</b> <span class="muted" style="font-size:13px">${esc(it.text)}</span></td><td class="num">${r.grade ?? '—'}</td><td class="num">${fmtK(r.k)}</td><td class="num">${fmt(r.psfk, 5)}</td><td class="num">${fmt(r.pct, 0)}</td>${withMinMax ? `<td class="num">${fmt(r.min, 3)}</td><td class="num">${fmt(r.max, 3)}</td>` : ''}</tr>`; }).join('')}
  </tbody><tfoot><tr><td></td><td>Суммарный показатель (среднее арифметическое, n = ${res.n})</td><td class="num">${fmt(res.meanGrade, 2)}</td><td class="num">${fmtK(res.sumK)}</td><td class="num">${fmt(res.meanPsfk, 3)}</td><td class="num">${fmt(res.meanPct, 2)}</td>${withMinMax ? `<td class="num">${fmt(2 * res.sumK / res.n, 3)}</td><td class="num">${fmt(res.maxMeanPsfk, 3)}</td>` : ''}</tr></tfoot></table></div>`;
}
function scaleNote(scale, lv) {
  if (!scale || !lv) return '';
  const L = levelInfo(lv.code);
  return `<div class="src">Уровень по шкале «${esc(scale.title)}»: значение ${scale.decimals != null ? 'округлено до ' + scale.decimals + ' знаков = ' + fmt(lv.compared, scale.decimals) : '= ' + fmtFull(lv.compared)}; интервал «${esc(L.name)}»: ${esc(scale.text[METHOD.LEVELS.indexOf(L)])}. ${esc(scale.src)}${scale.note ? ' — ' + esc(scale.note) : ''}</div>`;
}
function calcDetails(set, res, scaleKey, snap) {
  const sc = scaleKey ? snap.scales[scaleKey] : null;
  return `<details class="calc"><summary>Как получено (${esc(set.name)})</summary>
    <ol>
      <li>Для каждой компетенции: <span class="formula">Псфк<sub>i</sub> = Оценка<sub>i</sub> × Кзнк<sub>i</sub></span>. Пример: ${esc(res.rows[0].id)}: ${res.rows[0].grade} × ${fmtK(res.rows[0].k)} = ${fmtFull(res.rows[0].psfk)}.</li>
      <li>Σ Псфк = ${fmtFull(res.sumPsfk)}; n = ${res.n}; суммарный показатель = Σ Псфк / n = <b>${fmtFull(res.meanPsfk)}</b> (среднее арифметическое, методика).</li>
      <li>Средний балл bcp = Σ Оценок / n = ${res.sumGrade} / ${res.n} = <b>${fmtFull(res.meanGrade)}</b>; степень сформированности = bcp / 5 × 100 = ${fmt(res.meanPct, 2)}%.</li>
      <li>Максимально возможный суммарный показатель (все оценки 5) = 5 × ΣКзнк / n = ${fmtFull(res.maxMeanPsfk)}; отношение = ${fmt(res.relPsfk, 2)}%.</li>
      <li>Уровень по bcp: ${lvBadge(res.levelBsr)} (${esc(snap.scales.bsr.src)}).</li>
      <li>Уровень по Псфк: ${sc ? lvBadge(res.levelPsfk) + ' (' + esc(sc.src) + ')' : 'шкала для данного набора Кзнк в методике не установлена — не определяется'}.</li>
    </ol>
    <p class="src">Набор Кзнк: ${esc(set.name)} — ${esc(set.src || '')}. Промежуточные значения не округляются.</p></details>`;
}

function resultView(a, r) {
  const s = a.snapshot;
  if (a.type === 'graduate') {
    const v = r.vpk, p = r.psk, t = r.total;
    return `<div class="grid g3">
      ${kpi('ВПК: суммарный Псфк', fmt(v.meanPsfk, 3), `${lvBadge(v.levelPsfk)}<br>bcp = ${fmt(v.meanGrade, 2)} ${lvBadge(v.levelBsr, false)}`)}
      ${p ? kpi('ПСК: суммарный Псфк', fmt(p.meanPsfk, 3), `${lvBadge(p.levelPsfk)}<br>bcp = ${fmt(p.meanGrade, 2)} ${lvBadge(p.levelBsr, false)}`) : kpi('ПСК', '—', 'не оценивались')}
      ${t ? kpi('Итоговая оценка (ВПК + ПСК)', fmt(t.meanPsfk, 3), `${lvBadge(t.levelPsfk)}<br>bcp = ${fmt(t.meanGrade, 2)} ${lvBadge(t.levelBsr, false)}`) : kpi('Итоговая оценка', '—', 'рассчитывается только при оценке ВПК и ПСК')}
    </div>
    ${r.warnings.map(w => `<div class="note warn">${esc(w)}</div>`).join('')}
    ${scaleNote(s.scales[s.sets.vpk.scale], v.levelPsfk)}
    ${headerBlock(a)}
    <h3>Раздел I. Военно-профессиональные компетенции</h3>${setTable(s.sets.vpk, v, true)}${calcDetails(s.sets.vpk, v, s.sets.vpk.scale, s)}
    ${competenceChart(v.rows, 'Псфк ВПК')}
    ${p ? `<h3>Профессионально-специализированные компетенции</h3>${setTable(s.sets.psk, p, true)}${calcDetails(s.sets.psk, p, s.sets.psk.scale, s)}` : ''}
    ${t ? `<details class="calc"><summary>Как получена итоговая оценка</summary><p><span class="formula">Итог = (Σ Псфк ВПК + Σ Псфк ПСК) / (n<sub>ВПК</sub> + n<sub>ПСК</sub>)</span> = (${fmtFull(v.sumPsf
```

```
k)} + ${fmtFull(p.sumPsfk)}) / ${t.n} = <b>${fmtFull(t.meanPsfk)}</b>.</p><p class="src">${esc(s.scales.psfkTotal21.note)}</p>${scaleNote(s.scales.psfkTotal21, t.levelPsfk)}</details>` : ''}
    ${graduateOther(a)}`;
  }
  if (a.type === 'practice') {
    const v = r.vpk;
    return `<div class="grid g4">
      ${kpi('Средняя оценка (bcp)', fmt(v.meanGrade, 2), lvBadge(v.levelBsr))}
      ${kpi('Степень сформированности', fmt(v.meanPct, 2) + '%', 'bcp / 5 × 100')}
      ${kpi('Уровень компетентности (Псфк)', fmt(v.meanPsfk, 3), `как в программе ЭВМ: «${fmt(v.meanPsfk, 3)} (${fmt(v.meanPct, 0)}%)»`)}
      ${kpi('Уровень по Псфк', '—', 'пороги для 13 ВПК в методике не установлены')}
    </div>
    ${scaleNote(s.scales.bsr, v.levelBsr)}
    ${(() => { const pv = typeof passports === 'function' ? Calc.passportCheck(v.rows, passports(), a.meta.course, s.scales.bsr) : []; return pv.length ? `<div class="note warn"><b>Паспорт компетенции:</b> ${pv.map(x => `${esc(x.id)} — уровень «${levelInfo(x.level).name}» (оценка ${x.grade}) не предусмотрен для ${a.meta.course} курса`).join('; ')}.</div>` : ''; })()}
    <p class="no-print"><a href="#/object/${a.objectId}/profile">Диагностический профиль курсанта →</a></p>
    <h3>Военно-профессиональные компетенции</h3>${setTable(s.sets.vpk, v, true)}${calcDetails(s.sets.vpk, v, null, s)}
    <h3>График уровня сформированности ВПК</h3>${competenceChart(v.rows)}
    <h3>Оценки ОК и ПК (в расчёт не входят)</h3>
    <div class="table-wrap"><table><thead><tr>${r.okpk.map(x => `<th class="c">${esc(x.id)}</th>`).join('')}</tr></thead><tbody><tr>${r.okpk.map(x => `<td class="c">${x.grade ?? '—'}</td>`).join('')}</tr></tbody></table></div>
    <div class="grid g2" style="margin-top:12px">${kpi('Оценка руководителя практики', esc(a.input.leaderMark || '—'), [a.meta.evaluator ? 'командир подразделения: ' + esc(a.meta.evaluator) : '', a.meta.evaluator2 ? 'руководитель от института: ' + esc(a.meta.evaluator2) : ''].filter(Boolean).join('<br>'))}${kpi('Итоговая оценка комиссии', esc(a.input.commissionMark || '—'), '')}</div>
    ${a.input.note ? `<p><b>Примечание:</b> ${esc(a.input.note)}</p>` : ''}
    ${(() => { const d = typeof discFor === 'function' ? discFor({ ...a, result: r }) : null; return d ? `<div class="note ${d.flag ? 'warn' : ''}"><b>Расхождение (правило пользователя, не из методики):</b> уровень по bcp соответствует оценке ${d.derived}, выставлено ${d.mark} — ${d.flag ? (d.diff > 0 ? 'оценка завышена' : 'оценка занижена') + ' на ' + Math.abs(d.diff) : 'расхождения нет'}.</div>` : ''; })()}`;
  }
  if (a.type === 'g0') {
    return `<div class="grid g3">${kpi('g₀', fmt(r.g0, 4), 'относительный уровень подготовленности на начальной стадии')}${kpi('L1 (аттестат)', r.L1, 'Σx = ' + r.sumX)}${kpi('L2 (вступит. испытания)', r.L2, 'Σy = ' + r.sumY)}</div>
    <p><span class="formula">g₀ = (Σx + Σy) / (5·(L1 + L2)) = (${r.sumX} + ${r.sumY}) / (5·(${r.L1} + ${r.L2})) = ${fmtFull(r.g0)}</span></p><p class="src">Формула (22), методика.</p>`;
  }
  if (a.type === 'trajectory') {
    const c = s.curriculum;
    return `<div class="grid g3">${kpi('g₀ (начальный)', fmt(a.input.g0, 4), '')}${kpi('g после последней дисциплины', fmt(r.final, 4), esc(r.rows[r.rows.length - 1].name))}${kpi('Формула', a.input.mode === '21' ? '(21)' : '(24)', 'p = ' + fmtFull(Number(a.input.p)) + ' ч⁻¹')}</div>
    <div class="table-wrap"><table><thead><tr><th>№</th><th>Дисциплина</th><th class="num">T1</th><th class="num">T2</th><th class="num">m</th><th class="num">${a.input.mode === '21' ? 'c1 / c2' : 'c'}</th><th>g₀ дисциплины</th><th class="num">Показатель степени</th><th class="num">g</th></tr></thead><tbody>
    ${r.rows.map((x, i) => { const d = c.disciplines[i], ab = (a.input.abilities || {})[d.id] || {}; return `<tr><td>${i + 1}</td><td>${esc(x.name)}</td><td class="num">${fmtFull(d.T1)}</td><td class="num">${fmtFull(d.T2)}</td><td class="num">${fmtFull(d.m)}</td><td class="num">${a.input.mode === '21' ? fmtFull(ab.c1) + ' / ' + fmtFull(ab.c2) : fmtFull(ab.c)}</td><td>${fmt(x.start, 4)} <span class="src">${x.startRule === '22' ? '(g₀)' : '(g_ot)'}</span></td><td class="num">${fmt(x.exponent, 5)}</td><td class="num"><b>${fmt(x.g, 4)}</b></td></tr>`; }).join('')}
    </tbody></table></div>
    ${lineChart({ labels: r.rows.map(x => x.name), series: [{ name: 'Уровень подготовленности g', color: '#007AFF', values: r.rows.map(x => x.g) }], yDigits: 3 })}
    <p class="src">Алгоритм формулы (22)–(24) / (21), методика. Промежуточные значения g не округляются (в контрольном примере методики округлены до 3 знаков — расхождение до 0,0006).</p>`;
  }
  if (a.type === 'ymodel') {
    const m = s.yModel;
    return `<div class="grid g3">${kpi('Y', fmt(r.Y, 4), 'отношение к эталону')}${kpi('Σ aᵢ·k_jᵢ', fmtFull(r.num), '')}${kpi('Σ aᵢ·kᵢ (эталон)', fmtFull(r.den), '')}</div>
    <div class="table-wrap"><table><thead><tr><th>Признак</th><th class="num">aᵢ</th><th class="num">kᵢ</th><th class="num">k_jᵢ</th><th class="num">aᵢ·k_jᵢ</th><th class="num">aᵢ·kᵢ</th></tr></thead><tbody>
    ${r.rows.map((x, i) => `<tr><td>${esc(m.features[i].name)}</td><td class="num">${fmtFull(x.a)}</td><td class="num">${fmtFull(x.k)}</td><td class="num">${fmtFull(x.v)}</td><td class="num">${fmtFull(x.av)}</td><td class="num">${fmtFull(x.ak)}</td></tr>`).join('')}
```

```
    </tbody></table></div><p class="src">Модель признаков: «${esc(m.name)}». Формула со схемы модели; тр
актовка переменных — по методике. Уровни для Y методикой не установлены.</p>`;
  }
  return '';
}
function graduateOther(a) {
  const i = a.input;
  const q = METHOD.PERSONAL_QUALITIES.map(x => `<tr><td>${esc(x.name)}</td><td class="num">${(i.qualitie
s || {})[x.id] ?? '—'}</td></tr>`).join('');
  const tr = METHOD.PERSONAL_TRAINING[a.meta.period === 2 ? 2 : 1].map(s => `<td class="c">${esc((i.trai
ning || {})[s] || '—')}</td>`).join('');
  return `<h3>II. Личностные качества</h3><div class="table-wrap"><table><tbody>${q}</tbody></table></di
v>
    <h3>III. Уровень личной подготовки</h3><div class="table-wrap"><table><thead><tr>${METHOD.PERSONAL_T
RAINING[a.meta.period === 2 ? 2 : 1].map(s => `<th class="c">${esc(s)}</th>`).join('')}</tr></thead><tbo
dy><tr>${tr}</tr></tbody></table></div>
    ${[['unit', 'Оценка подразделения'], ['rewards', 'Поощрения'], ['penalties', 'Взыскания'], ['shortco
mings', 'Недостатки и причины'], ['recommendations', 'Рекомендации']].filter(([k]) => (i.notes || {})
[k]).map(([k, l]) => `<p><b>${l}:</b> ${esc(i.notes[k])}</p>`).join('')}
    <h3>IV. Оценка по видам деятельности</h3><div class="table-wrap"><table><thead><tr>${METHOD.ACTIVITY
_KINDS.map(s => `<th class="c">${esc(s)}</th>`).join('')}</tr></thead><tbody><tr>${METHOD.ACTIVITY_KIND
S.map(s => `<td class="c">${esc((i.activity || {})[s] || '—')}</td>`).join('')}</tr></tbody></table></di
v>
    <h3>V. Вывод по отзыву</h3><p>${esc(i.conclusion || '—')}</p>
    ${signersBlock(a)}
    <h3>Колонка выпускника</h3>${METHOD.GRADUATE_COLUMN.map((q, n) => { const v = (i.graduateColumn ||
{})[q.id], own = (i.graduateColumn || {})[q.id + 'own']; return `<p><b>${n + 1}. ${esc(q.text)}</b><br>
${esc(v || '—')}${own ? ' — ' + esc(own) : ''}</p>`; }).join('')}`;
}
function headerBlock(a) {
  if (a.type !== 'graduate') return '';
  const h = a.meta.header || {};
  const rows = METHOD.REVIEW_HEADER.map(([k, l]) => [l, h[k] ?? (k === 'position' ? a.meta.position :
'')]).filter(r => r[1]);
  if (h.family) rows.push(['Семейное положение', h.family]);
  if (a.meta.serviceChange) rows.push([a.meta.serviceChange === 'transfer' ? 'Перевод' : 'Увольнение',
a.meta.serviceChangeNote || '—']);
  return rows.length ? `<h3>Сведения о выпускнике</h3><dl class="spec">${rows.map(r => `<dt>${esc(r[0])}
</dt><dd>${esc(r[1])}</dd>`).join('')}</dl>` : '';
}
function signersBlock(a) {
  const sg = a.meta.signers || {};
  const rows = METHOD.REVIEW_SIGNERS.filter(([k]) => sg[k]);
  return rows.length ? `<h3>Подписи</h3><dl class="spec">${rows.map(([k, l]) => `<dt>${esc(l)}</dt><dd>
${esc(sg[k])}</dd>`).join('')}</dl>` : '';
}

route(/^\/a\/([\w-]+)$/, id => {
  const a = findA(id); if (!a) return notFound('Запись');
  if (a.status === 'draft') return `<div class="card"><h1>${esc(typeTitle(a.type))}</h1><p>${statusBadge
(a)} ${esc(objName(a.objectId))}</p><div class="note warn">Расчёт не утверждён. Черновик не является рез
ультатом оценки.</div><a class="btn primary" href="#/a/${id}/edit/1">Продолжить заполнение</a></div>`;
  const metaLine = [a.type === 'graduate' ? `${a.meta.period} год службы` : '', a.type === 'practice' ?
`${a.meta.course} курс` : '', a.meta.evaluator ? 'оценщик: ' + a.meta.evaluator : '', a.meta.unit || '',
(a.meta.header || {}).position || a.meta.position || '', a.meta.evaluator2 ? 'руководитель от института:
' + a.meta.evaluator2 : ''].filter(Boolean).join(' · ');
  return `<div class="row"><h1 style="margin:0">${esc(typeTitle(a.type))}</h1><span class="spacer"></spa
n>${statusBadge(a)}</div>
  <p class="muted"><a href="#/object/${a.objectId}">${esc(objName(a.objectId))}</a> · дата: ${fmtDate(a.
meta.date)}${metaLine ? ' · ' + esc(metaLine) : ''}<br>Утверждён: ${fmtDT(a.finalizedAt)} · методика v
${esc(a.snapshot.methodVersion)} (снимок от ${fmtDT(a.snapshot.takenAt)})</p>
  <div class="row no-print" style="margin-bottom:16px"><a class="btn primary" href="#/a/${id}/report" ti
tle="Отчёт с формулами и основаниями — для печати или PDF">Отчёт для печати</a><button class="btn" data-
act="reopen" data-id="${id}" title="Создаётся новая редакция; прежний результат сохранится в истории">Ис
править оценки</button><button class="btn" data-act="verify" data-id="${id}" title="Пересчитать по сохра
нённому снимку методики и сравнить с утверждённым результатом">Пересчитать и сверить</button><button cla
ss="btn" data-act="export-one" data-id="${id}" title="Скачать запись в формате JSON">Скачать JSON</butto
n><span class="spacer"></span><button class="btn danger" data-act="del-a" data-id="${id}">Удалить</butto
n></div>
  <div class="card">${resultView(a, a.result)}</div>
  ${a.revisions.length ? `<div class="card"><h2>Предыдущие редакции (${a.revisions.length})</h2><div cla
ss="table-wrap"><table><thead><tr><th>Утверждена</th><th>Методика</th><th class="num">Показатель</th></t
r></thead><tbody>${a.revisions.map(rv => { const h = Model.headline({ ...a, result: rv.result }); return
```

```
    `<tr><td>${fmtDT(rv.finalizedAt)}</td><td>v${esc(rv.snapshot.methodVersion)}</td><td class="num">${h ? esc(h.label) + ' ' + fmt(h.value, 3) : '—'}</td></tr>`; }).join('')}</tbody></table></div></div>` : ''}`;
});

/* ---------- Отчёт ---------- */
const FORMULA_REFS = {
  graduate: [['Псфк = Оценка × Кзнк', 'методика'], ['Суммарный показатель ВПК (ПСК) — среднее арифметическое Псфк', 'методика'], ['Итоговая оценка = Σ всех Псфк / (n_ВПК + n_ПСК)', 'принятое правило (формула в методике не приведена) (согласуется с таблицей «Итоговая оценка»)'], ['Уровни по Псфк: таблицы ВПК, ПСК, итоговая', 'методика'], ['Уровни по bcp: 4,5 / 3,75 / 2,7', 'методика'], ['Кзнк = λ = Σs / ΣΣs, s = x_max − x', 'методика (для ВПК)']],
  practice: [['Псфк = Оценка × Кзнк', 'методика'], ['Степень сформированности, % = Оценка / 5 × 100', 'методика'], ['Мин./макс. Псфк = 2·Кзнк / 5·Кзнк', 'методика'], ['Уровень компетентности = среднее Псфк; ср. оценка = среднее оценок', 'методика'], ['Уровни по bcp', 'методика']],
  g0: [['g₀ = (Σx + Σy) / (5·(L1 + L2))', 'методика']],
  trajectory: [['g = 1 − (1 − g₀)·exp(−p·(m·T1 + T2)/(1 − c))', 'методика'], ['g = 1 − (1 − g₀)·exp(−p·(m·T1/(1 − c1) + T2/(1 − c2)))', 'методика'], ['g_ot = (1/m)·Σ g_ij', 'методика'], ['Алгоритм', 'методика']],
  ymodel: [['Y = Σ aᵢ·k_jᵢ / Σ aᵢ·kᵢ', 'методика (трактовка переменных принята по тексту методики)']]
};
route(/^\/a\/([\w-]+)\/report$/, id => {
  const a = findA(id); if (!a || a.status !== 'final') return '<div class="card empty">Отчёт доступен только для утверждённого расчёта.</div>';
  const o = state.objects.find(x => x.id === a.objectId) || {};
  const usedSrc = { graduate: ['S1', 'S4'], practice: ['S2', 'S4'], g0: ['S5'], trajectory: ['S5', 'S6'], ymodel: ['S6'] }[a.type] || [];
  return `<div class="row no-print" style="margin-bottom:12px"><button class="btn primary" data-act="print">Печать / сохранить в PDF</button></div>
  <div class="report">
  <div class="card"><h1>Отчёт: ${esc(typeTitle(a.type))}</h1>
    <dl class="spec"><dt>Объект оценки</dt><dd>${esc(o.name || '—')}</dd><dt>Группа / подразделение</dt><dd>${esc(o.group || '—')}</dd><dt>Специальность</dt><dd>${esc(o.specialty || '—')}</dd>
    ${a.type === 'graduate' ? `<dt>Период</dt><dd>по окончании ${a.meta.period === 2 ? 'двух лет' : 'первого года'} службы</dd>` : ''}${a.type === 'practice' ? `<dt>Практика</dt><dd>${a.meta.course} курс${a.meta.unit ? ', ' + esc(a.meta.unit) : ''}</dd>` : ''}
    <dt>Дата</dt><dd>${fmtDate(a.meta.date)}</dd><dt>Оценщик / исполнитель</dt><dd>${esc(a.meta.evaluator || '—')}</dd>
    <dt>Утверждён</dt><dd>${fmtDT(a.finalizedAt)}</dd><dt>Версия методики</dt><dd>${esc(a.snapshot.methodVersion)} (снимок параметров сохранён в записи)</dd><dt>Идентификатор записи</dt><dd class="mono">${esc(a.id)}</dd></dl></div>
  <div class="card"><h2>Результаты</h2>${resultView(a, a.result).replace(/<details class="calc">/g, '<details class="calc" open>')}</div>
  <div class="card"><h2>Применённые формулы и правила</h2><div class="table-wrap"><table><thead><tr><th>Формула / правило</th><th>Основание</th></tr></thead><tbody>${FORMULA_REFS[a.type].map(f => `<tr><td>${esc(f[0])}</td><td>${esc(f[1])}</td></tr>`).join('')}</tbody></table></div>
    ${a.type === 'graduate' || a.type === 'practice' ? `<h3>Шкала уровней по среднему баллу</h3><table><tbody>${METHOD.LEVELS.map((L, i) => `<tr><td>«${L.name}» (${L.mark})</td><td>${esc(a.snapshot.scales.bsr.text[i])}</td></tr>`).join('')}</tbody></table>` : ''}</div>
  <div class="card"><h2>Основания расчёта</h2><ul>${usedSrc.map(k => `<li>${esc(METHOD.SOURCES[k].full)}</li>`).join('')}</ul>
  <p class="muted">Отчёт сформирован приложением «ПФК ВПК» ${fmtDT(new Date().toISOString())}.</p></div></div>`;
});
actions.print = () => window.print();

/* ============================================================================
   МОДУЛЬ 7. СПРАВОЧНИКИ МЕТОДИКИ И НАСТРАИВАЕМЫЕ ПАРАМЕТРЫ
   ============================================================================ */
const FORMULAS = [
  { id: 'F1', name: 'Показатель сформированности компетенции', expr: 'Псфк<sub>i</sub> = Оценка<sub>i</sub> × Кзнк<sub>i</sub>',
    vars: 'Оценка — индивидуальная оценка компетенции командиром (руководителем практики), целое 1…5; Кзнк — коэффициент значимости компетенции (> 0).',
    cond: 'Отзыв на выпускника (ВПК, ПСК), анкета практики (ВПК).', order: 'Вычисляется для каждой компетенции набора.', round: 'Без округления; в таблицах — 5 знаков (программа ЭВМ) или 3 знака.', missing: 'При пропуске оценки показатель не вычисляется, запись не может быть утверждена.', src: 'методика', example: 'ВПК-1: 4 × 0,06355 = 0,2542.' },
  { id: 'F2', name: 'Суммарный показатель сформированности ВПК (ПСК)', expr: 'Псфк<sub>сум</sub> = (1/n)·Σ Псфк<sub>i</sub>',
    vars: 'n — число компетенций набора (16 ВПК, 5 ПСК, 13 ВПК практики).', cond: 'Все оценки набора выставлены.', order: 'После F1.', round: 'Для сравнения с границами уровней — до 3 знаков (границы заданы с 3 знаками).', missing: 'Не вычисляется при неполном наборе.', src: 'методика', example: 'Контрольный пример 1: Σ = 3,26556; /13 = 0,251.' },
```

```
  { id: 'F3', name: 'Итоговая оценка сформированности компетенций', expr: 'Псфк<sub>итог</sub> = (Σ Псфк<sub>ВПК</sub> + Σ Псфк<sub>ПСК</sub>) / (n<sub>ВПК</sub> + n<sub>ПСК</sub>)',
    vars: 'Суммы по F1 для ВПК и ПСК.', cond: 'Оценены и ВПК, и ПСК.', order: 'После F1.', round: 'Сравнение — до 3 знаков.', missing: 'Без ПСК не вычисляется.', src: 'методика (таблица границ); формула не приведена — принята по принятому правилу, согласуется с границами 0,236 и 0,170', example: 'Все оценки 5: (5,00005 + 1,60685)/21 = 0,3146 ≤ 0,315.' },
  { id: 'F4', name: 'Средний балл и степень сформированности', expr: 'b<sub>ср</sub> = (1/n)·Σ Оценка<sub>i</sub>;  Степень,% = b<sub>ср</sub> / 5 × 100',
    vars: 'Оценки 1…5.', cond: 'Все оценки набора выставлены.', order: 'Параллельно F2.', round: 'В таблицах — 2 знака.', missing: 'Не вычисляется при неполном наборе.', src: 'методика', example: 'Контрольный пример 2: 60/13 = 4,62; 92,31%.' },
  { id: 'F5', name: 'Минимальное и максимальное значения показателя', expr: 'Псфк<sub>min,i</sub> = 2·Кзнк<sub>i</sub>;  Псфк<sub>max,i</sub> = 5·Кзнк<sub>i</sub>',
    vars: 'Кзнк.', cond: 'Графики «уровень сформированности ВПК».', order: '—', round: '3 знака в таблицах.', missing: '—', src: 'методика (график)', example: 'ВПК-1: 0,127 и 0,318.' },
  { id: 'F6', name: 'Определение уровня сформированности', expr: 'уровень = «высокий», если x ≥ a; «средний», если x ≥ b; «низкий», если x ≥ c; иначе «критический»',
    vars: 'x — bcp или суммарный Псфк; a, b, c — границы шкалы (см. «Шкалы»).', cond: 'Шкалы Псфк применимы только к наборам Кзнк методики, для которых они установлены.', order: 'После F2–F4.', round: 'Псфк — до 3 знаков; bcp — без округления.', missing: '—', src: 'методика', example: 'bcp = 4,62 → «высокий» (отлично).' },
  { id: 'F7', name: 'Начальный уровень подготовленности', expr: 'g<sub>0</sub> = (Σ x<sub>l</sub> + Σ y<sub>l</sub>) / (5·(L<sub>1</sub> + L<sub>2</sub>))',
    vars: 'x — оценки аттестата (L1 шт.), y — оценки вступительных экзаменов (L2 шт.), 5-балльная шкала.', cond: 'Дисциплины без предшествующих.', order: '—', round: 'Без округления.', missing: 'Должна быть хотя бы одна оценка.', src: 'методика', example: '(3·11+4·6+5·1)+(4·1+5·2) / 5·21 = 76/105 = 0,7238 (в методике напечатано 0,713).' },
  { id: 'F8', name: 'Уровень после изучения дисциплины с предшествующими', expr: 'g<sub>ot</sub> = (1/m)·Σ g<sub>i<sub>j</sub></sub>',
    vars: 'g предшествующих (базовых) дисциплин, m — их число.', cond: 'Дисциплина имеет предшествующие.', order: 'Дисциплины упорядочены: каждая опирается на дисциплины с меньшими номерами.', round: 'Без округления.', missing: '—', src: 'методика', example: 'D6: (0,727 + 0,728)/2 = 0,7275.' },
  { id: 'F9', name: 'Уровень подготовленности по дисциплине', expr: 'g = 1 − (1 − g<sub>0</sub>)·e<sup>−p·(m·T<sub>1</sub> + T<sub>2</sub>)/(1 − c)</sup> []; g = 1 − (1 − g<sub>0</sub>)·e<sup>−p·(m·T<sub>1</sub>/(1 − c<sub>1</sub>) + T<sub>2</sub>/(1 − c<sub>2</sub>))</sup> []',
    vars: 'T1 — объём дисциплины (ч), T2 — самостоятельная работа (ч), m — относительное методическое мастерство преподавателя (0 ≤ m < 1 по методике), c, c1, c2 — способности обучающегося (0 ≤ c < 1), p — интенсивность (ч⁻¹).', cond: 'По умолчанию — p = 10⁻⁴ (рабочая формула алгоритма и контрольного примера) — альтернативный режим.', order: 'g0 по F7 или F8, затем F9.', round: 'Без округления (в контрольном примере промежуточные g округлены до 3 знаков).', missing: 'Все параметры обязательны.', src: 'методика', example: 'g1 = 1 − 0,287·e^(−0,0001·(0,8·100+80)/0,5) = 0,722.' },
  { id: 'F10', name: 'Уровень компетентности выпускника', expr: 'Y<sub>j</sub><sup>k</sup> = Σ a<sub>i</sub><sub>k<sub>ji</sub> / Σ a<sub>i</sub><sub>k<sub>i</sub></sub>',
    vars: 'a — веса признаков, k_ji — значения признаков выпускника j, k_i — эталонные значения (трактовка по тексту методики).', cond: 'Признаки, веса и эталон задаются пользователем — в методике не определены.', order: '—', round: 'Без округления.', missing: 'Все значения обязательны; Σaᵢkᵢ ≠ 0.', src: 'методика', example: '—' },
  { id: 'F11', name: 'Коэффициент конкордации (связанные ранги)', expr: 'W = S / ( (1/12)·m²(n³ − n) − m·ΣT<sub>i</sub> ),  T<sub>i</sub> = (1/12)·Σ(t<sub>l</sub>³ − t<sub>l</sub>)',
    vars: 'S — сумма квадратов отклонений сумм рангов от среднего m(n+1)/2; n — число факторов; m — число экспертов; t — размеры связок.', cond: 'Экспертная оценка значимости.', order: 'После переформирования рангов.', round: '—', missing: 'Матрица должна быть заполнена.', src: 'методика, этапы 3–5', example: 'W = 12175 / (136000 − 74310) = 0,197 ≈ 0,2.' },
  { id: 'F12', name: 'Значимость коэффициента конкордации (критерий Пирсона)', expr: 'χ² = S / ( (1/12)·m·n(n + 1) − ΣT<sub>i</sub>/(n − 1) );  значим, если χ² ≥ χ²<sub>табл</sub>(n − 1; α)',
    vars: 'α = 0,05.', cond: '—', order: 'После F11.', round: '—', missing: '—', src: 'методика. В формуле методики напечатан знак «+», но численный результат 59,21 получен со знаком «−» (со знаком «+» было бы 17,37); реализовано по расчёту.', example: 'χ² = 12175 / (453,33 − 247,7) = 59,21 ≥ 24,996.' },
  { id: 'F13', name: 'Показатель весомости (коэффициент значимости)', expr: 's<sub>ij</sub> = x<sub>max</sub> − x<sub>ij</sub>;  λ<sub>i</sub> = Σ<sub>j</sub> s<sub>ij</sub> / ΣΣ s<sub>ij</sub>',
    vars: 'x_ij — исходный ранг фактора i у эксперта j; x_max = 6.', cond: 'Использовать результаты, только если W значим (F12).', order: 'Этап 7.', round: 'Кзнк в методике — 5 знаков.', missing: 'Не допускается (в примере методики пустая ячейка обработана как 0).', src: 'методика', example: 'ВПК-4: 100/1495 = 0,06689.' },
  { id: 'F14', name: 'Место в группе', expr: 'Место<sub>k</sub> = 1 + |{ j : Псфк<sub>j</sub> > Псфк<sub>k</sub> }|',
    vars: 'Суммарный Псфк курсантов группы.', cond: 'Сводная таблица по группе.', order: '—', round: 'Сравниваются неокруглённые значения.', missing: 'Курсанты без утверждённого расчёта не ранжируются.', src: 'методика (столбец «Место»)', example: 'Шесть курсантов с 0,313 — место 1.' }
];

route(/^\/method(?:\/(\w+))?$/, tab => {
  tab = tab || 'model';
```

```
  const tabs = [['model', 'Модель'], ['comp', 'Компетенции'], ['scales', 'Шкалы и уровни'], ['formulas', 'Формулы'], ['scheme', 'Схема контроля'], ['terms', 'Термины'], ['sources', 'Основания и уточнения'], ['ref', 'Справочные данные']];
  let body = '';
  if (tab === 'model') body = `
    <div class="card"><h2>Система полифункционального контроля</h2>
    <p>Целостная совокупность взаимосвязанных элементов (целевых, методологических, технологических, оценочных), функционирование которых направлено на диагностику, оценку и коррекцию уровня сформированности ВПК. Шесть блоков: <b>целевой</b>, <b>методологический</b> (подходы: акмеологический, личностно-ориентированный, контекстный, системный, компетентностно-деятельностный, квалиметрический), <b>содержательно-структурный</b> (5 компонентов ВПК), <b>функциональных направлений</b> (диагностическое, статистическое, содержательное, сопровождающее), <b>аналитико-коррекционный</b>, <b>результативный</b> (четыре уровня).</p></div>
    <div class="card"><h2>Модель полифункционального контроля</h2>
    <dl class="spec">
      <dt>Входной блок</dt><dd>Абитуриент (школа, колледж, военная служба, участие в боевых действиях), этапы подготовки 1–5 курсов по семестрам, выпускник (1–2 год службы).</dd>
      <dt>Блок полифункционального контроля</dt><dd><b>Мониторинг</b>: контрольные работы, тактические летучки, курсовые работы, групповые упражнения, тактико-строевые занятия, КШВИ, оценки за занятия по ПДП. <b>Диагностика</b>: ЕГЭ, профессиональный отбор, входная диагностика, тестирование, зачёты, экзамены, отзывы за войсковую практику, отчёты НИР, дипломная работа, ГИА, отзывы за 1 и 2 год службы.</dd>
      <dt>Математический блок</dt><dd>g₀ — уровень на начальной стадии; g_ot — относительный уровень после каждой следующей дисциплины; g — уровень по отдельной дисциплине (/24); Y — уровень компетентности.</dd>
      <dt>Блок ИКТ</dt><dd>Модули ПО №1–6, в т.ч. «Программа для расчёта уровня компетентности специалиста по результатам практики» и «Программа для расчёта уровня сформированности ВПК у выпускников … по результатам анализа отзывов».</dd>
      <dt>Результативный блок</dt><dd>Эмпирические оценочные показатели; квалиметрические числовые значения и графики; информационно-коммуникационные графики и диаграммы.</dd>
      <dt>Выход</dt><dd>Уровень сформированности ВПК: «высокий» (отлично), «средний» (хорошо), «низкий» (удовлетворительно), «критический» (неудовлетворительно); коррекция программы подготовки.</dd></dl>
    <p class="muted">Соответствие разделам приложения: «Отзыв на выпускника» и «Практика» — модули №4 и №6; g₀ — №1; траектория g — №2, №3; Y — №5.</p></div>
    <div class="card"><h2>Комплекс функций, виды, методы и формы контроля</h2>
    <dl class="spec"><dt>Регулятивно-оценочные функции</dt><dd>диагностическая, аналитико-информационная, управленческая</dd><dt>Развивающе-формирующие функции</dt><dd>обучающая, мотивационная, воспитательная</dd>
    <dt>Виды контроля</dt><dd>диагностический, текущий, периодический (тематический), промежуточный (рубежный), итоговый</dd><dt>Типы контроля</dt><dd>внешний, взаимный, самоконтроль</dd>
    <dt>Традиционные методы</dt><dd>устный, письменный, программируемый, комбинируемый, практический</dd><dt>Комплексные военно-профессиональные</dt><dd>в составе воинской части, подразделений, экипажа (расчёта)</dd>
    <dt>Формы контроля ВПК</dt><dd>курсовая работа, экзамен, зачёт, тест, портфолио, тактическая летучка, групповое упражнение, нормативы по боевой и специальной подготовке, вождение боевых машин, самостоятельная работа, войсковая стажировка, контрольная работа, устный опрос, боевая стрельба, КШВИ, ТСЗ, КШТ, КШУ</dd></dl>
    <p class="src">Эти элементы описывают организацию контроля; правил их числового агрегирования методика не содержит, поэтому в расчёты они не входят.</p></div>`;
  if (tab === 'scheme') body = schemeTab();
  if (tab === 'terms') body = `<div class="card"><h2>Термины и обозначения</h2><dl class="spec">${Object.entries(METHOD.GLOSSARY).map(([k, v]) => `<dt><b>${esc(k)}</b></dt><dd>${esc(v)}</dd>`).join('')}</dl>
    <p class="muted">Подчёркнутые пунктиром термины в таблицах и значок ⓘ у показателей показывают эти пояснения при наведении.</p></div>
    <div class="card"><h2>Уровни сформированности</h2>${METHOD.LEVELS.map(L => `<p>${lvBadge({ code: L.code })} — ${esc(L.descr)}.</p>`).join('')}</div>`;
  if (tab === 'comp') body = Object.values(METHOD.COEF_SETS).map(set => { const groups = [...new Set(set.items.map(c => c.group || ''))];
    return `<div class="card"><h2>${esc(set.name)}</h2><p class="src">${esc(set.src)}</p>
    ${groups.map(g => { const its = set.items.filter(c => (c.group || '') === g); return `<details open style="margin-bottom:8px"><summary style="cursor:pointer;font-weight:600;padding:6px 0">${esc(g ? g[0].toUpperCase() + g.slice(1) : 'Компетенции')} <span class="muted" style="font-weight:400">(${its.length}; Σ${term('Кзнк')} = ${fmtK(its.reduce((s, c) => s + c.k, 0))})</span></summary>
      <div class="table-wrap"><table><thead><tr><th>Индекс</th><th>Формулировка</th><th class="num">${term('Кзнк')}</th></tr></thead><tbody>
      ${its.map(c => `<tr><td><b>${esc(c.id)}</b></td><td>${esc(c.text)}</td><td class="num">${fmtK(c.k)}</td></tr>`).join('')}</tbody></table></div></details>`; }).join('')}
    <p><b>Сумма Кзнк набора: ${fmtK(set.items.reduce((s, c) => s + c.k, 0))}</b>. Иерархия: набор → вид профессиональной деятельности → компетенция. Промежуточные суммы по видам деятельности приведены справочно — правил расчёта показателей по видам деятельности методика не содержит.</p>
    <p class="muted">Шкала оценки: ${METHOD.COMPETENCE_SCALE.map(esc).join('; ')}. Правило оценивания: оценку выставляет командир (руководитель практики); ${term('Псфк')} = Оценка × ${term('Кзнк')}.</p></div>`; }).join('') +
    `<div class="card"><h2>ОК и ПК анкеты войсковой практики</h2><p class="src">методика. Кзнк не установлены — в расчёт не входят.</p><div class="table-wrap"><table><tbody>${METHOD.PRACTICE_OKPK.map(c => `<t
```

```
r><td><b>${esc(c.id)}</b></td><td>${esc(c.text)}</td></tr>`).join('')}</tbody></table></div></div>
    <div class="card"><h2>Компоненты ВПК</h2><div class="table-wrap"><table><thead><tr><th>Компонент</th><th>Показатели</th><th>Индикаторы</th><th>Методы контроля</th></tr></thead><tbody>${METHOD.COMPONENTS.map(r => `<tr>${r.map(x => `<td>${esc(x)}</td>`).join('')}</tr>`).join('')}</tbody></table></div>
    <p class="src">Матрица компонентов задаёт содержание контроля; числовые веса компонентов и правила свёртки в методике не приведены — в расчётах не используется.</p></div>`;
  if (tab === 'scales') body = Object.values(METHOD.SCALES).map(sc => `<div class="card"><h2>${esc(sc.title)}</h2>
    <div class="table-wrap"><table><thead><tr><th>Уровень</th><th>Интервал (как в методике)</th><th>Правило в приложении</th></tr></thead><tbody>
    ${METHOD.LEVELS.map((L, i) => `<tr><td>${lvBadge({ code: L.code })}</td><td>${esc(sc.text[i])}</td><td>${i < 3 ? `x ≥ ${fmtFull(sc.mins[i])}` : `x < ${fmtFull(sc.mins[2])}`}${sc.decimals != null ? `, x округляется до ${sc.decimals} знаков` : ''}</td></tr>`).join('')}</tbody></table></div>
    <p class="src">${esc(sc.src)}</p>${sc.note ? `<div class="note warn">${esc(sc.note)}</div>` : ''}</div>`).join('') +
    `<div class="card"><h2>Качественные характеристики уровней </h2><table><tbody>${METHOD.LEVELS.map(L => `<tr><td>${lvBadge({ code: L.code })}</td><td>${esc(L.descr)}</td></tr>`).join('')}</tbody></table>
    <div class="note">На стыке перекрывающихся интервалов (в методике знак «≤» с обеих сторон) присваивается верхний уровень.</div>
    <h3>Достижимые уровни по курсам (методика, пример паспорта ВПК-11)</h3><table><thead><tr><th>Уровень</th><th class="c">1</th><th class="c">2</th><th class="c">3</th><th class="c">4</th><th class="c">5</th></tr></thead><tbody>
    <tr><td>высокий</td><td></td><td></td><td></td><td></td><td class="c">+</td><td class="c">+</td></tr><tr><td>средний</td><td></td><td class="c">+</td><td class="c">+</td><td class="c">+</td><td class="c">+</td></tr><tr><td>низкий</td><td class="c">+</td><td class="c">+</td><td class="c">+</td><td class="c">+</td><td class="c">+</td><td class="c">+</td></tr><tr><td>критический</td><td class="c">+</td><td></td><td></td><td></td><td></td><td></td><td></td></tr></tbody></table></div>`;
  if (tab === 'formulas') body = FORMULAS.map(f => `<div class="card" id="${f.id}"><h2>${f.id}. ${esc(f.name)}</h2><div class="formula">${f.expr}</div>
    <dl class="spec"><dt>Переменные и допустимые значения</dt><dd>${esc(f.vars)}</dd><dt>Условия применения</dt><dd>${esc(f.cond)}</dd><dt>Порядок вычислений</dt><dd>${esc(f.order)}</dd><dt>Округление</dt><dd>${esc(f.round)}</dd><dt>Пропущенные данные</dt><dd>${esc(f.missing)}</dd><dt>Основание</dt><dd>${esc(f.src)}</dd><dt>Контрольный пример</dt><dd>${esc(f.example)}</dd></dl></div>`).join('');
  if (tab === 'sources') body = `<div class="card"><h2>Основания расчёта</h2><table><tbody>${Object.entries(METHOD.SOURCES).map(([k, s]) => `<tr><td><b>${esc(s.short)}</b></td><td>${esc(s.full)}</td></tr>`).join('')}</tbody></table></div>
    <div class="card"><h2>Выявленные расхождения и принятые решения</h2><div class="table-wrap"><table><thead><tr><th>№</th><th>Расхождение</th><th>Решение в приложении</th></tr></thead><tbody>${DISCREPANCIES.map((d, i) => `<tr><td>${i + 1}</td><td>${esc(d[0])}</td><td>${esc(d[1])}</td></tr>`).join('')}</tbody></table></div></div>`;
  if (tab === 'ref') body = `<div class="card"><h2>Сквозная матрица формирования ВПК</h2><div class="table-wrap"><table><thead><tr><th>Этап подготовки</th><th>Диагностируемые компетенции</th><th>Формы контроля</th><th class="num">Весовой коэффициент</th></tr></thead><tbody>${METHOD.STAGE_WEIGHTS.map(r => `<tr><td>${esc(r[0])}</td><td>${esc(r[1])}</td><td>${esc(r[2])}</td><td class="num">${fmtFull(r[3])}</td></tr>`).join('')}</tbody></table></div>
    <div class="note warn">Весовые коэффициенты этапов приведены справочно: правило их применения (свёртки результатов по этапам) в методике не описано, поэтому в расчётах они не используются.</div></div>
    <div class="card"><h2>Практики</h2><table><thead><tr><th>Вид практики</th><th class="num">з.е.</th><th class="num">Часов</th><th>Семестр</th></tr></thead><tbody>
    ${[['Практика по получению первичных профессиональных умений и навыков', 3, 108, '4'], ['Учебная практика в несении службы', 10, 360, 'с 1 по 8'], ['Научно-исследовательская работа', 1, 36, '8'], ['Войсковая практика', 5, 180, '7'], ['Практика по получению профессиональных умений и опыта профессиональной деятельности', 4, 144, '10'], ['Преддипломная практика', 5, 180, '9']].map(r => `<tr><td>${esc(r[0])}</td><td class="num">${r[1]}</td><td class="num">${r[2]}</td><td>${esc(r[3])}</td></tr>`).join('')}</tbody></table></div>`;
  return `<h1>Справочник методики</h1><p class="muted">Версия правил расчёта: ${METHOD.VERSION}. Справочник содержит модель, перечни компетенций, шкалы уровней, формулы, схему контроля, термины и основания расчёта.</p>
  <nav class="tabs no-print">${tabs.map(t => `<a href="#/method/${t[0]}" class="${t[0] === tab ? 'active' : ''}">${t[1]}</a>`).join('')}</nav>${body}`;
});

const DISCREPANCIES = [
  ['в таблице Псфк для ВПК граница «высокий» = 0,272, тогда как остальные границы равны bcp × ΣКзнк / n (0,169; 0,235), и по этому правилу граница должна быть 0,281.', 'Применяется таблица дословно (принятое правило); при расхождении с уровнем по bcp выводится предупреждение.'],
  ['интервалы уровней Псфк перекрываются (знак «≤» на обеих границах).', 'На стыке присваивается верхний уровень.'],
  ['формула итоговой оценки сформированности компетенций (ВПК + ПСК) не приведена.', 'Итог = среднее все x 21 Псфк (принятое правило); согласуется с границами 0,236 и 0,170 таблицы.'],
  ['в тексте — оценка «по четырёхбалльной шкале», в форме отзыва — 5-балльная схема (1–5).', 'Используется 5-балльная схема формы отзыва (с ней согласованы пороги bcp и Псфк).'],
  ['максимальный Псфк ПСК = 0,32137, верхняя граница таблицы — 0,321.', 'Сравнение выполняется со значением, округлённым до 3 знаков (0,321).'],
```

```
  ['Кзнк ВПК-14 в форме 1-го года напечатан как «0,000,06355».', 'Принято 0,06355 (по форме 2-го года и расчёту по методике: 95/1495).'],
  ['формулировки ВПК-8 и ВПК-10 в форме отзыва отличаются от перечня квалификационных требований в разделе I.', 'В форме оценки используются формулировки формы отзыва; в анкете практики — формулировки анкеты.'],
  ['анкета практики содержит ОК и ПК, но Кзнк для них не определены; программа ЭВМ обрабатывает только 13 ВПК.', 'ОК и ПК фиксируются, но в расчёт не входят.'],
  ['пороги уровней Псфк для набора 13 ВПК не установлены.', 'Уровень для практики определяется по bcp; Псфк выводится без уровня (как в программе ЭВМ: «0,251 (80%)»).'],
  ['у курсанта №20 при оценке 5,00 указано 92,31%.', 'Опечатка в методике; приложение вычисляет % = bcp/5 × 100.'],
  ['в формуле χ² напечатан знак «+» перед ΣT/(n−1), но результат 59,21 получен со знаком «−».', 'Реализовано по расчёту (со знаком «−»), что соответствует стандартной формуле для связанных рангов.'],
  ['в сводной матрице рангов пустая ячейка (фактор 16, эксперт 19) обработана как 0.', 'В приложении пустые ячейки не допускаются; контрольный тест воспроизводит пример с 0.'],
  ['пример g₀ для K1 — (3·11+4·6+5·1)+(4·1+5·2) / 5·21 напечатан как 0,713, арифметически равен 0,7238.', 'Формула реализована как есть; в контрольном примере g по дисциплинам использовано значение 0,713 из методики.'],
  ['формула (21) и схема модели — c1, c2 отдельно; рабочая формула (24) алгоритма и контрольный пример — единый c и (m·T1 + T2)/(1 − c).', 'По умолчанию — (24); (21) доступна как режим (принятое правило).'],
  ['p = 0,005 ч⁻¹ и p = 10⁻⁴ ч⁻¹ (контрольный пример); графики визуально соответствуют p ≈ 0,002.', 'p задаётся пользователем; по умолчанию 10⁻⁴ для (24) и 0,005 для (21).'],
  ['значения для K2 и K4 не воспроизводятся из g₀ (0,892 и 0,836) и способностей K1 воспроизводится полностью, K3 — с отклонением ≤ 0,001.', 'Контрольный тест построен по K1.'],
  ['m в пределах 0 ≤ m < 1, однако по смыслу m = M/M_max может быть равен 1.', 'Допускается 0 ≤ m ≤ 1.'],
  ['формула Y = Σaᵢk_jᵢ / Σaᵢkᵢ без расшифровки переменных; перечень признаков, веса и эталон не заданы.', 'Реализована с настраиваемой моделью признаков; трактовка помечена в интерфейсе.'],
  ['весовые коэффициенты этапов 0,2 / 0,3 / 0,4 / 0,1 без правила применения.', 'Приведены справочно, в расчётах не используются.']
];

/* ---------- Наборы Кзнк ---------- */
route(/^\/coef$/, () => {
  const builtins = Object.values(METHOD.COEF_SETS);
  return `<h1>Коэффициенты Кзнк</h1>
  <p class="muted">Наборы методики защищены от изменения. Пользовательские наборы создаются копированием или по результатам экспертной оценки. Для пользовательских наборов пороги Псфк не установлены — уровень определяется по bcp.</p>
  <div class="card"><h2>Наборы методики</h2><table><thead><tr><th>Набор</th><th class="num">n</th><th class="num">ΣКзнк</th><th></th></tr></thead><tbody>
  ${builtins.map(s => `<tr><td><a href="#/coef/${s.id}">${esc(s.name)}</a><div class="src">${esc(s.src)}</div></td><td class="num">${s.items.length}</td><td class="num">${fmtK(s.items.reduce((a, c) => a + c.k, 0))}</td><td class="no-print"><button class="btn sm" data-act="coef-copy" data-id="${s.id}">Создать копию</button> <button class="btn sm" data-act="expert-new" data-id="${s.id}">Новая экспертиза</button></td></tr>`).join('')}</tbody></table></div>
  <div class="card"><h2>Пользовательские наборы (${state.coefSets.length})</h2>${state.coefSets.length ? `<table><thead><tr><th>Набор</th><th>На основе</th><th>Создан</th></tr></thead><tbody>${state.coefSets.map(s => `<tr class="click" data-act="go" data-href="#/coef/${s.id}"><td>${esc(s.name)}</td><td>${esc(METHOD.COEF_SETS[s.baseSetId].name)}</td><td>${fmtDT(s.createdAt)}</td></tr>`).join('')}</tbody></table>` : '<p class="muted">Нет.</p>'}</div>
  <div class="card"><h2>Экспертные оценки значимости (${state.expertStudies.length})</h2>${state.expertStudies.length ? `<table><thead><tr><th>Название</th><th>Факторы</th><th class="num">Экспертов</th><th>Изменена</th></tr></thead><tbody>${state.expertStudies.map(s => `<tr class="click" data-act="go" data-href="#/expert/${s.id}"><td>${esc(s.name)}</td><td>${esc(METHOD.COEF_SETS[s.baseSetId].name)}</td><td class="num">${s.m}</td><td>${fmtDT(s.updatedAt)}</td></tr>`).join('')}</tbody></table>` : '<p class="muted">Нет. Нажмите «Новая экспертиза» у набора методики, чтобы ввести матрицу рангов.</p>'}</div>`;
});
actions['coef-copy'] = el => {
  const b = METHOD.COEF_SETS[el.dataset.id];
  const s = { id: Store.uid(), builtin: false, baseSetId: b.id, name: b.name + ' — копия', items: Model.clone(b.items), scale: null, src: 'Пользовательский набор (копия набора методики, значения могут быть изменены пользователем).', createdAt: Store.nowIso() };
  state.coefSets.push(s); persist(); location.hash = '#/coef/' + s.id;
};
route(/^\/coef\/([\w-]+)$/, id => {
  const s = Model.coefSet(state, id); if (!s) return notFound('Набор');
  const ed = !s.builtin;
  const used = state.assessments.filter(a => a.status === 'draft' && (a.meta.vpkSetId === id || a.meta.pskSetId === id)).length;
  return `<h1>${esc(s.name)}</h1><p class="src">${esc(s.src)}</p>
  <div class="card">${ed ? `<div class="field"><label class="f" for="cs-name">Название</label><input type="text" id="cs-name" value="${esc(s.name)}" maxlength="120"></div>` : ''}
```

```
  <div class="table-wrap"><table><thead><tr><th>Индекс</th><th>Формулировка</th><th class="num">Кзнк</th></tr></thead><tbody>
  ${s.items.map((c, i) => `<tr><td><b>${esc(c.id)}</b></td><td>${esc(c.text)}</td><td class="num">${ed ? `<input type="text" class="cell" style="width:100px" id="cs-k-${i}" value="${esc(String(c.k).replace('.', ','))}" aria-label="Кзнк ${esc(c.id)}">` : fmtK(c.k)}</td></tr>`).join('')}</tbody>
  <tfoot><tr><td></td><td>Сумма</td><td class="num">${fmtK(s.items.reduce((a, c) => a + c.k, 0))}</td></tr></tfoot></table></div>
  ${ed ? `<div class="note warn">Изменение набора не влияет на утверждённые расчёты (в них сохранён снимок). Черновиков, использующих набор: ${used} — при следующем открытии они будут пересчитаны по новым значениям.</div>
  <div class="row"><button class="btn primary" data-act="coef-save" data-id="${id}">Сохранить</button><span class="spacer"></span><button class="btn danger" data-act="coef-del" data-id="${id}">Удалить набор</button></div>` : '<p class="muted">Набор методики не редактируется.</p>'}</div>`;
});
actions['coef-save'] = el => {
  const s = state.coefSets.find(x => x.id === el.dataset.id); if (!s) return;
  const vals = s.items.map((c, i) => parseNum($('#cs-k-' + i).value));
  const bad = vals.findIndex(v => !isNumV(v) || v <= 0);
  if (bad >= 0) { $('#cs-k-' + bad).classList.add('invalid'); toast(`Кзнк для ${s.items[bad].id} должен быть положительным числом`, true); return; }
  const name = $('#cs-name').value.trim(); if (!name) { toast('Укажите название', true); return; }
  s.name = name; s.items.forEach((c, i) => c.k = vals[i]); persist(); toast('Набор сохранён'); render();
};
actions['coef-del'] = async el => {
  const id = el.dataset.id;
  const used = state.assessments.filter(a => a.status === 'draft' && (a.meta.vpkSetId === id || a.meta.pskSetId === id)).length;
  if (used) { toast(`Набор используется в черновиках (${used}). Смените набор в них или удалите черновики.`, true); return; }
  if (!await confirmDlg('Удалить набор?', 'Утверждённые расчёты сохранят свой снимок коэффициентов.', 'Удалить', true)) return;
  state.coefSets = state.coefSets.filter(x => x.id !== id); persist(); location.hash = '#/coef';
};

/* ---------- Экспертная оценка значимости ---------- */
actions['expert-new'] = el => {
  const b = METHOD.COEF_SETS[el.dataset.id];
  const st = { id: Store.uid(), name: 'Экспертиза: ' + b.name, baseSetId: b.id, m: 20, xmax: METHOD.G_MODEL.xmaxExpert, matrix: b.items.map(() => Array(20).fill('')), createdAt: Store.nowIso(), updatedAt: Store.nowIso() };
  state.expertStudies.push(st); persist(); location.hash = '#/expert/' + st.id;
};
route(/^\/expert\/([\w-]+)$/, id => {
  const st = state.expertStudies.find(x => x.id === id); if (!st) return notFound('Экспертиза');
  const base = METHOD.COEF_SETS[st.baseSetId];
  let res = null, err = null;
  const filled = st.matrix.every(r => r.every(v => v !== '' && v != null));
  if (filled) { try { res = Calc.expert(st.matrix.map(r => r.map(Number)), Number(st.xmax)); } catch (e) { err = e.message; } }
  return `<h1>${esc(st.name)}</h1><p class="muted">Метод экспертных оценок, методика: ранжирование факторов (компетенций) экспертами; ранг 1 — наивысшая значимость, равнозначным факторам — одинаковый ранг.</p>
  <div class="card"><div class="grid g3"><div class="field"><label class="f" for="ex-name">Название</label><input type="text" id="ex-name" value="${esc(st.name)}"></div>
  <div class="field"><label class="f" for="ex-m">Число экспертов m</label><input type="number" id="ex-m" min="1" max="200" value="${st.m}"></div>
  <div class="field"><label class="f" for="ex-x">x<sub>max</sub> (s = x<sub>max</sub> − x)</label><input type="number" id="ex-x" min="2" max="100" value="${esc(st.xmax)}"><div class="hint">В методике x_max = 6 при ранжировании 1…5.</div></div></div>
  <div class="row"><button class="btn" data-act="ex-apply" data-id="${id}">Применить параметры</button><span class="spacer"></span><button class="btn danger" data-act="ex-del" data-id="${id}">Удалить экспертизу</button></div></div>
  <div class="card"><h2>Матрица рангов (факторы × эксперты)</h2>
  <div class="table-wrap"><table id="exMatrix"><thead><tr><th>Фактор</th>${Array.from({ length: st.m }, (_, j) => `<th class="c">${j + 1}</th>`).join('')}</tr></thead><tbody>
  ${base.items.map((c, i) => `<tr><td><b>${esc(c.id)}</b></td>${st.matrix[i].map((v, j) => `<td class="c" style="padding:3px"><input type="text" inputmode="numeric" class="cell" style="width:40px" data-ex="${i},${j}" value="${esc(v)}" aria-label="${esc(c.id)}, эксперт ${j + 1}"></td>`).join('')}</tr>`).join('')}
  </tbody></table></div><div class="row" style="margin-top:10px"><button class="btn primary" data-act="ex-calc" data-id="${id}">Сохранить матрицу и рассчитать</button><span class="muted">Можно вставить столбцы из таблицы (Ctrl+V в первую ячейку).</span></div></div>
  ${err ? `<div class="note err">${esc(err)}</div>` : ''}${!filled ? '<div class="note warn">Матрица заполнена не полностью — расчёт не выполняется.</div>' : ''}
```

```
  ${res ? expertResult(st, base, res) : ''}`;
});
function expertResult(st, base, r) {
  return `<div class="card"><h2>Результаты</h2>
  <div class="grid g4">${kpi('W (конкордация)', fmt(r.W, 3), r.W < 0.3 ? 'слабая согласованность' : r.W < 0.7 ? 'умеренная согласованность' : 'высокая согласованность')}${kpi('χ² расчётный', fmt(r.chi2, 2), `df = ${r.df}`)}${kpi('χ² табличный (α = 0,05)', fmt(r.chi2crit, 3), '')}${kpi('Вывод', r.significant ? 'значим' : 'не значим', r.significant ? 'результаты могут использоваться' : 'результаты не имеют смысла')}</div>
  <p class="src">Градации согласованности по W — ориентировочные (в методике W = 0,2 названо «слабой степенью согласованности»).</p>
  <div class="table-wrap"><table><thead><tr><th>Место</th><th>Фактор</th><th class="num">Сумма рангов</th><th class="num">d</th><th class="num">d²</th><th class="num">Σs</th><th class="num">λ (Кзнк)</th></tr></thead><tbody>
  ${r.order.map((i, p) => `<tr><td>${p + 1}</td><td>${esc(base.items[i].id)}</td><td class="num">${fmtFull(r.sums[i])}</td><td class="num">${fmtFull(r.d[i])}</td><td class="num">${fmtFull(r.d[i] * r.d[i])}</td><td class="num">${r.sRow[i]}</td><td class="num"><b>${fmtK(r.lambda[i])}</b></td></tr>`).join('')}</tbody>
  <tfoot><tr><td></td><td>Итого</td><td class="num">${fmtFull(r.sums.reduce((a, b) => a + b, 0))}</td><td></td><td class="num">S = ${fmtFull(r.S)}</td><td class="num">${r.sTot}</td><td class="num">1</td></tr></tfoot></table></div>
  <details class="calc"><summary>Переформированные ранги и T<sub>j</sub></summary><div class="table-wrap"><table><thead><tr><th></th>${r.T.map((_, j) => `<th class="c">${j + 1}</th>`).join('')}</tr></thead><tbody>${r.ranks.map((row, i) => `<tr><td>${esc(base.items[i].id)}</td>${row.map(v => `<td class="c">${fmtFull(v)}</td>`).join('')}</tr>`).join('')}<tr><td><b>T<sub>j</sub></b></td>${r.T.map(v => `<td class="c">${fmtFull(v)}</td>`).join('')}</tr></tbody></table></div><p>ΣT = ${fmtFull(r.sumT)}; среднее сумм рангов m(n+1)/2 = ${fmtFull(r.mean)}.</p></details>
  <div class="row" style="margin-top:12px"><button class="btn primary" data-act="ex-save-set" data-id="${st.id}" ${r.significant ? '' : 'disabled title="Коэффициент конкордации незначим"'}>Сохранить λ как набор Кзнк</button></div></div>`;
}
document.addEventListener('paste', e => {
  const t = e.target; if (!t.dataset || !t.dataset.ex) return;
  const text = (e.clipboardData || window.clipboardData).getData('text'); if (!/[\t\n]/.test(text)) return;
  e.preventDefault();
  const [i0, j0] = t.dataset.ex.split(',').map(Number);
  text.trim().split(/\r?\n/).forEach((line, di) => line.split('\t').forEach((v, dj) => { const el = $(`[data-ex="${i0 + di},${j0 + dj}"]`); if (el) el.value = v.trim(); }));
});
function readMatrix(st) {
  $$('[data-ex]').forEach(el => { const [i, j] = el.dataset.ex.split(',').map(Number); const v = el.value.trim(); st.matrix[i][j] = v === '' ? '' : (Number.isInteger(Number(v)) ? Number(v) : v); });
}
actions['ex-calc'] = el => {
  const st = state.expertStudies.find(x => x.id === el.dataset.id); readMatrix(st);
  st.name = $('#ex-name').value.trim() || st.name; st.updatedAt = Store.nowIso(); persist(); render();
};
actions['ex-apply'] = el => {
  const st = state.expertStudies.find(x => x.id === el.dataset.id); readMatrix(st);
  const m = parseInt($('#ex-m').value, 10), x = parseInt($('#ex-x').value, 10);
  if (!(m >= 1 && m <= 200)) { toast('Число экспертов: 1…200', true); return; }
  if (!(x >= 2)) { toast('x_max должен быть ≥ 2', true); return; }
  st.matrix = st.matrix.map(r => Array.from({ length: m }, (_, j) => r[j] ?? ''));
  st.m = m; st.xmax = x; st.name = $('#ex-name').value.trim() || st.name; st.updatedAt = Store.nowIso(); persist(); render();
};
actions['ex-del'] = async el => {
  if (!await confirmDlg('Удалить экспертизу?', 'Матрица рангов будет удалена. Созданные из неё наборы Кзнк сохранятся.', 'Удалить', true)) return;
  state.expertStudies = state.expertStudies.filter(x => x.id !== el.dataset.id); persist(); location.hash = '#/coef';
};
actions['ex-save-set'] = el => {
  const st = state.expertStudies.find(x => x.id === el.dataset.id); const base = METHOD.COEF_SETS[st.baseSetId];
  const r = Calc.expert(st.matrix.map(x => x.map(Number)), Number(st.xmax)); if (!r.significant) return;
  const s = { id: Store.uid(), builtin: false, baseSetId: base.id, name: `${base.name} — Кзнк по эксперт изе «${st.name}»`, items: base.items.map((c, i) => ({ ...c, k: Calc.roundTo(r.lambda[i], 5) })), scale: null,
    src: `Пользовательский набор: λ по экспертизе «${st.name}» (m = ${st.m}, W = ${fmt(r.W, 3)}, χ² = ${fmt(r.chi2, 2)}), округлено до 5 знаков как в методике.`, createdAt: Store.nowIso() };
  state.coefSets.push(s); persist(); toast('Набор Кзнк создан'); location.hash = '#/coef/' + s.id;
};
```

```
2934
2935 /* ---------- Учебные планы (для расчёта g) ---------- */
2936 route(/^\/curricula$/, () => `<h1>Учебные планы</h1><p class="muted">Исходные данные алгоритма: дисципли
     ны, объём T1, самостоятельная работа T2, коэффициент мастерства преподавателя m, предшествующие (базовы
     е) дисциплины.</p>
2937   <div class="row no-print" style="margin-bottom:12px"><button class="btn primary" data-act="cur-new">+
     Новый учебный план</button><button class="btn" data-act="cur-example">Создать из контрольного примера</b
     utton></div>
2938   <div class="card">${state.curricula.length ? `<table><thead><tr><th>Название</th><th class="num">Дисци
     плин</th><th>Изменён</th></tr></thead><tbody>${state.curricula.map(c => `<tr class="click" data-act="go"
     data-href="#/curriculum/${c.id}"><td>${esc(c.name)}</td><td class="num">${c.disciplines.length}</td><td>
     ${fmtDT(c.updatedAt)}</td></tr>`).join('')}</tbody></table>` : '<div class="empty">Учебные планы не созд
     аны.</div>'}</div>`);
2939 actions['cur-new'] = () => { const c = { id: Store.uid(), name: 'Новый учебный план', disciplines: [], c
     reatedAt: Store.nowIso(), updatedAt: Store.nowIso() }; state.curricula.push(c); persist(); location.hash
     = '#/curriculum/' + c.id; };
2940 actions['cur-example'] = () => {
2941   const ids = Array.from({ length: 6 }, () => Store.uid());
2942   const rows = [[100, 80, []], [100, 70, []], [120, 90, [0]], [80, 50, [0]], [100, 80, [1]], [90, 70,
     [3, 4]]];
2943   const c = { id: Store.uid(), name: 'Контрольный пример (условные дисциплины D1–D6)', disciplines: row
     s.map((r, i) => ({ id: ids[i], name: 'D' + (i + 1), T1: r[0], T2: r[1], m: 0.8, preds: r[2].map(k => ids
     [k]) })), createdAt: Store.nowIso(), updatedAt: Store.nowIso(), note: 'Данные контрольного примера. Спос
     обности K1: D1 0,5; D2 0,5; D3 0,4; D4 0,4; D5 0,3; D6 0,4; g₀ = 0,713 →: 0,722; 0,722; 0,731; 0,727; 0,
     728; 0,734.' };
2944   state.curricula.push(c); persist(); location.hash = '#/curriculum/' + c.id;
2945 };
2946 route(/^\/curriculum\/([\w-]+)$/, id => {
2947   const c = state.curricula.find(x => x.id === id); if (!c) return notFound('Учебный план');
2948   return `<h1>Учебный план</h1>${c.note ? `<div class="note">${esc(c.note)}</div>` : ''}
2949   <div class="card"><div class="field"><label class="f" for="cu-name">Название</label><input type="text"
     id="cu-name" value="${esc(c.name)}"></div>
2950   <div class="table-wrap"><table><thead><tr><th>№</th><th>Дисциплина</th><th class="c">T1, ч</th><th cla
     ss="c">T2, ч</th><th class="c">m (0…1)</th><th>Опирается на №</th><th></th></tr></thead><tbody id="cuRow
     s">
2951   ${c.disciplines.map((d, i) => `<tr><td>${i + 1}</td><td><input type="text" data-cu="name" value="${esc
     (d.name)}" aria-label="Название дисциплины ${i + 1}"></td><td class="c"><input type="text" class="cell"
     data-cu="T1" value="${esc(d.T1)}" aria-label="T1"></td><td class="c"><input type="text" class="cell" dat
     a-cu="T2" value="${esc(d.T2)}" aria-label="T2"></td><td class="c"><input type="text" class="cell" data-c
     u="m" value="${esc(String(d.m).replace('.', ','))}" aria-label="m"></td><td><input type="text" data-cu
     ="preds" style="width:110px" value="${esc((d.preds || []).map(p => c.disciplines.findIndex(x => x.id ===
     p) + 1).join(', '))}" aria-label="Предшествующие"></td><td><button class="btn sm danger" data-act="cu-de
     lrow" data-i="${i}" aria-label="Удалить строку">×</button></td></tr>`).join('')}
2952   </tbody></table></div>
2953   <p class="hint">Дисциплина может опираться только на дисциплины с меньшими номерами. Если дисциплина ч
     итается в нескольких семестрах, части считаются разными дисциплинами.</p>
2954   <div class="row"><button class="btn" data-act="cu-addrow" data-id="${id}">+ Дисциплина</button><button
     class="btn primary" data-act="cu-save" data-id="${id}">Сохранить</button><span class="spacer"></span><bu
     tton class="btn danger" data-act="cu-del" data-id="${id}">Удалить план</button></div></div>`;
2955 });
2956 function readCurriculum(c) {
2957   const rows = $$('#cuRows tr'); const errs = [];
2958   const out = rows.map((tr, i) => {
2959     const g = k => $(`[data-cu="${k}"]`, tr).value.trim();
2960     const d = { ...c.disciplines[i], id: c.disciplines[i].id, name: g('name') || 'Дисциплина ' + (i +
     1), T1: parseNum(g('T1')), T2: parseNum(g('T2')), m: parseNum(g('m')) };
2961     if (!isNumV(d.T1) || d.T1 < 0) errs.push(`строка ${i + 1}: T1`); if (!isNumV(d.T2) || d.T2 < 0) err
     s.push(`строка ${i + 1}: T2`); if (!isNumV(d.m) || d.m < 0 || d.m > 1) errs.push(`строка ${i + 1}: m`);
2962     const nums = g('preds') ? g('preds').split(/[,; ]+/).filter(Boolean).map(Number) : [];
2963     if (nums.some(n => !Number.isInteger(n) || n < 1 || n >= i + 1)) errs.push(`строка ${i + 1}: предшес
     твующие должны быть номерами строк выше`);
2964     d.preds = nums.filter(n => Number.isInteger(n) && n >= 1 && n < i + 1).map(n => c.disciplines[n -
     1].id);
2965     return d;
2966   });
2967   return { out, errs };
2968 }
2969 actions['cu-addrow'] = el => { const c = state.curricula.find(x => x.id === el.dataset.id); const { out
     } = readCurriculum(c); c.disciplines = out; c.disciplines.push({ id: Store.uid(), name: 'Дисциплина ' +
     (out.length + 1), T1: 0, T2: 0, m: 0.8, preds: [] }); render(); };
2970 actions['cu-delrow'] = el => {
2971   const c = state.curricula.find(x => location.hash.includes(x.id)); const { out } = readCurriculum(c);
2972   const i = +el.dataset.i, rid = out[i].id; c.disciplines = out.filter((_, k) => k !== i); c.discipline
     s.forEach(d => d.preds = d.preds.filter(p => p !== rid)); render();
```

```
};
actions['cu-save'] = el => {
  const c = state.curricula.find(x => x.id === el.dataset.id); const { out, errs } = readCurriculum(c);
  if (errs.length) { toast('Исправьте: ' + errs.join('; '), true); return; }
  c.disciplines = out; c.name = $('#cu-name').value.trim() || c.name; c.updatedAt = Store.nowIso(); pers
ist(); toast('Учебный план сохранён'); render();
};
actions['cu-del'] = async el => {
  if (!await confirmDlg('Удалить учебный план?', 'Утверждённые расчёты сохранят копию плана.', 'Удалит
ь', true)) return;
  state.curricula = state.curricula.filter(x => x.id !== el.dataset.id); persist(); location.hash = '#/c
urricula';
};

/* ---------- Модели признаков (для Y) ---------- */
route(/^\/ymodels$/, () => `<h1>Модели Y</h1>
  <div class="note warn">Формула Y = Σaᵢk_jᵢ / Σaᵢkᵢ приведена на схеме модели. Перечень ключевых показа
телей «определяется заранее на основе профессиональных стандартов, паспортов компетенций и моделей выпус
кника» — в предоставленных материалах он не задан, поэтому признаки, веса aᵢ и эталонные значения kᵢ вво
дит пользователь.</div>
  <div class="row no-print" style="margin-bottom:12px"><button class="btn primary" data-act="ym-new">+ Н
овая модель</button></div>
  <div class="card">${state.yModels.length ? `<table><thead><tr><th>Название</th><th class="num">Признак
ов</th><th>Изменена</th></tr></thead><tbody>${state.yModels.map(c => `<tr class="click" data-act="go" da
ta-href="#/ymodel/${c.id}"><td>${esc(c.name)}</td><td class="num">${c.features.length}</td><td>${fmtDT
(c.updatedAt)}</td></tr>`).join('')}</tbody></table>` : '<div class="empty">Модели не созданы.</div>'}</
div>`);
actions['ym-new'] = () => { const c = { id: Store.uid(), name: 'Новая модель признаков', features: [], c
reatedAt: Store.nowIso(), updatedAt: Store.nowIso() }; state.yModels.push(c); persist(); location.hash =
'#/ymodel/' + c.id; };
route(/^\/ymodel\/([\w-]+)$/, id => {
  const c = state.yModels.find(x => x.id === id); if (!c) return notFound('Модель');
  return `<h1>Модель признаков</h1><div class="card"><div class="field"><label class="f" for="ym-name">Н
азвание</label><input type="text" id="ym-name" value="${esc(c.name)}"></div>
  <div class="table-wrap"><table><thead><tr><th>№</th><th>Признак</th><th class="c">Вес aᵢ (≥ 0)</th><th
class="c">Эталон kᵢ</th><th>Пояснение / основание</th><th></th></tr></thead><tbody id="ymRows">
  ${c.features.map((f, i) => `<tr><td>${i + 1}</td><td><input type="text" data-ym="name" value="${esc(f.
name)}" aria-label="Признак ${i + 1}"></td><td class="c"><input type="text" class="cell" data-ym="a" val
ue="${esc(String(f.a).replace('.', ','))}" aria-label="Вес"></td><td class="c"><input type="text" class
="cell" data-ym="k" value="${esc(String(f.k).replace('.', ','))}" aria-label="Эталон"></td><td><input ty
pe="text" data-ym="note" value="${esc(f.note || '')}" aria-label="Пояснение"></td><td><button class="btn
sm danger" data-act="ym-delrow" data-i="${i}" aria-label="Удалить">×</button></td></tr>`).join('')}
  </tbody></table></div>
  <div class="row"><button class="btn" data-act="ym-addrow" data-id="${id}">+ Признак</button><button cl
ass="btn primary" data-act="ym-save" data-id="${id}">Сохранить</button><span class="spacer"></span><butt
on class="btn danger" data-act="ym-del" data-id="${id}">Удалить модель</button></div></div>`;
});
function readY(c) {
  const errs = [];
  const out = $$('#ymRows tr').map((tr, i) => { const g = k => $(`[data-ym="${k}"]`, tr).value.trim();
    const f = { id: c.features[i].id, name: g('name') || 'Признак ' + (i + 1), a: parseNum(g('a')), k: p
arseNum(g('k')), note: g('note') };
    if (!isNumV(f.a) || f.a < 0) errs.push(`строка ${i + 1}: вес`); if (!isNumV(f.k)) errs.push(`строка
${i + 1}: эталон`); return f; });
  return { out, errs };
}
actions['ym-addrow'] = el => { const c = state.yModels.find(x => x.id === el.dataset.id); c.features = r
eadY(c).out; c.features.push({ id: Store.uid(), name: '', a: 1, k: 5, note: '' }); render(); };
actions['ym-delrow'] = el => { const c = state.yModels.find(x => location.hash.includes(x.id)); c.featur
es = readY(c).out.filter((_, k) => k !== +el.dataset.i); render(); };
actions['ym-save'] = el => {
  const c = state.yModels.find(x => x.id === el.dataset.id); const { out, errs } = readY(c);
  if (errs.length) { toast('Исправьте: ' + errs.join('; '), true); return; }
  c.features = out; c.name = $('#ym-name').value.trim() || c.name; c.updatedAt = Store.nowIso(); persist
(); toast('Модель сохранена'); render();
};
actions['ym-del'] = async el => {
  if (!await confirmDlg('Удалить модель?', 'Утверждённые расчёты сохранят копию модели.', 'Удалить', tru
e)) return;
  state.yModels = state.yModels.filter(x => x.id !== el.dataset.id); persist(); location.hash = '#/ymode
ls';
};

/* ================================================================== */
```

```
   МОДУЛЬ 9. ГРУППОВОЙ ВВОД И ИМПОРТ CSV
   ========================================================================== */
const normName = s => String(s || '').toLowerCase().replace(/ё/g, 'е').replace(/\s+/g, ' ').replace(/\s*\.\s*/g, '.').trim();
function batchSets(type, b) {
  if (type === 'practice') return { vpk: Model.coefSet(state, b.vpkSetId || 'src-vpk13') };
  return { vpk: Model.coefSet(state, b.vpkSetId || 'src-vpk16'), psk: b.pskEnabled ? Model.coefSet(state, b.pskSetId || 'src-psk5') : null };
}
function batchComps(type, b) { const s = batchSets(type, b); return [...s.vpk.items, ...(s.psk ? s.psk.items : [])]; }
/* Создание записи оценки из табличных данных (групповой ввод, импорт) */
function createAssessment(type, objectId, meta, grades, extra, finalize) {
  const a = Model.newAssessment(state, type, objectId);
  Object.assign(a.meta, meta);
  const sets = batchSets(type, { vpkSetId: a.meta.vpkSetId, pskSetId: a.meta.pskSetId, pskEnabled: type === 'graduate' ? !!extra.pskEnabled : false });
  for (const c of sets.vpk.items) if (Calc.isGrade(grades[c.id])) a.input.vpk[c.id] = grades[c.id];
  if (type === 'graduate') { a.input.pskEnabled = !!extra.pskEnabled; if (sets.psk) for (const c of sets.psk.items) if (Calc.isGrade(grades[c.id])) a.input.psk[c.id] = grades[c.id]; }
  if (type === 'practice') { a.input.leaderMark = extra.leader || ''; a.input.commissionMark = extra.commission || ''; }
  a.snapshot = Model.snapshot(state, a.type, a.meta, a.input);
  if (finalize) { const r = Model.compute(a); if (r.ok && r.complete) { a.result = r.result; a.status = 'final'; a.finalizedAt = Store.nowIso(); } }
  state.assessments.push(a);
  return a;
}

/* ---------- Групповой ввод ---------- */
function batchState() {
  if (!state.batchDraft) state.batchDraft = { type: 'practice', group: '', course: 4, period: 1, date: new Date().toISOString().slice(0, 10), evaluator: '', evaluator2: '', vpkSetId: '', pskSetId: '', pskEnabled: true, rows: {} };
  return state.batchDraft;
}
function batchRowStats(b, objId) {
  const comps = batchComps(b.type, b), row = (b.rows[objId] || {}).g || {};
  const filled = comps.filter(c => Calc.isGrade(row[c.id]));
  if (filled.length !== comps.length) return { n: filled.length, total: comps.length };
  const sum = filled.reduce((s, c) => s + row[c.id], 0), ps = filled.reduce((s, c) => s + row[c.id] * c.k, 0);
  return { n: filled.length, total: comps.length, meanGrade: sum / comps.length, meanPsfk: ps / comps.length };
}
route(/^\/batch$/, () => {
  const b = batchState();
  const groups = [...new Set(state.objects.map(o => o.group).filter(Boolean))].sort((a, c) => a.localeCompare(c, 'ru'));
  const objs = state.objects.filter(o => !b.group || o.group === b.group).sort((a, c) => a.name.localeCompare(c.name, 'ru'));
  const comps = batchComps(b.type, b);
  const isP = b.type === 'practice';
  const bset = isP ? 'src-vpk13' : 'src-vpk16';
  return `<h1>Ввод по группе</h1><p class="muted">Ввод оценок 1–5 сразу по всем курсантам группы (по образцу сводной таблицы методики). Можно вставить блок из Excel (Ctrl+V в первую ячейку). Таблица сохраняется автоматически до создания записей.</p>
  <div class="card"><div class="grid g4">
    <div class="field"><label class="f" for="bt-type">Вид оценки</label><select id="bt-type" data-bt="type"><option value="practice" ${isP ? 'selected' : ''}>Анкета войсковой практики</option><option value="graduate" ${!isP ? 'selected' : ''}>Отзыв на выпускника (раздел I)</option></select></div>
    <div class="field"><label class="f" for="bt-group">Группа</label><select id="bt-group" data-bt="group"><option value="">все объекты</option>${groups.map(g => `<option ${b.group === g ? 'selected' : ''}>${esc(g)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="bt-sub">${isP ? 'Курс' : 'Период'}</label><select id="bt-sub" data-bt="${isP ? 'course' : 'period'}">${isP ? `<option value="4" ${b.course == 4 ? 'selected' : ''}>4 курс</option><option value="5" ${b.course == 5 ? 'selected' : ''}>5 курс</option>` : `<option value="1" ${b.period == 1 ? 'selected' : ''}>1 год службы</option><option value="2" ${b.period == 2 ? 'selected' : ''}>2 года службы</option>`}</select></div>
    <div class="field"><label class="f" for="bt-date">Дата</label><input type="date" id="bt-date" data-bt="date" value="${esc(b.date)}"></div>
    <div class="field"><label class="f" for="bt-ev">${isP ? 'Командир подразделения' : 'Непосредственный начальник'}</label><input type="text" id="bt-ev" data-bt="evaluator" value="${esc(b.evaluator)}"></div>
```

```
    ${isP ? `<div class="field"><label class="f" for="bt-ev2">Руководитель практики от института</label><input type="text" id="bt-ev2" data-bt="evaluator2" value="${esc(b.evaluator2)}"></div>` : `<div class="field"><label class="f"> </label><label><input type="checkbox" data-bt="pskEnabled" ${b.pskEnabled ? 'checked' : ''}> Оценивать ПСК</label></div>`}
    <div class="field"><label class="f" for="bt-set">Набор Кзнк ВПК</label><select id="bt-set" data-bt="vpkSetId">${setOptions(bset, b.vpkSetId || bset)}</select></div>
  </div></div>
  ${objs.length ? `<div class="card"><div class="table-wrap"><table id="btTable"><thead><tr><th>Курсант</th>${comps.map(c => `<th class="c" title="${esc(c.text)}">${esc(c.id.replace('ВПК-', 'В').replace('ПСК-', 'П'))}</th>`).join('')}${isP ? '<th class="c" title="Оценка руководителя практики">Рук.</th><th class="c" title="Итоговая оценка комиссии">Ком.</th>' : ''}<th class="num">${term('bcp')}</th><th class="num">${term('Псфк')}</th></tr></thead><tbody>
    ${objs.map((o, ri) => { const row = b.rows[o.id] || {}; const g = row.g || {}; const st = batchRowStats(b, o.id);
      return `<tr data-btrow="${o.id}"><td style="white-space:nowrap">${esc(o.name)}</td>${comps.map((c, ci) => `<td class="c" style="padding:2px"><input class="cell${g[c.id] != null && !Calc.isGrade(g[c.id]) ? ' invalid' : ''}" style="width:34px;padding:3px" inputmode="numeric" maxlength="1" data-bg="${ri},${ci}" data-obj="${o.id}" data-comp="${esc(c.id)}" value="${esc(g[c.id] ?? '')}" aria-label="${esc(o.name)}, ${esc(c.id)}"></td>`).join('')}
      ${isP ? ['leader', 'commission'].map((k, q) => `<td class="c" style="padding:2px"><input class="cell" style="width:34px;padding:3px" inputmode="numeric" maxlength="1" data-bg="${ri},${comps.length + q}" data-obj="${o.id}" data-mark="${k}" value="${esc(row[k] ?? '')}" aria-label="${esc(o.name)}, ${k === 'leader' ? 'оценка руководителя' : 'оценка комиссии'}"></td>`).join('') : ''}
      <td class="num" data-st="g">${st.meanGrade != null ? fmt(st.meanGrade, 2) : st.n + '/' + st.total}</td><td class="num" data-st="p">${fmt(st.meanPsfk, 3)}</td></tr>`; }).join('')}
  </tbody></table></div>
  <div class="row" style="margin-top:12px"><button class="btn primary" data-act="bt-commit" data-final="1">Создать записи: полные — утвердить, неполные — черновики</button><button class="btn" data-act="bt-commit" data-final="0">Создать всё черновиками</button><span class="spacer"></span><button class="btn danger" data-act="bt-clear">Очистить таблицу</button></div>
  <p class="hint">Допустимые значения — 1…5 (оценки руководителя и комиссии — 2…5). Колонки: ${comps.map(c => esc(c.id)).join(', ')}. Разделы II–V отзыва и ОК/ПК анкеты заполняются в карточке записи.</p></div>` : '<div class="card empty">Нет объектов оценки в выбранной группе. <a href="#/objects">Добавить объекты</a> или <a href="#/import">импортировать CSV</a>.</div>'}`;
});
document.addEventListener('change', e => {
  const k = e.target.dataset && e.target.dataset.bt; if (!k) return;
  const b = batchState();
  let v = e.target.type === 'checkbox' ? e.target.checked : e.target.value;
  if (k === 'type' && v !== b.type) { b.rows = {}; b.vpkSetId = ''; }
  if (k === 'course' || k === 'period') v = Number(v);
  b[k] = v; persist(); render();
});
function bgSet(input, raw) {
  const b = batchState(), obj = input.dataset.obj; if (!b.rows[obj]) b.rows[obj] = { g: {} };
  const v = String(raw).trim(), n = v === '' ? undefined : (/^\d$/.test(v) ? Number(v) : v);
  input.value = v;
  if (input.dataset.mark) { b.rows[obj][input.dataset.mark] = v; input.classList.toggle('invalid', v !== '' && !Calc.isGrade(n, 2, 5)); }
  else { if (n === undefined) delete b.rows[obj].g[input.dataset.comp]; else b.rows[obj].g[input.dataset.comp] = n; input.classList.toggle('invalid', v !== '' && !Calc.isGrade(n)); }
  const tr = input.closest('tr'), st = batchRowStats(b, obj);
  $('[data-st=g]', tr).textContent = st.meanGrade != null ? fmt(st.meanGrade, 2) : st.n + '/' + st.total;
  $('[data-st=p]', tr).textContent = fmt(st.meanPsfk, 3);
}
document.addEventListener('input', e => { if (e.target.dataset && e.target.dataset.bg) { bgSet(e.target, e.target.value); persistSoon(); } });
document.addEventListener('keydown', e => {
  const t = e.target; if (!t.dataset || !t.dataset.bg) return;
  const [r, c] = t.dataset.bg.split(',').map(Number);
  const mv = { ArrowRight: [0, 1], ArrowLeft: [0, -1], ArrowDown: [1, 0], ArrowUp: [-1, 0], Enter: [1, 0] }[e.key];
  if (!mv) return; e.preventDefault();
  const el = $(`[data-bg="${r + mv[0]},${c + mv[1]}"]`); if (el) { el.focus(); el.select(); }
});
document.addEventListener('paste', e => {
  const t = e.target; if (!t.dataset || !t.dataset.bg) return;
  const text = (e.clipboardData || window.clipboardData).getData('text'); if (!/[\t\n]/.test(text)) return;
  e.preventDefault();
  const [r0, c0] = t.dataset.bg.split(',').map(Number); let cnt = 0;
  text.replace(/\r/g, '').replace(/\n$/, '').split('\n').forEach((line, dr) => line.split('\t').forEach((v, dc) => { const el = $(`[data-bg="${r0 + dr},${c0 + dc}"]`); if (el) { bgSet(el, v); cnt++; } }));
```

```
  persist(); toast(`Вставлено значений: ${cnt}`);
});
actions['bt-clear'] = async () => { if (!await confirmDlg('Очистить таблицу?', 'Все введённые в таблицу оценки будут удалены (созданные ранее записи не затрагиваются).', 'Очистить', true)) return; batchState().rows = {}; persist(); render(); };
actions['bt-commit'] = async el => {
  const b = batchState(), final = el.dataset.final === '1';
  const comps = batchComps(b.type, b);
  const ids = Object.keys(b.rows).filter(id => state.objects.some(o => o.id === id && (!b.group || o.group === b.group)) && (Object.keys(b.rows[id].g || {}).length || b.rows[id].leader || b.rows[id].commission));
  const bad = []; let complete = 0;
  for (const id of ids) {
    const r = b.rows[id];
    if (Object.values(r.g || {}).some(v => !Calc.isGrade(v)) || ['leader', 'commission'].some(k => r[k] && !Calc.isGrade(Number(r[k]), 2, 5))) bad.push(objName(id));
    else if (comps.every(c => Calc.isGrade(r.g[c.id]))) complete++;
  }
  if (!ids.length) { toast('В таблице нет введённых оценок', true); return; }
  if (bad.length) { toast('Недопустимые значения у: ' + bad.join(', '), true); return; }
  const msg = final ? `Будет создано записей: ${ids.length}. Утверждено (все оценки заполнены): ${complete}; черновиков: ${ids.length - complete}.` : `Будет создано черновиков: ${ids.length}.`;
  if (!await confirmDlg('Создать записи?', `<p>${msg}</p><p class="muted">Вид: ${esc(typeTitle(b.type))}; ${b.type === 'practice' ? b.course + ' курс' : b.period + ' год службы'}; дата ${fmtDate(b.date)}.</p>`, 'Создать')) return;
  const meta = { date: b.date, evaluator: b.evaluator, ...(b.type === 'practice' ? { course: b.course, evaluator2: b.evaluator2, vpkSetId: b.vpkSetId || 'src-vpk13' } : { period: b.period, vpkSetId: b.vpkSetId || 'src-vpk16' }) };
  for (const id of ids) { const r = b.rows[id]; createAssessment(b.type, id, { ...meta }, r.g || {}, { pskEnabled: b.pskEnabled, leader: r.leader, commission: r.commission }, final); delete b.rows[id]; }
  persist(); toast(`Создано записей: ${ids.length}`); location.hash = '#/history';
};

/* ---------- Импорт CSV ---------- */
function parseCsv(text) {
  text = text.replace(/^/, '');
  const first = text.split(/\r?\n/)[0] || '';
  const delim = [';', '\t', ','].map(d => [d, first.split(d).length]).sort((a, b) => b[1] - a[1])[0][0];
  const rows = []; let row = [], cell = '', q = false;
  for (let i = 0; i < text.length; i++) {
    const ch = text[i];
    if (q) { if (ch === '"') { if (text[i + 1] === '"') { cell += '"'; i++; } else q = false; } else cell += ch; continue; }
    if (ch === '"') q = true;
    else if (ch === delim) { row.push(cell); cell = ''; }
    else if (ch === '\n' || ch === '\r') { if (ch === '\r' && text[i + 1] === '\n') i++; row.push(cell); rows.push(row); row = []; cell = ''; }
    else cell += ch;
  }
  if (cell !== '' || row.length) { row.push(cell); rows.push(row); }
  return rows.filter(r => r.some(c => c.trim() !== ''));
}
const IMPORT_FIXED = ['Фамилия И.О.', 'Группа', 'Курс/год', 'Дата (ГГГГ-ММ-ДД)', 'Оценщик'];
function importColumns(type) {
  const comps = type === 'practice' ? METHOD.COEF_SETS['src-vpk13'].items : [...METHOD.COEF_SETS['src-vpk16'].items, ...METHOD.COEF_SETS['src-psk5'].items];
  return [...IMPORT_FIXED, ...comps.map(c => c.id), ...(type === 'practice' ? ['Оценка руководителя', 'Оценка комиссии'] : [])];
}
route(/^\/import$/, () => {
  const im = ui.importState || (ui.importState = { type: 'practice', rows: null, fileName: '' });
  return `<h1>Импорт из файла</h1>
  <div class="card"><h2>1. Шаблон</h2><div class="grid g3"><div class="field"><label class="f" for="im-type">Вид оценки</label><select id="im-type" data-im="type"><option value="practice" ${im.type === 'practice' ? 'selected' : ''}>Анкета войсковой практики</option><option value="graduate" ${im.type === 'graduate' ? 'selected' : ''}>Отзыв на выпускника (раздел I)</option></select></div></div>
  <button class="btn" data-act="im-template">Скачать шаблон CSV</button>
  <p class="hint">Шаблон открывается в Excel. Заполните строки (одна строка — один курсант) и загрузите книгу .xlsx как есть (берётся первый лист) либо сохраните как CSV. Поле «Курс/год»: для практики — 4 или 5, для отзыва — 1 или 2. Пустая ячейка ПСК у отзыва означает, что ПСК не оцениваются, если у строки пусты все ПСК.</p></div>
  <div class="card"><h2>2. Загрузка</h2><label class="btn primary" for="imFile">Выбрать файл (.xlsx или .csv)…</label><input type="file" id="imFile" accept=".csv,.txt,.xlsx,text/csv" hidden> ${im.fileName ? `<span class="muted">${esc(im.fileName)}</span>` : ''}</div>
```

```
  ${im.rows ? importPreview(im) : ''}`;
});
function importAnalyze(type, table) {
  const cols = importColumns(type);
  const head = table[0].map(h => h.trim());
  const idx = cols.map(c => head.findIndex(h => h.toLowerCase() === c.toLowerCase()));
  const need = ['Фамилия И.О.'];
  const missingCols = cols.filter((c, i) => idx[i] < 0 && (need.includes(c) || /^(ВПК|ПСК)-/.test(c)));
  const comps = type === 'practice' ? METHOD.COEF_SETS['src-vpk13'].items : [...METHOD.COEF_SETS['src-vpk16'].items, ...METHOD.COEF_SETS['src-psk5'].items];
  const get = (r, c) => { const i = idx[cols.indexOf(c)]; return i < 0 ? '' : String(r[i] ?? '').trim(); };
  const rows = table.slice(1).map((r, n) => {
    const errs = [], grades = {};
    const name = get(r, 'Фамилия И.О.'), group = get(r, 'Группа');
    if (!name) errs.push('нет ФИО');
    const sub = Number(get(r, 'Курс/год') || (type === 'practice' ? 4 : 1));
    if (type === 'practice' ? ![4, 5].includes(sub) : ![1, 2].includes(sub)) errs.push('курс/год');
    let date = String(excelDate(get(r, 'Дата (ГГГГ-ММ-ДД)'))); const dm = date.match(/^(\d{1,2})\.(\d{1,2})\.(\d{4})$/); if (dm) date = `${dm[3]}-${dm[2].padStart(2, '0')}-${dm[1].padStart(2, '0')}`;
    if (date && !/^\d{4}-\d{2}-\d{2}$/.test(date)) errs.push('дата');
    for (const c of comps) { const v = get(r, c.id); if (v === '') continue; const g = Number(v.replace(',', .')); if (!Calc.isGrade(g)) errs.push(`${c.id}: «${v}»`); else grades[c.id] = g; }
    const leader = get(r, 'Оценка руководителя'), commission = get(r, 'Оценка комиссии');
    for (const [k, v] of [['оценка руководителя', leader], ['оценка комиссии', commission]]) if (v && !Calc.isGrade(Number(v), 2, 5)) errs.push(k);
    const pskEnabled = type === 'graduate' && METHOD.COEF_SETS['src-psk5'].items.some(c => grades[c.id] != null);
    const needComps = type === 'practice' ? METHOD.COEF_SETS['src-vpk13'].items : [...METHOD.COEF_SETS['src-vpk16'].items, ...(pskEnabled ? METHOD.COEF_SETS['src-psk5'].items : [])];
    const complete = needComps.every(c => grades[c.id] != null);
    const cand = state.objects.filter(o => normName(o.name) === normName(name) && (!group || normName(o.group) === normName(group)));
    return { line: n + 2, name, group, sub, date: date || new Date().toISOString().slice(0, 10), evaluator: get(r, 'Оценщик'), grades, leader, commission, pskEnabled, complete, errs, match: cand.length === 1 ? cand[0].id : null, ambiguous: cand.length > 1 };
  });
  return { rows, missingCols };
}
function importPreview(im) {
  const { rows, missingCols } = im.analysis;
  const okRows = rows.filter(r => !r.errs.length && !r.ambiguous);
  return `<div class="card"><h2>3. Проверка (${rows.length} строк)</h2>
  ${missingCols.length ? `<div class="note err">Не найдены столбцы: ${esc(missingCols.join(', '))}. Используйте шаблон.</div>` : ''}
  <div class="table-wrap"><table><thead><tr><th>Строка</th><th>Фамилия И.О.</th><th>Группа</th><th>Объект</th><th class="num">Оценок</th><th>Полнота</th><th>Ошибки</th></tr></thead><tbody>
  ${rows.map(r => `<tr><td>${r.line}</td><td>${esc(r.name)}</td><td>${esc(r.group)}</td><td>${r.ambiguous ? '<span class="badge b-draft">неоднозначно</span>' : r.match ? 'существующий' : '<span class="badge b-info">будет создан</span>'}</td><td class="num">${Object.keys(r.grades).length}</td><td>${r.complete ? 'полная' : 'неполная'}</td><td style="color:var(--danger)">${esc(r.errs.join('; ')) || (r.ambiguous ? 'несколько объектов с таким ФИО — уточните группу' : '')}</td></tr>`).join('')}
  </tbody></table></div>
  <div class="row" style="margin-top:12px"><label><input type="checkbox" id="im-final" checked> Утвердить полные строки (неполные — черновиками)</label><span class="spacer"></span>
  <button class="btn primary" data-act="im-commit" ${okRows.length && !missingCols.length ? '' : 'disabled'}>Импортировать ${okRows.length} строк</button></div>
  ${rows.length - okRows.length ? `<p class="hint">Строки с ошибками (${rows.length - okRows.length}) импортированы не будут.</p>` : ''}</div>`;
}
document.addEventListener('change', async e => {
  const d = e.target.dataset || {};
  if (d.im) { ui.importState[d.im] = e.target.value; ui.importState.rows = null; ui.importState.fileName = ''; render(); return; }
  if (e.target.id !== 'imFile' || !e.target.files.length) return;
  const f = e.target.files[0]; e.target.value = '';
  const table = parseCsv(await f.text());
  if (table.length < 2) { toast('Файл пуст или не содержит строк данных', true); return; }
  ui.importState.rows = table; ui.importState.fileName = f.name; ui.importState.analysis = importAnalyze(ui.importState.type, table); render();
});
actions['im-template'] = () => download(`шаблон_${ui.importState.type === 'practice' ? 'практика' : 'отзыв'}.csv`, toCsv([importColumns(ui.importState.type)]), 'text/csv;charset=utf-8');
actions['im-commit'] = async () => {
```

```
  const im = ui.importState, type = im.type, final = $('#im-final').checked;
  const rows = im.analysis.rows.filter(r => !r.errs.length && !r.ambiguous);
  const created = rows.filter(r => !r.match).length;
  if (!await confirmDlg('Импортировать?', `<p>Будет создано записей оценки: ${rows.length}; новых объектов оценки: ${created}.</p>`, 'Импортировать')) return;
  for (const r of rows) {
    let id = r.match;
    if (!id) { const o = { id: Store.uid(), name: r.name, group: r.group, specialty: '', year: '', notes: 'импорт CSV', createdAt: Store.nowIso() }; state.objects.push(o); id = o.id; }
    const meta = { date: r.date, evaluator: r.evaluator, ...(type === 'practice' ? { course: r.sub } : { period: r.sub }) };
    createAssessment(type, id, meta, r.grades, { pskEnabled: r.pskEnabled, leader: r.leader, commission: r.commission }, final);
  }
  persist(); ui.importState = { type, rows: null, fileName: '' }; toast(`Импортировано записей: ${rows.length}`); location.hash = '#/history';
};

/* ======================================================================
   МОДУЛЬ 10. АНАЛИТИКА: ВЫПУСК ↔ ВОЙСКА, СРАВНЕНИЕ ГРУПП, РАСХОЖДЕНИЯ
   ====================================================================== */
const groupList = () => [...new Set(state.objects.map(o => o.group).filter(Boolean))].sort((a, b) => a.localeCompare(b, 'ru'));
const latest = arr => arr.sort((a, b) => (b.meta.date || '').localeCompare(a.meta.date || '') || (b.finalizedAt || '').localeCompare(a.finalizedAt || ''))[0];
const rowsOfResult = r => [r.vpk, r.psk].filter(Boolean).flatMap(x => x.rows);

/* ---------- Выпуск ↔ войска ---------- */
function followupPairs() {
  const F = ui.follow;
  const out = [];
  for (const o of state.objects) {
    if (F.group && o.group !== F.group) continue;
    const fin = state.assessments.filter(a => a.objectId === o.id && a.status === 'final');
    const pr = latest(fin.filter(a => a.type === 'practice' && (!F.course || String(a.meta.course) === F.course)));
    const gr = latest(fin.filter(a => a.type === 'graduate' && String(a.meta.period) === F.period));
    if (!pr || !gr) continue;
    out.push({ o, pr, gr, cmp: Calc.followup(rowsOfResult(pr.result), rowsOfResult(gr.result)) });
  }
  return out;
}
route(/^\/followup$/, () => {
  const F = ui.follow;
  const pairs = followupPairs();
  const ids = [...new Set(pairs.flatMap(p => p.cmp.rows.map(r => r.id)))];
  const byComp = ids.map(id => { const rs = pairs.map(p => p.cmp.rows.find(r => r.id === id)).filter(Boolean); return { id, n: rs.length, a: rs.reduce((s, r) => s + r.a, 0) / rs.length, b: rs.reduce((s, r) => s + r.b, 0) / rs.length, lower: rs.filter(r => r.diff < 0).length }; });
  return `<h1>Сопоставление: практика и служба</h1>
  <p class="muted">Оценки компетенций, полученные курсантом на войсковой практике, сопоставляются с оценками командира в отзыве после 1 (2) лет службы — по одинаковым индексам компетенций (13 ВПК). Основание: методика (у 15–23% выпускников оценки в войсках отличаются от результатов обучения).</p>
  <div class="card no-print"><div class="grid g3">
    <div class="field"><label class="f" for="fu-g">Группа</label><select id="fu-g" data-fu="group"><option value="">все</option>${groupList().map(g => `<option ${F.group === g ? 'selected' : ''}>${esc(g)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="fu-c">Практика</label><select id="fu-c" data-fu="course"><option value="">последняя (4 или 5 курс)</option><option value="4" ${F.course === '4' ? 'selected' : ''}>4 курс</option><option value="5" ${F.course === '5' ? 'selected' : ''}>5 курс</option></select></div>
    <div class="field"><label class="f" for="fu-p">Отзыв</label><select id="fu-p" data-fu="period"><option value="1" ${F.period === '1' ? 'selected' : ''}>после 1 года службы</option><option value="2" ${F.period === '2' ? 'selected' : ''}>после 2 лет службы</option></select></div>
  </div><div class="row"><button class="btn sm" data-act="fu-csv" ${pairs.length ? '' : 'disabled'}>CSV</button><button class="btn sm" data-act="print" ${pairs.length ? '' : 'disabled'}>Печать / PDF</button></div></div>
  ${pairs.length ? `<div class="card"><h2>По выпускникам (${pairs.length})</h2><div class="table-wrap"><table><thead><tr><th>Выпускник</th><th>Группа</th><th>Практика</th><th>Отзыв</th><th class="num">Общих компетенций</th><th class="num">${term('bcp')} на практике</th><th class="num">${term('bcp')} в отзыве</th><th class="num">Δ (отзыв − практика)</th><th class="num">Выше / ниже / равно</th></tr></thead><tbody>
    ${pairs.map(p => `<tr class="click" data-act="go" data-href="#/followup/${p.o.id}"><td>${esc(p.o.name)}</td><td>${esc(p.o.group || '—')}</td><td>${p.pr.meta.course} курс, ${fmtDate(p.pr.meta.date)}</td><td>${p.gr.meta.period} год, ${fmtDate(p.gr.meta.date)}</td><td class="num">${p.cmp.n}</td><td class="num">${fmt(p.cmp.meanA, 2)}</td><td class="num">${fmt(p.cmp.meanB, 2)}</td><td class="num"><b>${p.cmp.n ?
```

```
(p.cmp.meanDiff > 0 ? '+' : '') + fmt(p.cmp.meanDiff, 2) : '—'}</b></td><td class="num">${p.cmp.higher} / ${p.cmp.lower} / ${p.cmp.same}</td></tr>`).join('')}
  </tbody></table></div></div>
  <div class="card"><h2>По компетенциям</h2><div class="table-wrap"><table><thead><tr><th>Компетенция</th><th class="num">Выпускников</th><th class="num">Средняя оценка на практике</th><th class="num">Средняя оценка в отзыве</th><th class="num">Δ</th><th class="num">Оценка в отзыве ниже</th></tr></thead><tbody>
    ${byComp.map(c => `<tr><td>${esc(c.id)}</td><td class="num">${c.n}</td><td class="num">${fmt(c.a, 2)}</td><td class="num">${fmt(c.b, 2)}</td><td class="num">${(c.b - c.a > 0 ? '+' : '') + fmt(c.b - c.a, 2)}</td><td class="num">${c.lower} (${fmt(c.lower / c.n * 100, 0)}%)</td></tr>`).join('')}</tbody></table></div>
    ${lineChart({ labels: byComp.map(c => c.id), yMin: 1, yMax: 5, yDigits: 0, ticks: 4, series: [{ name: 'Средняя оценка на практике', color: '#8E8E93', values: byComp.map(c => c.a) }, { name: 'Средняя оценка в отзыве', color: '#007AFF', values: byComp.map(c => c.b) }] })}
    <p class="src">Выводятся разности оценок; критерий «подтвердил / не подтвердил оценку» методикой не установлен. Для каждого выпускника берётся последняя утверждённая практика выбранного курса и последний отзыв за выбранный период. Значения Псфк не сравниваются: в отзыве и анкете практики разные наборы компетенций.</p></div>`
  : '<div class="card empty">Нет выпускников, у которых есть и утверждённая анкета практики, и утверждённый отзыв за выбранный период.</div>'}`;
});
route(/^\/followup\/([\w-]+)$/, id => {
  const o = state.objects.find(x => x.id === id); if (!o) return objMissing();
  const p = followupPairs().find(x => x.o.id === id) || (() => { const keep = ui.follow.group; ui.follow.group = ''; const r = followupPairs().find(x => x.o.id === id); ui.follow.group = keep; return r; })();
  if (!p) return `<div class="card">${emptyState('Сравнивать пока не с чем', 'Нужны утверждённые анкета практики и отзыв из войск с общими компетенциями.')}</div>`;
  return `<div class="row no-print" style="margin-bottom:12px"><button class="btn" data-act="print">Печать / PDF</button></div>
  <h1>${esc(o.name)}: практика и служба</h1><p class="muted">Практика: <a href="#/a/${p.pr.id}">${p.pr.meta.course} курс, ${fmtDate(p.pr.meta.date)}</a> · отзыв: <a href="#/a/${p.gr.id}">${p.gr.meta.period} год службы, ${fmtDate(p.gr.meta.date)}</a></p>
  <div class="card"><div class="grid g4">${kpi('bcp на практике', fmt(p.cmp.meanA, 2), p.cmp.n + ' общих компетенций')}${kpi('bcp в отзыве', fmt(p.cmp.meanB, 2), '')}${kpi('Δ', (p.cmp.meanDiff > 0 ? '+' : '') + fmt(p.cmp.meanDiff, 2), 'отзыв − практика')}${kpi('Выше / ниже / равно', `${p.cmp.higher} / ${p.cmp.lower} / ${p.cmp.same}`, 'число компетенций')}</div>
  <div class="table-wrap"><table><thead><tr><th>Компетенция</th><th class="num">Практика</th><th class="num">Отзыв</th><th class="num">Δ</th></tr></thead><tbody>${p.cmp.rows.map(r => `<tr><td>${esc(r.id)}</td><td class="num">${r.a}</td><td class="num">${r.b}</td><td class="num" style="${r.diff < 0 ? 'color:var(--danger);font-weight:600' : ''}">${(r.diff > 0 ? '+' : '') + r.diff}</td></tr>`).join('')}</tbody></table></div>
  ${lineChart({ labels: p.cmp.rows.map(r => r.id), yMin: 1, yMax: 5, yDigits: 0, ticks: 4, series: [{ name: 'Практика', color: '#8E8E93', values: p.cmp.rows.map(r => r.a) }, { name: 'Отзыв', color: '#007AFF', values: p.cmp.rows.map(r => r.b) }] })}</div>`;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.fu; if (!k) return; ui.follow[k] = e.target.value; render(); });
actions['fu-csv'] = () => {
  const rows = [['Выпускник', 'Группа', 'Курс практики', 'Дата практики', 'Период отзыва', 'Дата отзыва', 'Компетенция', 'Оценка на практике', 'Оценка в отзыве', 'Разность']];
  for (const p of followupPairs()) p.cmp.rows.forEach(r => rows.push([p.o.name, p.o.group || '', p.pr.meta.course, p.pr.meta.date, p.gr.meta.period, p.gr.meta.date, r.id, r.a, r.b, r.diff]));
  download(`выпуск-войска_${new Date().toISOString().slice(0, 10)}.csv`, toCsv(rows), 'text/csv;charset=utf-8');
};

/* ---------- Сравнение групп (КГ / ЭГ) ---------- */
function groupItems(type, group, sub) {
  return state.assessments.filter(a => a.status === 'final' && a.type === type && (state.objects.find(o => o.id === a.objectId) || {}).group === group
    && (!sub || String(type === 'practice' ? a.meta.course : a.meta.period) === sub))
    .map(a => ({ meanGrade: a.result.vpk.meanGrade, meanPsfk: a.result.vpk.meanPsfk, level: ((type === 'graduate' ? (a.result.vpk.levelPsfk || a.result.vpk.levelBsr) : a.result.vpk.levelBsr) || {}).code, leader: a.input.leaderMark, commission: a.input.commissionMark, set: a.snapshot.sets.vpk.id }));
}
route(/^\/groups$/, () => {
  const G = ui.groupsCmp, gl = groupList();
  const sel = [G.a, G.b, G.c].filter(Boolean);
  const stats = sel.map(g => ({ g, items: groupItems(G.type, g, G.sub) })).map(x => ({ ...x, s: Calc.groupStats(x.items) }));
  const sets = new Set(stats.flatMap(x => x.items.map(i => i.set)));
  const colors = ['#8E8E93', '#007AFF', '#FF9500'];
  const opt = (k, label) => `<div class="field"><label class="f" for="gc-${k}">${label}</label><select id="gc-${k}" data-gc="${k}"><option value="">—</option>${gl.map(g => `<option ${G[k] === g ? 'selected' :
```

```
''}>${esc(g)}</option>`).join('')}</select></div>`;
3298   return `<h1>Сравнение групп</h1><p class="muted">Средний балл и показатель сформированности по группам — например, контрольной (КГ) и экспериментальной (ЭГ), как на. Учитываются утверждённые расчёты.</p>
3299   <div class="card no-print"><div class="grid g3">
3300     <div class="field"><label class="f" for="gc-type">Вид оценки</label><select id="gc-type" data-gc="type"><option value="practice" ${G.type === 'practice' ? 'selected' : ''}>Войсковая практика</option><option value="graduate" ${G.type === 'graduate' ? 'selected' : ''}>Отзыв на выпускника</option></select></div>
3301     <div class="field"><label class="f" for="gc-sub">${G.type === 'practice' ? 'Курс' : 'Год службы'}</label><select id="gc-sub" data-gc="sub"><option value="">все</option>${(G.type === 'practice' ? ['4', '5'] : ['1', '2']).map(v => `<option ${G.sub === v ? 'selected' : ''}>${v}</option>`).join('')}</select></div><div></div>
3302     ${opt('a', 'Группа 1 (например, КГ)')}${opt('b', 'Группа 2 (например, ЭГ)')}${opt('c', 'Группа 3 (не обязательно)')}
3303   </div><button class="btn sm" data-act="print" ${stats.length ? '' : 'disabled'}>Печать / PDF</button></div>
3304   ${sets.size > 1 ? '<div class="note warn">В группах использованы разные наборы Кзнк — показатели Псфк несопоставимы.</div>' : ''}
3305   ${stats.length ? `<div class="card"><div class="table-wrap"><table><thead><tr><th>Показатель</th>${stats.map(x => `<th class="num">${esc(x.g)}</th>`).join('')}</tr></thead><tbody>
3306     <tr><td>Число утверждённых расчётов</td>${stats.map(x => `<td class="num">${x.s.n}</td>`).join('')}</tr>
3307     <tr><td>Средний балл (${term('bcp')})</td>${stats.map(x => `<td class="num"><b>${fmt(x.s.meanGrade, 2)}</b></td>`).join('')}</tr>
3308     <tr><td>Средний ${term('Псфк')} ВПК</td>${stats.map(x => `<td class="num">${fmt(x.s.meanPsfk, 3)}</td>`).join('')}</tr>
3309     <tr><td>Псфк: минимум — максимум</td>${stats.map(x => `<td class="num">${fmt(x.s.minPsfk, 3)} — ${fmt(x.s.maxPsfk, 3)}</td>`).join('')}</tr>
3310     ${G.type === 'practice' ? `<tr><td>Средняя оценка руководителя практики</td>${stats.map(x => `<td class="num">${fmt(x.s.meanLeader, 2)}</td>`).join('')}</tr><tr><td>Средняя итоговая оценка комиссии</td>${stats.map(x => `<td class="num">${fmt(x.s.meanCommission, 2)}</td>`).join('')}</tr>` : ''}
3311     ${METHOD.LEVELS.map(L => `<tr><td>Уровень «${L.name}»</td>${stats.map(x => `<td class="num">${x.s.counts[L.code]}${x.s.n ? ' (' + fmt(x.s.counts[L.code] / x.s.n * 100, 0) + '%)' : ''}</td>`).join('')}</tr>`).join('')}
3312   </tbody></table></div>
3313   <h3>Средний балл групп</h3>${barChart(stats.map((x, i) => ({ label: x.g, value: x.s.meanGrade, color: colors[i] })), 0, 5, 2)}
3314   <p class="src">Уровень: ${G.type === 'practice' ? 'по bcp' : 'по таблице Псфк ВПК (для пользовательских наборов — по bcp)'}. Проверка статистической значимости различий в методике не описана и не выполняется.</p></div>` : '<div class="card empty">Выберите группы для сравнения.</div>'}`;
3315 });
3316 function barChart(items, lo, hi, digits) {
3317   const W = 760, H = 60 + items.length * 44, pl = 160, pr = 70;
3318   const x = v => pl + (v - lo) / (hi - lo) * (W - pl - pr);
3319   return `<svg class="chart" viewBox="0 0 ${W} ${H}" role="img" aria-label="Столбчатая диаграмма">${items.map((it, i) => { const y = 20 + i * 44;
3320     return `<text x="${pl - 8}" y="${y + 20}" text-anchor="end">${esc(it.label)}</text>${isNumV(it.value) ? `<rect x="${pl}" y="${y + 4}" width="${Math.max(0, x(it.value) - pl)}" height="26" fill="${it.color}" rx="3"><title>${esc(it.label)}: ${fmt(it.value, digits)}</title></rect><text x="${x(it.value) + 6}" y="${y + 22}">${fmt(it.value, digits)}</text>` : `<text x="${pl + 4}" y="${y + 22}">нет данных</text>`}`; }).join('')}
3321     <line x1="${pl}" x2="${pl}" y1="14" y2="${H - 20}" stroke="#9aa3ad"/></svg>`;
3322 }
3323 document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.gc; if (!k) return; ui.groupsCmp[k] = e.target.value; if (k === 'type') ui.groupsCmp.sub = ''; render(); });
3324
3325 /* ---------- Правило расхождений (пользовательское) ---------- */
3326 const discRule = () => state.settings && state.settings.discrepancy ? state.settings.discrepancy : { enabled: false, compareTo: 'commission', minDiff: 1 };
3327 function discFor(a) {
3328   const R = discRule(); if (!R.enabled || a.type !== 'practice' || !a.result) return null;
3329   return Calc.discrepancy(a.result.vpk.levelBsr && a.result.vpk.levelBsr.code, R.compareTo === 'leader' ? a.input.leaderMark : a.input.commissionMark, R.minDiff);
3330 }
3331 function discSettingsCard() {
3332   const R = discRule();
3333   return `<div class="card no-print"><h3 style="margin-top:0">Правило выявления расхождений <span class="badge b-draft">не из методики</span></h3>
3334   <p class="hint">В методике расхождения между расчётом и выставленными оценками выявлялись «визуально»; формального правила методика не даёт. Здесь правило задаёте вы: уровень по bcp переводится в оценку (высокий — 5, средний — 4, низкий — 3, критический — 2) и сравнивается с выбранной оценкой.</p>
3335   <div class="grid g3"><div class="field"><label><input type="checkbox" data-dr="enabled" ${R.enabled ? 'checked' : ''}> Показывать расхождения</label></div>
```

```
  <div class="field"><label class="f" for="dr-c">Сравнивать с</label><select id="dr-c" data-dr="compareTo"><option value="commission" ${R.compareTo === 'commission' ? 'selected' : ''}>итоговой оценкой комиссии</option><option value="leader" ${R.compareTo === 'leader' ? 'selected' : ''}>оценкой руководителя практики</option></select></div>
  <div class="field"><label class="f" for="dr-m">Расхождение, если разница ≥</label><select id="dr-m" data-dr="minDiff">${[1, 2].map(v => `<option ${R.minDiff == v ? 'selected' : ''}>${v}</option>`).join('')}</select></div></div></div>`;
}
document.addEventListener('change', e => {
  const k = e.target.dataset && e.target.dataset.dr; if (!k) return;
  if (!state.settings) state.settings = {};
  const R = { ...discRule() }; R[k] = e.target.type === 'checkbox' ? e.target.checked : (k === 'minDiff' ? Number(e.target.value) : e.target.value);
  state.settings.discrepancy = R; persist(); render();
});
function discBadge(d) {
  if (!d) return '—';
  return d.flag ? `<span class="badge b-draft" style="color:var(--danger)" title="Расчётный уровень соответствует оценке ${d.derived}">${d.diff > 0 ? 'завышена' : 'занижена'} на ${Math.abs(d.diff)}</span>` : '<span class="badge b-final">нет</span>';
}

/* ======================================================================
   МОДУЛЬ 11. МОДЕЛЬ ПОДГОТОВЛЕННОСТИ g: ГРУППА, ЧУВСТВИТЕЛЬНОСТЬ, ОПТИМИЗАЦИЯ
   ====================================================================== */
const PALETTE = ['#007AFF', '#FF9500', '#34C759', '#FF3B30', '#AF52DE', '#30B0C7', '#A2845E', '#FF2D55', '#636366', '#00C7BE'];
/* Универсальная вставка блока из Excel в сетку полей: data-grid="имя" data-rc="r,c" */
document.addEventListener('paste', e => {
  const t = e.target; if (!t.dataset || !t.dataset.grid || !t.dataset.rc) return;
  const text = (e.clipboardData || window.clipboardData).getData('text'); if (!/[\t\n]/.test(text)) return;
  e.preventDefault();
  const [r0, c0] = t.dataset.rc.split(',').map(Number); let n = 0;
  text.replace(/\r/g, '').replace(/\n$/, '').split('\n').forEach((line, dr) => line.split('\t').forEach((v, dc) => {
    const el = $(`[data-grid="${t.dataset.grid}"][data-rc="${r0 + dr},${c0 + dc}"]`); if (el) { el.value = v.trim(); el.dispatchEvent(new Event('input', { bubbles: true })); n++; } }));
  toast(`Вставлено значений: ${n}`);
});
const parseList = s => String(s || '').split(/[;\s]+/).filter(Boolean).map(parseNum);

/* ---------- Прогноз g по группе ---------- */
function gb() {
  if (!state.gbatch) state.gbatch = { curriculumId: '', mode: '24', p: METHOD.G_MODEL.p24, group: '', rows: {} };
  return state.gbatch;
}
function gbCompute() {
  const G = gb(), c = state.curricula.find(x => x.id === G.curriculumId); if (!c) return null;
  const objs = state.objects.filter(o => !G.group || o.group === G.group).sort((a, b) => a.name.localeCompare(b.name, 'ru'));
  return { c, objs, res: objs.map(o => {
    const row = G.rows[o.id] || {}; const ab = {};
    for (const d of c.disciplines) ab[d.id] = G.mode === '21' ? { c1: (row.c1 || {})[d.id], c2: (row.c2 || {})[d.id] } : { c: (row.c || {})[d.id] };
    try { const r = Calc.trajectory({ disciplines: c.disciplines, g0: row.g0, abilities: ab, p: Number(G.p), mode: G.mode }); return { o, ok: true, r, ab }; }
    catch (e) { return { o, ok: false, err: e.message }; }
  }) };
}
route(/^\/gbatch$/, () => {
  const G = gb(), c = state.curricula.find(x => x.id === G.curriculumId);
  const objs = state.objects.filter(o => !G.group || o.group === G.group).sort((a, b) => a.name.localeCompare(b.name, 'ru'));
  const cols = c ? c.disciplines.flatMap(d => G.mode === '21' ? [[d, 'c1'], [d, 'c2']] : [[d, 'c']]) : [];
  const comp = c && objs.length ? gbCompute() : null;
  const okRes = comp ? comp.res.filter(x => x.ok) : [];
  return `<h1>Прогноз уровня подготовленности по группе</h1><p class="muted">Алгоритм для нескольких обучающихся с общим учебным планом: у каждого свой начальный уровень g₀ и способности по дисциплинам (как в контрольном примере). Можно вставить блок из Excel.</p>
  <div class="card"><div class="grid g4">
```

```
    <div class="field"><label class="f" for="gb-c">Учебный план</label><select id="gb-c" data-gbk="curriculumId"><option value="">— выберите —</option>${state.curricula.map(x => `<option value="${x.id}" ${x.id === G.curriculumId ? 'selected' : ''}>${esc(x.name)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="gb-g">Группа</label><select id="gb-g" data-gbk="group"><option value="">все объекты</option>${groupList().map(g => `<option ${G.group === g ? 'selected' : ''}>${esc(g)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="gb-m">Формула</label><select id="gb-m" data-gbk="mode"><option value="24" ${G.mode === '24' ? 'selected' : ''}>(24), единый c</option><option value="21" ${G.mode === '21' ? 'selected' : ''}>(21), c1 и c2</option></select></div>
    <div class="field"><label class="f" for="gb-p">${term('p')}, ч⁻¹</label><input type="text" id="gb-p" data-gbk="p" value="${esc(String(G.p).replace('.', ','))}"></div>
  </div>${!state.curricula.length ? '<div class="note warn">Нет учебных планов. <a href="#/curricula">Создайте план</a>.</div>' : ''}</div>
  ${c && objs.length ? `<div class="card"><h2>Исходные данные</h2><div class="table-wrap"><table><thead><tr><th>Обучающийся</th><th class="c">${term('g₀')}</th>${cols.map(([d, k]) => `<th class="c" title="${esc(d.name)}">${esc(d.name.slice(0, 10))}${G.mode === '21' ? ' ' + k : ''}</th>`).join('')}</tr></thead><tbody>
    ${objs.map((o, ri) => { const row = G.rows[o.id] || {}; const g0s = state.assessments.filter(a => a.type === 'g0' && a.status === 'final' && a.objectId === o.id);
      return `<tr><td style="white-space:nowrap">${esc(o.name)}${g0s.length && row.g0 == null ? ' <button class="btn sm" data-act="gb-takeg0" data-id="${o.id}" title="Взять g₀ из утверждённого расчёта">g₀</button>' : ''}</td><td class="c" style="padding:2px"><input class="cell" style="width:60px" data-grid="gb" data-rc="${ri},0" data-gb="${o.id}|g0" value="${esc(row.g0 != null ? String(row.g0).replace('.', ',') : '')}" aria-label="g0 ${esc(o.name)}"></td>
      ${cols.map(([d, k], ci) => `<td class="c" style="padding:2px"><input class="cell" style="width:50px" data-grid="gb" data-rc="${ri},${ci + 1}" data-gb="${o.id}|${k}|${d.id}" value="${esc((row[k] || {})[d.id] != null ? String(row[k][d.id]).replace('.', ',') : '')}" aria-label="${k} ${esc(d.name)} ${esc(o.name)}"></td>`).join('')}</tr>`; }).join('')}
  </tbody></table></div><div class="row" style="margin-top:10px"><button class="btn primary" data-act="gb-calc">Рассчитать</button></div></div>
  <div class="card" id="gbRes"><h2>Уровни подготовленности по дисциплинам</h2>${gbResultHtml(comp)}</div>` : c ? '<div class="card empty">Нет объектов в выбранной группе.</div>' : ''}`;
});
function gbResultHtml(comp) {
  if (!comp) return '';
  const ok = comp.res.filter(x => x.ok), bad = comp.res.filter(x => !x.ok);
  return `${ok.length ? `<div class="table-wrap"><table><thead><tr><th>Обучающийся</th><th class="num">g₀</th>${comp.c.disciplines.map(d => `<th class="num">${esc(d.name)}</th>`).join('')}</tr></thead><tbody>
    ${ok.map(x => `<tr><td>${esc(x.o.name)}</td><td class="num">${fmt((gb().rows[x.o.id] || {}).g0, 3)}</td>${x.r.map(r => `<td class="num">${fmt(r.g, 3)}</td>`).join('')}</tr>`).join('')}</tbody></table></div>
    ${lineChart({ labels: comp.c.disciplines.map(d => d.name), yDigits: 3, series: ok.slice(0, 10).map((x, i) => ({ name: x.o.name, color: PALETTE[i % PALETTE.length], values: x.r.map(r => r.g) })) })}
    ${ok.length > 10 ? '<p class="hint">На графике — первые 10 обучающихся.</p>' : ''}
    <div class="row" style="margin-top:10px"><button class="btn" data-act="gb-save">Сохранить как утверждённые расчёты (${ok.length})</button></div>` : '<p class="muted">Нет обучающихся с полными данными.</p>'}
    ${bad.length ? `<div class="note warn">Не рассчитаны (неполные или неверные данные): ${bad.map(x => esc(x.o.name) + ' — ' + esc(x.err)).join('; ')}</div>` : ''}`;
}
document.addEventListener('change', e => {
  const k = e.target.dataset && e.target.dataset.gbk; if (!k) return;
  const G = gb(); let v = e.target.value;
  if (k === 'p') { v = parseNum(v); if (!isNumV(v) || v <= 0) { toast('p должно быть положительным числом', true); return; } }
  if (k === 'mode' && v !== G.mode) G.p = v === '21' ? METHOD.G_MODEL.p21 : METHOD.G_MODEL.p24;
  G[k] = v; persist(); render();
});
document.addEventListener('input', e => {
  const d = e.target.dataset; if (!d || !d.gb) return;
  const [obj, k, did] = d.gb.split('|'); const G = gb(); if (!G.rows[obj]) G.rows[obj] = {};
  const raw = e.target.value.trim(), v = raw === '' ? undefined : parseNum(raw);
  e.target.classList.toggle('invalid', raw !== '' && !(isNumV(v) && v >= 0 && (k === 'g0' ? v <= 1 : v < 1)));
  if (k === 'g0') G.rows[obj].g0 = v; else { if (!G.rows[obj][k]) G.rows[obj][k] = {}; if (v === undefined) delete G.rows[obj][k][did]; else G.rows[obj][k][did] = v; }
  persistSoon();
});
actions['gb-takeg0'] = el => { const a = latest(state.assessments.filter(x => x.type === 'g0' && x.status === 'final' && x.objectId === el.dataset.id)); if (!a) return; const G = gb(); if (!G.rows[el.dataset.id]) G.rows[el.dataset.id] = {}; G.rows[el.dataset.id].g0 = a.result.g0; persist(); render(); };
actions['gb-calc'] = () => { persist(); $('#gbRes').innerHTML = '<h2>Уровни подготовленности по дисциплинам</h2>' + gbResultHtml(gbCompute()); };
actions['gb-save'] = async () => {
```

```
  const comp = gbCompute(); const ok = comp.res.filter(x => x.ok); const G = gb();
  if (!await confirmDlg('Сохранить расчёты?', `<p>Для ${ok.length} обучающихся будут созданы утверждённые записи «${esc(typeTitle('trajectory'))}» по плану «${esc(comp.c.name)}».</p>`, 'Сохранить')) return;
  for (const x of ok) {
    const a = Model.newAssessment(state, 'trajectory', x.o.id);
    Object.assign(a.input, { curriculumId: comp.c.id, g0: G.rows[x.o.id].g0, mode: G.mode, p: Number(G.p), abilities: x.ab });
    a.snapshot = Model.snapshot(state, 'trajectory', a.meta, a.input);
    const r = Model.compute(a); if (!r.ok) continue;
    a.result = r.result; a.status = 'final'; a.finalizedAt = Store.nowIso(); state.assessments.push(a);
  }
  persist(); toast('Расчёты сохранены'); location.hash = '#/history';
};

/* ---------- Чувствительность модели ---------- */
route(/^\/sensitivity$/, () => {
  const S = ui.sens || (ui.sens = { g0: '0,5', p: '0,005', mode: '21', c: '0,5', m: '0,7; 0,8; 0,9', shares: '0; 50; 100', from: '50', to: '200', step: '10' });
  let out = '', err = '';
  try {
    const mValues = parseList(S.m), shares = parseList(S.shares).map(x => x / 100);
    if (!mValues.length || mValues.some(v => !isNumV(v) || v < 0 || v > 1)) throw new Error('Значения m: числа 0…1 через «;»');
    if (!shares.length || shares.some(v => !isNumV(v) || v < 0)) throw new Error('Доли самостоятельной работы: числа ≥ 0 (в %) через «;»');
    const r = Calc.sensitivity({ g0: parseNum(S.g0), p: parseNum(S.p), mode: S.mode, c: parseNum(S.c), mValues, shares, T1from: parseNum(S.from), T1to: parseNum(S.to), T1step: parseNum(S.step) });
    const lab = r.T1s.map(String);
    out = `<div class="card"><h2>g от объёма дисциплины T1 при разных m (T2 = ${fmt(shares[0] * 100, 0)}% от T1)</h2><p class="src">Аналог графиков методики.</p>${lineChart({ labels: lab, yDigits: 3, series: r.byM.map((s, i) => ({ name: 'm = ' + fmtFull(s.m), color: PALETTE[i % PALETTE.length], values: s.values })) })}</div>
    <div class="card"><h2>g от T1 при разной доле самостоятельной работы (m = ${fmtFull(r.mMid)})</h2><p class="src">Аналог графиков методики.</p>${lineChart({ labels: lab, yDigits: 3, series: r.byShare.map((s, i) => ({ name: 'T2 = ' + fmt(s.share * 100, 0) + '% от T1', color: PALETTE[i % PALETTE.length], values: s.values })) })}</div>
    ${r.ratio.length ? `<div class="card"><h2>Отношение уровней подготовленности g(T2) / g(T2 = 0)</h2><p class="src">Аналог графиков методики; максимум отношения показывает объём дисциплины с наибольшим эффектом самостоятельной работы.</p>${lineChart({ labels: lab, yDigits: 3, series: r.ratio.map((s, i) => ({ name: 'T2 = ' + fmt(s.share * 100, 0) + '%', color: PALETTE[i % PALETTE.length], values: s.values })) })}
    <p>${r.ratio.map(s => { const mx = Math.max(...s.values); return `T2 = ${fmt(s.share * 100, 0)}%: максимум ${fmt(mx, 3)} при T1 = ${r.T1s[s.values.indexOf(mx)]} ч`; }).join('; ')}.</p></div>` : ''}
    <div class="card"><h3>Таблица значений</h3><div class="table-wrap"><table><thead><tr><th class="num">T1, ч</th>${r.byM.map(s => `<th class="num">m = ${fmtFull(s.m)}</th>`).join('')}${r.byShare.map(s => `<th class="num">T2 = ${fmt(s.share * 100, 0)}%</th>`).join('')}</tr></thead><tbody>${r.T1s.map((T, i) => `<tr><td class="num">${T}</td>${r.byM.map(s => `<td class="num">${fmt(s.values[i], 4)}</td>`).join('')}${r.byShare.map(s => `<td class="num">${fmt(s.values[i], 4)}</td>`).join('')}</tr>`).join('')}</tbody></table></div></div>`;
  } catch (e) { err = e.message; }
  const f = (k, l, hint = '') => `<div class="field"><label class="f" for="sn-${k}">${l}</label><input type="text" id="sn-${k}" data-sn="${k}" value="${esc(S[k])}">${hint ? `<div class="hint">${hint}</div>` : ''}</div>`;
  return `<h1>Чувствительность g</h1><p class="muted">Как уровень подготовленности g зависит от объёма дисциплины, методического мастерства преподавателя и доли самостоятельной работы.</p>
  <div class="card no-print"><div class="grid g4">${f('g0', term('g₀'))}${f('p', term('p') + ', ч⁻¹')}
    <div class="field"><label class="f" for="sn-mode">Формула</label><select id="sn-mode" data-sn="mode"><option value="21" ${S.mode === '21' ? 'selected' : ''}>(21), c1 = c2 = c</option><option value="24" ${S.mode === '24' ? 'selected' : ''}>(24)</option></select></div>
    ${f('c', term('c') + ' (способности)', 'слабые 0,5; сильные 0,9; средние 0,7')}${f('m', term('m') + ' — значения через «;»')}${f('shares', 'T2, % от T1 — через «;»')}${f('from', 'T1 от, ч')}${f('to', 'T1 до, ч')}${f('step', 'шаг, ч')}</div>
  <div class="note">По умолчанию — параметры методики (g₀ = 0,5; p = 0,005 ч⁻¹). Графики методики визуально соответствуют p ≈ 0,002 — это расхождение в самом методикае; при необходимости измените p.</div></div>
  ${err ? `<div class="note err">${esc(err)}</div>` : out}`;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.sn; if (!k) return; ui.sens[k] = e.target.value; render(); });

/* ---------- Оптимизация учебного плана ---------- */
route(/^\/optimize$/, () => {
  const O = ui.opt || (ui.opt = { curriculumId: '', result: null });
  const c = state.curricula.find(x => x.id === O.curriculumId);
```

```
  const cfg = c ? (c.opt || (c.opt = { limits: {}, useGamma: true, gamma: 3, students: 'typical', g0: 0.7, c: 0.5, iterations: 2000, seed: 1 })) : null;
  const sems = c ? [...new Set(c.disciplines.map(d => d.semester).filter(Number.isInteger))].sort((a, b) => a - b) : [];
  const G = gb(); const gbOk = c && G.curriculumId === c.id && G.mode === '24';
  return `<h1>Оптимизация учебного плана</h1><p class="muted">Задача: распределить время занятий с преподавателем T1 и самостоятельной работы T2 по дисциплинам так, чтобы максимизировать уровень подготовленности обучающихся по выпускающим дисциплинам (по которым предусмотрена ГИА), при ограничениях (26) T ≥ T<sub>min</sub>, (27) Σ(T1 + T2) по семестру ≤ T<sup>(k)</sup> и, при необходимости, (28) (T1 + T2)/T2 = γ. Метод случайного поиска; расчёт g — по формуле (24).</p>
  <div class="card"><div class="field"><label class="f" for="op-c">Учебный план</label><select id="op-c" data-op="curriculumId"><option value="">— выберите —</option>${state.curricula.map(x => `<option value="${x.id}" ${x.id === O.curriculumId ? 'selected' : ''}>${esc(x.name)}</option>`).join('')}</select></div></div>
  ${c ? `<div class="card"><h2>1. Дисциплины: семестр, минимальные объёмы, выпускающие</h2><div class="table-wrap"><table><thead><tr><th>№</th><th>Дисциплина</th><th class="num">T1</th><th class="num">T2</th><th class="c">Семестр</th><th class="c">T1<sub>min</sub></th><th class="c">T2<sub>min</sub></th><th class="c">Выпускающая (ГИА)</th></tr></thead><tbody>
    ${c.disciplines.map((d, i) => `<tr><td>${i + 1}</td><td>${esc(d.name)}</td><td class="num">${fmtFull(d.T1)}</td><td class="num">${fmtFull(d.T2)}</td><td class="c"><input class="cell" data-opd="${i}|semester" value="${esc(d.semester ?? '')}" aria-label="Семестр"></td><td class="c"><input class="cell" data-opd="${i}|T1min" value="${esc(d.T1min ?? '')}" aria-label="T1 min"></td><td class="c"><input class="cell" data-opd="${i}|T2min" value="${esc(d.T2min ?? '')}" aria-label="T2 min"></td><td class="c"><input type="checkbox" data-opd="${i}|final" ${d.final ? 'checked' : ''} aria-label="Выпускающая"></td></tr>`).join('')}
  </tbody></table></div><p class="hint">Изменения сохраняются в учебном плане. Дисциплины, читаемые в нескольких семестрах, считаются разными.</p></div>
  <div class="card"><h2>2. Ограничения и обучающиеся</h2>
    <h3>Время учебных занятий по семестрам T<sup>(k)</sup>, ч</h3>${sems.length ? `<div class="row">${sems.map(k => `<label>Семестр ${k}: <input type="text" class="cell" style="width:80px" data-opl="${k}" value="${esc(cfg.limits[k] ?? '')}"></label>`).join('')}</div>` : '<p class="muted">Укажите семестры дисциплин.</p>'}
    <div class="grid g4" style="margin-top:12px">
      <div class="field"><label><input type="checkbox" data-opc="useGamma" ${cfg.useGamma ? 'checked' : ''}> Соотношение (28): (T1 + T2)/T2 = γ</label></div>
      <div class="field"><label class="f">γ (в среднем 3, методика)</label><input type="text" data-opc="gamma" value="${esc(String(cfg.gamma).replace('.', ','))}"></div>
      <div class="field"><label class="f">Итераций</label><input type="text" data-opc="iterations" value="${esc(cfg.iterations)}"></div>
      <div class="field"><label class="f">Зерно случайного поиска</label><input type="text" data-opc="seed" value="${esc(cfg.seed)}"><div class="hint">Одинаковое зерно — одинаковый результат.</div></div>
    </div>
    <div class="grid g3"><div class="field"><label class="f">Обучающиеся</label><select data-opc="students"><option value="typical" ${cfg.students === 'typical' ? 'selected' : ''}>типовой обучающийся (g₀, c)</option><option value="group" ${cfg.students === 'group' ? 'selected' : ''} ${gbOk ? '' : 'disabled'}>из «Анализ → Прогноз по группе» (формула 24)</option></select></div>
    <div class="field"><label class="f">g₀ типового</label><input type="text" data-opc="g0" value="${esc(String(cfg.g0).replace('.', ','))}"></div><div class="field"><label class="f">c типового (для всех дисциплин)</label><input type="text" data-opc="c" value="${esc(String(cfg.c).replace('.', ','))}"></div></div>
    <p class="hint">p = ${fmtFull(METHOD.G_MODEL.p24)} ч⁻¹ (формула 24).</p>
    <button class="btn primary" data-act="op-run">Выполнить оптимизацию</button></div>
  <div id="opRes">${O.result && O.result.cid === c.id ? optResultHtml(c, O.result) : ''}</div>` : ''}`;
});
function optStudents(c, cfg) {
  if (cfg.students === 'group') {
    const comp = gbCompute(); if (!comp) return [];
    return comp.res.filter(x => x.ok).map(x => { const row = gb().rows[x.o.id]; const cs = c.disciplines.map(d => row.c[d.id]); return { g0: row.g0, c: cs.reduce((s, v) => s + v, 0) / cs.length, name: x.o.name }; });
  }
  return [{ g0: cfg.g0, c: cfg.c }];
}
function optResultHtml(c, R) {
  if (R.error) return `<div class="note err">${esc(R.error)}</div>`;
  const r = R.r;
  return `<div class="card"><h2>3. Результат</h2>
    <div class="grid g4">${kpi('Существующий план: g по выпускающим', fmt(r.baseScore, 4), 'средний по обучающимся')}${kpi('Оптимальный план', fmt(r.bestScore, 4), `прирост ${(r.bestScore - r.baseScore >= 0 ? '+' : '') + fmt(r.bestScore - r.baseScore, 4)}`)}${kpi('Итераций', r.iterations, 'зерно ' + r.seed)}${kpi('Улучшений найдено', r.trace.length, '')}</div>
    ${r.semUse.some(s => s.used > s.limit + 1e-9) ? `<div class="note warn">Существующий план превышает лимит: ${r.semUse.filter(s => s.used > s.limit + 1e-9).map(s => `семестр ${s.semester} — ${fmtFull(s.used)} из ${s.limit} ч`).join('; ')}.</div>` : ''}
```

```
    ${R.studentsNote ? `<p class="src">${esc(R.studentsNote)}</p>` : ''}
    <div class="table-wrap"><table><thead><tr><th>Дисциплина</th><th class="c">Семестр</th><th class="num">Сущ. T1</th><th class="num">Сущ. T2</th><th class="num">Опт. T1</th><th class="num">Опт. T2</th><th class="num">Δ (T1 + T2)</th></tr></thead><tbody>
    ${c.disciplines.map((d, i) => `<tr><td>${esc(d.name)}${d.final ? ' <span class="badge b-info">ГИА</span>' : ''}</td><td class="c">${d.semester}</td><td class="num">${fmtFull(r.baseT[i][0])}</td><td class="num">${fmtFull(r.baseT[i][1])}</td><td class="num"><b>${fmt(r.best[i][0], 1)}</b></td><td class="num"><b>${fmt(r.best[i][1], 1)}</b></td><td class="num">${fmt(r.best[i][0] + r.best[i][1] - r.baseT[i][0] - r.baseT[i][1], 1)}</td></tr>`).join('')}
    </tbody></table></div>
    ${r.trace.length > 1 ? lineChart({ labels: r.trace.map(t => String(t.it)), yDigits: 4, series: [{ name: 'Лучшее значение целевой функции (по номеру итерации)', color: '#007AFF', values: r.trace.map(t => t.score) }] }) : ''}
    <p class="src">Метод случайного поиска не гарантирует оптимум и даёт нижнюю границу максимума целевой функции. Значения округлены до 0,1 ч только при отображении.</p>
    <div class="row"><button class="btn" data-act="op-save" data-id="${c.id}">Создать учебный план из оптимального варианта</button></div></div>`;
}
document.addEventListener('change', e => {
  const d = e.target.dataset || {};
  if (d.op) { ui.opt[d.op] = e.target.value; ui.opt.result = null; render(); return; }
  const c = ui.opt && state.curricula.find(x => x.id === ui.opt.curriculumId); if (!c) return;
  if (d.opd) { const [i, k] = d.opd.split('|'); const dd = c.disciplines[+i];
    if (k === 'final') dd.final = e.target.checked;
    else { const v = e.target.value.trim() === '' ? undefined : parseNum(e.target.value); const ok = v === undefined || (isNumV(v) && v >= 0 && (k !== 'semester' || Number.isInteger(v) && v >= 1)); e.target.classList.toggle('invalid', !ok); if (!ok) return; dd[k] = v; if (k === 'semester') { persist(); render(); return; } }
    persist(); return; }
  if (d.opl) { const v = parseNum(e.target.value); e.target.classList.toggle('invalid', !(isNumV(v) && v > 0)); c.opt.limits[d.opl] = isNumV(v) ? v : undefined; persist(); return; }
  if (d.opc) { const k = d.opc; let v = e.target.type === 'checkbox' ? e.target.checked : e.target.value;
    if (['gamma', 'g0', 'c', 'iterations', 'seed'].includes(k)) { v = parseNum(v); if (!isNumV(v)) { e.target.classList.add('invalid'); return; } e.target.classList.remove('invalid'); }
    c.opt[k] = v; persist(); }
});
actions['op-run'] = async () => {
  const c = state.curricula.find(x => x.id === ui.opt.curriculumId); const cfg = c.opt;
  const students = optStudents(c, cfg);
  const box = $('#opRes'); box.innerHTML = '<div class="card"><p>Выполняется расчёт…</p></div>';
  await new Promise(r => setTimeout(r, 30));
  let R;
  try {
    if (!students.length) throw new Calc.CalcError('Нет обучающихся с полными данными');
    if (students.some(s => !(s.g0 >= 0 && s.g0 <= 1) || !(s.c >= 0 && s.c < 1))) throw new Calc.CalcError('g₀ должно быть 0…1, c — 0…1 (не включая 1)');
    const r = Calc.optimizePlan({ disciplines: c.disciplines, limits: cfg.limits, gamma: cfg.useGamma ? cfg.gamma : null, students, p: METHOD.G_MODEL.p24, iterations: Math.round(cfg.iterations), seed: Math.round(cfg.seed) });
    R = { cid: c.id, r, studentsNote: cfg.students === 'group' ? `Обучающиеся: ${students.length} (для каждого способность c — среднее по дисциплинам из прогноза по группе).` : `Типовой обучающийся: g₀ = ${fmtFull(cfg.g0)}, c = ${fmtFull(cfg.c)}.` };
  } catch (e) { R = { cid: c.id, error: e instanceof Calc.CalcError ? e.message : 'Ошибка расчёта: ' + e.message }; }
  ui.opt.result = R; box.innerHTML = optResultHtml(c, R);
};
actions['op-save'] = el => {
  const c = state.curricula.find(x => x.id === el.dataset.id), r = ui.opt.result.r;
  const ids = c.disciplines.map(() => Store.uid()); const map = new Map(c.disciplines.map((d, i) => [d.id, ids[i]]));
  const n = { id: Store.uid(), name: c.name + ' — оптимальный вариант', createdAt: Store.nowIso(), updatedAt: Store.nowIso(),
    note: `Получен оптимизацией из плана «${c.name}»: зерно ${r.seed}, итераций ${r.iterations}, целевая функция ${fmt(r.baseScore, 4)} → ${fmt(r.bestScore, 4)}.`,
    disciplines: c.disciplines.map((d, i) => ({ ...Model.clone(d), id: ids[i], T1: Calc.roundTo(r.best[i][0], 1), T2: Calc.roundTo(r.best[i][1], 1), preds: (d.preds || []).map(p => map.get(p)) })), opt: Model.clone(c.opt) };
  state.curricula.push(n); persist(); toast('Создан учебный план с оптимальными объёмами'); location.hash = '#/curriculum/' + n.id;
};

/* ================================================================
   МОДУЛЬ 12. ПЕЧАТНЫЕ БЛАНКИ, ОТЧЁТЫ О ПРАКТИКЕ, ДУБЛИ ОБЪЕКТОВ
```

```
   ========================================================================== */
const line = (w = 260) => `<span style="display:inline-block;min-width:${w}px;border-bottom:1px solid #000"> </span>`;
const filled = (v, w = 260) => v ? `<span style="display:inline-block;min-width:${w}px;border-bottom:1px solid #000">${esc(v)}</span>` : line(w);

/* ---------- Печатные бланки ---------- */
function reviewBlank(period, o) {
  const p2 = period === 2;
  const vpk = METHOD.COEF_SETS['src-vpk16'].items, psk = METHOD.COEF_SETS['src-psk5'].items;
  const tbl = (items, title, sumTitle) => `<p><b>${title}</b></p><p style="font-size:12px"><u>Оценку проводить по следующей схеме:</u> 5 — высокий уровень сформированности; 4 — выше среднего; 3 — средний; 2 — ниже среднего; 1 — низкий.</p>
    <table class="bl"><thead><tr><th>№ п/п</th><th>Показатели сформированности</th><th>Оценка</th><th>Кзнк</th><th>Псфк</th></tr></thead><tbody>
    ${items.map((c, i) => `<tr><td class="c">${i + 1}</td><td>${esc(c.text)}</td><td style="width:60px"></td><td class="c">${fmtK(c.k)}</td><td style="width:70px"></td></tr>`).join('')}
    <tr><td></td><td><b>${sumTitle}</b></td><td></td><td></td><td></td><td></td></tr></tbody></table>`;
  return `<div class="blank">
  <p style="text-align:right;font-size:12px"><b>Высылается</b> в институт <b>после ${p2 ? '2 лет' : '1 года'} службы к 1 декабря.</b><br>В случаях: <b>перевода</b> — указывать, в какую в/ч убыл выпускник; <b>увольнения</b> — по каким основаниям уволен.</p>
  <p style="text-align:center"><b>СЛУЖЕБНЫЙ ОТЗЫВ</b><br>на выпускника (по окончании ${p2 ? 'двух лет' : 'первого года'} службы)</p>
  <p>${filled(o && o.year, 80)} года выпуска, факультета ${line(120)}, роты ${line(50)} взвода ${line(50)}</p>
  <p>Воинское звание ${line(200)}</p><p>Фамилия, имя, отчество ${filled(o && o.name, 360)}</p>
  <p>Первичная должность по прибытии в часть (подразделение) ${line(260)}</p><p>Занимаемая штатная должность в настоящее время ${line(300)}</p>
  <p>Место службы (округ, город, номер войсковой части) ${line(320)}</p>
  ${tbl(vpk, 'I. Уровень сформированности военно-профессиональных компетенций (ВПК)', 'Суммарный показатель сформированности ВПК')}
  ${tbl(psk, 'Уровень сформированности профессионально-специализированных компетенций (ПСК)', 'Суммарный показатель сформированности ПСК')}
  <table class="bl"><tbody><tr><td><b><i>ИТОГОВАЯ ОЦЕНКА СФОРМИРОВАННОСТИ КОМПЕТЕНЦИЙ</i></b></td><td style="width:140px"></td></tr></tbody></table>
  <p class="src">Колонку «Оценка» заполняет командир; Кзнк и Псфк — учебно-методический отдел института.</p>
  <p><b>II. Личностные качества</b></p><p style="font-size:12px"><u>Оценку качеств проводить по схеме</u> (нужное подчеркнуть): 5 — высокий уровень развития; 4 — выше среднего; 3 — удовлетворительный (средний); 2 — обеспечивающий минимальную деятельность; 1 — качества не развиты или отсутствуют.</p>
  <table class="bl"><tbody>${METHOD.PERSONAL_QUALITIES.map(q => `<tr><td><b>${esc(q.name)}:</b> ${esc(q.text)}</td>${[1, 2, 3, 4, 5].map(g => `<td class="c" style="width:28px">${g}</td>`).join('')}</tr>`).join('')}</tbody></table>
  <p><b>III. Уровень личной подготовки молодого офицера</b> (оценивается: отл., хор., удовл., неуд.)</p>
  <table class="bl"><thead><tr>${METHOD.PERSONAL_TRAINING[p2 ? 2 : 1].map(s => `<th>${esc(s)} подготовка</th>`).join('')}</tr></thead><tbody><tr>${METHOD.PERSONAL_TRAINING[1].map(() => '<td style="height:28px"></td>').join('')}</tr></tbody></table>
  ${['Оценка подразделения (влияние на неё деятельности офицера)', 'Количество поощрений (кем, сколько раз?)', 'Наличие дисциплинарных проступков, взысканий (в т.ч. снятых)', 'Укажите, пожалуйста, подробнее характерные недостатки, которые имеются в подготовке офицера. Каковы, на ваш взгляд, их причины?', 'Есть ли у Вас рекомендации по улучшению подготовки выпускников института?'].map((q, i) => `<p>${i + 1}. ${esc(q)}<br>${line(640)}<br>${line(640)}</p>`).join('')}
  <p><b>IV. Общая оценка по видам профессиональной деятельности офицера за год</b> (оценивается: отл., хор., удовл., неуд.)</p>
  <table class="bl"><tbody>${METHOD.ACTIVITY_KINDS.map((s, i) => `<tr><td>${i + 1}. ${esc(s)}</td><td style="width:120px"></td></tr>`).join('')}</tbody></table>
  <p><b>V. Выводы по отзыву</b> (нужное подчеркнуть): ${METHOD.CONCLUSIONS.map(esc).join(' · ')}</p>
  <p>С отзывом ознакомлен ${line(260)}</p>
  ${METHOD.REVIEW_SIGNERS.map(([, l]) => `<p>${esc(l.replace(' («СОГЛАСЕН»)', ''))}${l.includes('СОГЛАСЕН') ? ' — СОГЛАСЕН:' : ''} ${line(320)}<br><span style="font-size:11px;margin-left:220px">(звание, подпись, фамилия)</span></p>`).join('')}
  <p>М. П.     «${line(30)}» ${line(120)} 20${line(30)} года</p>
  <p style="text-align:center;margin-top:18px"><b>КОЛОНКА ВЫПУСКНИКА</b><br>(заполняется выпускником)</p>
  ${METHOD.GRADUATE_COLUMN.map((q, i) => `<p>${i + 1}. ${esc(q.text)}${q.options ? '<br>' + q.options.map((x, k) => `${k + 1}) ${esc(x)}`).join('; ') : ''}<br>${line(640)}${q.options ? '' : '<br>' + line(640)}</p>`).join('')}
  <p style="text-align:center"><i>Спасибо за искренние ответы, успехов Вам в дальнейшей службе!</i></p>
</div>`;
}
function anketaBlank(o) {
  const items = [...METHOD.PRACTICE_OKPK, ...METHOD.COEF_SETS['src-vpk13'].items];
```

```
  return `<div class="blank"><p style="text-align:center"><b>Анкета<br>экспертной оценки уровня достижения компетенций курсантом — стажёром командира взвода в период прохождения войсковой практики</b></p>
  <p>курсант ${filled(o && o.name, 420)}</p><p style="font-size:12px">(необходимо в строке напротив компетенции обвести оценку от 1 до 5)</p>
  <table class="bl"><thead><tr><th>Индекс компетенции</th><th>Формулировка компетенции</th><th colspan="5">Показатель</th></tr></thead><tbody>
  ${items.map(c => `<tr><td>${esc(c.id)}</td><td>${esc(c.text)}</td>${[1, 2, 3, 4, 5].map(g => `<td class="c" style="width:24px">${g}</td>`).join('')}</tr>`).join('')}</tbody></table>
  <p style="margin-top:16px">Должность ${line(420)}</p><p>${line(200)} ${line(160)} ${line(200)}<br><span style="font-size:11px">звание                         подпись                     ФИО</span></p></div>`;
}
route(/^\/blanks$/, () => {
  const B = ui.blanks || (ui.blanks = { kind: 'review1', fill: 'none', obj: '', group: '' });
  const objs = [...state.objects].sort((a, b) => a.name.localeCompare(b.name, 'ru'));
  let targets = [null];
  if (B.fill === 'obj') targets = objs.filter(o => o.id === B.obj);
  if (B.fill === 'group') targets = objs.filter(o => o.group === B.group);
  const make = o => B.kind === 'anketa' ? anketaBlank(o) : reviewBlank(B.kind === 'review2' ? 2 : 1, o);
  return `<h1>Бланки для печати</h1><p class="muted no-print">Пустые формы для заполнения в войсках: служебный отзыв и анкета экспертной оценки на войсковой практике. Кзнк в отзыв вписываются автоматически. Заполненные бланки затем вводятся через «Ввод по группе» или «Новая оценка».</p>
  <div class="card no-print"><div class="grid g4">
    <div class="field"><label class="f" for="bl-k">Бланк</label><select id="bl-k" data-bl="kind"><option value="review1" ${B.kind === 'review1' ? 'selected' : ''}>Служебный отзыв — 1 год службы</option><option value="review2" ${B.kind === 'review2' ? 'selected' : ''}>Служебный отзыв — 2 года службы</option><option value="anketa" ${B.kind === 'anketa' ? 'selected' : ''}>Анкета войсковой практики</option></select></div>
    <div class="field"><label class="f" for="bl-f">Заполнить ФИО</label><select id="bl-f" data-bl="fill"><option value="none" ${B.fill === 'none' ? 'selected' : ''}>нет (пустой бланк)</option><option value="obj" ${B.fill === 'obj' ? 'selected' : ''}>для одного объекта</option><option value="group" ${B.fill === 'group' ? 'selected' : ''}>для всей группы (по бланку на человека)</option></select></div>
    ${B.fill === 'obj' ? `<div class="field"><label class="f" for="bl-o">Объект</label><select id="bl-o" data-bl="obj"><option value="">—</option>${objs.map(o => `<option value="${o.id}" ${o.id === B.obj ? 'selected' : ''}>${esc(o.name)}</option>`).join('')}</select></div>` : ''}
    ${B.fill === 'group' ? `<div class="field"><label class="f" for="bl-g">Группа</label><select id="bl-g" data-bl="group"><option value="">—</option>${groupList().map(g => `<option ${B.group === g ? 'selected' : ''}>${esc(g)}</option>`).join('')}</select></div>` : ''}
  </div><button class="btn primary" data-act="print" ${targets.length ? '' : 'disabled'}>Печать / PDF (${targets.length} шт.)</button></div>
  ${targets.length ? targets.map((o, i) => `<div class="card report${i < targets.length - 1 ? ' page-break' : ''}">${make(o)}</div>`).join('') : '<div class="card empty">Выберите объект или группу.</div>'}`;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.bl; if (!k) return; ui.blanks[k] = e.target.value; render(); });

/* ---------- Отчёт руководителя об итогах войсковой практики ---------- */
const PREPORT_ITEMS = [
  ['where', 'В каком соединении, воинской части войск национальной гвардии и в каких воинских должностях выполняли обязанности курсанты'],
  ['program', 'Как выполнена программа войсковой практики'],
  ['readiness', 'Подготовленность курсантов, их командирские качества и методические навыки'],
  ['participation', 'Участие курсантов в несении боевой службы, проведении занятий по боевой подготовке, в информационно-воспитательной и спортивно-массовой работе (общие показатели)'],
  ['prosCons', 'Положительные стороны и недостатки в подготовке курсантов, выявленные при проведении войсковой практики'],
  ['discipline', 'Состояние воинской дисциплины'],
  ['officers', 'Работа офицеров с курсантами в период войсковой практики'],
  ['personal', 'Личное участие в обучении и воспитании курсантов в период войсковой практики'],
  ['proposals', 'Предложения по дальнейшему улучшению войсковой практики и внесению дополнений в программу войсковой практики курсантов'],
  ['examples', 'Наиболее характерные примеры из практики работы офицеров по организации боевой службы, обучению и воспитанию личного состава, которые целесообразно использовать в военном институте'],
  ['graduates', 'Анализ работы выпускников военного института, положительные стороны в их подготовке, трудности, конкретные предложения по изменению программ учебных дисциплин и улучшению подготовки курсантов']
];
const PREPORT_DOCS = ['Отзывы непосредственных руководителей войсковой практики, утверждённые командиром воинской части', 'Заполненные бланки «Анкета экспертной оценки уровня достижения компетенций…»', 'Обобщённые выводы и предложения командира воинской части', 'Выписки из приказов командира воинской части об организации и итогах войсковой практики', 'Журналы учёта и контроля руководителя и помощника руководителя войсковой практики'];
```

```
route(/^\/preports$/, () => `<h1>Отчёты о практике</h1><p class="muted">Отчёт руководителя практики от военного института, представляемый в учебный отдел в срок не более 2 рабочих дней после прибытия. Краткая характеристика курсантов дополняется результатами расчёта по анкетам.</p>
  <div class="row no-print" style="margin-bottom:12px"><button class="btn primary" data-act="pr-new">+ Новый отчёт</button></div>
  <div class="card">${(state.practiceReports || []).length ? `<table><thead><tr><th>Группа</th><th>Курс</th><th>Дата</th><th>Руководитель</th></tr></thead><tbody>${state.practiceReports.map(r => `<tr class="click" data-act="go" data-href="#/preport/${r.id}"><td>${esc(r.group || '—')}</td><td>${r.course}</td><td>${fmtDate(r.date)}</td><td>${esc(r.author || '—')}</td></tr>`).join('')}</tbody></table>` : '<div class="empty">Отчётов нет.</div>'}</div>`);
actions['pr-new'] = () => { if (!state.practiceReports) state.practiceReports = []; const r = { id: Store.uid(), group: groupList()[0] || '', course: 4, date: new Date().toISOString().slice(0, 10), author: '', items: {}, chars: {}, docs: {}, createdAt: Store.nowIso() }; state.practiceReports.push(r); persist(); location.hash = '#/preport/' + r.id; };
route(/^\/preport\/([\w-]+)$/, id => {
  const R = (state.practiceReports || []).find(x => x.id === id); if (!R) return notFound('Отчёт');
  const list = state.assessments.filter(a => a.status === 'final' && a.type === 'practice' && String(a.meta.course) === String(R.course) && (state.objects.find(o => o.id === a.objectId) || {}).group === R.group);
  const per = new Map(); list.forEach(a => { const p = per.get(a.objectId); if (!p || (a.meta.date || '') > (p.meta.date || '')) per.set(a.objectId, a); });
  const rows = [...per.values()].sort((a, b) => objName(a.objectId).localeCompare(objName(b.objectId), 'ru'));
  const ranks = Calc.rank(rows.map(a => a.result.vpk.meanPsfk));
  const st = Calc.groupStats(rows.map(a => ({ meanGrade: a.result.vpk.meanGrade, meanPsfk: a.result.vpk.meanPsfk, level: (a.result.vpk.levelBsr || {}).code, leader: a.input.leaderMark, commission: a.input.commissionMark })));
  return `<div class="row no-print" style="margin-bottom:12px"><button class="btn primary" data-act="print">Печать / PDF</button><span class="spacer"></span><button class="btn danger" data-act="pr-del" data-id="${id}">Удалить отчёт</button></div>
  <div class="report"><div class="card"><h1 style="text-align:center">ОТЧЁТ<br><span style="font-size:16px;font-weight:600">об итогах войсковой практики курсантов ${R.course} курса</span></h1>
  <div class="grid g4 no-print">
    <div class="field"><label class="f">Группа</label><select data-pr="${id}|group">${groupList().map(g => `<option ${R.group === g ? 'selected' : ''}>${esc(g)}</option>`).join('')}</select></div>
    <div class="field"><label class="f">Курс</label><select data-pr="${id}|course"><option ${R.course == 4 ? 'selected' : ''}>4</option><option ${R.course == 5 ? 'selected' : ''}>5</option></select></div>
    <div class="field"><label class="f">Дата</label><input type="date" data-pr="${id}|date" value="${esc(R.date)}"></div>
    <div class="field"><label class="f">Руководитель практики от института</label><input type="text" data-pr="${id}|author" value="${esc(R.author)}"></div></div>
  <p class="only-print">Группа: ${esc(R.group)}; дата: ${fmtDate(R.date)}.</p>
  ${PREPORT_ITEMS.map(([k, l], i) => `<div class="field"><label class="f" style="color:var(--text);font-weight:600">${i + 1}. ${esc(l)}</label><textarea class="no-print" data-pr="${id}|items|${k}">${esc(R.items[k] || '')}</textarea><p class="only-print">${esc(R.items[k] || '—')}</p></div>`).join('')}
  <h3>12. Краткая характеристика на каждого курсанта с выставлением оценки</h3>
  ${rows.length ? `<p class="muted">По утверждённым анкетам экспертной оценки (${rows.length}): средний балл группы ${fmt(st.meanGrade, 2)}, средний Псфк ${fmt(st.meanPsfk, 3)}.</p>
  <div class="table-wrap"><table><thead><tr><th>Курсант</th><th class="num">${term('bcp')}</th><th class="num">${term('Псфк')}</th><th class="num">Место</th><th>Уровень (по bcp)</th><th class="c">Оценка руководителя</th><th>Положительные и отрицательные черты</th></tr></thead><tbody>
  ${rows.map((a, i) => `<tr><td>${esc(objName(a.objectId))}</td><td class="num">${fmt(a.result.vpk.meanGrade, 2)}</td><td class="num">${fmt(a.result.vpk.meanPsfk, 3)}</td><td class="num">${ranks[i]}</td><td>${lvBadge(a.result.vpk.levelBsr, false)}</td><td class="c">${esc(a.input.leaderMark || '—')}</td><td><textarea class="no-print" style="min-height:44px" data-pr="${id}|chars|${a.objectId}">${esc(R.chars[a.objectId] || '')}</textarea><span class="only-print">${esc(R.chars[a.objectId] || '')}</span></td></tr>`).join('')}</tbody></table></div>` : '<p class="muted">Утверждённых анкет практики для этой группы и курса нет — заполните их через «Ввод по группе».</p>'}
  <h3>Вместе с отчётом представляются</h3>${PREPORT_DOCS.map((d, i) => `<p><label><input type="checkbox" data-pr="${id}|docs|${i}" ${R.docs[i] ? 'checked' : ''}> ${esc(d)}</label></p>`).join('')}
  <p style="margin-top:24px">Руководитель войсковой практики от военного института ${filled(R.author, 280)}</p></div></div>`;
});
document.addEventListener('input', e => { if (e.target.dataset && e.target.dataset.pr && e.target.tagName === 'TEXTAREA') prSet(e.target, false); });
document.addEventListener('change', e => { if (e.target.dataset && e.target.dataset.pr) prSet(e.target, true); });
function prSet(el, rerender) {
  const [id, ...path] = el.dataset.pr.split('|'); const R = state.practiceReports.find(x => x.id === id); if (!R) return;
  setPath(R, path.join('|'), el.type === 'checkbox' ? el.checked : (path[0] === 'course' ? Number(el.value) : el.value));
  persistSoon(); if (rerender && (path[0] === 'group' || path[0] === 'course')) { persist(); render(); }
}
```

```
actions['pr-del'] = async el => { if (!await confirmDlg('Удалить отчёт?', 'Текст отчёта будет удалён. Расчёты по анкетам не затрагиваются.', 'Удалить', true)) return; state.practiceReports = state.practiceReports.filter(x => x.id !== el.dataset.id); persist(); location.hash = '#/preports'; };

/* ---------- Дубли объектов оценки ---------- */
function duplicateGroups() {
  const m = new Map();
  for (const o of state.objects) { const k = normName(o.name); if (!m.has(k)) m.set(k, []); m.get(k).push(o); }
  return [...m.values()].filter(g => g.length > 1);
}
function duplicatesCard() {
  const groups = duplicateGroups();
  return `<div class="card"><h2>Поиск дублей объектов оценки</h2><p class="hint">Объекты с одинаковыми ФИО (без учёта регистра, пробелов и «ё»). Такое бывает при объединении данных с разных компьютеров или импорте. При слиянии все расчёты переносятся на выбранный основной объект, пустые сведения дополняются из дубликатов, дубликаты удаляются. Однофамильцы с одинаковыми инициалами — не дубли: проверяйте группу.</p>
  ${groups.length ? groups.map((g, gi) => `<div class="note"><div class="table-wrap"><table><thead><tr><th>Основной</th><th>Фамилия И.О.</th><th>Группа</th><th>Специальность</th><th class="num">Расчётов</th><th>Создан</th></tr></thead><tbody>${g.map((o, i) => `<tr><td class="c"><input type="radio" name="dup-${gi}" value="${o.id}" ${i === 0 ? 'checked' : ''} aria-label="Основной: ${esc(o.name)}"></td><td>${esc(o.name)}</td><td>${esc(o.group || '—')}</td><td>${esc(o.specialty || '—')}</td><td class="num">${state.assessments.filter(a => a.objectId === o.id).length}</td><td>${fmtDT(o.createdAt)}</td></tr>`).join('')}</tbody></table></div>
    <button class="btn sm" data-act="dup-merge" data-g="${gi}">Объединить</button></div>`).join('') : '<p class="muted">Дублей не найдено.</p>'}</div>`;
}
actions['dup-merge'] = async el => {
  const g = duplicateGroups()[+el.dataset.g]; if (!g) return;
  const mainId = ($(`input[name="dup-${el.dataset.g}"]:checked`) || {}).value; const main = g.find(o => o.id === mainId); const others = g.filter(o => o.id !== mainId);
  const n = state.assessments.filter(a => others.some(o => o.id === a.objectId)).length;
  if (!await confirmDlg('Объединить объекты?', `<p>Основной: <b>${esc(main.name)}</b> (${esc(main.group || 'без группы')}). Будет перенесено расчётов: ${n}; удалено объектов: ${others.length}.</p>`, 'Объединить')) return;
  for (const o of others) {
    state.assessments.forEach(a => { if (a.objectId === o.id) a.objectId = main.id; });
    (state.controlRecords || []).forEach(r => { if (r.objectId === o.id) r.objectId = main.id; });
    for (const k of ['group', 'specialty', 'year', 'notes']) if (!main[k] && o[k]) main[k] = o[k];
    for (const st of [state.batchDraft, state.gbatch]) if (st && st.rows && st.rows[o.id] && !st.rows[main.id]) st.rows[main.id] = st.rows[o.id];
  }
  state.objects = state.objects.filter(o => !others.includes(o));
  persist(); toast('Объекты объединены'); render();
};

/* ======================================================================
   МОДУЛЬ 13. БЛОК ПОЛИФУНКЦИОНАЛЬНОГО КОНТРОЛЯ: ЖУРНАЛ ФОРМ КОНТРОЛЯ,
   КАРТА ЭТАПОВ, СХЕМА СВЯЗЕЙ
   ====================================================================== */
const C = METHOD.CONTROL;
const formName = id => (C.forms.find(f => f.id === id) || { name: id || '—' }).name;
const pairName = (list, id) => (list.find(x => x[0] === id) || [id, '—'])[1];
const schemeLinks = () => (state.settings && state.settings.schemeLinks) || C.links;
const stageName = id => (C.stages.find(s => s.id === id) || { name: '—' }).name;
const markText = r => r.markType === 'grade' ? String(r.mark) : r.markType === 'score' ? fmtFull(r.mark) + ' б.' : esc(r.mark);
const recordsOf = objId => (state.controlRecords || []).filter(r => r.objectId === objId);

function formOptions(selected, kind, family) {
  const L = schemeLinks(); const used = new Set(); let html = '<option value="">— выберите —</option>';
  const grp = (title, ids) => { const list = ids.filter(id => !used.has(id)); if (!list.length) return; list.forEach(id => used.add(id)); html += `<optgroup label="${esc(title)}">${list.map(id => `<option value="${id}" ${id === selected ? 'selected' : ''}>${esc(formName(id))}</option>`).join('')}</optgroup>`; };
  if (kind && L[kind]) grp(`По схеме — вид «${pairName(C.kinds, kind)}»`, L[kind]);
  if (family && L[family]) grp(`По схеме — методы «${pairName(C.methodFamilies, family)}»`, L[family]);
  grp('Формы контроля ВПК (схема «Формы контроля»)', C.forms.filter(f => f.src === 'forms').map(f => f.id));
  grp('Формы блока контроля модели (мониторинг и диагностика)', C.forms.filter(f => f.src === 'model').map(f => f.id));
  return html;
}
```

```
const sel = (attr, list, val, empty) => `<select ${attr}>${empty ? `<option value="">${esc(empty)}</option>` : ''}${list.map(([v, t]) => `<option value="${esc(v)}" ${String(v) === String(val ?? '') ? 'selected' : ''}>${esc(t)}</option>`).join('')}</select>`;

/* Общие поля записи (одиночная запись и ведомость) */
function recordFields(r, withObject) {
  const objs = [...state.objects].sort((a, b) => a.name.localeCompare(b.name, 'ru'));
  const st = C.stages.find(s => s.id === r.stage) || C.stages[1];
  return `<div class="grid g4">
    ${withObject ? `<div class="field"><label class="f">Объект оценки *</label>${sel('data-jr="objectId"', objs.map(o => [o.id, o.name + (o.group ? ' — ' + o.group : '')]), r.objectId, '— выберите —')}</div>` : ''}
    <div class="field"><label class="f">Этап *</label>${sel('data-jr="stage" data-num="1"', C.stages.map(s => [s.id, s.name]), r.stage)}</div>
    <div class="field"><label class="f">Семестр</label>${sel('data-jr="semester" data-num="1"', st.semesters.map(x => [x, x + ' семестр']), r.semester, '—')}</div>
    <div class="field"><label class="f">Дата</label><input type="date" data-jr="date" value="${esc(r.date || '')}"></div>
    <div class="field"><label class="f">Вид контроля</label>${sel('data-jr="kind"', C.kinds, r.kind, '—')}</div>
    <div class="field"><label class="f">Тип контроля</label>${sel('data-jr="type"', C.types, r.type)}</div>
    <div class="field"><label class="f">Методы контроля</label>${sel('data-jr="methodFamily"', C.methodFamilies, r.methodFamily, '—')}</div>
    <div class="field"><label class="f">Метод</label>${sel('data-jr="method"', (C.methods[r.methodFamily] || []).map(m => [m, m]), r.method, '—')}</div>
    <div class="field" style="grid-column:span 2"><label class="f">Форма контроля *</label><select data-jr="form">${formOptions(r.form, r.kind, r.methodFamily)}</select><div class="hint">Сверху — формы, связанные по схеме с выбранным видом и методами.</div></div>
    <div class="field"><label class="f">Блок схемы модели</label>${sel('data-jr="block"', [['monitoring', 'мониторинг'], ['diagnostics', 'диагностика']], r.block)}</div>
    <div class="field"><label class="f">Дисциплина / тема</label><input type="text" data-jr="discipline" value="${esc(r.discipline || '')}"></div>
    <div class="field"><label class="f">Вид результата</label>${sel('data-jr="markType"', C.markTypes, r.markType)}</div>
  </div>
  <details ${r.comps && r.comps.length ? 'open' : ''}><summary style="cursor:pointer;color:var(--accent)">Контролируемые ВПК (${(r.comps || []).length})</summary><div class="row" style="margin-top:6px">${METHOD.COEF_SETS['src-vpk16'].items.map(c => `<label title="${esc(c.text)}" style="font-size:13px"><input type="checkbox" data-jrc="${c.id}" ${(r.comps || []).includes(c.id) ? 'checked' : ''}> ${c.id}</label>`).join('')}</div></details>`;
}
function markInput(r, attr) {
  if (r.markType === 'grade') return `<select ${attr}><option value="">—</option>${[5, 4, 3, 2].map(g => `<option ${r.mark === g ? 'selected' : ''}>${g}</option>`).join('')}</select>`;
  if (r.markType === 'pass') return `<select ${attr}><option value="">—</option>${['зачтено', 'не зачтено'].map(g => `<option ${r.mark === g ? 'selected' : ''}>${g}</option>`).join('')}</select>`;
  return `<input type="text" ${attr} value="${esc(r.mark ?? '')}">`;
}
const newRecord = () => ({ objectId: '', stage: 1, semester: 1, date: new Date().toISOString().slice(0, 10), kind: '', type: 'external', methodFamily: '', method: '', form: '', block: 'monitoring', discipline: '', markType: 'grade', mark: '', comps: [], note: '' });

/* ---------- Журнал ---------- */
route(/^\/journal$/, () => {
  const F = ui.jf || (ui.jf = { group: '', stage: '', block: '', form: '', q: '' });
  const jr = ui.jr || (ui.jr = newRecord());
  const q = F.q.toLowerCase().trim();
  const list = (state.controlRecords || []).filter(r => { const o = state.objects.find(x => x.id === r.objectId) || {};
    return (!F.group || o.group === F.group) && (F.stage === '' || String(r.stage) === F.stage) && (!F.block || r.block === F.block) && (!F.form || r.form === F.form) && (!q || (o.name || '').toLowerCase().includes(q) || (r.discipline || '').toLowerCase().includes(q)); })
    .sort((a, b) => (b.date || '').localeCompare(a.date || ''));
  const st = Calc.journalStats(list);
  return `<h1>Журнал контроля</h1><p class="muted">Блок полифункционального контроля модели: результаты мониторинга и диагностики по этапам подготовки — курсовые, контрольные, летучки, зачёты, экзамены, нормативы, ГИА и другие формы. Записи дают <b>эмпирические оценочные показатели</b> результативного блока; в расчёт уровня сформированности ВПК они не входят — правил такого пересчёта методика не содержит.</p>
  <div class="row no-print" style="margin-bottom:12px"><a class="btn" href="#/journal/sheet">Ведомость группы</a><button class="btn" data-act="jr-csv" ${list.length ? '' : 'disabled'}>CSV</button></div>
  <div class="card no-print"><h2>${jr.id ? 'Изменить запись' : 'Новая запись'}</h2>${recordFields(jr, true)}
```

```
    <div class="grid g4"><div class="field"><label class="f">Результат *</label>${markInput(jr, 'data-jr="mark"')}</div><div class="field" style="grid-column:span 3"><label class="f">Примечание</label><input type="text" data-jr="note" value="${esc(jr.note || '')}"></div></div>
    <div class="row"><button class="btn primary" data-act="jr-save">${jr.id ? 'Сохранить изменения' : 'Добавить запись'}</button>${jr.id ? '<button class="btn" data-act="jr-cancel">Отменить</button>' : ''}</div></div>
  <div class="card"><div class="grid g4 no-print">
    <div class="field"><label class="f">Группа</label>${sel('data-jf="group"', groupList().map(g => [g, g]), F.group, 'все')}</div>
    <div class="field"><label class="f">Этап</label>${sel('data-jf="stage"', C.stages.map(s => [s.id, s.name]), F.stage, 'все')}</div>
    <div class="field"><label class="f">Блок</label>${sel('data-jf="block"', [['monitoring', 'мониторинг'], ['diagnostics', 'диагностика']], F.block, 'все')}</div>
    <div class="field"><label class="f">Поиск (ФИО, дисциплина)</label><input type="search" data-jf="q" value="${esc(F.q)}"></div></div>
  <p class="muted">Записей: ${st.n}; оценок 2–5: ${st.grades}${st.grades ? `, средний балл ${fmt(st.meanGrade, 2)} (эмпирический показатель)` : ''}; зачётов: ${st.passed} из ${st.pass}.</p>
  ${list.length ? `<div class="table-wrap"><table><thead><tr><th>Дата</th><th>Курсант</th><th>Этап / сем.</th><th>Блок</th><th>Форма</th><th>Вид</th><th>Тип</th><th>Дисциплина</th><th class="c">Результат</th><th>ВПК</th><th class="no-print"></th></tr></thead><tbody>
  ${list.slice(0, 500).map(r => `<tr><td>${fmtDate(r.date)}</td><td>${esc(objName(r.objectId))}</td><td>${esc(stageName(r.stage))}${r.semester ? ', ' + r.semester : ''}</td><td>${r.block === 'monitoring' ? 'мониторинг' : 'диагностика'}</td><td>${esc(formName(r.form))}</td><td>${esc(r.kind ? pairName(C.kinds, r.kind) : '—')}</td><td>${esc(pairName(C.types, r.type))}</td><td>${esc(r.discipline || '—')}</td><td class="c"><b>${markText(r)}</b></td><td style="font-size:12px">${esc((r.comps || []).join(', '))}</td><td class="no-print" style="white-space:nowrap"><button class="btn sm" data-act="jr-edit" data-id="${r.id}">Изм.</button> <button class="btn sm danger" data-act="jr-del" data-id="${r.id}" aria-label="Удалить">×</button></td></tr>`).join('')}
  </tbody></table></div>${list.length > 500 ? '<p class="hint">Показаны первые 500 записей — уточните фильтр.</p>' : ''}` : '<div class="empty">Записей нет.</div>'}</div>`;
});
function jrOnChange(el, rec) {
  const k = el.dataset.jr; let v = el.value;
  if (el.dataset.num) v = v === '' ? null : Number(v);
  if (k === 'mark') v = rec.markType === 'grade' ? (v === '' ? '' : Number(v)) : rec.markType === 'score' ? (v.trim() === '' ? '' : parseNum(v)) : v;
  rec[k] = v;
  if (k === 'stage') { const st = C.stages.find(s => s.id === v); rec.semester = st && st.semesters.length ? st.semesters[0] : null; }
  if (k === 'methodFamily') rec.method = '';
  if (k === 'markType') rec.mark = '';
  if (k === 'form') { const f = C.forms.find(x => x.id === v); if (f) rec.block = f.block; const mt = ['ege', 'profotbor'].includes(v) ? 'score' : v === 'zachet' ? 'pass' : 'grade'; if (mt !== rec.markType) { rec.markType = mt; rec.mark = ''; } }
  return ['stage', 'methodFamily', 'markType', 'form', 'kind'].includes(k);
}
const curRec = () => location.hash.includes('sheet') ? ui.js.rec : ui.jr;
document.addEventListener('change', e => {
  const d = e.target.dataset || {};
  if (d.jr) { if (jrOnChange(e.target, curRec())) render(); return; }
  if (d.jrc) { const rec = curRec(); const set = new Set(rec.comps || []); if (e.target.checked) set.add(d.jrc); else set.delete(d.jrc); rec.comps = [...set]; return; }
  if (d.jf) { ui.jf[d.jf] = e.target.value; render(); }
});
document.addEventListener('input', e => { const d = e.target.dataset || {}; if (d.jr && e.target.tagName === 'INPUT' && e.target.type === 'text') jrOnChange(e.target, curRec()); });
actions['jr-save'] = () => {
  const r = ui.jr; const errs = Calc.validateRecord(r);
  if (errs.length) { toast('Исправьте: ' + errs.join('; '), true); return; }
  if (!state.controlRecords) state.controlRecords = [];
  if (r.id) { const i = state.controlRecords.findIndex(x => x.id === r.id); state.controlRecords[i] = { ...Model.clone(r), updatedAt: Store.nowIso() }; toast('Запись изменена'); }
  else { state.controlRecords.push({ ...Model.clone(r), id: Store.uid(), createdAt: Store.nowIso() }); toast('Запись добавлена'); }
  ui.jr = { ...newRecord(), objectId: r.objectId, stage: r.stage, semester: r.semester, kind: r.kind, type: r.type, methodFamily: r.methodFamily, method: r.method, form: r.form, block: r.block, discipline: r.discipline, markType: r.markType, date: r.date, comps: r.comps };
  persist(); render();
};
actions['jr-edit'] = el => { ui.jr = Model.clone(state.controlRecords.find(x => x.id === el.dataset.id)); render(); window.scrollTo(0, 0); };
actions['jr-cancel'] = () => { ui.jr = newRecord(); render(); };
actions['jr-del'] = async el => { if (!await confirmDlg('Удалить запись журнала?', '', 'Удалить', true)) return; state.controlRecords = state.controlRecords.filter(x => x.id !== el.dataset.id); persist(); rend
```

```
er(); };
actions['jr-csv'] = () => {
  const rows = [['Дата', 'Курсант', 'Группа', 'Этап', 'Семестр', 'Блок', 'Форма контроля', 'Вид контрол
я', 'Тип контроля', 'Методы', 'Метод', 'Дисциплина', 'Вид результата', 'Результат', 'ВПК', 'Примечани
e']];
  for (const r of state.controlRecords || []) { const o = state.objects.find(x => x.id === r.objectId) |
| {};
    rows.push([r.date, o.name, o.group || '', stageName(r.stage), r.semester ?? '', r.block === 'monitor
ing' ? 'мониторинг' : 'диагностика', formName(r.form), r.kind ? pairName(C.kinds, r.kind) : '', pairName
(C.types, r.type), r.methodFamily ? pairName(C.methodFamilies, r.methodFamily) : '', r.method || '', r.d
iscipline || '', pairName(C.markTypes, r.markType), r.markType === 'score' ? csvNum(r.mark) : r.mark,
(r.comps || []).join(', '), r.note || '']); }
  download(`журнал_контроля_${new Date().toISOString().slice(0, 10)}.csv`, toCsv(rows), 'text/csv;charse
t=utf-8');
};

/* ---------- Ведомость группы ---------- */
route(/^\/journal\/sheet$/, () => {
  const S = ui.js || (ui.js = { group: groupList()[0] || '', rec: newRecord(), marks: {} });
  const objs = state.objects.filter(o => o.group === S.group).sort((a, b) => a.name.localeCompare(b.nam
e, 'ru'));
  return `<h1>Ведомость группы</h1><p class="muted">Одна форма контроля — результаты всех курсантов груп
пы. Можно вставить столбец из Excel.</p>
  <div class="card"><div class="field" style="max-width:300px"><label class="f">Группа</label>${sel('dat
a-jsg="1"', groupList().map(g => [g, g]), S.group)}</div>${recordFields(S.rec, false)}</div>
  ${objs.length ? `<div class="card"><table><thead><tr><th>№</th><th>Курсант</th><th>Результат (${esc(pa
irName(C.markTypes, S.rec.markType))})</th></tr></thead><tbody>
  ${objs.map((o, i) => `<tr><td>${i + 1}</td><td>${esc(o.name)}</td><td><input type="text" class="cell"
style="width:120px" data-grid="js" data-rc="${i},0" data-jsm="${o.id}" value="${esc(S.marks[o.id] ??
'')}" aria-label="Результат ${esc(o.name)}"></td></tr>`).join('')}</tbody></table>
  <p class="hint">Оценка: 2–5; зачёт: «зачтено» / «не зачтено» (или «з» / «нз»); баллы: число. Пустые ст
роки не сохраняются.</p>
  <button class="btn primary" data-act="js-save">Сохранить ведомость</button></div>` : '<div class="card
empty">В группе нет объектов.</div>'}`;
});
document.addEventListener('change', e => { if (e.target.dataset && e.target.dataset.jsg) { ui.js.group =
e.target.value; ui.js.marks = {}; render(); } });
document.addEventListener('input', e => { const id = e.target.dataset && e.target.dataset.jsm; if (id) u
i.js.marks[id] = e.target.value; });
actions['js-save'] = () => {
  const S = ui.js, recs = [], bad = [];
  for (const [objId, raw] of Object.entries(S.marks)) {
    const v = String(raw).trim(); if (!v) continue;
    let mark = v;
    if (S.rec.markType === 'grade') mark = Number(v);
    if (S.rec.markType === 'pass') mark = /^(з|зач|зачтено|\+)$/i.test(v) ? 'зачтено' : /^(нз|н\/з|не за
чтено|-)$/i.test(v) ? 'не зачтено' : v;
    if (S.rec.markType === 'score') mark = parseNum(v);
    const r = { ...Model.clone(S.rec), objectId: objId, mark, id: Store.uid(), createdAt: Store.nowIso()
};
    const errs = Calc.validateRecord(r); if (errs.length) bad.push(objName(objId) + ': ' + errs.join(',
')); else recs.push(r);
  }
  if (bad.length) { toast('Исправьте: ' + bad.join('; '), true); return; }
  if (!recs.length) { toast('Нет результатов для сохранения', true); return; }
  if (!state.controlRecords) state.controlRecords = [];
  state.controlRecords.push(...recs); S.marks = {}; persist(); toast(`Сохранено записей: ${recs.length}
`); location.hash = '#/journal';
};

/* ---------- Карта этапов курсанта (по схеме модели) ---------- */
route(/^\/object\/([\w-]+)\/map$/, id => {
  const o = state.objects.find(x => x.id === id); if (!o) return objMissing();
  const recs = recordsOf(id);
  const fin = state.assessments.filter(a => a.objectId === id && a.status === 'final');
  const ap = o.applicant || {};
  const cell = (stage, block) => {
    const rs = recs.filter(r => r.stage === stage && r.block === block).sort((a, b) => (a.semester || 0)
- (b.semester || 0) || (a.date || '').localeCompare(b.date || ''));
    if (!rs.length) return '<span class="muted">—</span>';
    return rs.slice(0, 12).map(r => `<div style="font-size:12.5px">${r.semester ? `<span class="muted">
${r.semester} с.</span> ` : ''}${esc(formName(r.form))}${r.discipline ? ' (' + esc(r.discipline) + ')' :
''}: <b>${markText(r)}</b></div>`).join('') + (rs.length > 12 ? `<div class="muted" style="font-size:12p
x">ещё ${rs.length - 12}</div>` : '');
```

```
  };
  const assessCell = stage => {
    const items = [];
    if (stage === 0) fin.filter(a => a.type === 'g0').forEach(a => items.push(`<a href="#/a/${a.id}">g₀ = ${fmt(a.result.g0, 3)}</a>`));
    if (stage >= 1 && stage <= 5) fin.filter(a => a.type === 'practice' && a.meta.course === stage).forEach(a => items.push(`<a href="#/a/${a.id}">Анкета практики: bcp ${fmt(a.result.vpk.meanGrade, 2)}, Псфк ${fmt(a.result.vpk.meanPsfk, 3)}</a> ${lvBadge(a.result.vpk.levelBsr, false)}`));
    if (stage === 6) fin.filter(a => a.type === 'graduate').sort((a, b) => a.meta.period - b.meta.period).forEach(a => items.push(`<a href="#/a/${a.id}">Отзыв, ${a.meta.period} год: Псфк ВПК ${fmt(a.result.vpk.meanPsfk, 3)}</a> ${lvBadge(a.result.vpk.levelPsfk || a.result.vpk.levelBsr, false)}`));
    if (stage === 6) fin.filter(a => a.type === 'ymodel').forEach(a => items.push(`<a href="#/a/${a.id}">Y = ${fmt(a.result.Y, 3)}</a>`));
    return items.length ? items.map(x => `<div style="font-size:12.5px;margin-bottom:4px">${x}</div>`).join('') : '<span class="muted">—</span>';
  };
  const empir = stage => { const s = Calc.journalStats(recs.filter(r => r.stage === stage)); return s.n ? `записей ${s.n}${s.grades ? `; ср. балл ${fmt(s.meanGrade, 2)}` : ''}${s.pass ? `; зачтено ${s.passed}/${s.pass}` : ''}` : '<span class="muted">—</span>'; };
  const traj = fin.filter(a => a.type === 'trajectory');
  const stages = C.stages;
  return objectChrome(o, 'map') + `<div class="row no-print" style="margin-bottom:12px"><button class="btn" data-act="print">Печать / PDF</button></div>
  <h1 class="only-print">${esc(o.name)}: карта этапов подготовки</h1><p class="muted">По схеме «Модель полифункционального контроля»: этапы подготовки → блок контроля (мониторинг, диагностика) → математический блок → результативный и выходной блоки.${o.group ? ' Группа: ' + esc(o.group) + '.' : ''}</p>
  <div class="card"><div class="table-wrap"><table style="min-width:1100px"><thead><tr><th style="width:150px">Блок модели</th>${stages.map(s => `<th>${esc(s.name)}${s.semesters.length ? `<div class="muted" style="font-weight:400">${s.semesters.join(' и ')} семестры</div>` : s.id === 6 ? '<div class="muted" style="font-weight:400">1 и 2 год службы</div>' : ''}</th>`).join('')}</tr></thead><tbody>
    <tr><td><b>Входной блок</b></td><td>${[ap.education ? 'до поступления: ' + esc(ap.education) : '', ...METHOD.APPLICANT.flags.filter(([k]) => (ap.flags || {})[k]).map(([, l]) => esc(l)), ap.experience ? esc(ap.experience) : ''].filter(Boolean).map(x => `<div style="font-size:12.5px">${x}</div>`).join('') || '<span class="muted">сведения не внесены</span>'}</td>${stages.slice(1).map(() => '<td class="muted">—</td>').join('')}</tr>
    <tr><td><b>Контроль: мониторинг</b></td>${stages.map(s => `<td>${cell(s.id, 'monitoring')}</td>`).join('')}</tr>
    <tr><td><b>Контроль: диагностика</b></td>${stages.map(s => `<td>${cell(s.id, 'diagnostics')}</td>`).join('')}</tr>
    <tr><td><b>Расчёты по методике</b></td>${stages.map(s => `<td>${assessCell(s.id)}</td>`).join('')}</tr>
    <tr><td><b>Математический блок: g по дисциплинам</b></td><td></td><td colspan="5">${traj.length ? traj.map(a => `<a href="#/a/${a.id}">${fmtDate(a.meta.date)}: g после последней дисциплины = ${fmt(a.result.final, 3)}</a>`).join('<br>') : '<span class="muted">—</span>'}</td><td></td></tr>
    <tr><td><b>Эмпирические показатели этапа</b><div class="src">из журнала; в уровень не входят</div></td>${stages.map(s => `<td style="font-size:12.5px">${empir(s.id)}</td>`).join('')}</tr>
  </tbody></table></div>
  <p class="src">Выходной блок: уровни сформированности приведены по каждому этапу отдельно. Единый уровень ВПК за весь период подготовки не вычисляется — правило объединения этапов в методике не установлено (весовые коэффициенты этапов в методике даны без правила применения).</p></div>`;
});

/* ---------- Схема связей в справочнике ---------- */
function schemeTab() {
  const L = schemeLinks(); const custom = !!(state.settings && state.settings.schemeLinks);
  const cols = [...C.kinds, ...C.methodFamilies];
  return `<div class="card"><h2>Комплекс функций контроля</h2><div class="table-wrap"><table><thead><tr><th>Группа функций</th><th>Функция</th><th>Где реализуется в приложении</th></tr></thead><tbody>
    ${C.functions.map(f => `<tr><td>${esc(f.group)}</td><td><b>${esc(f.name)}</b></td><td>${esc(f.where)}</td></tr>`).join('')}</tbody></table></div>
    <p class="src">Соответствие функций разделам приложения — пояснение для пользователя; в методике оно не приведено.</p>
    <dl class="spec"><dt>Типы контроля</dt><dd>${C.types.map(t => esc(t[1])).join(', ')}</dd><dt>Традиционные методы</dt><dd>${C.methods.traditional.map(esc).join(', ')}</dd><dt>Комплексные военно-профессиональные</dt><dd>${C.methods.complex.map(esc).join(', ')}</dd></dl></div>
  <div class="card"><h2>Связи «вид контроля / методы → форма контроля»</h2>
    <div class="note warn">Связи прочитаны со стрелок схемы визуально; часть цветных линий пересекается, поэтому возможны неточности. Проверьте и при необходимости исправьте отметки — изменения сохраняются и используются в журнале для подсказки форм.${custom ? ' <b>Сейчас действуют изменённые пользователем связи.</b>' : ''}</div>
    <div class="table-wrap"><table><thead><tr><th>Форма контроля</th>${cols.map(c => `<th class="c" style="white-space:normal;font-size:12px">${esc(c[1])}</th>`).join('')}</tr></thead><tbody>
    ${C.forms.filter(f => f.src === 'forms').map(f => `<tr><td>${esc(f.name)}</td>${cols.map(([k]) => `<td class="c"><input type="checkbox" data-link="${k}|${f.id}" ${(L[k] || []).includes(f.id) ? 'checked' :
```

```
''} aria-label="${esc(f.name)} — ${esc(k)}"></td>`).join('')}</tr>`).join('')}</tbody></table></div>
3883     <div class="row" style="margin-top:8px">${custom ? '<button class="btn sm" data-act="links-reset">Вернуть связи, прочитанные со схемы</button>' : ''}</div>
3884     <p class="src">Формы блока контроля схемы модели (ЕГЭ, профотбор, входная диагностика, НИР, дипломная работа, ГИА, учебные сборы, проверка на классность, оценки по ПДП) в схеме не связаны с видами контроля и доступны в журнале отдельной группой.</p></div>`;
3885 }
3886 document.addEventListener('change', e => {
3887   const k = e.target.dataset && e.target.dataset.link; if (!k) return;
3888   const [col, form] = k.split('|'); if (!state.settings) state.settings = {};
3889   const L = Model.clone(schemeLinks()); const s = new Set(L[col] || []); if (e.target.checked) s.add(form); else s.delete(form); L[col] = [...s];
3890   state.settings.schemeLinks = L; persist(); render();
3891 });
3892 actions['links-reset'] = () => { delete state.settings.schemeLinks; persist(); render(); };
3893
3894 /* ========================================================================
3895    МОДУЛЬ 14. ГРУППЫ РИСКА, ПРОФИЛЬ КУРСАНТА, ПАСПОРТА КОМПЕТЕНЦИЙ,
3896    МАТРИЦА «ДИСЦИПЛИНЫ ↔ ВПК», СТАТИСТИКА
3897    ======================================================================== */
3898 const LV_ORDER = { crit: 0, low: 1, mid: 2, high: 3 };
3899 const passports = () => (state.settings && state.settings.passports) || METHOD.PASSPORTS;
3900 const finalsOf = (objId, type) => state.assessments.filter(a => a.objectId === objId && a.status === 'final' && (!type || a.type === type))
3901   .sort((a, b) => (a.meta.date || '').localeCompare(b.meta.date || '') || (a.finalizedAt || '').localeCompare(b.finalizedAt || ''));
3902
3903 /* ---------- Группы риска (методика: предупреждения об отставании, «группы риска») ----------
3904    Признаки опираются только на уровни методики и данные расчётов. */
3905 function riskOf(o) {
3906   const out = [];
3907   const pr = finalsOf(o.id, 'practice'), gr = finalsOf(o.id, 'graduate');
3908   const lastPr = pr[pr.length - 1], lastGr = gr[gr.length - 1];
3909   if (lastPr && lastPr.result.vpk.levelBsr && LV_ORDER[lastPr.result.vpk.levelBsr.code] <= 1)
3910     out.push({ sev: lastPr.result.vpk.levelBsr.code === 'crit' ? 2 : 1, text: `Практика ${lastPr.meta.course} курса: уровень «${levelInfo(lastPr.result.vpk.levelBsr.code).name}» (bcp ${fmt(lastPr.result.vpk.meanGrade, 2)})`, href: '#/a/' + lastPr.id });
3911   if (lastGr) { const lv = lastGr.result.vpk.levelPsfk || lastGr.result.vpk.levelBsr;
3912     if (lv && LV_ORDER[lv.code] <= 1) out.push({ sev: lv.code === 'crit' ? 2 : 1, text: `Отзыв (${lastGr.meta.period} год службы): уровень ВПК «${levelInfo(lv.code).name}»`, href: '#/a/' + lastGr.id }); }
3913   const p4 = latest(pr.filter(a => a.meta.course === 4)), p5 = latest(pr.filter(a => a.meta.course === 5));
3914   if (p4 && p5 && p4.snapshot.sets.vpk.id === p5.snapshot.sets.vpk.id && p5.result.vpk.meanPsfk < p4.result.vpk.meanPsfk - 1e-12)
3915     out.push({ sev: 1, text: `Снижение показателя: 4 курс ${fmt(p4.result.vpk.meanPsfk, 3)} → 5 курс ${fmt(p5.result.vpk.meanPsfk, 3)}`, href: '#/object/' + o.id });
3916   if (lastPr && lastGr) {
3917     const cmp = Calc.followup(rowsOfResult(lastPr.result), rowsOfResult(lastGr.result));
3918     if (cmp.n) { const S = METHOD.SCALES.bsr, la = Calc.levelBy(S, cmp.meanA).code, lb = Calc.levelBy(S, cmp.meanB).code;
3919       if (LV_ORDER[lb] < LV_ORDER[la]) out.push({ sev: 1, text: `В войсках уровень ниже, чем на практике: «${levelInfo(la).name}» → «${levelInfo(lb).name}» (по ${cmp.n} общим ВПК)`, href: '#/followup/' + o.id }); }
3920   }
3921   const last = [lastPr, lastGr].filter(Boolean);
3922   for (const a of last) { const crit = rowsOfResult(a.result).filter(r => r.grade != null && r.grade <= 2).map(r => r.id);
3923     if (crit.length) out.push({ sev: 2, text: `${a.type === 'practice' ? 'Практика' : 'Отзыв'}: компетенции на критическом уровне (оценка 1–2): ${crit.join(', ')}`, href: '#/a/' + a.id }); }
3924   if (lastPr) { const v = Calc.passportCheck(lastPr.result.vpk.rows, passports(), lastPr.meta.course, METHOD.SCALES.bsr).filter(x => LV_ORDER[x.level] < Math.min(...x.allowed.map(l => LV_ORDER[l])));
3925     if (v.length) out.push({ sev: 1, text: `Ниже уровня, допустимого паспортом для ${lastPr.meta.course} курса: ${v.map(x => x.id).join(', ')}`, href: '#/a/' + lastPr.id }); }
3926   const bad = recordsOf(o.id).filter(r => (r.markType === 'grade' && r.mark === 2) || (r.markType === 'pass' && r.mark === 'не зачтено'));
3927   if (bad.length) out.push({ sev: 1, text: `Журнал контроля: неудовлетворительных результатов — ${bad.length} (${[...new Set(bad.map(r => formName(r.form)))].join(', ')})`, href: '#/object/' + o.id + '/map' });
3928   return out;
3929 }
3930 function riskList(group) {
3931   return state.objects.filter(o => !group || o.group === group).map(o => ({ o, items: riskOf(o) })).filter(x => x.items.length)
```

```
    .sort((a, b) => Math.max(...b.items.map(i => i.sev)) - Math.max(...a.items.map(i => i.sev)) || b.items.length - a.items.length);
}
route(/^\/risk$/, () => {
  const R = ui.risk || (ui.risk = { group: '' });
  const list = riskList(R.group);
  return `<h1>Группы риска</h1><p class="muted">Предупреждения об отставании (методика: портал «автоматически генерирует предупреждения об отставании по ключевым индикаторам … для раннего выявления групп риска»). Признаки строятся только на уровнях и данных методики:</p>
  <ul class="muted" style="margin-top:0"><li>уровень «низкий» или «критический» по последней анкете практики (по bcp) или по отзыву;</li><li>снижение суммарного Псфк от 4 к 5 курсу;</li><li>уровень в войсках ниже, чем на практике, по общим ВПК;</li><li>компетенции с оценкой 1–2 (критический уровень компетенции);</li><li>уровень компетенции ниже допустимого паспортом для курса;</li><li>неудовлетворительные результаты в журнале контроля.</li></ul>
  <div class="card no-print"><div class="row"><div class="field" style="min-width:220px"><label class="f">Группа</label>${sel('data-risk="group"', groupList().map(g => [g, g]), R.group, 'все')}</div><span class="spacer"></span><button class="btn" data-act="print">Печать / PDF</button></div></div>
  <div class="card">${list.length ? `<p><b>Курсантов с признаками риска: ${list.length}</b> из ${state.objects.filter(o => !R.group || o.group === R.group).length}.</p><div class="table-wrap"><table><thead><tr><th>Курсант</th><th>Группа</th><th>Признаки</th></tr></thead><tbody>
  ${list.map(x => `<tr><td style="white-space:nowrap"><a href="#/object/${x.o.id}/profile">${esc(x.o.name)}</a></td><td>${esc(x.o.group || '—')}</td><td>${x.items.map(i => `<div style="margin-bottom:4px"><span class="badge ${i.sev === 2 ? 'b-draft' : 'b-info'}" style="${i.sev === 2 ? 'color:var(--danger)' : ''}">${i.sev === 2 ? 'критично' : 'внимание'}</span> <a href="${i.href}">${esc(i.text)}</a></div>`).join('')}</td></tr>`).join('')}
  </tbody></table></div>` : emptyState('Признаков риска не выявлено', 'По введённым данным ни у кого нет признаков отставания. Список обновляется после каждой утверждённой оценки.')}</div>`;
});
document.addEventListener('change', e => { if (e.target.dataset && e.target.dataset.risk) { ui.risk.group = e.target.value; render(); } });
function riskSummaryCard() {
  const list = riskList('');
  if (!list.length) return '';
  const crit = list.filter(x => x.items.some(i => i.sev === 2)).length;
  return `<div class="note ${crit ? 'err' : 'warn'}"><b>Группы риска:</b> признаки отставания у ${list.length} курсантов${crit ? `, из них критические — у ${crit}` : ''}: ${list.slice(0, 5).map(x => `<a href="#/object/${x.o.id}/profile">${esc(x.o.name)}</a>`).join(', ')}${list.length > 5 ? ' …' : ''}. <a href="#/risk">Подробнее</a></div>`;
}

/* ---------- Профиль курсанта (диагностический профиль) ---------- */
function radarChart(labels, series, max = 5) {
  const W = 520, H = 460, cx = 260, cy = 220, R = 170, n = labels.length;
  const pt = (i, v) => { const a = -Math.PI / 2 + 2 * Math.PI * i / n, r = R * v / max; return [cx + r * Math.cos(a), cy + r * Math.sin(a)]; };
  let g = '';
  for (let k = 1; k <= max; k++) g += `<polygon points="${labels.map((_, i) => pt(i, k).join(',')).join(' ')}" fill="none" stroke="#e5e7eb"/><text x="${cx + 3}" y="${cy - R * k / max + 4}" style="font-size:10px">${k}</text>`;
  labels.forEach((l, i) => { const [x, y] = pt(i, max), [lx, ly] = pt(i, max + 0.55); g += `<line x1="${cx}" y1="${cy}" x2="${x}" y2="${y}" stroke="#e5e7eb"/><text x="${lx}" y="${ly + 4}" text-anchor="middle" style="font-size:10.5px">${esc(l)}</text>`; });
  for (const s of series) {
    const pts = s.values.map((v, i) => isNumV(v) ? pt(i, v) : pt(i, 0));
    g += `<polygon points="${pts.map(p => p.join(',')).join(' ')}" fill="${s.color}" fill-opacity="${s.fill ?? 0.12}" stroke="${s.color}" stroke-width="2" ${s.dash ? 'stroke-dasharray="5 4"' : ''}/>`;
    g += s.values.map((v, i) => isNumV(v) ? `<circle cx="${pts[i][0]}" cy="${pts[i][1]}" r="3" fill="${s.color}"><title>${esc(s.name)} — ${esc(labels[i])}: ${fmt(v, 2)}</title></circle>` : '').join('');
  }
  return `<svg class="chart" viewBox="0 0 ${W} ${H}" style="max-width:560px;margin:auto" role="img" aria-label="Лепестковая диаграмма профиля">${g}</svg><div class="legend" style="justify-content:center">${series.map(s => `<span><i style="background:${s.color}"></i>${esc(s.name)}</span>`).join('')}</div>`;
}
function groupMeanByComp(a) {
  const o = state.objects.find(x => x.id === a.objectId) || {};
  const peers = state.assessments.filter(x => x.status === 'final' && x.type === a.type && (state.objects.find(q => q.id === x.objectId) || {}).group === o.group
    && (a.type === 'practice' ? x.meta.course === a.meta.course : x.meta.period === a.meta.period) && x.snapshot.sets.vpk.id === a.snapshot.sets.vpk.id);
  const latestPer = new Map(); peers.forEach(x => { const p = latestPer.get(x.objectId); if (!p || (x.meta.date || '') > (p.meta.date || '')) latestPer.set(x.objectId, x); });
  const list = [...latestPer.values()]; const m = {};
  for (const r of rowsOfResult(a.result)) { const gs = list.map(x => (rowsOfResult(x.result).find(q => q.id === r.id) || {}).grade).filter(isNumV); m[r.id] = gs.length ? gs.reduce((s, v) => s + v, 0) / gs.le
```

```
ngth : null; }
  return { m, n: list.length };
}
/* ---------- Краткие названия компетенций и автоматическая характеристика ----------
   Краткое название — сокращённый пересказ формулировки компетенции для чтения профиля.
   Характеристика строится только по введённым оценкам: что выше и что ниже, где рост
   и где снижение. Это описание данных, а не вывод методики. */
const COMP_SHORT = {
  'ВПК-1': 'Соблюдение законодательства и учёт последствий решений', 'ВПК-2': 'Профессиональная подготовка и саморазвитие',
  'ВПК-3': 'Бдительность, исполнительность, инициативность', 'ВПК-4': 'Поддержание боевой готовности подразделения',
  'ВПК-5': 'Руководство подразделением при выполнении задач', 'ВПК-6': 'Подведение итогов деятельности подразделения',
  'ВПК-7': 'Внешний вид и строевая выправка подчинённых', 'ВПК-8': 'Проведение занятий и обучение подчинённых',
  'ВПК-9': 'Физическая подготовка личного состава', 'ВПК-10': 'Воспитательная и морально-психологическая работа',
  'ВПК-11': 'Эксплуатация и учёт вооружения и техники', 'ВПК-12': 'Личное владение вооружением и технико й',
  'ВПК-13': 'Обеспечение довольствием и забота о подчинённых', 'ВПК-14': 'Учёт личного состава',
  'ВПК-15': 'Руководство хозяйством подразделения', 'ВПК-16': 'Обеспечение безопасности военной службы',
  'ПСК-1': 'Нормы международного гуманитарного права', 'ПСК-2': 'Правовое обучение и консультирование',
  'ПСК-3': 'Отбор кандидатов на военную службу', 'ПСК-4': 'Представление к званиям и должностям', 'ПСК-5': 'Работа в юридическом подразделении'
};
const compShort = id => COMP_SHORT[id] || id;
function compInfo(a, id) { return (a.snapshot.sets.vpk.items.concat(a.snapshot.sets.psk ? a.snapshot.sets.psk.items : []).find(c => c.id === id)) || {}; }
const listJoin = arr => arr.length <= 1 ? (arr[0] || '') : arr.slice(0, -1).join(', ') + ' и ' + arr[arr.length - 1];
function characterize(o, a, rows, gm, gn) {
  const S = METHOD.SCALES.bsr, mean = a.result.vpk.meanGrade, lv = a.result.vpk.levelBsr;
  const d = r => isNumV(gm[r.id]) && gn > 1 ? r.grade - gm[r.id] : 0;
  const byK = (x, y) => y.k - x.k;
  const strong = rows.filter(r => r.grade === 5 || (r.grade >= 4 && d(r) >= 0.5)).sort((x, y) => y.grade - x.grade || d(y) - d(x) || y.k - x.k);
  const weak = rows.filter(r => r.grade <= 3 || (r.grade < 5 && d(r) <= -0.5)).sort((x, y) => x.grade - y.grade || byK(x, y));
  const kMed = [...rows.map(r => r.k)].sort((x, y) => x - y)[Math.floor(rows.length / 2)];
  const fmtItem = r => { const ci = compInfo(a, r.id); return { r, html: `<b>${esc(compShort(r.id))}</b> (${esc(r.id)}) — оценка ${r.grade}${isNumV(gm[r.id]) && gn > 1 ? `, по группе ${fmt(gm[r.id], 1)}` : ''}${ci.group ? `<div class="vpk-desc">${esc(ci.group)}</div>` : ''}` }; };
  const kind = a.type === 'practice' ? `войсковой практики ${a.meta.course} курса` : `отзыва за ${a.meta.period} год службы`;
  const P = [];
  P.push(`По результатам ${kind} (${fmtDate(a.meta.date)}) средняя оценка ${esc(o.name)} — <b>${fmt(mean, 2)}</b> из 5, степень сформированности ${fmt(a.result.vpk.meanPct, 1)}%, уровень ${lv ? `«${esc(levelInfo(lv.code).name)}»` : 'не определён'}.`);
  const fives = rows.filter(r => r.grade === 5).length, lows = rows.filter(r => r.grade <= 3).length;
  P.push(`Из ${rows.length} компетенций на «отлично» оценено ${fives}, на «удовлетворительно» и ниже — ${lows}.${gn > 1 ? ` Профиль сравнивается со средним по группе «${esc(o.group)}» (${gn} чел.).` : ''}`);
  if (strong.length) P.push(`Сильнее всего проявляются: ${listJoin(strong.slice(0, 3).map(r => esc(compShort(r.id)).toLowerCase()))}.`);
  if (weak.length) {
    const imp = weak.filter(r => r.k >= kMed);
    P.push(`Требуют внимания: ${listJoin(weak.slice(0, 3).map(r => esc(compShort(r.id)).toLowerCase()))}.${imp.length ? ` Среди них компетенции с высоким коэффициентом значимости: ${imp.map(r => r.id).join(', ')} — их развитие сильнее всего повлияет на итоговый показатель.` : ''}`);
  } else P.push('Компетенций с заметным отставанием нет.');
  // Динамика: предыдущая оценка того же вида и сопоставление практики с отзывом
  const same = finalsOf(o.id, a.type).filter(x => x.id !== a.id && (x.meta.date || '') < (a.meta.date || ''));
  const prev = same[same.length - 1]; const dyn = [];
  if (prev) { const c = Calc.followup(rowsOfResult(prev.result), rows);
    if (c.n) { const up = c.rows.filter(x => x.b > x.a).map(x => x.id), dn = c.rows.filter(x => x.b < x.a).map(x => x.id);
      P.push(`По сравнению с предыдущей оценкой (${fmtDate(prev.meta.date)}) средний балл ${c.meanB > c.meanA ? 'вырос' : c.meanB < c.meanA ? 'снизился' : 'не изменился'}: ${fmt(c.meanA, 2)} → ${fmt(c.meanB, 2)}.${up.length ? ` Рост: ${up.join(', ')}.` : ''}${dn.length ? ` Снижение: ${dn.join(', ')}.` : ''}`);
      dn.forEach(id => dyn.push(id)); } }
  if (a.type === 'graduate') { const pr = finalsOf(o.id, 'practice'); const lp = pr[pr.length - 1];
    if (lp) { const c = Calc.followup(rowsOfResult(lp.result), rows); if (c.n) P.push(`В войсках средний балл по общим компетенциям ${c.meanB > c.meanA ? 'выше' : c.meanB < c.meanA ? 'ниже' : 'такой же'}, чем
```

```
на практике ${lp.meta.course} курса: ${fmt(c.meanA, 2)} → ${fmt(c.meanB, 2)}.`); } }
  const bad = recordsOf(o.id).filter(r => (r.markType === 'grade' && r.mark === 2) || (r.markType === 'pass' && r.mark === 'не зачтено'));
  if (bad.length) P.push(`В журнале контроля есть неудовлетворительные результаты: ${bad.length}.`);
  return { strong: strong.map(fmtItem), weak: weak.map(fmtItem), text: P, dyn };
}
route(/^\/object\/([\w-]+)\/profile$/, id => {
  const o = state.objects.find(x => x.id === id); if (!o) return objMissing();
  const cand = finalsOf(id).filter(a => a.type === 'practice' || a.type === 'graduate');
  if (!cand.length) return objectChrome(o, 'profile') + `<div class="card">${emptyState('Профиль пока не построен', 'Профиль строится по утверждённой анкете войсковой практики или отзыву из войск.', `<a class="btn primary" href="#/new?obj=${id}">Новая оценка</a>`)}</div>`;
  const P = ui.prof && ui.prof.obj === id ? ui.prof : (ui.prof = { obj: id, aid: cand[cand.length - 1].id });
  const a = cand.find(x => x.id === P.aid) || cand[cand.length - 1];
  const rows = rowsOfResult(a.result); const { m: gm, n: gn } = groupMeanByComp(a);
  const labels = rows.map(r => r.id.replace('ВПК-', 'В').replace('ПСК-', 'П'));
  const S = METHOD.SCALES.bsr;
  const minG = Math.min(...rows.map(r => r.grade));
  const lag = rows.filter(r => r.grade < 5 && (r.grade === minG || (isNumV(gm[r.id]) && r.grade < gm[r.id]))).sort((x, y) => y.k - x.k);
  const pv = a.type === 'practice' ? Calc.passportCheck(rows, passports(), a.meta.course, S) : [];
  const risks = riskOf(o);
  return objectChrome(o, 'profile') + `<div class="row no-print" style="margin-bottom:12px"><button class="btn primary" data-act="print">Печать / PDF</button>
  <div class="field" style="margin:0 0 0 auto;min-width:280px"><select data-prof="aid" aria-label="Оценка">${cand.map(x => `<option value="${x.id}" ${x.id === a.id ? 'selected' : ''}>${esc(Model.TYPES[x.type].short)}: ${x.type === 'practice' ? x.meta.course + ' курс' : x.meta.period + ' год службы'}, ${fmtDate(x.meta.date)}</option>`).join('')}</select></div></div>
  <div class="report"><div class="card"><h1>Диагностический профиль: ${esc(o.name)}</h1><p class="muted">${esc(o.group || '')} · ${esc(typeTitle(a.type))}, ${a.type === 'practice' ? a.meta.course + ' курс' : a.meta.period + ' год службы'}, ${fmtDate(a.meta.date)}</p>
  <div class="grid g4">${kpi('bcp', fmt(a.result.vpk.meanGrade, 2), lvBadge(a.result.vpk.levelBsr))}${kpi('Псфк ВПК', fmt(a.result.vpk.meanPsfk, 3), a.type === 'graduate' ? lvBadge(a.result.vpk.levelPsfk) : 'уровень по bcp')}${kpi('Степень сформированности', fmt(a.result.vpk.meanPct, 1) + '%', '')}${kpi('Сравнение с группой', gn > 1 ? gn + ' чел.' : '—', gn > 1 ? 'средние по той же оценке группы' : 'нет данных по группе')}</div>
  <div class="grid" style="margin-top:12px;align-items:center"><div>${radarChart(labels, [{ name: 'Максимум (5)', color: '#8E8E93', values: rows.map(() => 5), fill: 0, dash: true }, ...(gn > 1 ? [{ name: 'Среднее по группе', color: '#FF9500', values: rows.map(r => gm[r.id]), fill: 0.05 }] : []), { name: o.name, color: '#007AFF', values: rows.map(r => r.grade), fill: 0.18 }])}</div>
  <div><div class="table-wrap"><table><thead><tr><th>Компетенция</th><th class="num">Оценка</th><th>Уровень компетенции</th><th class="num">Группа</th><th class="num">${term('Кзнк')}</th></tr></thead><tbody>
  ${rows.map(r => `<tr><td><b>${esc(r.id)}</b> · ${esc(compShort(r.id))}<div class="vpk-desc">${esc(compInfo(a, r.id).text || '')}</div></td><td class="num"><b>${r.grade}</b></td><td>${lvBadge({ code: Calc.levelOfGrade(r.grade, S) }, false)}</td><td class="num">${fmt(gm[r.id], 2)}</td><td class="num">${fmtK(r.k)}</td></tr>`).join('')}</tbody></table></div></div></div></div>
  ${(() => { const C = characterize(o, a, rows, gm, gn); return `<div class="card"><h2>Характеристика</h2><div class="narr">${C.text.map(t => `<p>${t}</p>`).join('')}</div>
  <div class="sw" style="margin-top:12px"><div class="sw-col plus"><h3>Сильные стороны</h3>${C.strong.length ? `<ul>${C.strong.slice(0, 6).map(x => `<li>${x.html}</li>`).join('')}</ul>` : '<p class="muted">Явно выделяющихся компетенций нет.</p>'}</div>
  <div class="sw-col minus"><h3>Зоны развития</h3>${C.weak.length ? `<ul>${C.weak.slice(0, 6).map(x => `<li>${x.html}</li>`).join('')}</ul>` : '<p class="muted">Отстающих компетенций нет.</p>'}</div></div>
  <p class="src" style="margin-top:10px">Характеристика составлена программой автоматически по введённым оценкам: сильная сторона — оценка 5 или на 0,5 балла выше среднего по группе; зона развития — оценка 3 и ниже или на 0,5 балла ниже среднего по группе. Текст требует подтверждения руководителем.</p></div>`; })()}
  <div class="card"><h2>Компетенции, требующие целенаправленной работы</h2>
  ${lag.length ? `<p>По образцу вывода методики: «необходимость целенаправленной работы по развитию у курсанта отстающих компетенций». В порядке убывания значимости (Кзнк):</p><ol>${lag.map(r => `<li><b>${esc(r.id)}</b> — оценка ${r.grade}${isNumV(gm[r.id]) && r.grade < gm[r.id] ? ` (ниже среднего по группе ${fmt(gm[r.id], 2)})` : ''}${r.grade === minG ? ', наименьшая оценка курсанта' : ''}; Кзнк ${fmtK(r.k)}</li>`).join('')}</ol>` : '<p>Все компетенции оценены на 5 или не ниже среднего по группе.</p>'}
  ${pv.length ? `<div class="note warn"><b>Паспорт компетенции:</b> ${pv.map(x => `${x.id} — уровень «${levelInfo(x.level).name}» не предусмотрен для ${a.meta.course} курса (допустимо: ${x.allowed.map(l => levelInfo(l).name).join(', ')})`).join('; ')}.</div>` : ''}
  ${risks.length ? `<h3>Признаки риска</h3>${risks.map(r => `<div class="note ${r.sev === 2 ? 'err' : 'warn'}">${esc(r.text)}</div>`).join('')}` : ''}
  <p class="src">Конкретные меры обучения методикой не формализованы — профиль показывает, на какие компетенции обратить внимание. Уровень отдельной компетенции — по шкале «средний балл за освоение компетенции»: 5 — высокий, 4 — средний, 3 — низкий, 1–2 — критический.</p></div></div>`;
});
```

```
document.addEventListener('change', e => { if (e.target.dataset && e.target.dataset.prof) { ui.prof.aid = e.target.value; render(); } });

/* ---------- Паспорта компетенций: допустимые уровни по курсам ---------- */
route(/^\/passports$/, () => {
  const P = passports(); const custom = !!(state.settings && state.settings.passports);
  const comps = METHOD.COEF_SETS['src-vpk16'].items;
  return `<h1>Паспорта компетенций</h1><p class="muted">«Уровни и последовательность формирования компетенции»: знак «+» — уровень, достижимый на данном курсе. В методике таблица приведена только для ВПК-11; для остальных компетенций отметьте уровни по паспортам компетенций вашей образовательной программы. Если таблица для компетенции не заполнена, проверка не выполняется.</p>
  ${custom ? '<div class="note">Действуют изменённые пользователем паспорта. <button class="btn sm" data-act="pp-reset">Вернуть значения методики (только ВПК-11)</button></div>' : ''}
  ${comps.map(c => { const p = P[c.id]; return `<details class="card" ${p ? 'open' : ''}><summary style="cursor:pointer"><b>${esc(c.id)}</b> ${p ? '<span class="badge b-info">задан</span>' : '<span class="muted">не задан</span>'} <span class="muted" style="font-size:13px">${esc(c.text.slice(0, 110))}…</span></summary>
    <div class="table-wrap" style="margin-top:8px"><table style="max-width:560px"><thead><tr><th>Степень сформированности</th>${[1, 2, 3, 4, 5].map(k => `<th class="c">${k} курс</th>`).join('')}</tr></thead><tbody>
    ${['high', 'mid', 'low', 'crit'].map(l => `<tr><td>«${levelInfo(l).name}»</td>${[1, 2, 3, 4, 5].map(k => `<td class="c"><input type="checkbox" data-pp="${c.id}|${k}|${l}" ${p && (p[k] || []).includes(l) ? 'checked' : ''} aria-label="${esc(c.id)}, ${k} курс, ${levelInfo(l).name}"></td>`).join('')}</tr>`).join('')}</tbody></table></div>
    ${p ? `<button class="btn sm" data-act="pp-clear" data-id="${c.id}">Убрать паспорт</button>` : ''}
    ${c.id === 'ВПК-11' ? '<p class="src">Значения по умолчанию — методика.</p>' : ''}</details>`; }).join('')}`;
});
document.addEventListener('change', e => {
  const k = e.target.dataset && e.target.dataset.pp; if (!k) return;
  const [id, course, lv] = k.split('|'); if (!state.settings) state.settings = {};
  const P = Model.clone(passports()); if (!P[id]) P[id] = {}; const s = new Set(P[id][course] || []);
  if (e.target.checked) s.add(lv); else s.delete(lv); P[id][course] = [...s];
  state.settings.passports = P; persist(); render();
});
actions['pp-clear'] = el => { if (!state.settings) state.settings = {}; const P = Model.clone(passports()); delete P[el.dataset.id]; state.settings.passports = P; persist(); render(); };
actions['pp-reset'] = () => { delete state.settings.passports; persist(); render(); };

/* ---------- Матрица «дисциплины ↔ ВПК» (методика: матрица формирования компетенций) ---------- */
route(/^\/matrix$/, () => {
  const M = ui.matrix || (ui.matrix = { curriculumId: state.curricula[0] ? state.curricula[0].id : '' });
  const c = state.curricula.find(x => x.id === M.curriculumId);
  const comps = METHOD.COEF_SETS['src-vpk16'].items;
  let body = '';
  if (c) {
    const recs = state.controlRecords || [];
    const cov = comps.map(k => {
      const ds = c.disciplines.filter(d => (d.comps || []).includes(k.id));
      const names = new Set(ds.map(d => normName(d.name)));
      const rs = recs.filter(r => (r.comps || []).includes(k.id) || (r.discipline && names.has(normName(r.discipline))));
      return { k, ds, st: Calc.journalStats(rs) };
    });
    const uncovered = cov.filter(x => !x.ds.length);
    body = `<div class="card"><h2>Матрица формирования компетенций: «${esc(c.name)}»</h2><div class="table-wrap"><table><thead><tr><th>Дисциплина</th>${comps.map(k => `<th class="c" title="${esc(k.text)}" style="padding:6px 3px;font-size:11.5px">${k.id.replace('ВПК-', 'В')}</th>`).join('')}</tr></thead><tbody>
    ${c.disciplines.map((d, i) => `<tr><td style="white-space:nowrap">${esc(d.name)}${d.semester ? ` <span class="muted">(${d.semester} с.)</span>` : ''}</td>${comps.map(k => `<td class="c" style="padding:3px"><input type="checkbox" data-mx="${i}|${k.id}" ${(d.comps || []).includes(k.id) ? 'checked' : ''} aria-label="${esc(d.name)} — ${k.id}"></td>`).join('')}</tr>`).join('')}
    <tr><td><b>Дисциплин</b></td>${cov.map(x => `<td class="c" style="${x.ds.length ? '' : 'background:var(--danger-soft);color:var(--danger)'}"><b>${x.ds.length}</b></td>`).join('')}</tr></tbody></table></div>
    ${uncovered.length ? `<div class="note warn">Не формируются ни одной дисциплиной плана: ${uncovered.map(x => x.k.id).join(', ')}.</div>` : '<div class="note ok">Каждая ВПК формируется хотя бы одной дисциплиной.</div>'}</div>
    <div class="card"><h2>Контроль по компетенциям (журнал)</h2><p class="muted">Записи журнала, где компетенция отмечена явно или дисциплина совпадает с дисциплиной матрицы. Эмпирические показатели — в уровень ВПК не входят.</p>
    <div class="table-wrap"><table><thead><tr><th>ВПК</th><th class="num">Дисциплин</th><th class="num">Записей журнала</th><th class="num">Оценок 2–5</th><th class="num">Средний балл</th><th class="num">Зачт
```

```
ено</th></tr></thead><tbody>
4099      ${cov.map(x => `<tr><td title="${esc(x.k.text)}">${esc(x.k.id)}</td><td class="num">${x.ds.length}</td><td class="num">${x.st.n}</td><td class="num">${x.st.grades}</td><td class="num">${fmt(x.st.meanGrade, 2)}</td><td class="num">${x.st.pass ? x.st.passed + '/' + x.st.pass : '—'}</td></tr>`).join('')}</tbody></table></div></div>`;
4100    }
4101    return `<h1>Дисциплины и ВПК</h1><p class="muted">Матрица соответствия (формирования) компетенций — часть образовательной программы. Отметьте, какие дисциплины учебного плана формируют каждую ВПК; отметки сохраняются в учебном плане.</p>
4102    <div class="card no-print"><div class="field" style="max-width:420px"><label class="f">Учебный план</label>${sel('data-mxc="1"', state.curricula.map(x => [x.id, x.name]), M.curriculumId, '— выберите —')}</div>${state.curricula.length ? '' : '<p class="muted">Учебных планов нет. <a href="#/curricula">Создайте план</a>.</p>'}</div>${body}`;
4103  });
4104  document.addEventListener('change', e => {
4105    const d = e.target.dataset || {};
4106    if (d.mxc) { ui.matrix.curriculumId = e.target.value; render(); return; }
4107    if (d.mx) { const c = state.curricula.find(x => x.id === ui.matrix.curriculumId); const [i, k] = d.mx.split('|'); const dd = c.disciplines[+i]; const s = new Set(dd.comps || []); if (e.target.checked) s.add(k); else s.delete(k); dd.comps = [...s]; c.updatedAt = Store.nowIso(); persist(); render(); }
4108  });
4109
4110  /* ---------- Статистика (методика: статистическое направление) ---------- */
4111  function scatter(points, fit, xl, yl) {
4112    if (!points.length) return '';
4113    const W = 560, H = 360, pl = 56, pb = 46, pt = 14, pr = 16;
4114    const xs = points.map(p => p[0]), ys = points.map(p => p[1]);
4115    let x0 = Math.min(...xs), x1 = Math.max(...xs), y0 = Math.min(...ys), y1 = Math.max(...ys);
4116    if (x1 === x0) { x0 -= 0.5; x1 += 0.5; } if (y1 === y0) { y0 -= 0.5; y1 += 0.5; }
4117    const px = (x1 - x0) * 0.08, py = (y1 - y0) * 0.08; x0 -= px; x1 += px; y0 -= py; y1 += py;
4118    const X = v => pl + (v - x0) / (x1 - x0) * (W - pl - pr), Y = v => pt + (1 - (v - y0) / (y1 - y0)) * (H - pt - pb);
4119    let g = '';
4120    for (let k = 0; k <= 4; k++) { const vx = x0 + (x1 - x0) * k / 4, vy = y0 + (y1 - y0) * k / 4;
4121      g += `<line x1="${X(vx)}" x2="${X(vx)}" y1="${pt}" y2="${H - pb}" stroke="#f1f3f5"/><text x="${X(vx)}" y="${H - pb + 16}" text-anchor="middle">${fmt(vx, 2)}</text><line x1="${pl}" x2="${W - pr}" y1="${Y(vy)}" y2="${Y(vy)}" stroke="#f1f3f5"/><text x="${pl - 6}" y="${Y(vy) + 4}" text-anchor="end">${fmt(vy, 2)}</text>`; }
4122    const cid = 'sc' + Math.random().toString(36).slice(2, 8);
4123    g += `<defs><clipPath id="${cid}"><rect x="${pl}" y="${pt}" width="${W - pl - pr}" height="${H - pt - pb}"/></clipPath></defs>`;
4124    if (fit) g += `<line clip-path="url(#${cid})" x1="${X(x0)}" y1="${Y(fit.intercept + fit.slope * x0)}" x2="${X(x1)}" y2="${Y(fit.intercept + fit.slope * x1)}" stroke="#FF9500" stroke-width="2" stroke-dasharray="6 4"/>`;
4125    g += points.map(p => `<circle cx="${X(p[0])}" cy="${Y(p[1])}" r="4.5" fill="#007AFF" fill-opacity=".75"><title>${esc(p[2] || '')}: ${fmt(p[0], 3)}; ${fmt(p[1], 3)}</title></circle>`).join('');
4126    g += `<text x="${(W + pl) / 2}" y="${H - 6}" text-anchor="middle">${esc(xl)}</text><text transform="translate(14,${(H - pb) / 2}) rotate(-90)" text-anchor="middle">${esc(yl)}</text>`;
4127    return `<svg class="chart" viewBox="0 0 ${W} ${H}" style="max-width:620px" role="img" aria-label="Диаграмма рассеяния">${g}</svg>`;
4128  }
4129  const STAT_PAIRS = {
4130    leader: { title: 'Псфк по анкете практики ↔ оценка руководителя практики', xl: 'Псфк (анкета)', yl: 'Оценка руководителя' },
4131    commission: { title: 'Псфк по анкете практики ↔ итоговая оценка комиссии', xl: 'Псфк (анкета)', yl: 'Оценка комиссии' },
4132    follow: { title: 'bcp на практике ↔ bcp в отзыве (по общим ВПК)', xl: 'bcp на практике', yl: 'bcp в отзыве' },
4133    journal: { title: 'Средний балл журнала контроля ↔ Псфк последней анкеты практики', xl: 'Средний балл журнала', yl: 'Псфк (анкета)' }
4134  };
4135  function statPoints(kind, group, course) {
4136    const inG = id => !group || (state.objects.find(o => o.id === id) || {}).group === group;
4137    const lastPr = () => { const m = new Map(); state.assessments.filter(a => a.status === 'final' && a.type === 'practice' && inG(a.objectId) && (!course || String(a.meta.course) === course)).forEach(a => { const p = m.get(a.objectId); if (!p || (a.meta.date || '') > (p.meta.date || '')) m.set(a.objectId, a); }); return [...m.values()]; };
4138    if (kind === 'leader' || kind === 'commission') return lastPr().map(a => [a.result.vpk.meanPsfk, Number(kind === 'leader' ? a.input.leaderMark : a.input.commissionMark), objName(a.objectId)]).filter(p => Calc.isGrade(p[1], 2, 5));
4139    if (kind === 'follow') return followupPairs().filter(p => inG(p.o.id) && p.cmp.n).map(p => [p.cmp.meanA, p.cmp.meanB, p.o.name]);
4140    if (kind === 'journal') return lastPr().map(a => { const s = Calc.journalStats(recordsOf(a.objectId)); return [s.meanGrade, a.result.vpk.meanPsfk, objName(a.objectId)]; }).filter(p => isNumV(p[0]));
```

```
  return [];
}
route(/^\/stats$/, () => {
  const T = ui.stats || (ui.stats = { kind: 'leader', group: '', course: '' });
  const pts = statPoints(T.kind, T.group, T.course);
  const pr = Calc.pearson(pts.map(p => p[0]), pts.map(p => p[1])), sp = Calc.spearman(pts.map(p => p[0]), pts.map(p => p[1]));
  const P = STAT_PAIRS[T.kind];
  const strength = r => { const a = Math.abs(r); return a < 0.3 ? 'слабая' : a < 0.7 ? 'умеренная' : 'сильная'; };
  return `<h1>Статистика</h1><div class="note warn"><b>Не из методики.</b> методика называет статистическое направление (факторный, кластерный, регрессионный анализ) без конкретных методов. Здесь применяются стандартные коэффициенты корреляции Пирсона (r) и Спирмена (ρ) и линия регрессии. Значимость не проверяется; при малом числе пар (менее 10) выводы ненадёжны.</div>
  <div class="card no-print"><div class="grid g3">
    <div class="field"><label class="f">Связь</label>${sel('data-st="kind"', Object.entries(STAT_PAIRS).map(([k, v]) => [k, v.title]), T.kind)}</div>
    <div class="field"><label class="f">Группа</label>${sel('data-st="group"', groupList().map(g => [g, g]), T.group, 'все')}</div>
    <div class="field"><label class="f">Курс практики</label>${sel('data-st="course"', [['4', '4 курс'], ['5', '5 курс']], T.course, 'последняя анкета')}</div></div></div>
  <div class="card"><h2>${esc(P.title)}</h2>
  ${pts.length >= 3 ? `<div class="grid g3">${kpi('Пар наблюдений', pts.length, pts.length < 10 ? 'мало для надёжных выводов' : '')}${kpi('r Пирсона', pr ? fmt(pr.r, 3) : '—', pr ? strength(pr.r) + (pr.r >= 0 ? ' прямая' : ' обратная') + ' связь' : 'нет разброса значений')}${kpi('ρ Спирмена', sp ? fmt(sp.rho, 3) : '—', 'ранговая')}</div>
  ${scatter(pts, pr, P.xl, P.yl)}${pr ? `<p class="src">Линия регрессии: y = ${fmt(pr.intercept, 3)} + ${fmt(pr.slope, 3)}·x. Градации «слабая / умеренная / сильная» (|r| < 0,3; < 0,7; ≥ 0,7) — общепринятые ориентиры, не из методики.</p>` : ''}` : `<div class="empty">Недостаточно данных: нужно не менее 3 пар (сейчас ${pts.length}).</div>`}</div>`;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.st; if (!k || !ui.stats) return; ui.stats[k] = e.target.value; render(); });

/* ========================================================================
   МОДУЛЬ 15. СЕРВИС: ИМПОРТ XLSX, КОРЗИНА, УЧЕБНЫЙ РЕЖИМ, СПРАВКА,
   НАПОМИНАНИЕ О РЕЗЕРВНОЙ КОПИИ
   ======================================================================== */

/* ---------- Чтение .xlsx без внешних библиотек (ZIP + XML первого листа) ---------- */
async function inflateRaw(bytes) {
  if (typeof DecompressionStream === 'undefined') throw new Error('Браузер не поддерживает распаковку .xlsx — сохраните файл как CSV');
  const ds = new DecompressionStream('deflate-raw');
  const out = new Response(new Blob([bytes]).stream().pipeThrough(ds));
  return new Uint8Array(await out.arrayBuffer());
}
async function unzip(buf) {
  const dv = new DataView(buf), u8 = new Uint8Array(buf);
  let eocd = -1; for (let i = u8.length - 22; i >= Math.max(0, u8.length - 65557); i--) if (dv.getUint32(i, true) === 0x06054b50) { eocd = i; break; }
  if (eocd < 0) throw new Error('Файл не является книгой Excel (.xlsx)');
  const n = dv.getUint16(eocd + 10, true); let p = dv.getUint32(eocd + 16, true);
  const files = {}, dec = new TextDecoder();
  for (let i = 0; i < n; i++) {
    if (dv.getUint32(p, true) !== 0x02014b50) throw new Error('Повреждённый архив .xlsx');
    const method = dv.getUint16(p + 10, true), csize = dv.getUint32(p + 20, true), nl = dv.getUint16(p + 28, true), el = dv.getUint16(p + 30, true), cl = dv.getUint16(p + 32, true), lo = dv.getUint32(p + 42, true);
    const name = dec.decode(u8.subarray(p + 46, p + 46 + nl));
    files[name] = { method, csize, lo }; p += 46 + nl + el + cl;
  }
  return async name => {
    const f = files[name]; if (!f) return null;
    const start = f.lo + 30 + dv.getUint16(f.lo + 26, true) + dv.getUint16(f.lo + 28, true);
    const data = u8.subarray(start, start + f.csize);
    return dec.decode(f.method === 0 ? data : await inflateRaw(data));
  };
}
async function readXlsx(buf) {
  const get = await unzip(buf);
  const xml = s => new DOMParser().parseFromString(s, 'application/xml');
  const shared = []; const ss = await get('xl/sharedStrings.xml');
```

```
  if (ss) for (const si of xml(ss).getElementsByTagName('si')) shared.push([...si.getElementsByTagName('t')].map(t => t.textContent).join(''));
  let sheetPath = 'xl/worksheets/sheet1.xml';
  const wb = await get('xl/workbook.xml'), rels = await get('xl/_rels/workbook.xml.rels');
  if (wb && rels) { const first = xml(wb).getElementsByTagName('sheet')[0]; const rid = first && (first.getAttribute('r:id') || first.getAttributeNS('http://schemas.openxmlformats.org/officeDocument/2006/relationships', 'id'));
    const rel = rid && [...xml(rels).getElementsByTagName('Relationship')].find(r => r.getAttribute('Id') === rid);
    if (rel) { const t = rel.getAttribute('Target'); sheetPath = t.startsWith('/') ? t.slice(1) : 'xl/' + t.replace(/^\.\//, ''); } }
  const sh = await get(sheetPath); if (!sh) throw new Error('В книге не найден первый лист');
  const rows = [];
  for (const row of xml(sh).getElementsByTagName('row')) {
    const r = Number(row.getAttribute('r')) - 1; const cells = rows[r] = rows[r] || [];
    for (const c of row.getElementsByTagName('c')) {
      const ref = c.getAttribute('r') || ''; const col = [...ref.replace(/\d+/g, '')].reduce((s, ch) => s * 26 + ch.charCodeAt(0) - 64, 0) - 1;
      const t = c.getAttribute('t'); const v = c.getElementsByTagName('v')[0];
      let val = '';
      if (t === 's') val = shared[Number(v ? v.textContent : -1)] ?? '';
      else if (t === 'inlineStr') val = [...c.getElementsByTagName('t')].map(x => x.textContent).join('');
      else val = v ? v.textContent : '';
      cells[col] = val;
    }
  }
  return Array.from({ length: rows.length }, (_, i) => Array.from({ length: (rows[i] || []).length }, (_, j) => (rows[i] || [])[j] ?? '')).filter(r => r.some(x => String(x).trim() !== ''));
}
/* Импорт: CSV или .xlsx */
document.addEventListener('change', async e => {
  if (e.target.id !== 'imFile' || !e.target.files.length) return;
  e.stopImmediatePropagation();
  const f = e.target.files[0]; e.target.value = '';
  let table;
  try { table = await withBusy('Чтение файла «' + f.name + '»…', async () => /\.xlsx$/i.test(f.name) ? await readXlsx(await f.arrayBuffer()) : parseCsv(await f.text())); }
  catch (err) { toast('Не удалось прочитать файл: ' + err.message, true); return; }
  if (table.length < 2) { toast('Файл пуст или не содержит строк данных', true); return; }
  ui.importState.rows = table; ui.importState.fileName = f.name; ui.importState.analysis = importAnalyze(ui.importState.type, table); render();
}, true);
/* Excel хранит даты числом дней от 30.12.1899 */
const excelDate = v => { const n = Number(v); if (!/^\d+(\.\d+)?$/.test(String(v)) || n < 20000 || n > 80000) return v; const d = new Date(Date.UTC(1899, 11, 30) + Math.round(n) * 86400000); return d.toISOString().slice(0, 10); };

/* ---------- Корзина: удалённое хранится 30 дней ---------- */
const TRASH_DAYS = 30;
function toTrash(kind, label, payload) {
  if (!state.trash) state.trash = [];
  const id = Store.uid(); state.trash.push({ id, kind, label, payload, deletedAt: Store.nowIso() }); ui.lastTrashId = id; return id;
}
/* Сообщение после удаления с кнопкой «Отменить» (возврат из корзины без перехода в неё) */
function undoToast(msg) { const id = ui.lastTrashId; toastAction(msg, 'Отменить', () => { if (restoreTrash(id)) { toast('Удаление отменено'); render(); } }, 8000); }
function purgeTrash() {
  if (!state.trash) return;
  const lim = Date.now() - TRASH_DAYS * 86400000; const n = state.trash.length;
  state.trash = state.trash.filter(t => new Date(t.deletedAt).getTime() >= lim);
  if (state.trash.length !== n) persist();
}
const TRASH_KINDS = { object: 'Объект оценки (с расчётами и журналом)', assessment: 'Расчёт', record: 'Запись журнала', curriculum: 'Учебный план', ymodel: 'Модель признаков', coefSet: 'Набор Кзнк', report: 'Отчёт о практике', expert: 'Экспертиза' };
const trashDlg = (what) => confirmDlg(`Удалить ${what}?`, `<p>Удалённое переместится в корзину и хранится ${TRASH_DAYS} дней — его можно восстановить (раздел «Корзина»).</p>`, 'Удалить', true);
actions['obj-del'] = async el => {
  const o = state.objects.find(x => x.id === el.dataset.id); if (!o) return;
  const as = state.assessments.filter(a => a.objectId === o.id), rs = (state.controlRecords || []).filter(r => r.objectId === o.id);
```

```
  if (!await confirmDlg('Удалить объект оценки?', `<p>Объект «${esc(o.name)}», его расчёты (${as.length}) и записи журнала (${rs.length}) переместятся в корзину и хранятся ${TRASH_DAYS} дней.</p>`, 'Удалить', true)) return;
  toTrash('object', o.name, { object: o, assessments: as, records: rs });
  state.objects = state.objects.filter(x => x.id !== o.id); state.assessments = state.assessments.filter(a => a.objectId !== o.id); state.controlRecords = (state.controlRecords || []).filter(r => r.objectId !== o.id);
  persist(); location.hash = '#/objects'; undoToast(`«${o.name}» удалён`);
};
actions['del-a'] = async el => {
  const a = findA(el.dataset.id); if (!a) return;
  if (!await trashDlg(a.status === 'draft' ? 'черновик' : 'расчёт')) return;
  toTrash('assessment', `${Model.TYPES[a.type].short}: ${objName(a.objectId)}, ${fmtDate(a.meta.date)}`, a);
  state.assessments = state.assessments.filter(x => x.id !== a.id); ui.editing = null; persist(); location.hash = '#/history'; undoToast(a.status === 'draft' ? 'Черновик удалён' : 'Оценка удалена');
};
actions['jr-del'] = async el => {
  const r = (state.controlRecords || []).find(x => x.id === el.dataset.id); if (!r || !await trashDlg('запись журнала')) return;
  toTrash('record', `${formName(r.form)}: ${objName(r.objectId)}, ${fmtDate(r.date)}`, r);
  state.controlRecords = state.controlRecords.filter(x => x.id !== r.id); persist(); render(); undoToast('Запись журнала удалена');
};
const simpleDel = (kind, arr, back, labelOf) => async el => {
  const item = (state[arr] || []).find(x => x.id === el.dataset.id); if (!item) return;
  if (kind === 'coefSet') { const used = state.assessments.filter(a => a.status === 'draft' && (a.meta.vpkSetId === item.id || a.meta.pskSetId === item.id)).length; if (used) { toast(`Набор используется в черновиках (${used}). Смените набор в них или удалите черновики.`, true); return; } }
  if (!await trashDlg(TRASH_KINDS[kind].toLowerCase())) return;
  toTrash(kind, labelOf(item), item); state[arr] = state[arr].filter(x => x.id !== item.id); persist(); location.hash = back; undoToast('Удалено: ' + labelOf(item));
};
actions['cu-del'] = simpleDel('curriculum', 'curricula', '#/curricula', x => x.name);
actions['ym-del'] = simpleDel('ymodel', 'yModels', '#/ymodels', x => x.name);
actions['coef-del'] = simpleDel('coefSet', 'coefSets', '#/coef', x => x.name);
actions['pr-del'] = simpleDel('report', 'practiceReports', '#/preports', x => `Отчёт о практике: ${x.group}, ${x.course} курс`);
actions['ex-del'] = simpleDel('expert', 'expertStudies', '#/coef', x => x.name);
const TRASH_ARR = { curriculum: 'curricula', ymodel: 'yModels', coefSet: 'coefSets', report: 'practiceReports', expert: 'expertStudies' };
route(/^\/trash$/, () => {
  const T = [...(state.trash || [])].sort((a, b) => b.deletedAt.localeCompare(a.deletedAt));
  return `<h1>Корзина</h1><p class="muted">Удалённые записи хранятся ${TRASH_DAYS} дней, затем удаляются окончательно. Полная очистка данных («Удалить все данные») корзину не использует.</p>
  <div class="card">${T.length ? `<div class="table-wrap"><table><thead><tr><th>Удалено</th><th>Что</th><th>Описание</th><th>Удаляется окончательно</th><th></th></tr></thead><tbody>
  ${T.map(t => `<tr><td>${fmtDT(t.deletedAt)}</td><td>${esc(TRASH_KINDS[t.kind] || t.kind)}</td><td>${esc(t.label)}</td><td>${fmtDate(new Date(new Date(t.deletedAt).getTime() + TRASH_DAYS * 86400000).toISOString())}</td><td style="white-space:nowrap"><button class="btn sm" data-act="tr-restore" data-id="${t.id}">Восстановить</button> <button class="btn sm danger" data-act="tr-purge" data-id="${t.id}">Удалить навсегда</button></td></tr>`).join('')}
  </tbody></table></div><button class="btn danger" style="margin-top:10px" data-act="tr-empty">Очистить корзину</button>` : emptyState('Корзина пуста', 'Удалённые курсанты, оценки и записи попадают сюда и хранятся 30 дней.')}</div>`;
});
function restoreTrash(tid) {
  const t = (state.trash || []).find(x => x.id === tid); if (!t) return false;
  const has = (arr, id) => (state[arr] || []).some(x => x.id === id);
  if (t.kind === 'object') {
    if (has('objects', t.payload.object.id)) { toast('Курсант уже существует', true); return false; }
    state.objects.push(t.payload.object);
    t.payload.assessments.forEach(a => { if (!has('assessments', a.id)) state.assessments.push(a); });
    if (!state.controlRecords) state.controlRecords = []; t.payload.records.forEach(r => { if (!has('controlRecords', r.id)) state.controlRecords.push(r); });
  } else if (t.kind === 'assessment' || t.kind === 'record') {
    if (!state.objects.some(o => o.id === t.payload.objectId)) { toast('Курсант удалён — сначала восстановите курсанта', true); return false; }
    const arr = t.kind === 'assessment' ? 'assessments' : 'controlRecords'; if (!state[arr]) state[arr] = [];
    if (!has(arr, t.payload.id)) state[arr].push(t.payload);
  } else { const arr = TRASH_ARR[t.kind]; if (!state[arr]) state[arr] = []; if (!has(arr, t.payload.id)) state[arr].push(t.payload); }
  state.trash = state.trash.filter(x => x.id !== t.id); persist(); return true;
```

```
}
actions['tr-restore'] = el => { if (restoreTrash(el.dataset.id)) { toast('Восстановлено'); render(); } };
actions['tr-purge'] = async el => { if (!await confirmDlg('Удалить навсегда?', 'Восстановить запись будет невозможно.', 'Удалить навсегда', true)) return; state.trash = state.trash.filter(x => x.id !== el.dataset.id); persist(); render(); };
actions['tr-empty'] = async () => { if (!await confirmDlg('Очистить корзину?', 'Все записи корзины будут удалены без возможности восстановления.', 'Очистить', true)) return; state.trash = []; persist(); render(); };

/* ---------- Учебный режим (отдельное хранилище) ---------- */
function modeBanner() {
  const b = $('#modeBanner'); if (!b) return;
  b.hidden = Store.mode() !== 'demo';
}
actions['mode-demo'] = async () => {
  if (!await confirmDlg('Перейти в учебный режим?', '<p>Учебный режим использует отдельное хранилище: рабочие данные не видны и не изменяются. В нём можно загрузить демонстрационные данные и контрольные примеры для обучения. Вернуться можно в любой момент.</p>', 'Перейти')) return;
  persist(); Store.setMode('demo'); state = Store.load(); location.hash = '#/'; modeBanner(); render(); toast('Учебный режим');
};
actions['mode-main'] = () => { persist(); Store.setMode('main'); state = Store.load(); location.hash = '#/'; modeBanner(); render(); toast('Основной режим: рабочие данные'); };

/* ---------- Справка по разделам ---------- */
const HELP = [
  [/^\/$/, 'Главная', 'Сводка: число объектов и расчётов, предупреждения (группы риска, резервная копия), последние расчёты и состояние исходных данных методики.'],
  [/^\/new/, 'Новая оценка', 'Выберите курсанта и вид оценки. Мастер ведёт по шагам: параметры → ввод данных → проверка → утверждение. Черновик сохраняется автоматически; неполные данные утвердить нельзя.'],
  [/^\/a\/[\w-]+\/edit/, 'Ввод данных оценки', 'Оценки 1–5 выставляются кнопками; незаполненные строки отмечены жёлтой полосой. На последнем шаге — предварительный результат; «Утвердить расчёт» фиксирует результат вместе со снимком методики.'],
  [/^\/a\//, 'Результат расчёта', 'Показатели, уровни, таблица по компетенциям и «Как получено» с промежуточными значениями. «Пересчитать и сверить» пересчитывает по сохранённым данным; «Изменить данные» создаёт новую редакцию, прежняя сохраняется.'],
  [/^\/batch/, 'Ввод по группе', 'Оценки всей группы в одной таблице. Вставка из Excel — Ctrl+V в первую ячейку; переход между ячейками — стрелки и Enter. Полные строки утверждаются, неполные становятся черновиками.'],
  [/^\/import/, 'Импорт из файла', 'Скачайте шаблон, заполните в Excel и загрузите как .xlsx или CSV. Перед импортом каждая строка проверяется; объекты сопоставляются по ФИО и группе.'],
  [/^\/journal/, 'Журнал контроля', 'Результаты форм контроля (экзамены, зачёты, летучки, курсовые, нормативы, ЕГЭ, ГИА…) по этапам и семестрам с видом, типом и методом контроля по схеме. Дают эмпирические показатели; в уровень ВПК не входят. «Ведомость группы» — одна форма на всю группу.'],
  [/^\/objects?\/[\w-]+\/map/, 'Карта этапов', 'Курсант по схеме модели: входной блок, мониторинг и диагностика по этапам, расчёты по методике, математический блок, эмпирические показатели.'],
  [/^\/objects?\/[\w-]+\/profile/, 'Профиль курсанта', 'Лепестковая диаграмма оценок по компетенциям в сравнении со средним по группе и максимумом; компетенции, требующие работы (по убыванию значимости); признаки риска.'],
  [/^\/objects?/, 'Курсанты и выпускники', 'Курсанты и выпускники. В карточке — расчёты, динамика, сравнение, профиль, карта этапов, отчёт. Сведения абитуриента — входной блок модели.'],
  [/^\/history/, 'Все оценки', 'Все расчёты с фильтрами по виду, статусу, дате и ФИО. Черновики открываются для продолжения ввода. CSV — исходные данные и результаты по фильтру.'],
  [/^\/summary/, 'Сводная по группе', 'Средние значения и место в группе (как в), распределение по уровням. Для практики можно включить пользовательское правило выявления расхождений.'],
  [/^\/reviews/, 'Отзывы из войск', 'Форма методички по отзывам: по органам военного управления — поступило отзывов и сколько оценено на 5/4/3/2. Основание оценки выберите сами.'],
  [/^\/risk/, 'Группы риска', 'Курсанты с признаками отставания: низкий или критический уровень, снижение показателя, компетенции с оценкой 1–2, нарушения паспорта, неудовлетворительные оценки в журнале.'],
  [/^\/followup/, 'Практика и служба', 'Сравнение оценок одних и тех же компетенций на практике и в отзыве через 1 (2) года.'],
  [/^\/groups/, 'Сравнение групп', 'Средние показатели 2–3 групп (например, контрольной и экспериментальной) и распределение по уровням.'],
  [/^\/gbatch/, 'Прогноз по группе', 'Алгоритм для всей группы: g₀ и способности каждого обучающегося по дисциплинам. Результат можно сохранить как расчёты.'],
  [/^\/sensitivity/, 'Чувствительность g', 'Графики зависимости уровня подготовленности от объёма дисциплины, мастерства преподавателя и доли самостоятельной работы.'],
  [/^\/optimize/, 'Оптимизация плана', 'Распределение часов T1/T2 по дисциплинам при ограничениях по семестрам, метод случайного поиска. Укажите семестры, минимальные объёмы, выпускающие дисциплины и лимиты.'],
  [/^\/matrix/, 'Дисциплины и ВПК', 'Отметьте, какие дисциплины формируют каждую ВПК; приложение покажет непокрытые компетенции и данные журнала по каждой ВПК.'],
```

```
  [/^\/stats/, 'Статистика', 'Коэффициенты корреляции между расчётным показателем и оценками, практикой и отзывом, журналом и анкетой. Метод не из методички — помечен.'],
  [/^\/blanks/, 'Бланки для печати', 'Служебный отзыв и анкета практики для заполнения в войсках — пусты е или с ФИО на всю группу. Печать в PDF через диалог печати.'],
  [/^\/preport/, 'Отчёты о практике', 'Отчёт руководителя практики по пунктам методички; характеристика курсантов дополняется результатами анкет.'],
  [/^\/method/, 'Справочник методики', 'Модель, компетенции, шкалы, спецификации формул, схема контроля, термины, основания и выявленные расхождения.'],
  [/^\/coef|^\/expert/, 'Коэффициенты Кзнк', 'Наборы коэффициентов значимости. Экспертиза: матрица ранго в → согласованность W, критерий χ², коэффициенты λ.'],
  [/^\/curricul/, 'Учебные планы', 'Дисциплины, часы T1 и T2, коэффициент m и предшествующие дисциплины — исходные данные для расчёта g.'],
  [/^\/ymodel/, 'Модели Y', 'Признаки, веса и эталон для уровня компетентности Y.'],
  [/^\/passports/, 'Паспорта компетенций', 'Уровни, достижимые на каждом курсе. Нарушения показываются в анкете практики, профиле и группах риска.'],
  [/^\/data/, 'Данные и резервные копии', 'Резервная копия в файл и восстановление, экспорт CSV/JSON, по иск дублей, учебный режим и демонстрационные данные.'],
  [/^\/trash/, 'Корзина', 'Удалённое хранится 30 дней и восстанавливается одной кнопкой.'],
  [/^\/tests/, 'Проверка расчётов', 'Контрольные примеры из методики и проверки граничных случаев вычисл ительного модуля.']
];
actions.help = () => {
  const path = (location.hash || '#/').slice(1).split('?')[0] || '/';
  const h = HELP.find(([re]) => re.test(path)) || ['', 'Справка', 'Выберите раздел в меню слева.'];
  dialog('Справка: ' + h[1], `<p>${esc(h[2])}</p><p class="muted" style="font-size:13px">Термины — «Спра вочник методики» → «Термины». Подробная инструкция — файл README.md рядом с приложением.</p>`, [{ label: 'Понятно', value: true, cls: 'primary' }]);
};
document.addEventListener('keydown', e => { if (e.key === 'F1') { e.preventDefault(); actions.help(); } });

/* ---------- Напоминание о резервной копии ---------- */
function backupReminder() {
  const fin = state.assessments.filter(a => a.status === 'final');
  if (!fin.length && !(state.controlRecords || []).length) return '';
  const last = state.meta.lastBackupAt;
  const since = fin.filter(a => !last || (a.finalizedAt || '') > last).length;
  const days = last ? Math.floor((Date.now() - new Date(last).getTime()) / 86400000) : null;
  if (last && days < 7 && since < 10) return '';
  const why = !last ? 'резервная копия ещё не создавалась' : days >= 7 ? `последняя резервная копия — ${days} дн. назад` : `после последней копии утверждено расчётов: ${since}`;
  return `<div class="note warn no-print"><b>Сделайте резервную копию:</b> ${why}. Данные в браузере мог ут быть утеряны при очистке истории или смене компьютера. <button class="btn sm" data-act="backup">Сохра нить копию</button></div>`;
}

function demoRng(seed) { let a = seed >>> 0; return () => { a |= 0; a = a + 0x6D2B79F5 | 0; let t = Mat h.imul(a ^ a >>> 15, 1 | a); t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t; return ((t ^ t >>> 14) >>> 0) / 4294967296; }; }
/* ====================================================================
   МОДУЛЬ 16. ДЕМОНСТРАЦИОННЫЕ ДАННЫЕ (60 ВЫМЫШЛЕННЫХ КУРСАНТОВ)
   Все записи помечены demo: true и удаляются одной кнопкой. Генерация
   детерминирована (фиксированное зерно). В двух анкетах использованы
   наборы оценок контрольных примеров методики.
   ==================================================================== */
/* 60 вымышленных курсантов: 4 взвода по 15 человек (41, 43 — экспериментальные; 42, 44 — контрольные) */
const DEMO_SURNAMES = ['Абрамов', 'Белов', 'Васильев', 'Григорьев', 'Дмитриев', 'Егоров', 'Жуков', 'Зайц ев', 'Ильин', 'Козлов', 'Лебедев', 'Морозов', 'Никитин', 'Орлов', 'Павлов',
  'Романов', 'Сергеев', 'Тарасов', 'Ушаков', 'Фёдоров', 'Харитонов', 'Цветков', 'Чернов', 'Шестаков', 'Щ ербаков', 'Юдин', 'Яковлев', 'Алексеев', 'Богданов', 'Воробьёв',
  'Гусев', 'Давыдов', 'Ершов', 'Журавлёв', 'Зуев', 'Исаев', 'Калинин', 'Лукин', 'Макаров', 'Назаров', 'О сипов', 'Попов', 'Родионов', 'Соболев', 'Титов',
  'Уваров', 'Филиппов', 'Хохлов', 'Цыганов', 'Чистяков', 'Шаров', 'Широков', 'Андреев', 'Борисов', 'Вино градов', 'Голубев', 'Денисов', 'Ефимов', 'Захаров', 'Киселёв'];
const DEMO_INITIALS = 'АБВГДЕИКЛМНОПРСТ';
const DEMO_PEOPLE = (() => {
  const r = demoRng(19820601);
  return DEMO_SURNAMES.map((sn, i) => {
    const ini = DEMO_INITIALS[Math.floor(r() * DEMO_INITIALS.length)] + '.' + DEMO_INITIALS[Math.floor(r () * DEMO_INITIALS.length)] + '.';
    const ab = Math.round((2.7 + 2.2 * (r() + r()) / 2) * 10) / 10; /* способность 2,7…4,9, чаще около с ередины */
    return [sn + ' ' + ini, (41 + Math.floor(i / 15)) + ' взвод', ab];
```

```
  });
})();
const DEMO_EG = ['41 взвод', '43 взвод'];
const hasDemo = () => state.objects.some(o => o.demo);
function loadDemo() {
  const rnd = demoRng(20260522);
  const grade = (ab, sh = 0, lo = 1) => Math.max(lo, Math.min(5, Math.round(ab + sh + (rnd() - 0.5) * 1.6)));
  const pick = arr => arr[Math.floor(rnd() * arr.length)];
  const v13 = METHOD.COEF_SETS['src-vpk13'].items, v16 = METHOD.COEF_SETS['src-vpk16'].items, p5 = METHOD.COEF_SETS['src-psk5'].items;
  const CONTROL_EX = { 1: [4, 4, 5, 5, 4, 4, 5, 5, 5, 5, 5, 5, 4], 2: [4, 3, 5, 5, 4, 4, 3, 5, 4, 4, 4, 3, 4] }; /* методика */
  const organs = ['Центральный округ', 'Северо-Западный округ', 'Южный округ', 'Уральский округ'];
  if (!state.controlRecords) state.controlRecords = []; if (!state.practiceReports) state.practiceReports = [];
  const objs = DEMO_PEOPLE.map(([name, group, ab], i) => {
    const o = { id: Store.uid(), demo: true, name, group, specialty: SPECIALTIES[0], year: '2023', notes: 'Демонстрационные данные (вымышленное лицо)', createdAt: Store.nowIso(),
      applicant: { education: pick(['школа', 'школа', 'колледж', 'лицей', 'кадетский корпус / училище']), experience: i % 5 === 3 ? 'военная служба по контракту, 2 года' : '', flags: { commissariat: rnd() < 0.7, camp: rnd() < 0.3, service: i % 5 === 3, combat: i % 9 === 3 } } };
    state.objects.push(o); return { o, ab, i };
  });
  const rec = (o, stage, semester, form, kind, markType, mark, extra = {}) => state.controlRecords.push({ id: Store.uid(), demo: true, objectId: o.id, stage, semester, date: extra.date || '', kind, type: 'external', methodFamily: extra.mf || '', method: '', form, block: (METHOD.CONTROL.forms.find(f => f.id === form) || {}).block || 'diagnostics', discipline: extra.disc || '', markType, mark, comps: extra.comps || [], note: '', createdAt: Store.nowIso() });
  const semDate = s => `${2018 + Math.floor((s - 1) / 2)}-${s % 2 ? '12-25' : '06-20'}`;
  for (const { o, ab, i } of objs) {
    /* Входной блок и журнал контроля по этапам */
    rec(o, 0, null, 'ege', 'diagnostic', 'score', Math.round(150 + (ab - 2.5) * 45 + rnd() * 20), { disc: 'сумма баллов', date: '2018-07-10' });
    rec(o, 0, null, 'profotbor', 'diagnostic', 'pass', ab > 2.9 || rnd() < 0.5 ? 'зачтено' : 'не зачтено', { date: '2018-07-20' });
    const plan = [[1, 1, 'zachet', 'final', 'pass', 'Физическая подготовка', ['ВПК-2', 'ВПК-9']], [1, 2, 'exam', 'final', 'grade', 'Тактика', ['ВПК-4', 'ВПК-5']], [1, 2, 'kontrolnaya', 'periodic', 'grade', 'Общевоинские уставы', ['ВПК-1', 'ВПК-3']],
      [2, 3, 'letuchka', 'current', 'grade', 'Тактика', ['ВПК-5']], [2, 4, 'kursovaya', 'intermediate', 'grade', 'Тактика', ['ВПК-4', 'ВПК-6']], [3, 5, 'normativy', 'current', 'grade', 'Огневая подготовка', ['ВПК-12']],
      [3, 6, 'exam', 'final', 'grade', 'Вооружение и военная техника', ['ВПК-11', 'ВПК-12']], [4, 7, 'gruppovoe', 'current', 'grade', 'Тактика', ['ВПК-5', 'ВПК-8']], [4, 8, 'tsz', 'current', 'grade', 'Тактико-специальная подготовка', ['ВПК-5', 'ВПК-7']],
      [5, 9, 'exam', 'final', 'grade', 'Управление повседневной деятельностью', ['ВПК-13', 'ВПК-14', 'ВПК-15']], [5, 10, 'gia', 'final', 'grade', 'Государственный экзамен', []], [5, 10, 'diplom', 'final', 'grade', 'Выпускная квалификационная работа', []]];
    for (const [stage, sem, form, kind, mt, disc, comps] of plan) rec(o, stage, sem, form, kind, mt, mt === 'pass' ? (grade(ab, 0, 2) >= 3 ? 'зачтено' : 'не зачтено') : grade(ab, 0, 2), { disc, comps, date: semDate(sem), mf: ['normativy', 'tsz', 'gruppovoe'].includes(form) ? 'complex' : 'traditional' });
    /* g0: оценки аттестата (18) и вступительных испытаний (3) */
    const g0a = Model.newAssessment(state, 'g0', o.id); g0a.meta.date = '2018-08-01'; g0a.meta.evaluator = 'приёмная комиссия (демо)';
    for (let k = 0; k < 18; k++) { const g = grade(ab, 0.1, 3); g0a.input.x[g] = (g0a.input.x[g] || 0) + 1; }
    for (let k = 0; k < 3; k++) { const g = grade(ab, 0, 3); g0a.input.y[g] = (g0a.input.y[g] || 0) + 1; }
    const r0 = Model.compute(g0a); g0a.result = r0.result; g0a.status = 'final'; g0a.finalizedAt = Store.nowIso(); state.assessments.push(g0a);
    /* Анкеты войсковой практики 4 и 5 курса (ЭГ — 41 и 43 взвод — улучшение к 5 курсу) */
    const eg = DEMO_EG.includes(o.group);
    const g4 = CONTROL_EX[i + 1] || v13.map(() => grade(ab, 0, 2));
    const m4 = Math.round(g4.reduce((s, v) => s + v, 0) / g4.length + (rnd() < 0.25 ? (rnd() < 0.5 ? -1 : 1) : 0));
    createAssessment('practice', o.id, { date: '2021-10-15', course: 4, evaluator: 'командир роты (демо)', evaluator2: 'руководитель практики (демо)', unit: 'в/ч 0000 (демо)' }, Object.fromEntries(v13.map((c, k) => [c.id, g4[k]])), { leader: String(Math.max(2, Math.min(5, m4))), commission: String(Math.max(2, Math.min(5, m4))) }, true);
    if (i % 20 !== 19) { /* трое отчислены до 5 курса */
      const g5 = v13.map((c, k) => Math.max(1, Math.min(5, g4[k] + (eg ? (rnd() < 0.45 ? 1 : 0) : (rnd() < 0.2 ? 1 : rnd() < 0.15 ? -1 : 0)))));
      const m5 = Math.max(2, Math.min(5, Math.round(g5.reduce((s, v) => s + v, 0) / g5.length)));
```

```
      createAssessment('practice', o.id, { date: '2022-05-16', course: 5, evaluator: 'командир роты (демо)', evaluator2: 'руководитель практики (демо)', unit: 'в/ч 0000 (демо)' }, Object.fromEntries(v13.map((c, k) => [c.id, g5[k]])), { leader: String(m5), commission: String(rnd() < 0.2 ? Math.max(2, Math.min(5, m5 + 1)) : m5) }, true);
    }
    /* Отзывы на выпускников: 1 год — 12 чел., 2 года — 6 чел. */
    const hdr = { gradYear: '2023', rank: 'лейтенант', position: 'командир взвода', primaryPosition: 'командир взвода', organ: organs[i % 4], city: '—', unitNo: '0000 (демо)' };
    if (i % 5 !== 4) { /* отзыв после 1 года — 48 чел. */
      const gg = Object.fromEntries([...v16, ...p5].map(c => [c.id, grade(ab, -0.2)]));
      const a = createAssessment('graduate', o.id, { date: '2023-12-01', period: 1, evaluator: 'командир роты (демо)', header: hdr, signers: { direct: 'капитан (демо)' } }, gg, { pskEnabled: true }, true);
      a.input.qualities = Object.fromEntries(METHOD.PERSONAL_QUALITIES.map(q => [q.id, grade(ab)])); a.input.conclusion = ab >= 4.4 ? METHOD.CONCLUSIONS[1] : ab >= 3.2 ? METHOD.CONCLUSIONS[2] : METHOD.CONCLUSIONS[3];
    }
    if (i % 5 < 2) createAssessment('graduate', o.id, { date: '2024-12-01', period: 2, evaluator: 'командир роты (демо)', header: hdr }, Object.fromEntries([...v16, ...p5].map(c => [c.id, grade(ab, 0.1)])), { pskEnabled: true }, true);
  }
  /* Учебный план (контрольный пример методика) с семестрами и матрицей ВПК; прогноз g для 41 и 42 взводов */
  const ids = Array.from({ length: 6 }, () => Store.uid());
  const rows = [[100, 80, [], 1, ['ВПК-1', 'ВПК-3']], [100, 70, [], 1, ['ВПК-2', 'ВПК-9']], [120, 90, [0], 2, ['ВПК-4', 'ВПК-5']], [80, 50, [0], 2, ['ВПК-6', 'ВПК-8']], [100, 80, [1], 2, ['ВПК-11', 'ВПК-12']], [90, 70, [3, 4], 3, ['ВПК-10', 'ВПК-13', 'ВПК-16']]];
  const cur = { id: Store.uid(), demo: true, name: 'Демо: учебный план D1–D6', createdAt: Store.nowIso(), updatedAt: Store.nowIso(), note: 'Демонстрационные данные на основе контрольного примера.',
    disciplines: rows.map((r, k) => ({ id: ids[k], name: 'D' + (k + 1), T1: r[0], T2: r[1], m: 0.8, preds: r[2].map(q => ids[q]), semester: r[3], T1min: 40, T2min: 20, final: k === 5, comps: r[4] })),
    opt: { limits: { 1: 380, 2: 520, 3: 160 }, useGamma: true, gamma: 3, students: 'typical', g0: 0.75, c: 0.5, iterations: 2000, seed: 1 } };
  state.curricula.push(cur);
  for (const { o } of objs.filter(x => ['41 взвод', '42 взвод'].includes(x.o.group))) {
    const g0 = state.assessments.find(a => a.type === 'g0' && a.objectId === o.id).result.g0;
    const ab = Object.fromEntries(ids.map(id => [id, { c: Math.round((0.2 + rnd() * 0.6) * 10) / 10 }]));
    const a = Model.newAssessment(state, 'trajectory', o.id); a.meta.date = '2019-06-30';
    Object.assign(a.input, { curriculumId: cur.id, g0, mode: '24', p: METHOD.G_MODEL.p24, abilities: ab });
    a.snapshot = Model.snapshot(state, 'trajectory', a.meta, a.input);
    const r = Model.compute(a); if (r.ok) { a.result = r.result; a.status = 'final'; a.finalizedAt = Store.nowIso(); state.assessments.push(a); }
  }
  for (const grp of DEMO_EG) state.practiceReports.push({ id: Store.uid(), demo: true, group: grp, course: 4, date: '2021-10-20', author: 'руководитель практики (демо)', createdAt: Store.nowIso(), chars: {}, docs: { 0: true, 1: true },
    items: { where: 'в/ч 0000 (демо), обязанности командира взвода', program: 'Программа войсковой практики выполнена в полном объёме (демонстрационный текст).' } });
  persist();
}
function removeDemo() {
  const ids = new Set(state.objects.filter(o => o.demo).map(o => o.id));
  state.objects = state.objects.filter(o => !o.demo);
  state.assessments = state.assessments.filter(a => !ids.has(a.objectId));
  state.controlRecords = (state.controlRecords || []).filter(r => !ids.has(r.objectId));
  state.curricula = state.curricula.filter(c => !c.demo);
  state.practiceReports = (state.practiceReports || []).filter(r => !r.demo);
  persist();
}
function demoCard() {
  const demo = Store.mode() === 'demo';
  return `<div class="card"><h2>Учебный режим и демонстрационные данные</h2>
  <p>${demo ? '<b>Сейчас включён учебный режим</b> — работа идёт в отдельном хранилище, рабочие данные не затрагиваются.' : 'Учебный режим открывает отдельное хранилище для обучения: рабочие данные в нём не видны и не изменяются.'}</p>
  <div class="row">${demo ? '<button class="btn" data-act="mode-main">Вернуться к рабочим данным</button>' : '<button class="btn" data-act="mode-demo">Перейти в учебный режим</button>'}
  ${hasDemo() ? '<button class="btn danger" data-act="demo-remove">Удалить демонстрационные данные</button>' : '<button class="btn primary" data-act="demo-load">Загрузить демонстрационные данные (60 человек)</button>'}</div>
  <p class="hint">Демонстрационные данные: 60 вымышленных курсантов четырёх взводов по 15 человек (41 и 43 — экспериментальные, 42 и 44 — контрольные); сведения абитуриентов; журнал контроля по всем этапам (ЕГЭ, профотбор, зачёты, экзамены, летучки, курсовые, нормативы, ТСЗ, ГИА, диплом); g₀; анкеты войсковой п
```

```
рактики 4 курса (60) и 5 курса (57 — трое отчислены); отзывы после 1 года (48 чел.) и 2 лет службы (24 чел.) по четырём округам; учебный план с матрицей ВПК; прогноз g для 41 и 42 взводов; отчёты о практике 41 и 43 взводов. В двух анкетах использованы оценки контрольных примеров методики. Все записи помечены как демонстрационные и удаляются одной кнопкой.</p></div>`;
}
actions['demo-load'] = async () => {
  if (Store.mode() !== 'demo') { const c = await dialog('Загрузить демонстрационные данные?', '<p>Рекомендуется загружать их в учебном режиме, чтобы не смешивать с рабочими данными. В основном режиме записи будут помечены как демонстрационные и удаляются одной кнопкой.</p>', [{ label: 'Отмена', value: null }, { label: 'В основной режим', value: 'main' }, { label: 'Перейти в учебный и загрузить', value: 'demo', cls: 'primary' }]);
    if (!c) return; if (c === 'demo') { persist(); Store.setMode('demo'); state = Store.load(); modeBanner(); } }
  if (hasDemo()) { toast('Демонстрационные данные уже загружены', true); return; }
  await withBusy('Загрузка демонстрационных данных: 60 человек…', () => loadDemo()); toast('Демонстрационные данные загружены: 60 человек'); location.hash = '#/'; render();
};
actions['demo-remove'] = async () => { if (!await confirmDlg('Удалить демонстрационные данные?', 'Будут удалены все записи, помеченные как демонстрационные (без корзины). Остальные данные не затрагиваются.', 'Удалить', true)) return; removeDemo(); toast('Демонстрационные данные удалены'); render(); };

/* ======================================================================
   МОДУЛЬ 17. ЕДИНЫЙ РАЗДЕЛ «ОТЧЁТЫ»
   Все печатные формы собраны в одном месте: по курсанту, по группе,
   документы практики, бланки и выгрузки. Сами отчёты строятся теми же
   экранами, что и раньше; раздел только помогает выбрать нужный.
   ====================================================================== */
route(/^\/reports$/, () => {
  const R = ui.rep || (ui.rep = { obj: '', group: '' });
  const objs = [...state.objects].sort((a, b) => a.name.localeCompare(b.name, 'ru'));
  if (R.obj && !objs.some(o => o.id === R.obj)) R.obj = '';
  const groups = groupList();
  const fin = state.assessments.filter(a => a.status === 'final');
  const o = objs.find(x => x.id === R.obj);
  const oFin = o ? fin.filter(a => a.objectId === o.id) : [];
  const item = (title, desc, btn, extra = '') => `<div class="rep-item"><div class="rep-t"><b>${title}</b><span class="muted">${desc}</span></div>${extra}${btn}</div>`;
  const go = (act, label, dis, data = '') => `<button class="btn ${dis ? '' : 'primary'} sm" data-act="${act}" ${data} ${dis ? 'disabled' : ''}>${label}</button>`;
  return `<div class="pagehead"><h1>Отчёты</h1></div>
  <p class="muted">Все печатные формы в одном месте. Выберите, по кому нужен отчёт, и нажмите «Открыть». На открытом отчёте кнопка «Печать / PDF» выводит его на принтер или в файл PDF.</p>
  ${!fin.length ? `<div class="note warn">Утверждённых оценок пока нет — отчёты будут пустыми. <a href="#/new">Новая оценка</a></div>` : ''}
  <div class="grid g2">
  <div class="card"><h2>${iconTile('people', '#5856D6')} По курсанту</h2>
    <div class="field"><label class="f" for="repObj">Курсант или выпускник</label><select id="repObj" data-rep="obj"><option value="">— выберите —</option>${objs.map(x => `<option value="${x.id}" ${x.id === R.obj ? 'selected' : ''}>${esc(x.name)}${x.group ? ' · ' + esc(x.group) : ''}</option>`).join('')}</select>
    ${o ? `<p class="hint">Утверждённых оценок: ${oFin.length}</p>` : ''}</div>
    ${item('Отчёт по курсанту', 'все утверждённые оценки, динамика по компетенциям', go('rep-open', 'Открыть', !o, 'data-to="report"'))}
    ${item('Диагностический профиль', 'диаграмма компетенций, отстающие компетенции, рекомендации', go('rep-open', 'Открыть', !o || !oFin.some(a => a.type === 'practice' || a.type === 'graduate'), 'data-to="profile"'))}
    ${item('Карта этапов подготовки', 'контроль и результаты по этапам: от абитуриента до службы', go('rep-open', 'Открыть', !o, 'data-to="map"'))}
    ${item('Отчёт по отдельной оценке', 'формулы, промежуточные значения, подписи', o && oFin.length ? `<select data-rep="aid" aria-label="Оценка" style="max-width:240px">${oFin.map(a => `<option value="${a.id}">${esc(Model.TYPES[a.type].short)}, ${fmtDate(a.meta.date)}</option>`).join('')}</select>` + go('rep-assess', 'Открыть', false) : go('rep-assess', 'Открыть', true))}
  </div>
  <div class="card"><h2>${iconTile('chart', '#FF9500')} По группе</h2>
    <div class="field"><label class="f" for="repGroup">Группа / подразделение</label><select id="repGroup" data-rep="group"><option value="">все группы</option>${groups.map(g => `<option ${g === R.group ? 'selected' : ''} value="${esc(g)}">${esc(g)}</option>`).join('')}</select></div>
    ${item('Сводная по группе', 'средние значения, место в группе, распределение по уровням', go('rep-group', 'Открыть', false, 'data-to="summary"'))}
    ${item('Группы риска', 'кто отстаёт и по каким признакам', go('rep-group', 'Открыть', false, 'data-to="risk"'))}
    ${item('Практика и служба', 'оценки практики против отзывов из войск', go('rep-group', 'Открыть', false, 'data-to="followup"'))}
```

```
    ${item('Отзывы из войск', 'итоги отзывов по органам управления (все группы)', go('rep-group', 'Открыть', false, 'data-to="reviews"'))}
    ${item('Сравнение групп', 'две-три группы рядом: средние и уровни', go('rep-group', 'Открыть', false, 'data-to="groups"'))}
  </div></div>
  <div class="grid g2">
  <div class="card"><h2>${iconTile('book', '#A2845E')} Документы</h2>
    ${item('Отчёты о практике', `отчёты руководителя практики (${(state.practiceReports || []).length})`, `<a class="btn primary sm" href="#/preports">Открыть</a>`)}
    ${item('Бланки для печати', 'пустые отзывы и анкеты для заполнения, в том числе на всю группу', `<a class="btn primary sm" href="#/blanks">Открыть</a>`)}
  </div>
  <div class="card"><h2>${iconTile('disk', '#007AFF')} Выгрузка данных</h2>
    ${item('Все оценки в Excel (CSV)', 'исходные оценки и результаты всех утверждённых расчётов', `<button class="btn primary sm" data-act="csv-all">Скачать</button>`)}
    ${item('Резервная копия', 'все данные в одном файле JSON', `<button class="btn primary sm" data-act="backup">Скачать</button>`)}
  </div></div>`;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.rep; if (!k || !ui.rep) return; ui.rep[k] = e.target.value; if (k !== 'aid') render(); });
actions['rep-open'] = el => { if (ui.rep.obj) location.hash = `#/object/${ui.rep.obj}/${el.dataset.to}`; };
actions['rep-assess'] = () => { const s = $('[data-rep=aid]'); if (s && s.value) location.hash = `#/a/${s.value}/report`; };
actions['rep-group'] = el => {
  const g = ui.rep.group, to = el.dataset.to;
  if (to === 'summary') ui.summary.group = g;
  if (to === 'risk') ui.risk = { ...(ui.risk || {}), group: g };
  if (to === 'followup') ui.follow.group = g;
  if (to === 'groups' && g) ui.groupsCmp.a = g;
  location.hash = '#/' + to;
};

/* ========================================================================
   МОДУЛЬ 8. ИСТОРИЯ, СВОДНЫЕ ОТЧЁТЫ, ИМПОРТ/ЭКСПОРТ, САМОПРОВЕРКА
   ======================================================================== */
route(/^\/history$/, () => {
  const f = ui.filters;
  if (hashQuery().has('draft')) f.status = 'draft';
  const q = f.q.toLowerCase().trim();
  const list = state.assessments.filter(a => (!f.type || a.type === f.type) && (!f.status || a.status === f.status)
    && (!q || objName(a.objectId).toLowerCase().includes(q) || (a.meta.evaluator || '').toLowerCase().includes(q))
    && (!f.from || (a.meta.date || '') >= f.from) && (!f.to || (a.meta.date || '') <= f.to))
    .sort((a, b) => (b.meta.date || '').localeCompare(a.meta.date || '') || b.updatedAt.localeCompare(a.updatedAt));
  return `<div class="pagehead"><h1>Все оценки</h1><span class="spacer"></span><a class="btn primary" href="#/new">${svgIcon('plus', 'bi')} Новая оценка</a></div>
  <p class="muted">Утверждённые оценки и черновики. Черновик открывается для продолжения ввода, утверждённая оценка — для просмотра результата.</p>
  <div class="card no-print"><div class="grid g4">
    <div class="field"><label class="f" for="hf-q">Поиск (ФИО, оценщик)</label><input type="search" id="hf-q" value="${esc(f.q)}" data-filter="q"></div>
    <div class="field"><label class="f" for="hf-type">Вид оценки</label><select id="hf-type" data-filter="type"><option value="">все</option>${Object.entries(Model.TYPES).map(([k, t]) => `<option value="${k}" ${f.type === k ? 'selected' : ''}>${esc(t.short)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="hf-st">Статус</label><select id="hf-st" data-filter="status"><option value="">все</option><option value="final" ${f.status === 'final' ? 'selected' : ''}>утверждён</option><option value="draft" ${f.status === 'draft' ? 'selected' : ''}>черновик</option></select></div>
    <div class="field"><label class="f">Период (дата оценки)</label><div class="row" style="flex-wrap:nowrap"><input type="date" value="${esc(f.from)}" data-filter="from" aria-label="с"><input type="date" value="${esc(f.to)}" data-filter="to" aria-label="по"></div></div>
  </div><div class="row"><button class="btn sm" data-act="hf-reset">Сбросить фильтры</button><span class="spacer"></span><button class="btn sm" data-act="csv-assess">CSV: исходные данные и результаты (по фильтру)</button></div></div>
  <div class="card"><p class="muted">Найдено: ${list.length}</p>${list.length ? `<div class="table-wrap"><table><thead><tr><th>Дата</th><th>Объект</th><th>Вид</th><th>Параметры</th><th>Статус</th><th class="num">Показатель</th><th>Уровень</th><th>Изменён</th></tr></thead><tbody>
  ${list.map(a => { const h = Model.headline(a); return `<tr class="click" data-act="go" data-href="${a.status === 'draft' ? `#/a/${a.id}/edit/1` : '#/a/' + a.id}"><td>${fmtDate(a.meta.date)}</td><td>${esc(objName(a.objectId))}</td><td>${esc(Model.TYPES[a.type].short)}</td><td>${a.type === 'practice' ? a.meta.c
```

```
ourse + ' курс' : a.type === 'graduate' ? a.meta.period + ' год' : ''}</td><td>${statusBadge(a)}${a.revi
sions.length ? ` <span class="badge b-info">ред. ${a.revisions.length + 1}</span>` : ''}</td><td class
="num">${h ? fmt(h.value, 3) : '—'}</td><td>${h && h.level !== undefined ? lvBadge(h.level, false) :
'—'}</td><td>${fmtDT(a.updatedAt)}</td></tr>`; }).join('')}
  </tbody></table></div>` : state.assessments.length ? emptyState('Ничего не найдено', 'Под выбранные ус
ловия не подходит ни одна оценка.', '<button class="btn" data-act="hf-reset">Сбросить фильтры</button>')
: emptyState('Оценок пока нет', 'Здесь появятся все анкеты, отзывы и расчёты — утверждённые и черновик
и.', '<a class="btn primary" href="#/new">Новая оценка</a><a class="btn" href="#/batch">Ввод по группе</
a>')}</div>`;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.filter; if (!
k) return; ui.filters[k] = e.target.value; if (location.hash.includes('draft')) location.hash = '#/histo
ry'; else render(); });
document.addEventListener('input', e => { if (e.target.dataset && e.target.dataset.filter === 'q') { ui.
filters.q = e.target.value; clearTimeout(ui.qt); ui.qt = setTimeout(() => { const pos = e.target.selecti
onStart; render(); const el = $('#hf-q'); if (el) { el.focus(); el.setSelectionRange(pos, pos); } }, 25
0); } });
actions['hf-reset'] = () => { ui.filters = { type: '', status: '', q: '', from: '', to: '' }; location.h
ash = '#/history'; render(); };

function assessmentRows(list) {
  const rows = [['ID записи', 'Объект', 'Группа', 'Вид оценки', 'Статус', 'Дата', 'Параметр', 'Оценщик',
'Версия методики', 'Набор Кзнк', 'Компетенция / показатель', 'Оценка', 'Кзнк', 'Псфк', 'Степень %']];
  for (const a of list) {
    const o = state.objects.find(x => x.id === a.objectId) || {};
    const param = a.type === 'practice' ? a.meta.course + ' курс' : a.type === 'graduate' ? a.meta.perio
d + ' год службы' : '';
    const base = [a.id, o.name || '', o.group || '', Model.TYPES[a.type].short, a.status === 'final' ?
'утверждён' : 'черновик', a.meta.date || '', param, a.meta.evaluator || '', a.snapshot.methodVersion];
    if (a.type === 'graduate' || a.type === 'practice') {
      const r = a.result || Model.compute(a).result; if (!r) continue;
      for (const [key, set] of [['vpk', a.snapshot.sets.vpk], ['psk', a.snapshot.sets.psk]]) {
        const res = r[key]; if (!res) continue;
        res.rows.forEach(x => rows.push([...base, set.name, x.id, x.grade ?? '', csvNum(x.k), csvNum(x.p
sfk), csvNum(x.pct)]));
        if (res.complete) rows.push([...base, set.name, 'СУММАРНЫЙ ПОКАЗАТЕЛЬ', csvNum(res.meanGrade),
'', csvNum(res.meanPsfk), csvNum(res.meanPct)]);
      }
      if (r.total) rows.push([...base, 'ВПК+ПСК', 'ИТОГОВАЯ ОЦЕНКА', csvNum(r.total.meanGrade), '', csvN
um(r.total.meanPsfk), '']);
      if (a.type === 'practice') (r.okpk || []).forEach(x => rows.push([...base, 'ОК/ПК (не в расчёте)',
x.id, x.grade ?? '', '', '', '']));
    } else {
      const h = Model.headline(a); rows.push([...base, '', h ? h.label : '', '', '', h ? csvNum(h.value)
: '', '']);
    }
  }
  return rows;
}
actions['csv-assess'] = () => {
  const f = ui.filters, q = f.q.toLowerCase().trim();
  const list = state.assessments.filter(a => (!f.type || a.type === f.type) && (!f.status || a.status ==
= f.status) && (!q || objName(a.objectId).toLowerCase().includes(q)) && (!f.from || (a.meta.date || '')
>= f.from) && (!f.to || (a.meta.date || '') <= f.to));
  if (!list.length) { toast('Нет записей для экспорта', true); return; }
  download(`расчёты_${new Date().toISOString().slice(0, 10)}.csv`, toCsv(assessmentRows(list)), 'text/cs
v;charset=utf-8');
};

/* ---------- Сводная по группе ---------- */
route(/^\/summary$/, () => {
  const S = ui.summary;
  const groups = [...new Set(state.objects.map(o => o.group).filter(Boolean))].sort((a, b) => a.localeCo
mpare(b, 'ru'));
  let list = state.assessments.filter(a => a.status === 'final' && a.type === S.type);
  if (S.group) list = list.filter(a => (state.objects.find(o => o.id === a.objectId) || {}).group === S.
group);
  if (S.sub) list = list.filter(a => String(S.type === 'practice' ? a.meta.course : a.meta.period) ===
S.sub);
  list.sort((a, b) => objName(a.objectId).localeCompare(objName(b.objectId), 'ru'));
  const sets = new Set(list.map(a => a.snapshot.sets.vpk.id));
  const vals = list.map(a => a.result.vpk.meanPsfk);
  const ranks = Calc.rank(vals);
```

```
  const counts = {}; list.forEach(a => { const lv = S.type === 'graduate' ? (a.result.vpk.levelPsfk || a.result.vpk.levelBsr) : a.result.vpk.levelBsr; if (lv) counts[lv.code] = (counts[lv.code] || 0) + 1; });
  const avg = list.length ? list.reduce((s, a) => s + a.result.vpk.meanGrade, 0) / list.length : NaN;
  ui.summaryRows = list.map((a, i) => ({ a, rank: ranks[i] }));
  return `<h1>Сводная по группе</h1><p class="muted">Средние значения критериев показателя сформированности компетенций и ранжирование (по образцу методики). Учитываются только утверждённые расчёты.</p>
  <div class="card no-print"><div class="grid g3">
    <div class="field"><label class="f" for="sm-type">Вид оценки</label><select id="sm-type" data-sum="type"><option value="practice" ${S.type === 'practice' ? 'selected' : ''}>Войсковая практика</option><option value="graduate" ${S.type === 'graduate' ? 'selected' : ''}>Отзыв на выпускника</option></select></div>
    <div class="field"><label class="f" for="sm-group">Группа / подразделение</label><select id="sm-group" data-sum="group"><option value="">все</option>${groups.map(g => `<option ${S.group === g ? 'selected' : ''}>${esc(g)}</option>`).join('')}</select></div>
    <div class="field"><label class="f" for="sm-sub">${S.type === 'practice' ? 'Курс' : 'Год службы'}</label><select id="sm-sub" data-sum="sub"><option value="">все</option>${(S.type === 'practice' ? ['4', '5'] : ['1', '2']).map(v => `<option ${S.sub === v ? 'selected' : ''}>${v}</option>`).join('')}</select></div>
  </div><div class="row"><button class="btn sm" data-act="csv-summary" ${list.length ? '' : 'disabled'}>Экспорт сводной в CSV</button><button class="btn sm" data-act="print" ${list.length ? '' : 'disabled'}>Печать / PDF</button></div></div>
  <div class="only-print"><p><b>${S.type === 'practice' ? 'Войсковая практика' : 'Отзывы на выпускников'}</b>; группа: ${esc(S.group || 'все')}; ${S.type === 'practice' ? 'курс' : 'год службы'}: ${esc(S.sub || 'все')}. Сформировано ${fmtDT(new Date().toISOString())}.</p></div>
  ${S.type === 'practice' ? discSettingsCard() : ''}
  ${sets.size > 1 ? '<div class="note warn">В выборке использованы разные наборы Кзнк — ранжирование по Псфк некорректно. Отфильтруйте выборку.</div>' : ''}
  ${!S.sub && list.length ? `<div class="note">В выборке объединены разные ${S.type === 'practice' ? 'курсы' : 'годы службы'}; один курсант может встречаться несколько раз.</div>` : ''}
  <div class="card">${list.length ? `<div class="table-wrap"><table><thead><tr><th>№</th><th>Курсант / выпускник</th><th>Группа</th><th>${S.type === 'practice' ? 'Курс' : 'Год'}</th><th class="num">Оценка руководителя (ср.)</th><th class="num">Степень сформированности, %</th><th class="num">Показатель сформированности</th><th class="num">Место</th><th>Уровень</th>${S.type === 'practice' ? '<th class="c">Оценка руководителя практики</th><th class="c">Оценка комиссии</th>' + (discRule().enabled ? '<th>Расхождение</th>' : '') : '<th class="num">Псфк ПСК</th><th>Уровень ПСК</th><th class="num">Итоговая оценка</th><th>Итоговый уровень</th>'}</tr></thead><tbody>
  ${list.map((a, i) => { const v = a.result.vpk; const lv = S.type === 'graduate' ? (v.levelPsfk || v.levelBsr) : v.levelBsr; const o = state.objects.find(x => x.id === a.objectId) || {};
    return `<tr class="click" data-act="go" data-href="#/a/${a.id}"><td>${i + 1}</td><td>${esc(o.name)}</td><td>${esc(o.group || '—')}</td><td>${S.type === 'practice' ? a.meta.course : a.meta.period}</td><td class="num">${fmt(v.meanGrade, 2)}</td><td class="num">${fmt(v.meanPct, 2)}</td><td class="num">${fmt(v.meanPsfk, 3)}</td><td class="num"><b>${ranks[i]}</b></td><td>${lvBadge(lv, false)}</td>${S.type === 'practice' ? `<td class="c">${esc(a.input.leaderMark || '—')}</td><td class="c">${esc(a.input.commissionMark || '—')}</td>${discRule().enabled ? `<td>${discBadge(discFor(a))}</td>` : ''}` : gradCols(a.result)}</tr>`; }).join('')}
  </tbody><tfoot><tr><td></td><td>Средний балл группы</td><td></td><td></td><td></td><td class="num">${fmt(avg, 2)}</td><td></td><td></td><td></td><td></td><td></td>${S.type === 'practice' ? '<td></td><td></td>' + (discRule().enabled ? `<td>${(() => { const ds = list.map(discFor).filter(Boolean); const f = ds.filter(d => d.flag); return ds.length ? `${f.length} из ${ds.length} (${fmt(f.length / ds.length * 100, 0)}%): завышено ${f.filter(d => d.diff > 0).length}, занижено ${f.filter(d => d.diff < 0).length}` : ''; })()}</td>` : '') : '<td></td><td></td><td></td><td></td><td></td>'}</tr></tfoot></table></div>
  <p class="src">Место: 1 + число курсантов с большим (неокруглённым) Псфк; равные значения делят место. Уровень: ${S.type === 'practice' ? 'по bcp (пороги Псфк для 13 ВПК не установлены)' : 'по таблице Псфк ВПК, для пользовательских наборов — по bcp'}. Правило выявления «разногласий» в методике не формализовано — при необходимости задайте его в блоке «Правило выявления расхождений».</p>
  <h3>Распределение по уровням</h3>${levelBars(counts)}
  ${sets.size === 1 ? `<h3>Показатель сформированности по курсантам</h3>${lineChart({ labels: list.map((a, i) => String(i + 1)), yMin: 0, yMax: 0.35, series: [{ name: 'Псфк ВПК', color: '#007AFF', values: vals }] })}` : ''}` : '<div class="empty">Нет утверждённых расчётов для выбранных условий.</div>'}</div>
  `;
});
document.addEventListener('change', e => { const k = e.target.dataset && e.target.dataset.sum; if (!k) return; ui.summary[k] = e.target.value; if (k === 'type') ui.summary.sub = ''; render(); });
function gradCols(r) {
  const lp = r.psk && (r.psk.levelPsfk || r.psk.levelBsr), lt = r.total && (r.total.levelPsfk || r.total.levelBsr);
  return `<td class="num">${r.psk ? fmt(r.psk.meanPsfk, 3) : '—'}</td><td>${r.psk ? lvBadge(lp, false) : '—'}</td><td class="num">${r.total ? fmt(r.total.meanPsfk, 3) : '—'}</td><td>${r.total ? lvBadge(lt, false) : '—'}</td>`;
}
actions['csv-summary'] = () => {
  const S = ui.summary; const rows = [['№', 'Курсант / выпускник', 'Группа', S.type === 'practice' ? 'Курс' : 'Год службы', 'Оценка руководителя (ср.)', 'Степень сформированности, %', 'Показатель сформированн
```

```
ости', 'Место', 'Уровень', ...(S.type === 'practice' ? ['Оценка руководителя практики', 'Оценка комисси
и'] : ['Псфк ПСК', 'Уровень ПСК', 'Итоговая оценка', 'Итоговый уровень'])]];
4635   (ui.summaryRows || []).forEach(({ a, rank }, i) => { const v = a.result.vpk; const lv = S.type === 'gr
aduate' ? (v.levelPsfk || v.levelBsr) : v.levelBsr; const o = state.objects.find(x => x.id === a.objectI
d) || {};
4636     rows.push([i + 1, o.name, o.group || '', S.type === 'practice' ? a.meta.course : a.meta.period, csvN
um(v.meanGrade), csvNum(v.meanPct), csvNum(v.meanPsfk), rank, lv ? levelInfo(lv.code).name : '', ...(S.t
ype === 'practice' ? [a.input.leaderMark || '', a.input.commissionMark || ''] : (() => { const r = a.res
ult, lp = r.psk && (r.psk.levelPsfk || r.psk.levelBsr), lt = r.total && (r.total.levelPsfk || r.total.le
velBsr); return [r.psk ? csvNum(r.psk.meanPsfk) : '', lp ? levelInfo(lp.code).name : '', r.total ? csvNu
m(r.total.meanPsfk) : '', lt ? levelInfo(lt.code).name : '']; })()]); });
4637   download(`сводная_${new Date().toISOString().slice(0, 10)}.csv`, toCsv(rows), 'text/csv;charset=utf-
8');
4638 };
4639
4640 /* ---------- Анализ отзывов на выпускников ---------- */
4641 const MARK_OF = { high: 5, mid: 4, low: 3, crit: 2 };
4642 function reviewLevel(r, basis) {
4643   if (basis === 'total') return r.total ? (r.total.levelPsfk || r.total.levelBsr) : null;
4644   if (basis === 'vpkPsfk') return r.vpk.levelPsfk || null;
4645   return r.vpk.levelBsr || null;
4646 }
4647 function reviewAnalysis() {
4648   const F = ui.reviews;
4649   const items = state.assessments.filter(a => a.status === 'final' && a.type === 'graduate' && String(a.
meta.period) === F.period
4650     && (!F.year || ((a.meta.header || {}).gradYear || '') === F.year));
4651   /* по одному (последнему по дате) отзыву на выпускника за выбранный период */
4652   const byObj = new Map();
4653   for (const a of items) { const p = byObj.get(a.objectId); if (!p || (a.meta.date || '') > (p.meta.date
|| '')) byObj.set(a.objectId, a); }
4654   const groups = new Map();
4655   for (const a of byObj.values()) {
4656     const organ = ((a.meta.header || {}).organ || '').trim() || '(не указан)';
4657     if (!groups.has(organ)) groups.set(organ, { organ, n: 0, marks: { 5: 0, 4: 0, 3: 0, 2: 0 }, undet:
0, transfer: 0, dismissal: 0 });
4658     const g = groups.get(organ); g.n++;
4659     const lv = reviewLevel(a.result, F.basis); if (lv) g.marks[MARK_OF[lv.code]]++; else g.undet++;
4660     if (a.meta.serviceChange === 'transfer') g.transfer++; if (a.meta.serviceChange === 'dismissal') g.d
ismissal++;
4661   }
4662   return { rows: [...groups.values()].sort((a, b) => a.organ.localeCompare(b.organ, 'ru')), dup: items.l
ength - byObj.size };
4663 }
4664 const sentKey = organ => `${ui.reviews.period}|${ui.reviews.year}|${organ}`;
4665 route(/^\/reviews$/, () => {
4666   const F = ui.reviews;
4667   const years = [...new Set(state.assessments.filter(a => a.type === 'graduate').map(a => (a.meta.header
|| {}).gradYear).filter(Boolean))].sort();
4668   const { rows, dup } = reviewAnalysis();
4669   const sent = state.reviewSent || {};
4670   const tot = rows.reduce((t, r) => { t.n += r.n; t.u += r.undet; for (const m of [5, 4, 3, 2]) t[m] +=
r.marks[m]; const v = Number(sent[sentKey(r.organ)]); if (Number.isFinite(v)) t.sent += v; return t; },
{ n: 0, u: 0, 5: 0, 4: 0, 3: 0, 2: 0, sent: 0 });
4671   const basisText = { total: 'по итоговой оценке сформированности компетенций (ВПК + ПСК)', vpkPsfk: 'по
суммарному Псфк ВПК (таблица методики)', bsr: 'по среднему баллу bcp ВПК' }[F.basis];
4672   return `<h1>Отзывы из войск</h1><p class="muted">Форма: «Анализ отзывов на выпускников, прослуживших 1
год (2 года) после выпуска». Учитываются утверждённые отзывы.</p>
4673   <div class="card no-print"><div class="grid g3">
4674     <div class="field"><label class="f" for="rv-p">Период</label><select id="rv-p" data-rv="period"><opt
ion value="1" ${F.period === '1' ? 'selected' : ''}>1 год службы</option><option value="2" ${F.period ==
= '2' ? 'selected' : ''}>2 года службы</option></select></div>
4675     <div class="field"><label class="f" for="rv-y">Год выпуска (из шапки отзыва)</label><select id="rv-
y" data-rv="year"><option value="">все</option>${years.map(y => `<option ${F.year === y ? 'selected' :
''}>${esc(y)}</option>`).join('')}</select></div>
4676     <div class="field"><label class="f" for="rv-b">«Уровень подготовки оценён» — основание</label><selec
t id="rv-b" data-rv="basis"><option value="total" ${F.basis === 'total' ? 'selected' : ''}>итоговая оцен
ка (ВПК + ПСК)</option><option value="vpkPsfk" ${F.basis === 'vpkPsfk' ? 'selected' : ''}>суммарный Псфк
ВПК</option><option value="bsr" ${F.basis === 'bsr' ? 'selected' : ''}>средний балл bcp ВПК</option></se
lect><div class="hint">В форме основание оценки 5/4/3/2 не указано — выберите его. Уровень переводится в
оценку: «высокий» — 5, «средний» — 4, «низкий» — 3, «критический» — 2.</div></div>
4677   </div><div class="row"><button class="btn sm" data-act="csv-reviews" ${rows.length ? '' : 'disabled'}>
Экспорт в CSV</button><button class="btn sm" data-act="print" ${rows.length ? '' : 'disabled'}>Печать /
PDF</button></div></div>
```

```
  <div class="card"><div class="only-print"><p><b>Анализ отзывов на выпускников, прослуживших ${F.period === '2' ? '2 года' : '1 год'} после выпуска</b>${F.year ? ', выпуск ' + esc(F.year) : ''}. Оценка — ${esc(basisText)}.</p></div>
  ${rows.length ? `<div class="table-wrap"><table><thead><tr><th rowspan="2">№</th><th rowspan="2">Орган военного управления</th><th rowspan="2" class="num">Количество выпускников, направленных для прохождения службы</th><th rowspan="2" class="num">Поступило отзывов, всего</th><th colspan="4" class="c">Уровень подготовки выпускников оценён</th><th rowspan="2">Примечание</th></tr><tr><th class="c">5</th><th class="c">4</th><th class="c">3</th><th class="c">2</th></tr></thead><tbody>
  ${rows.map((r, i) => `<tr><td>${i + 1}</td><td>${esc(r.organ)}</td><td class="num"><input type="number" min="0" class="cell no-print" data-sent="${esc(r.organ)}" value="${esc(sent[sentKey(r.organ)] ?? '')}" aria-label="Направлено: ${esc(r.organ)}"><span class="only-print">${esc(sent[sentKey(r.organ)] ?? '—')}</span></td><td class="num">${r.n}</td>${[5, 4, 3, 2].map(m => `<td class="c">${r.marks[m]}</td>`).join('')}<td>${[r.undet ? `уровень не определён: ${r.undet}` : '', r.transfer ? `переведено: ${r.transfer}` : '', r.dismissal ? `уволено: ${r.dismissal}` : ''].filter(Boolean).join('; ') || ''}</td></tr>`).join('')}
  </tbody><tfoot><tr><td></td><td>Итого</td><td class="num">${tot.sent || '—'}</td><td class="num">${tot.n}</td>${[5, 4, 3, 2].map(m => `<td class="c">${tot[m]}</td>`).join('')}<td>${tot.u ? 'уровень не определён: ' + tot.u : ''}</td></tr></tfoot></table></div>
  <p class="src">Орган военного управления — из поля «Округ (орган военного управления)» шапки отзыва. Количество направленных выпускников вводится вручную (в отзывах этих сведений нет). На каждого выпускника учитывается последний отзыв за период${dup ? ` (не учтено повторных: ${dup})` : ''}. Оценка ${esc(basisText)}${F.basis === 'total' ? '; без ПСК итоговая оценка не определяется' : ''}.</p>` : '<div class="empty">Нет утверждённых отзывов за выбранный период.</div>'}</div>`;
});
document.addEventListener('change', e => {
  const d = e.target.dataset || {};
  if (d.rv) { ui.reviews[d.rv] = e.target.value; render(); }
  if (d.sent !== undefined) { const v = e.target.value === '' ? undefined : Number(e.target.value); if (!state.reviewSent) state.reviewSent = {};
    if (v !== undefined && !(Number.isInteger(v) && v >= 0)) { e.target.classList.add('invalid'); toast('Количество — целое неотрицательное число', true); return; }
    if (v === undefined) delete state.reviewSent[sentKey(d.sent)]; else state.reviewSent[sentKey(d.sent)] = v; persist(); render(); }
});
actions['csv-reviews'] = () => {
  const { rows } = reviewAnalysis(); const sent = state.reviewSent || {};
  const out = [['№', 'Орган военного управления', 'Количество выпускников, направленных для прохождения службы', 'Поступило отзывов, всего', 'Оценено 5', 'Оценено 4', 'Оценено 3', 'Оценено 2', 'Уровень не определён', 'Переведено', 'Уволено']];
  rows.forEach((r, i) => out.push([i + 1, r.organ, sent[sentKey(r.organ)] ?? '', r.n, r.marks[5], r.marks[4], r.marks[3], r.marks[2], r.undet, r.transfer, r.dismissal]));
  download(`анализ_отзывов_${ui.reviews.period}год_${new Date().toISOString().slice(0, 10)}.csv`, toCsv(out), 'text/csv;charset=utf-8');
};

/* ---------- Данные и резервные копии ---------- */
route(/^\/data$/, () => {
  const st = Store.status();
  const size = (() => { try { return new Blob([JSON.stringify(state)]).size; } catch (e) { return 0; } })();
  return `<h1>Данные и резервные копии</h1>
  <div class="card"><h2>Хранение</h2>
  <p>Данные сохраняются в хранилище браузера (localStorage) на этом компьютере, отдельно для каждого браузера. Состояние: <b>${st.available ? 'доступно' : 'недоступно'}</b>${st.error ? ' — ' + esc(st.error) : ''}. Объём данных: ${(size / 1024).toFixed(1)} КБ.</p>
  <div class="note warn">При открытии файла через file:// некоторые браузеры (или режим «инкогнито», очистка истории, политики безопасности) могут не сохранять данные или хранить их отдельно для разных папок. Гарантированный способ сохранить работу — регулярная резервная копия в файл JSON.</div>
  <p class="muted">Последняя резервная копия: ${state.meta.lastBackupAt ? fmtDT(state.meta.lastBackupAt) : 'не создавалась'}.</p></div>
  <div class="card"><h2>Резервная копия</h2><div class="row"><button class="btn primary" data-act="backup">Сохранить резервную копию (JSON)</button>
    <label class="btn" for="importFile">Восстановить из файла…</label><input type="file" id="importFile" accept=".json,application/json" hidden></div>
  <p class="hint">Резервная копия содержит объекты оценки, все расчёты со снимками методики, редакции, пользовательские наборы Кзнк, экспертизы, учебные планы, модели признаков, журнал контроля, отчёты о практике и настройки.</p></div>
  <div class="card"><h2>Экспорт для анализа</h2><div class="row"><button class="btn" data-act="csv-all">CSV: все расчёты (исходные данные и результаты)</button><button class="btn" data-act="csv-objects">CSV: объекты оценки</button><button class="btn" data-act="json-results">JSON: утверждённые результаты</button></div>
  <p class="hint">CSV — разделитель «;», десятичная запятая, кодировка UTF-8 с BOM (открывается в Microsoft Excel).</p></div>
  ${typeof demoCard === 'function' ? demoCard() : ''}
```

```
  <div class="card"><h2>Очистка</h2><button class="btn danger" data-act="wipe">Удалить все данные</button> <a class="btn" href="#/trash">Корзина</a><p class="hint">Перед удалением будет предложено сохранить резервную копию.</p></div>`;
});
actions.backup = () => {
  state.meta.lastBackupAt = Store.nowIso(); persist();
  download(`ПФК-ВПК_резервная_копия_${new Date().toISOString().slice(0, 16).replace(/[:T]/g, '-')}.json`, JSON.stringify(Store.exportPayload(state), null, 1), 'application/json');
  toast('Резервная копия сохранена'); render();
};
actions['csv-all'] = () => { if (!state.assessments.length) { toast('Нет расчётов', true); return; } download(`расчёты_все_${new Date().toISOString().slice(0, 10)}.csv`, toCsv(assessmentRows(state.assessments)), 'text/csv;charset=utf-8'); };
actions['csv-objects'] = () => download(`объекты_${new Date().toISOString().slice(0, 10)}.csv`, toCsv([['ID', 'Фамилия И.О.', 'Группа', 'Специальность', 'Год/курс', 'Примечание'], ...state.objects.map(o => [o.id, o.name, o.group, o.specialty, o.year, o.notes])]), 'text/csv;charset=utf-8');
actions['json-results'] = () => {
  const out = state.assessments.filter(a => a.status === 'final').map(a => ({ id: a.id, object: objName(a.objectId), type: a.type, date: a.meta.date, meta: a.meta, methodVersion: a.snapshot.methodVersion, input: a.input, result: a.result }));
  download(`результаты_${new Date().toISOString().slice(0, 10)}.json`, JSON.stringify({ app: 'ПФК-ВПК калькулятор', exportedAt: Store.nowIso(), results: out }, null, 1), 'application/json');
};
document.addEventListener('change', async e => {
  if (e.target.id !== 'importFile' || !e.target.files.length) return;
  const file = e.target.files[0]; e.target.value = '';
  const text = await file.text();
  const p = Store.parseImport(text);
  if (!p.ok) { toast('Импорт невозможен: ' + p.error, true); return; }
  const d = p.data;
  const choice = await dialog('Восстановление данных', `<p>Файл «${esc(file.name)}»: объектов ${d.objects.length}, расчётов ${d.assessments.length}, наборов Кзнк ${d.coefSets.length}, учебных планов ${d.curricula.length}, моделей Y ${d.yModels.length}, записей журнала контроля ${(d.controlRecords || []).length}.</p><p><b>Заменить</b> — текущие данные будут удалены. <b>Объединить</b> — добавятся записи с новыми идентификаторами, существующие не изменятся.</p>`,
    [{ label: 'Отмена', value: null }, { label: 'Объединить', value: 'merge' }, { label: 'Заменить', value: 'replace', cls: 'danger' }]);
  if (!choice) return;
  if (choice === 'replace') state = d;
  else for (const k of ['objects', 'assessments', 'coefSets', 'expertStudies', 'curricula', 'yModels', 'practiceReports', 'controlRecords']) { const ids = new Set(state[k].map(x => x.id)); state[k].push(...d[k].filter(x => !ids.has(x.id))); }
  persist(); toast('Данные восстановлены'); render();
});
actions.wipe = async () => {
  const c = await dialog('Удалить все данные?', '<p>Будут удалены все объекты, расчёты, редакции и пользовательские справочники. Действие необратимо.</p>', [{ label: 'Отмена', value: null }, { label: 'Сначала сохранить копию', value: 'backup' }, { label: 'Удалить всё', value: 'wipe', cls: 'danger' }]);
  if (c === 'backup') { actions.backup(); return; }
  if (c !== 'wipe') return;
  state = Store.empty(); persist(); toast('Данные удалены'); location.hash = '#/';
};

/* ---------- Самопроверка ---------- */
route(/^\/tests$/, () => {
  const t0 = performance.now(); const res = SelfTests.run(); const dt = performance.now() - t0;
  const storage = (() => { try { const k = 'pfk-selftest'; localStorage.setItem(k, JSON.stringify({ v: 1 })); const ok = JSON.parse(localStorage.getItem(k)).v === 1; localStorage.removeItem(k); return ok; } catch (e) { return false; } })();
  res.push({ group: 'Среда', name: 'Запись и чтение localStorage', ok: storage, detail: storage ? '' : 'хранилище недоступно — используйте резервные копии' });
  const failed = res.filter(r => !r.ok);
  return `<h1>Проверка расчётов</h1>
  <div class="note ${failed.length ? 'err' : 'ok'}"><b>${res.length - failed.length} из ${res.length} проверок пройдено</b> за ${dt.toFixed(0)} мс. ${failed.length ? 'Обнаружены ошибки — результаты расчётов могут быть недостоверны.' : 'Контрольные примеры методики воспроизводятся.'}</div>
  <div class="card"><div class="table-wrap"><table><thead><tr><th>Группа</th><th>Проверка</th><th>Результат</th></tr></thead><tbody>
  ${res.map(r => `<tr><td>${esc(r.group)}</td><td>${esc(r.name)}${r.detail ? `<div class="src" style="color:var(--danger)">${esc(r.detail)}</div>` : ''}</td><td>${r.ok ? '<span class="badge b-final">пройдено</span>' : '<span class="badge b-draft" style="color:var(--danger)">ошибка</span>'}</td></tr>`).join('')}
  </tbody></table></div></div>`;
});
```

```

/* ---------- Запуск ---------- */
$('#menuBtn').addEventListener('click', () => { const s = $('#side'); s.classList.toggle('open'); $('#menuBtn').setAttribute('aria-expanded', s.classList.contains('open')); });
$('#sideScrim').addEventListener('click', () => { $('#side').classList.remove('open'); $('#menuBtn').setAttribute('aria-expanded', 'false'); });
document.addEventListener('keydown', e => { if (e.key === 'Escape' && $('#side').classList.contains('open')) { $('#side').classList.remove('open'); $('#menuBtn').setAttribute('aria-expanded', 'false'); } });
/* Поиск раздела в меню: по названиям и пояснениям всех экранов */
$('#navSearch').addEventListener('input', () => { const path = (location.hash || '#/').slice(1).split('?')[0] || '/'; renderNav(path, locate(path)); });
$('#navSearch').addEventListener('keydown', e => {
  if (e.key === 'Enter') { const a = $('#nav a[href^="#/"]'); if (a) { e.target.value = ''; location.hash = a.getAttribute('href'); } }
  if (e.key === 'Escape') { e.target.value = ''; e.target.dispatchEvent(new Event('input')); }
});
$('#nav').addEventListener('click', e => { if (e.target.closest('a[href^="#/"]') && $('#navSearch').value) { $('#navSearch').value = ''; } });
window.addEventListener('hashchange', () => { if (!/^#\/a\/[\w-]+\/edit/.test(location.hash)) { if (ui.editing) persist(); ui.editing = null; ui.snapNote = null; } render(); });
window.addEventListener('beforeunload', () => { clearTimeout(ui.saveTimer); Store.save(state); });
window.addEventListener('storage', e => { if (e.key && e.key.startsWith('pfk-vpk-calculator')) { state = Store.load(); render(); toast('Данные обновлены в другой вкладке'); } });
renderStorage();
if (typeof purgeTrash === 'function') purgeTrash();
if (typeof modeBanner === 'function') modeBanner();
render();
setTimeout(() => { if (typeof backupReminder === 'function' && backupReminder()) toast('Напоминание: сделайте резервную копию данных (раздел «Данные и резервные копии»)'); }, 600);

</script>
</body>
</html>

```